

From Principal Subspaces to Principal Components with Linear Autoencoders

Elad Plaut

Abstract—The autoencoder is an effective unsupervised learning model which is widely used in deep learning. It is well known that an autoencoder with a single fully-connected hidden layer, a linear activation function and a squared error cost function trains weights that span the same subspace as the one spanned by the principal component loading vectors, but that they are not identical to the loading vectors. In this paper, we show how to recover the loading vectors from the autoencoder weights.

I. BACKGROUND

PRINCIPAL COMPONENT ANALYSIS (PCA) is a linear transformation that transforms a set of observations to a new coordinate system in which the values of the first coordinate have the largest possible variance, and the values of each succeeding coordinate have the largest possible variance under the constraint that they are uncorrelated with the preceding coordinates. They are often found by either computing the eigendecomposition of the covariance matrix or by computing the singular value decomposition of the observations.

By keeping only the first few principal components, PCA can be used for dimensionality reduction. The decorrelation of the coordinates is also a useful property, and PCA is sometimes used as a preprocessing step for whitening a dataset before using it as an input into an optimization problem such as a neural network classifier.

One of the properties of PCA is that out of all possible linear transformations, the reconstructions of the observations from the leading principal components have the least total squared error. Autoencoders are neural networks that aim to minimize the error of reconstructions of observations. It is well known that an autoencoder with a single fully-connected hidden layer, a linear activation function and a squared error cost function is closely related to PCA - its weights span the principal subspace, which is the subspace spanned by the first loading vectors [1], [2]. However, they are not equal to the loading vectors.

This paper proposes a simple method for recovering the loading vectors from the weights of a linear autoencoder. This allows the usage of autoencoders for computing PCA, unlike existing methods which merely find the principal subspace. After recovering the loading vectors, the solution has the following properties: (i) it is unique; (ii) in the transformed data, different coordinates are uncorrelated; (iii) the coordinates are sorted in descending order of variance; and (iv) the solutions for reduction to different dimensions are nested: when reducing the data from dimension n to dimension m_1 , the first m_2 vectors ($m_2 < m_1$) are the same as the solution for reduction from dimension n to m_2 . These properties do not hold for a general basis for the principal subspace, which is what the autoencoder weights converge to.

Even though any method for computing PCA using autoencoders is highly inefficient for small datasets, for large datasets such a method is desirable because of its various advantages compared to the standard methods. Autoencoders can be trained by a variety of stochastic optimization methods that have been developed for training deep neural networks. These optimizers can handle high-dimensional training data such as images, and a large number of them. They are suitable for the online learning scenario in which new data arrives over time, and they do not require subtracting from each example the element-wise mean of the entire training set. While several online PCA methods have been proposed for meeting these demands [3], [4], [5], our method is the first to simply recover the loading vectors from the weights of an autoencoder.

II. INTRODUCTION

A. Principal Component Analysis

Let $\{\mathbf{y}_i\}_{i=1}^N$ be a set of N observation vectors, each of dimension n . We assume that $n \leq N$, which is the more common scenario in machine learning, but a similar analysis can be done for the case of $n > N$. Let $\mathbf{Y} \in \mathbb{R}^{n \times N}$ be a matrix whose columns are $\{\mathbf{y}_i\}_{i=1}^N$,

$$\mathbf{Y} = \begin{bmatrix} | & & | \\ \mathbf{y}_1 & \cdots & \mathbf{y}_N \\ | & & | \end{bmatrix}.$$

The element-wise average of the N observations is an n dimensional signal which may be written as:

$$\bar{\mathbf{y}} = \frac{1}{N} \sum_{i=1}^N \mathbf{y}_i = \frac{1}{N} \mathbf{Y} \mathbf{1}_N,$$

where $\mathbf{1}_N \in \mathbb{R}^{N \times 1}$ is a column vector of all-ones. Let $\mathbf{Y}_0$ be a matrix whose columns are the centered observations (we center each observation $\mathbf{y}_i$ by subtracting $\bar{\mathbf{y}}$ from it):

$$\mathbf{Y}_0 = \mathbf{Y} - \bar{\mathbf{y}} \mathbf{1}_N^T.$$

A linear transformation of a finite dimensional vector may be expressed as a matrix multiplication:

$$\mathbf{x}_i = \mathbf{W}^T \mathbf{y}_i,$$

where $\mathbf{y}_i \in \mathbb{R}^n$, $\mathbf{x}_i \in \mathbb{R}^m$, and $\mathbf{W} \in \mathbb{R}^{n \times m}$. Each element j in the vector $\mathbf{x}_i$ is an inner product between $\mathbf{y}_i$ and the j -th column of $\mathbf{W}$, which we denote by $\mathbf{w}_j$.

Let $\mathbf{X} \in \mathbb{R}^{m \times N}$ be a matrix whose columns are the set of N vectors of transformed observations, let $\bar{\mathbf{x}} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i = \frac{1}{N} \mathbf{X} \mathbf{1}_N$ be the element-wise average, and $\mathbf{X}_0 = \mathbf{X} - \bar{\mathbf{x}} \mathbf{1}_N^T$ the centered matrix. Clearly, $\mathbf{X} = \mathbf{W}^T \mathbf{Y}$ and $\mathbf{X}_0 = \mathbf{W}^T \mathbf{Y}_0$.

When the matrix W^T represents the transformation that applies principal component analysis, we denote $W = P$, and the columns of P , denoted $\{\mathbf{p}_j\}_{j=1}^n$, are referred to as *loading vectors*. The transformed vectors $\{\mathbf{x}_i\}_{i=1}^N$ are referred to as *principal components* or *scores*. The first loading vector is defined as the unit vector with which the inner products of the observations have the greatest variance:

$$\mathbf{p}_1 = \max_{\mathbf{w}_1} \mathbf{w}_1^T Y_0 Y_0^T \mathbf{w}_1 \text{ s.t. } \mathbf{w}_1^T \mathbf{w}_1 = 1. \quad (1)$$

The solution to (1) is known to be the eigenvector of the sample covariance matrix $Y_0 Y_0^T$ corresponding to its largest eigenvalue. We normalize the eigenvector and disregard its sign. Next, $\mathbf{p}_2$ is the unit vector which has the largest variance of inner products between it and the observations after removing the orthogonal projections of the observations onto $\mathbf{p}_1$. It may be found by solving:

$$\mathbf{p}_2 = \max_{\mathbf{w}_2} \mathbf{w}_2^T (Y_0 - \mathbf{p}_1 \mathbf{p}_1^T Y_0) (Y_0 - \mathbf{p}_1 \mathbf{p}_1^T Y_0)^T \mathbf{w}_2 \text{ s.t. } \mathbf{w}_2^T \mathbf{w}_2 = 1. \quad (2)$$

The solution to (2) is known to be the eigenvector corresponding to the largest eigenvalue under the constraint that it is not collinear with $\mathbf{p}_1$. Similarly, the remaining loading vectors are equal to the remaining eigenvectors of $Y_0 Y_0^T$ corresponding to descending eigenvalues. The eigenvalues of $Y_0 Y_0^T$, which is a positive semi-definite matrix, are non-negative. They are not necessarily distinct, but since it is a symmetric matrix it has n eigenvectors that are all orthogonal, and it is always diagonalizable. Thus, the matrix P may be computed by diagonalizing the covariance matrix:

$$Y_0 Y_0^T = P \Lambda P^{-1} = P \Lambda P^T,$$

where $\Lambda = X_0 X_0^T$ is a diagonal matrix whose diagonal elements $\{\lambda_i\}_{i=1}^n$ are sorted in descending order, the columns of P are orthonormal, i.e. $P^{-1} = P^T$. The transformation back to the observations is $Y = PX$. The fact that the covariance matrix of X is diagonal means that PCA is a decorrelation transformation. By dividing each coordinate by the square root of its corresponding eigenvalue, PCA can be used as a whitening transformation, which is sometimes used as a preprocessing step in order to cause optimization problems to converge more easily.

B. Dimensionality reduction

PCA is often used as a method for dimensionality reduction, the process of reducing the number of variables in a model in order to avoid the curse of dimensionality. This is done by simply keeping the first m principal components ($m < n$), i.e., applying the truncated transformation

$$X_m = P_m^T Y,$$

where each column of $X_m \in \mathbb{R}^{m \times N}$ is a vector whose elements are the first m principal components, and P_m is a matrix whose columns are the first m loading vectors,

$$P_m = \begin{bmatrix} | & & | \\ \mathbf{p}_1 & \cdots & \mathbf{p}_m \\ | & & | \end{bmatrix} \in \mathbb{R}^{n \times m}.$$

Intuitively, by keeping only m principal components, we are losing information, and we minimize this loss of information by maximizing their variances. Many iterative algorithms (e.g., QR algorithm, Jacobi algorithm, and the power method) can efficiently find the m largest eigenvalues of $Y_0 Y_0^T$ and their corresponding eigenvectors without having to fully diagonalize the matrix. Yet, for high dimensional data, computing $Y_0 Y_0^T$ may be prohibitive.

C. Singular-value decomposition

Any matrix $Y_0 \in \mathbb{R}^{n \times N}$ may be factorized as $Y_0 = U \Sigma V^T$, where $U \in \mathbb{R}^{n \times n}$ and $V \in \mathbb{R}^{N \times N}$ are both orthogonal matrices and $\Sigma \in \mathbb{R}^{n \times N}$ is a matrix whose elements are non-negative real numbers on the diagonal and zero elsewhere. The diagonal elements $\{\sigma_i\}_{i=1}^n$, referred to as singular values, are sorted in descending order, and the columns of U and V are referred to, respectively, as left and right singular vectors. Assuming $n \leq N$ and Y_0 is full-rank, the columns of U are an orthonormal basis for $\mathbb{R}^n$. If $n > N$ (the number of observation is smaller than the dimension of each observation), then the first n columns of U are an orthonormal basis for the column space of Y_0 , and the remaining $N - n$ columns are an orthonormal basis for its nullspace.

The covariance matrix may be written as:

$$Y_0 Y_0^T = U \Sigma V^T V \Sigma^T U^T = U \Sigma \Sigma^T U^{-1},$$

where $\Sigma \Sigma^T$ is a diagonal matrix. Thus, singular-value decomposition of Y_0 is equivalent to eigendecomposition of $Y_0 Y_0^T$. The singular values of Y_0 are the square roots of the eigenvalues of $Y_0 Y_0^T$, and the left singular vectors of Y_0 are the eigenvectors of $Y_0 Y_0^T$. Eigendecomposition is unique up to the scale of the eigenvectors, which we normalize, and to permutations of the eigenvectors and their corresponding eigenvalues, which we sort in descending order. Therefore, the left singular vectors of Y_0 must be equal to the loading vectors of Y (up to their sign, which we disregard).

Iterative algorithms (e.g., QR algorithm, one-sided Jacobi algorithm) can efficiently find the m largest singular values and their corresponding left singular vectors without having to fully decompose the matrix.

D. PCA and large sets of high-dimensional data

When the number of observations N is small enough to fit in memory, SVD is often the preferred method for computing the loading vectors, as it avoids computing the covariance matrix $Y_0 Y_0^T$, which is desirable especially when n is large.

When the number of observations N is large but the dimension n is sufficiently small, $Y_0 Y_0^T = \sum_{i=1}^N (\mathbf{y}_i - \bar{\mathbf{y}})(\mathbf{y}_i - \bar{\mathbf{y}})^T$ may be computed sequentially, with a memory requirement of $O(n^2)$ instead of $O(nN)$. Thus, there is no need to load the entire dataset into memory.

However, when the number of observations is large and each observation is high-dimensional (i.e., both N and n are large), this too may be infeasible. Online methods [3], [4], [5] iterate through the dataset one example at a time or in minibatches. When applied to images, the images are more often divided

into small patches, and PCA is applied to patches rather than to the entire images; this is known as local PCA [6].

E. Minimum total squared reconstruction error

Interestingly, P_m is also a solution to:

$$\min_{W \in \mathbb{R}^{n \times m}} \|Y_0 - WW^T Y_0\|_F^2 \text{ s.t. } W^T W = I_{m \times m}, \quad (3)$$

where F denotes the Frobenius norm. According to this formulation, the m leading loading vectors are an orthonormal basis which spans the m dimensional subspace onto which the projections of the centered observations have the minimum squared difference from the original centered observations. In other words, P_m compresses each centered vector of length n into a vector of length m (where $m \leq n$) in such a way that minimizes the sum of total squared reconstruction errors.

It is well known [7] and easily verified that P_m indeed solves (3). Yet, the minimizer of (3) is not unique: $W = P_m Q$ is also a solution, where $Q \in \mathbb{R}^{m \times m}$ is any orthogonal matrix, $Q^T = Q^{-1}$. Multiplying P_m from the right by Q transforms the first m loading vectors into a different orthonormal basis for the same subspace.

F. Autoencoders

A neural network that is trained to learn the identity function is called an autoencoder. Its output layer has the same number of nodes as the input layer, and the cost function is some measure of the reconstruction error. Autoencoders are unsupervised learning models, and they are often used for the purpose of dimensionality reduction [8]. A simple autoencoder that implements dimensionality reduction is a feed-forward autoencoder with at least one layer that has a smaller number of nodes, which functions as a bottleneck. After training the neural network using backpropagation, it is separated into two parts: the layers up to the bottleneck are used as an encoder, and the remaining layers are used as a decoder.

In the simplest case, there is only one hidden layer (the bottleneck), and the layers in the network are fully connected. A vector $\mathbf{y}_i \in \mathbb{R}^{n \times 1}$ passes through the hidden layer, which outputs $\mathbf{x}_i \in \mathbb{R}^{m \times 1}$ according to the mapping $\mathbf{x}_i = a(W_1 \mathbf{y}_i + \mathbf{b}_1)$, where $W_1 \in \mathbb{R}^{m \times n}$ is referred to as the weight matrix of the first layer, $\mathbf{b}_1 \in \mathbb{R}^{m \times 1}$ is referred to as the bias vector of the first layer, and $m < n$. The function a , referred to as the activation function, operates element-wise and is typically a non-linear function such as the rectified linear unit (ReLU). The second layer maps $\mathbf{x}_i$ to $\hat{\mathbf{y}}_i \in \mathbb{R}^{n \times 1}$ according to $\hat{\mathbf{y}}_i = a(W_2 \mathbf{x}_i + \mathbf{b}_2) = a(W_2 a(W_1 \mathbf{y}_i + \mathbf{b}_1) + \mathbf{b}_2)$, where $W_2 \in \mathbb{R}^{n \times m}$ and $\mathbf{b}_2 \in \mathbb{R}^{n \times 1}$ are the weight matrix and bias vector of the second layer. The parameters $W_1, \mathbf{b}_1, W_2, \mathbf{b}_2$ are found by minimizing some cost function measuring the difference between the output $\hat{\mathbf{y}}_i$ and the input $\mathbf{y}_i$. Using backpropagation with an optimizer such as stochastic gradient descent, each data sample from $\{\mathbf{y}_i\}_{i=1}^N$ is fed through the network to compute $\mathbf{x}_i$ and $\hat{\mathbf{y}}_i$, which are then used to compute the gradients and to update the parameters.

G. Linear autoencoders

In the case that no non-linear activation function is used, $\mathbf{x}_i = W_1 \mathbf{y}_i + \mathbf{b}_1$ and $\hat{\mathbf{y}}_i = W_2 \mathbf{x}_i + \mathbf{b}_2$. If the cost function is the total squared difference between output and input, then training the autoencoder on the input data matrix Y solves:

$$\min_{W_1, \mathbf{b}_1, W_2, \mathbf{b}_2} \|Y - (W_2 (W_1 Y + \mathbf{b}_1 \mathbf{1}_N^T) + \mathbf{b}_2 \mathbf{1}_N^T)\|_F^2. \quad (4)$$

In [1], it is shown that if we set the partial derivative with respect to $\mathbf{b}_2$ to zero and insert the solution into (4), then the problem becomes:

$$\min_{W_1, W_2} \|Y_0 - W_2 W_1 Y_0\|_F^2$$

Thus, for any $\mathbf{b}_1$, the optimal $\mathbf{b}_2$ is such that the problem becomes independent of $\mathbf{b}_1$ and of $\mathbf{y}$. Therefore, we may focus only on the weights W_1, W_2 . In [2] it is shown that when setting the gradients to zero, W_1 is the left Moore-Penrose pseudoinverse of W_2 (and W_2 is the right pseudoinverse of W_1):

$$W_1 = W_2^\dagger = (W_2^T W_2)^{-1} W_2^T$$

Thus, the minimization remains with respect to a single matrix:

$$\min_{W_2 \in \mathbb{R}^{n \times m}} \|Y_0 - W_2 W_2^\dagger Y_0\|_F^2 \quad (5)$$

The matrix $W_2 W_2^\dagger = W_2 (W_2^T W_2)^{-1} W_2^T$ is the orthogonal projection operator onto the column space of W_2 when its columns are not necessarily orthonormal. This problem is very similar to (3), but without the orthonormality constraint.

In [1] and [2] it is shown that W_2 is a minimizer of (5) if and only if its column space is spanned by the first m loading vectors of Y . This can also be shown by applying the QR decomposition to W_2 , which transforms the problem (5) into the one in (3). As a result, it is possible to solve (3) by first solving the unconstrained problem (5), and then orthonormalizing the columns of the solution, e.g. using the Gram-Schmidt process. However, this does not recover the loading vectors P_m , but rather $P_m Q$ for some unknown orthogonal matrix Q .

The linear autoencoder is said to apply PCA to the input data in the sense that its output is a projection of the data onto the low dimensional principal subspace. However, unlike actual PCA, the coordinates of the output of the bottleneck are correlated and are not sorted in descending order of variance. In addition, the solutions for reduction to different dimensions are not nested: when reducing the data from dimension n to dimension m_1 , the first m_2 vectors ($m_2 < m_1$) are not an optimal solution to reduction from dimension n to m_2 , which therefore requires training an entirely new autoencoder.

Several methods have been proposed for neural networks that compute the exact loading vectors [9], [10], [11], [12], [13]. However, they require specific algorithms for iteratively updating the weights, and as such are similar to online PCA methods. No method has so far been proposed for recovering the loading vectors from a simple linear autoencoder that is independent of the optimization method used for training it.

III. METHOD

Hypothesis: The first m loading vectors of Y are the first m left singular vectors of the matrix W_2 which minimizes (5).

Note: an earlier version of this work attempted to prove this statement, but was found to be erroneous.

In other words, instead of computing the first m left singular vectors of $Y_0 \in \mathbb{R}^{n \times N}$, we may train a linear autoencoder on the (non-centered) dataset Y and then compute the first m left singular vectors of $W_2 \in \mathbb{R}^{n \times m}$, where typically $m \ll N$. The loading vectors may also be recovered from the weights of the hidden layer, W_1 . If $W_2 = U\Sigma V^T$, then $W_1 = W_2^\dagger = V\Sigma^\dagger U^T$, and $W_1^T = U(\Sigma^\dagger)^T V^T$. Thus, the first m left singular vectors of $W_1^T \in \mathbb{R}^{n \times m}$ are also equal to the first m loading vectors of Y .

IV. EXPERIMENTS

An important advantage of applying PCA using a linear autoencoder is that it is very simple to implement using popular machine learning frameworks. We release a sample implementation of a linear autoencoder and the recovery of the loading vectors from its weights which uses the Keras library. It can be found at <https://github.com/plaut/linear-ae-pca>.

As in any neural network, we initialize the values of the parameters, including W_2 , to random numbers. Notice that the optimization is convex over W_2 for a fixed W_1 and it is convex over W_1 for a fixed W_2 , but it is not jointly convex and has many saddle points which may be far from optimal. The optimization may benefit from a more sophisticated update than basic stochastic gradient descent, and we chose to use the Adam optimizer. Weight decay regularization, which penalizes unreasonable factorizations, was also found to be beneficial.

A. MNIST

We trained a linear autoencoder on the MNIST training dataset [16], which contains 60,000 grayscale images of handwritten digits, each of size 28×28 . Fig. 1 shows a few examples of images from the dataset. We set the dimensions of the network for reduction from a dimension of $28 \times 28 = 784$ to a dimension of 16. Then, we applied our method for recovering the loading vectors from the weights of the autoencoder.

For comparison, Fig. 2(a) shows the first 16 loading vectors as computed using the standard method of applying SVD to the entire dataset after centering it. Fig. 2(b) shows the columns of W_2 , and Fig. 2(c) shows the columns of U_m (the first m left singular vectors of W_2). It is evident that although the columns of W_2 are entirely different from the loading vectors, their left

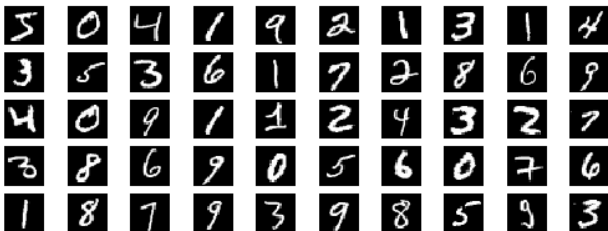


Fig. 1. Examples of images from the MNIST dataset.

singular vectors are approximately equal to them up to sign. Seven of the vectors have opposite signs, which is reflected in their inverted gray levels.

After computing the loading vectors, we applied three different transformations to the centered: $P_m^T Y_0$, $W_2^T Y_0$, and $U_m^T Y_0$. Fig. 3 shows the covariance matrix in the transformed coordinates for the three transformations. As expected, U_m^T transformed the data to coordinates in which the covariance is a diagonal matrix with descending diagonal elements (similarly to P_m^T), while W_2^T did not.

In order to further reduce the dimensionality to $m_2 < 16$, all we need to do is keep the first m_2 rows of $U_m^T Y_0$.

B. CUB-200-2011

Computing PCA using a linear autoencoder only becomes advantageous when handling a large set of large images. We applied the same technique to the CUB-200-2011 dataset [17], which contains 11,788 color images of birds. The images were resized to 256×256 . Fig. 4 shows a few examples of images from the dataset. The autoencoder was set for dimensionality reduction from a dimension of $256 \times 256 \times 3 = 196,608$ to a dimension of 36. Then, the first 36 loading vectors of the dataset were recovered by applying SVD to the weight matrix W_2 and taking the first 36 left singular vectors of it, they are shown in Fig. 5.

In order to verify that the resulting transformation applies PCA, we centered the dataset and calculated the covariance in the transformed coordinates, shown in Fig. 6. As expected, it is approximately a diagonal matrix with descending elements on the diagonal.

This dataset was too large to fit in memory, so we did not compare the results to applying SVD to the entire dataset. One might suspect that the last loading vectors in Fig. 5 are evidence of underfitting due to their high spatial frequencies. In order to rule this out, we computed the loading vectors of a subset of 1,000 examples from the dataset by applying SVD to all the 1,000 examples after centering them, and they exhibited the same appearance.

V. CONCLUSION

It is in fact possible to use a linear autoencoder not only to project data onto the principal subspace, but to actually perform principal component analysis. Recovering the loading vectors amounts to simply applying SVD to the weight matrix of one of the two layers. The solution is independent of the optimization algorithm used to train the neural network. The advantages of implementing PCA this way are that is able to:

- Process high-dimensional data.
- Process datasets with large numbers of observations.
- Avoid the need to center the data to element-wise zero mean.
- Process new data online as it arrives.
- Be implemented easily using very few lines of code, using the latest algorithms, software frameworks and hardware that are optimized for training neural networks.

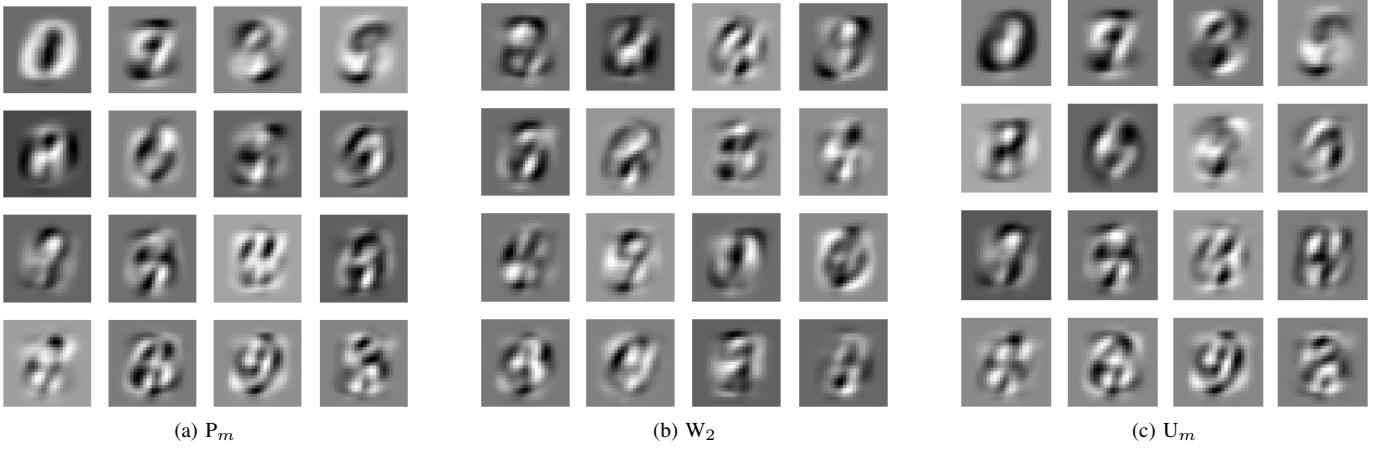


Fig. 2. (a) The first 16 loading vectors of MNIST, computed by applying SVD to the entire dataset, (b) the weights of a linear autoencoder trained on the dataset, (c) the left singular vectors of the autoencoder weights. Notice how the left singular vectors in (c) are very close to the loading vectors in (a) up to their sign (inversion of the gray levels in some of the images).

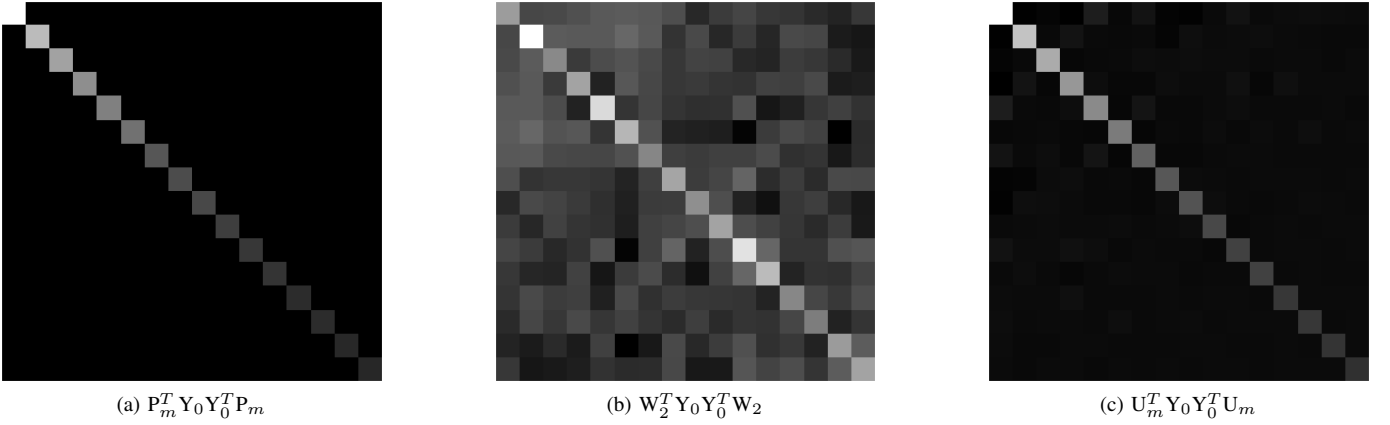


Fig. 3. The covariance matrix of the data in the transformed coordinates, according to (a) the loading vectors computed by applying SVD to the entire dataset, (b) the weights of the linear autoencoder, and (c) the left singular vectors of the autoencoder weights.

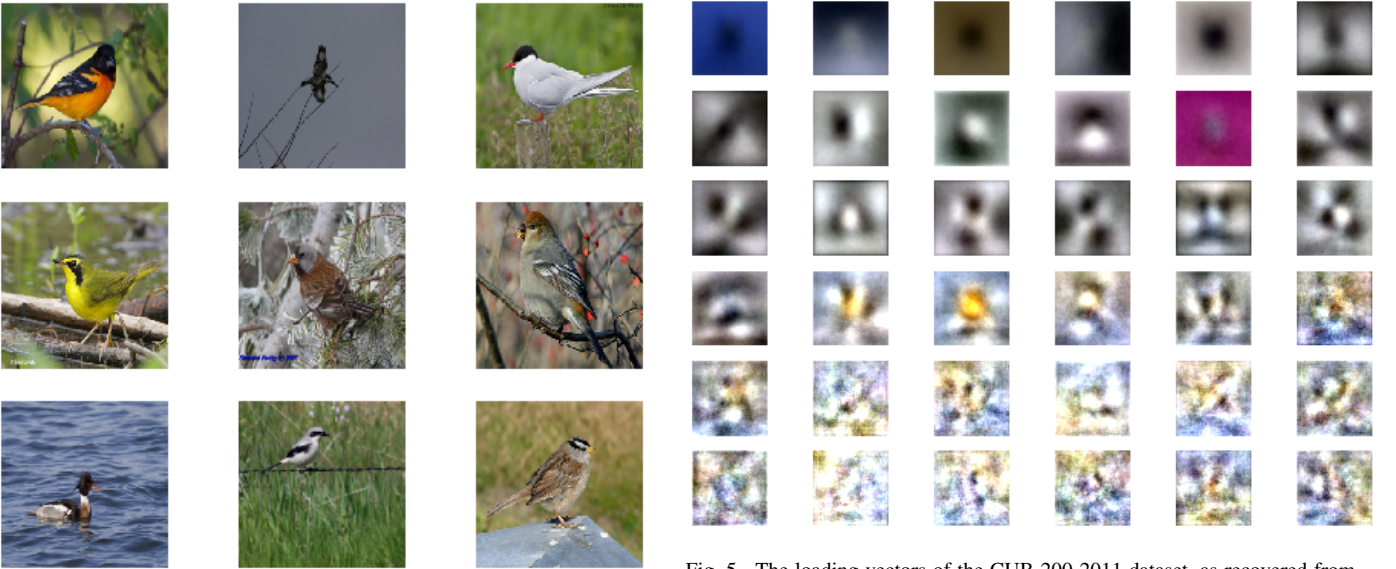


Fig. 4. Examples of images from the CUB-200-2011 dataset.

Fig. 5. The loading vectors of the CUB-200-2011 dataset, as recovered from the weight matrix W_2 .

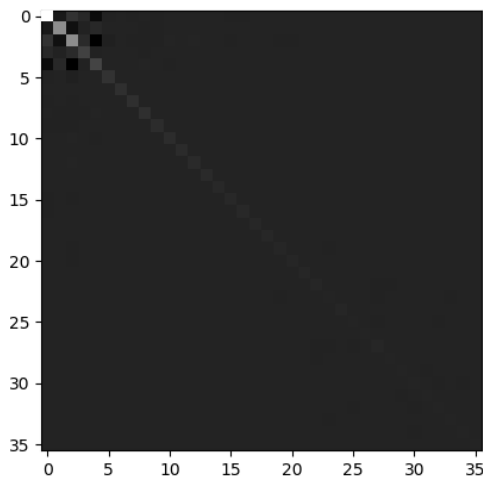


Fig. 6. The covariance matrix of the CUB-200-2011 dataset in the transformed coordinates. As expected, the covariance is approximately diagonal with descending elements on the diagonal.

REFERENCES

- [1] H. Bourlard and Y. Kamp, "Auto-association by multilayer perceptrons and singular value decomposition," *Biological Cybernetics*, vol. 59(4-5), pp. 291–294, 1988.
- [2] P. Baldi and K. Hornik, "Neural networks and principal component analysis: Learning from examples without local minima," *Neural Networks*, vol. 2(1), pp. 53–58, 1989.
- [3] M. Warmuth and D. Kuzmin, "Randomized online PCA algorithms with regret bounds that are logarithmic in the dimension," *Journal of Machine Learning Research*, vol. 9, pp. 2287–2320, 2008.
- [4] J. Fend, H. Xu, S. Mannor, and S. Yan, "Online PCA for contaminated data," *Advances in Neural Information Processing Systems*, pp. 764–772, 2013.
- [5] J. Fend, H. Xu, and S. Yan, "Online robust PCA via stochastic optimization," *Advances in Neural Information Processing Systems*, pp. 404–412, 2013.
- [6] N. Kambhata and T. Leen, "Dimension reduction by local principal component analysis," *Neural Computation*, vol. 9(7), pp. 1493–1516, 1997.
- [7] C. Eckardt and G. Young, "The approximation of one matrix by another of lower rank," *Psychometrika*, vol. 1, pp. 211–218, 1936.
- [8] G. Hinton and R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313(5786), pp. 504–507, 2006.
- [9] E. Oja, "Neural networks, principal components, and subspaces," *International Journal of Neural Systems*, vol. 1(01), pp. 61–68, 1989.
- [10] J. Rubner and P. Tavan, "A self-organizing network for principal-component analysis," *EPL (Europhysics Letters)*, vol. 10(7), p. 693, 1989.
- [11] S. Kung and K. Diamantaras, "A neural network learning algorithm for adaptive principal component extraction (APEX)," *ICASSP*, pp. 861–864, 1990.
- [12] E. Oja, "Principal components, minor components, and linear neural networks," *Neural Networks*, vol. 5(6), pp. 927–935, 1992.
- [13] L. Xu, "Least mean square error reconstruction principle for self-organizing neural-nets," *Neural Networks*, vol. 6(5), pp. 627–648, 1993.
- [14] D. A. Freedman, "Statistical models: Theory and practice." *Cambridge University Press*, 2009.
- [15] A. Antoulas, "Approximation of large-scale dynamical systems," *SIAM*, vol. 6, pp. 37–38, 2005.
- [16] Y. LeCun, C. Cortes, and C. Burges, "MNIST handwritten digit database," 2010.
- [17] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "The Caltech-UCSD birds-200-2011 dataset," no. CNS-TR-2011-001, 2011.

Machine Learning and Factor-Based Portfolio Optimization^{*}

Thomas Conlon^a, John Cotter^b, and Iason Kynigakis^c

Current Version: July 08, 2021

^{*} The authors gratefully acknowledge the support of Science Foundation Ireland under grant number 16/SPP/3347 and 17/SPP/5447.

^a Smurfit Graduate Business School, University College Dublin, Dublin, Ireland. Email: conlon.thomas@ucd.ie

^b Smurfit Graduate Business School, University College Dublin, Dublin, Ireland. Email: john.cotter@ucd.ie

^c Smurfit Graduate Business School, University College Dublin, Dublin, Ireland. Email: iason.kynigakis@ucd.ie

Machine Learning and Factor-Based Portfolio Optimization

Abstract

We examine machine learning and factor-based portfolio optimization. We find that factors based on autoencoder neural networks exhibit a weaker relationship with commonly used characteristic-sorted portfolios than popular dimensionality reduction techniques. Machine learning methods also lead to covariance and portfolio weight structures that diverge from simpler estimators. Minimum-variance portfolios using latent factors derived from autoencoders and sparse methods outperform simpler benchmarks in terms of risk minimization. These effects are amplified for investors with an increased sensitivity to risk-adjusted returns, during high volatility periods or when accounting for tail risk.

JEL classifications: C38, C4, C45, C5, C58, G1, G11

Keywords: Autoencoder; Covariance matrix; Dimensionality reduction; Factor models; Machine learning; Minimum-variance; Principal component analysis; Partial least squares; Portfolio optimization; Sparse principal component analysis; Sparse partial least squares.

1. Introduction

In this paper we examine the characteristics and benefits of latent factors generated from machine learning dimensionality reduction techniques for asset allocation. The analysis is conducted under the framework of factor-based covariance matrices used to construct minimum-variance portfolios.

Machine learning is well suited to many theoretical and empirical problems and has been shown to have the potential to dramatically improve performance in financial applications. In an early study Rapach, Strauss and Zhou (2013) use the elastic net to examine international stock return predictability, while a comprehensive comparison of machine learning methodologies is provided by Gu, Kelly and Xiu (2020), Bianchi, Büchner and Tamoni (2021) and Wu, Chen, Yang and Tindall (2020) for the equity, bond and hedge fund markets, respectively.

Our relation to this literature is twofold. First, we contribute to the studies that construct factors through machine learning. Specifically, we examine the economic value of latent factors generated using a variety of supervised and unsupervised dimensionality reduction methods and their relation to popular characteristics. In addition to classical approaches, such as principal component analysis (PCA) and partial least squares (PLS), their respective regularized versions that induce sparsity through a penalty in the objective function are also considered. We also investigate the performance of factors generated by autoencoders; a type of unsupervised neural network used for dimensionality reduction. The second contribution stems from the framework through which we bridge the gap between machine learning and finance. Particularly, we explore the impact that the proposed latent factors have on the structure of factor-based covariance matrices and to the composition and performance of minimum-variance portfolios.

The assumption that asset returns are driven by a set of observed (Sharpe, 1963; Chen, Roll and Ross, 1986) or latent (Connor and Korajczyk, 1988) factors has a long history in finance. More recently, the advantages of factor construction using machine learning has been shown by Feng, Giglio, and Xiu (2020). They combine the lasso with Fama-MacBeth regressions to systematically evaluate the contribution of an additional factor to a set of preexisting factors, finding that many proposed factors

do not add much incremental value. Kelly, Pruitt and Su (2019) propose Instrumental PCA (IPCA), where factors are latent, and the time-varying loadings depend on characteristics. They show that a small number of factors can explain the cross section of average returns more accurately than other leading factor models. Subsequently, Gu, Kelly and Xiu (2021), extend the analysis, without the linearity assumption of IPCA, using machine learning. Feng, Polson, and Xu (2020), build factors that maximize the fit in the cross-section of returns using a deep feed-forward neural network, while Huang, Li and Zhou (2019) find that using reduced-rank regression outperforms popular observed factor models.

PLS applications can also be found in the finance literature. For example, Light, Maslov and Rytchkov (2017) develop a PLS-based approach to aggregate information about expected returns from a large number of firm characteristics into a few composite variables that predict the cross-section of expected stock returns. Lettau and Pelger (2020) introduce risk premia PCA, which identifies factors with small time-series variation that are useful in the cross-section of returns. Kozak, Nagel, and Santosh (2020) find that a stochastic discount factor with a small number of principal components, using Bayesian shrinkage to select a subset of characteristics, provides good out-of-sample explanatory power for average returns.

Machine learning methods have also previously been used in asset allocation. Callot, Caner, Önder, and Ulasan (2019) use nodewise regression to directly estimate the sparse precision matrix. Minimum-variance portfolios based on their proposed approach exhibit lower variances and higher Sharpe ratios compared to commonly used covariance estimators. D'Hondt, DeWinne, Ghysels and Raymond (2020), consider a mean-variance strategy, where the returns are estimated by various machine learning models and the covariance using a dynamic shrinkage estimator, to assess the benefits of robo-investing strategies. They find that machine learning methods lead to portfolios with higher returns that vastly outperform other passive investments during the financial crisis. Lassance, DeMiguel and Vrin (2020), examine the performance of factor-risk-parity portfolios by choosing a set of uncorrelated factors using independent component analysis. They show that portfolios based on independent components provide greater diversification benefits and outperform those using principal components and other benchmarks.

We add to the literature which shows that simpler benchmarks such as the equal-weighted (EW) strategy can be outperformed. In this study we focus on modelling the covariance as a means to improve asset allocation. The improved performance can be attributed to two aspects. First, factor-based covariance matrices tend to significantly reduce the risk of a portfolio consisting of individual stocks. This finding remains robust in an out-of-sample setting, using different risk measures, across covariance and factor specifications, for a varying number of assets, alternative portfolio objective formulations and when transaction costs are taken into account. Second, we demonstrate that using machine learning can lead to significant economic gains. For example, using a factor-implied covariance based on machine learning, can lead to a decrease in out-of-sample portfolio standard deviation of up to 29% and an increase in the Sharpe ratio of over 25%.

The mean-variance criterion of Markowitz (1952) requires knowledge of the mean and covariance matrix of the assets in the portfolio, which in practice are derived from the data and carry considerable estimation risk (Broadie, 1993; Kan and Zhou, 2007). It has been argued (see DeMiguel, Garlappi and Uppal, 2009; Duchin and Levy, 2009) that portfolios selected according to this framework are unlikely to outperform the equal-weighted portfolio, since they are notorious for producing extreme weights that fluctuate considerably over time and perform poorly out-of-sample.¹

We focus instead on minimum-variance portfolios, which correspond to a risk averse investor who aims to minimize the variance without taking the expected return into consideration. This optimization framework is well motivated by Merton (1980), since it requires only estimates of the covariance matrix, which are often considered to be more accurate than the estimates of the means that have been found to be the principal source of estimation risk (Best and Grauer, 1991; Chopra and Turner, 1993).² Although the minimum-variance framework avoids the problem of estimation error associated with expected returns, its performance remains crucially dependent on the quality of the estimated covariance matrix

¹ Extensive effort has been devoted (see e.g., Kritzman, Page and Turkington, 2010; Hjalmarsson and Manchev, 2012; Branger, Lucivjanska and Weissensteiner, 2019; Platanakis, Sutcliffe and Ye, 2021) to the issue of reducing expected return and covariance estimation error with the goal of improving the performance of the Markowitz model relative to simple benchmarks.

² The literature (see Chan, Karceski, Lakonishok, 1999; Ledoit and Wolf, 2003; Kempf and Memmel, 2006; Frahm, 2008; Ledoit and Wolf, 2017; Carroll, Conlon, Cotter, Salvador, 2017; Moura, Santos and Ruiz, 2020, among others) has frequently advocated the use of constrained and unconstrained minimum-variance portfolios.

(DeMiguel, Garlappi, Nogales and Uppal, 2009; Ardia, Bolliger, Boudt and Gagnon-Fleury, 2017). To lessen the impact of covariance misspecification on the optimal weights, we impose a factor structure on the covariance matrix, which reduces the number of parameters to be estimated.³

In addition to having observed or latent factors, factor models can be static, such as in the arbitrage pricing theory (APT) of Ross (1976), or dynamic (see Sargent and Sims, 1977; Geweke, 1977). It has been shown that introducing factor structure to the covariance matrix can improve portfolio performance (Green and Hollifield, 1992; Chan, Karceski and Lakonishok, 1999). The performance of factor-implied covariances has also been examined by MacKinlay and Pastor (2000) for tangency portfolios and by Moskowitz (2003) for minimum-variance portfolios. The benefits of using the factor model-based approach to estimate the covariance matrix have also been investigated by Fan, Fan and Lv (2008) and Fan, Liao and Mincheva (2011; 2013) who propose estimators of the covariance for exact and approximate factor models, respectively. More recently, De Nard, Ledoit and Wolf (2019), use a factor framework and evaluate portfolios for different estimates of the error covariance matrix. Our contribution to the literature of factor-based portfolio optimization arises from conducting a comparative analysis of a static and several dynamic specifications of the covariance matrix, based on observed or latent factors. The structure of a dynamic covariance matrix can differ based on whether the factor loadings, the factor covariance matrix or the residual covariance matrix are allowed to vary over time.

To determine the effects of using factors based on machine learning in covariance estimation and portfolio optimization, we first explore the potential links between the latent factors and popular factor proxies, examine the structure of the factor-implied covariance matrices and analyze the properties of portfolio weights. The results show that machine learning yields factors that cause the covariances and

³ Alternative solutions involve imposing short-selling constraints (Hui, Kwan and Lee, 1993; Jagannathan and Ma, 2003), limiting turnover via norm constraints (DeMiguel, Garlappi, Nogales and Uppal, 2009) or penalizing the objective function, such as in Olivares-Nadal and DeMiguel (2018). Another approach uses either shrinkage estimators of the covariance matrix (Frahm and Memmel 2010; Ledoit and Wolf, 2012), which tend to shrink the covariance matrix towards a specific target covariance or sparse estimators that derive a regularized version of the precision matrix (Friedman, Hastie and Tibshirani, 2008; Callot, Caner, Önder, and Ulasan, 2019). Using higher frequency data can also reduce estimation error (Palczewski and Palczewski, 2014; Fan, Furger and Xiu, 2016).

portfolio weights to diverge from those based on commonly used estimators. Latent factors produced by PCA and PLS-type methods exhibit a stronger connection with well-known factors (such as those from the Fama and French (2015) five-factor model) throughout the out-of-sample period, compared to factors based on autoencoders. Furthermore, the covariance matrices whose structure deviates most from the sample estimator are based on unsupervised methods or allow the residual covariance matrix to be time-varying. Portfolios based on machine learning also have weights that are smaller, vary less over time and are more diversified, than models based on observed factors. Covariance matrices based on unsupervised methods also lead to portfolios with lower turnover and thus reduced sensitivity to transaction costs.

We evaluate the different factor and covariance specifications by constructing minimum-variance portfolios based on individual stock return data for a sample period spanning 60 years. Overall, our findings suggest that machine learning adds value to factor-based asset allocation. In the baseline case, machine learning leads to portfolios that significantly outperform the equal-weighted benchmark, which DeMiguel, Garlappi and Uppal (2009) show to be a very stringent benchmark, and improves upon portfolios based on the sample estimator or observed factors. The best-performing methods to generate the covariance matrix are autoencoders and sparse principal component analysis, which can lead to portfolios with higher risk-adjusted return and standard deviation that is 3% lower per annum than the equal-weighted one.

In addition, machine learning can improve factor-based portfolio optimization when performance is measured using alternative risk metrics. Covariance matrices based on autoencoders and sparse PCA outperform the equal-weighted portfolio by up to 2.9%, 1.26% and 1.57% per annum, in terms of mean absolute deviation, Value-at-Risk and Conditional Value-at-Risk, respectively. Furthermore, the performance according to certainty equivalent return improves relative to the benchmark as the investor's aversion to risk increases, which falls in line with the optimization goal. In particular, investors with moderate or conservative risk preferences using machine learning factors would realize significant utility gains that are between 2.5% and 4.5% higher than those of the EW portfolio on an annual basis.

Similar to recent studies, the results indicate that shallow learning outperforms deeper learning, which can be attributed to the small size of the data set and the low signal-to-noise ratio. When we consider factors based on neural networks with one to four hidden layers, we find that for the baseline optimization framework the shallowest network outperforms those with deeper architectures. Additionally, unsupervised methods tend to perform better than supervised methods. When we compare PLS and sparse PLS with PCA and sparse PCA, the results show that the PCA-type approaches significantly outperform the equal-weighted benchmark more often. This decline in portfolio performance is potentially associated with the high degree of turnover of strategies based on autoencoders with more hidden layers and supervised methods. Otherwise, the ranking among factors persists across specifications of the covariance matrix. When comparing the results across the alternative specifications of the factor-based covariance matrix, the differences become less pronounced, with approaches that allow the loadings or the residual covariance matrix to vary over time yielding higher risk-adjusted performance.

The performance of portfolios based on machine learning is amplified during periods of high volatility, while the benefits of factor-based optimization become evident during different inflation and credit spread subperiods. All machine learning portfolios generate positive breakeven transaction costs, indicating they outperform the equal-weighted benchmark. The results for machine learning portfolios remain robust when short-selling is allowed, unlike those for observed factors that show a significant increase in portfolio risk. Including a proportional transaction costs penalty to the minimum-variance objective, leads to a decrease in monthly turnover, which benefits primarily the portfolios with high rebalancing requirements such as those based on observed factors, unsupervised methods or deeper autoencoders. The factor-based portfolios continue to outperform the benchmark after increasing the number of assets. Turnover increases considerably with the size of the portfolio, while the breakeven transaction costs decrease. Machine learning models still generate the highest performance across different portfolio sizes. Finally, the results for alternative datasets based on industry or book-to-market sorted portfolios, tend to favor covariances based on observed factors or supervised methods. Overall, the results indicate that machine learning improves factor-based portfolio optimization.

2. Methodology

In this section we introduce the machine learning methods for dimensionality reduction used to construct the latent factors, we then describe the different specifications under which the factor-based covariance matrices are estimated and finally, present the optimization framework used to derive the portfolios.

2.1 Latent Factors via Machine Learning

The presence of factor structure in asset returns has been widely accepted in the economic literature. The capital asset pricing model (CAPM) of Sharpe (1964) and Lintner (1965) implies a simple factor structure. Other early studies that support the presence of a factor structure in asset returns include the APT of Ross (1976) and the intertemporal CAPM of Merton (1973). Factors can be observable quantities, such as macroeconomic indicators (Chen, Roll and Ross, 1986) or observable proxies, where factors are the returns of portfolios constructed by sorting stocks based on firm characteristics, such as the single index model of Sharpe (1963) or the three-factor model of Fama and French (1993) and its variations.

A factor model for the returns of every asset, $r_{i,t}$, with $i = 1, \dots, N$ assets, $t = 1, \dots, T$ observations and $k = 1, \dots, K$ observed factors takes the form

$$r_{i,t} = a_i + \beta_i F_t + u_{i,t}, \quad (1)$$

where $\beta_i = (\beta_{i,1}, \dots, \beta_{i,K})$ are the time-invariant factor loadings for factors $F_t = (f_{t,1}, \dots, f_{t,K})$, a_i is the time-invariant intercept and $u_{i,t}$ is the error term for asset i at date t . The intercepts and factor loadings can be estimated by ordinary least squares (OLS) using the different factor representations.

Factors can also be latent quantities, which are derived from the data using dimensionality reduction techniques. When factors are latent, principal component analysis is a very common approach to reduce dimensionality. The studies of Chamberlain and Rothschild (1983) and Connor and Korajczyk (1988) are among the first to use latent factors in applications of the APT.

The general form of a latent factor model is given by

$$r_{i,t} = a_i + \beta_i(X_t W) + u_{i,t} = a_i + \beta_i F_t + u_{i,t}, \quad (2)$$

where $X_t = (x_{1,t}, \dots, x_{p,t})$ is the $T \times p$ matrix of predictors and $W = (w_1, \dots, w_K)$ is the $p \times K$ matrix of weights, with $K \ll p$. Each w_k is the vector of weights used to construct the k^{th} latent factor, f_k . The $T \times K$ matrix of latent factors is given by $F_t = X_t W$.

In the following sections we describe the classical dimensionality reduction techniques used to generate the latent factors, along with their extensions from the machine learning literature, which rely on regularization and neural networks. The alternative methods we consider are similar in that the dimensionality of the data is reduced by mapping the set of p predictors to a smaller set of K combinations of the original variables.

2.1.1 Principal Component Analysis and Partial Least Squares

The two most commonly used dimensionality reduction techniques are principal component analysis and partial least squares. They are both designed to uncover a lower dimensional linear combination of the original predictor set, however, PCA derives the latent factors in an unsupervised way, based only on information from the predictors X_t , while in the case of PLS the factors are constructed in a supervised way, by using information from both the predictors X_t and the response R_t . The methods differ in the way the latent factor matrix, F_t , is extracted, since PCA produces the weight matrix W reflecting the covariance structure between predictors, while PLS computes weights that account for the covariation between the predictors and the response.

Principal component analysis can be viewed as a regression-type problem where the goal is to find the first K principal component weight vectors by minimizing:

$$\underset{W}{\operatorname{argmin}} \|X_t - X_t W W'\|^2, \quad \text{s.t. } W'W = I_K, \quad (3)$$

where I_K is a $K \times K$ identity matrix. The solution to this problem is most often obtained via singular value decomposition: $X_t = UDV'$, by setting $W = V$. The columns of $V = (v_1, \dots, v_K)$ are the principal components weights. Each v_j is used to derive the k^{th} principal component, $f_k = X_t v_k$, thus, $F_t V$ is the dimension reduced version of the original predictors. The derived variable f_1 is the first principal

component of X_t and has the largest sample variance amongst all linear combinations of the columns of X_t .

Partial least squares, introduced by Wold (1966), identifies the features in a supervised way, by constructing linear combinations based on both R_t and X_t . Specifically, PLS decomposes the matrix of predictors X_t and the matrix of asset returns R_t into the form: $X_t = F_t P' + E_t$ and $R_t = F_t Q' + H_t$, where the matrices P and Q are the loadings, while E and H are the residuals. In order to find the PLS component matrix F_t , the columns of the weight matrix W need to be obtained through consecutive optimization problems. The criterion to find the k^{th} estimated weight vector w_k is

$$\underset{w}{\operatorname{argmax}}[w'(X_t' R_t R_t' X_t)w], \quad \text{s.t. } w'w = 1, \quad w'\Sigma_{XX}w_k = 0, \quad (4)$$

where Σ_{XX} is the covariance of X_t . The latent factor matrix is then given by $F_t = X_t W$. The version of PLS we employ is SIMPLS proposed by de Jong (1993). If $K = p$ then PLS would give a solution equivalent to the OLS estimates.

PCA and PLS have the drawback that for each latent factor the weights are typically non-zero, which leads to difficulties in high dimensional settings. To address this issue, we consider methods that produce modified latent factors with sparse weights, such that each latent factor is a combination of only a few of the original variables.

Sparse principal component analysis (SPCA), developed by Zou, Hastie and Tibshirani (2006), is based on the regression/reconstruction property of PCA and produces modified principal components with sparse weights, such that each principal component is a linear combination of only a few of the original predictors. They show how PCA can be viewed in terms of a ridge regression problem and by adding the l_1 penalty, they convert it to an elastic net regression, which allows for the estimation of sparse principal components. The following regression criterion is proposed to derive the sparse principal component weights

$$\underset{W, C}{\operatorname{argmin}}[\|X_t - X_t W C'\|^2 + \lambda_1 \|w\|_1 + \lambda_2 \|w\|^2], \quad \text{s.t. } W'W = I_K, \quad (5)$$

where W and C are both $p \times K$. If $\lambda_1 = \lambda_2 = 0$, $T > p$ and we restrict $C = W$, then the minimizer of the objective function is exactly the first K weight vectors of ordinary PCA. When $p \gg T$, in order to obtain a unique solution, $\lambda_2 > 0$ is required. The l_1 penalty on c_k induces sparseness of the weights, with larger values of λ_1 leading to sparser solutions.

Sparse partial least squares (SPLS) is an extension of PLS that imposes the l_1 penalty to promote sparsity onto a surrogate weight vector c instead of the original weight vector w , while keeping w and c close to each other (Chun and Keles, 2010). The first SPLS weight vector solves

$$\operatorname{argmin}_{w,c} \left[-\frac{1}{2} w' M w + \frac{1}{2} (c - w)' M (c - w) + \lambda_1 \|c\|_1 + \lambda_2 \|c\|^2 \right], \quad \text{s.t. } w' w = 1, \quad (6)$$

where $M = X_t' R_t R_t' X_t$, λ_1 and λ_2 are non-negative tuning parameters. To solve SPLS a large λ_2 value is usually required and setting $\lambda_2 = \infty$ yields a solution that has the form of the soft threshold estimator by Zou and Hastie (2005). This reduces the number of tuning parameters to two, the tuning parameter λ_1 and the number of latent factors K .

2.1.2 Autoencoder Neural Networks

Another approach we use to construct the latent factors is based on autoencoders (Bourlard and Kamp, 1988; LeCun, Boser, Denker, Henderson, Howard, Hubbard, Jackel, 1989; Hinton and Zemel, 1994), which are a type of unsupervised neural network that can be used for dimensionality reduction. Autoencoders have a similar structure to feed-forward neural networks, which have been shown to be universal approximators for any continuous function (Hornik, Stinchcombe and White, 1989; Cybenko, 1989). However, an autoencoder differs in that the number of inputs is the same as the number of outputs and that it is used in an unsupervised context. Autoencoders have also been shown to be nonlinear generalizations of PCA. The goal of PCA and autoencoders is to learn a parsimonious representation of the original input data, X_t , through a bottleneck structure. The difference is that PCA reduces the dimensionality, by mapping the original p inputs into $K \ll p$ factors in a linear way, while autoencoders use non-linear activation functions to discover non-linear representations of the data.

An autoencoder is trying to learn an approximation to the identity function so as the output $\hat{X}_t$ is similar to the input X_t . The network consists of two parts: an encoder and a decoder. The encoder creates a compressed representation of X_t when the input variables pass through the units in the hidden layers, which are then decompressed to the output layer through the decoder. By placing constraints on the network, such as limiting the number of hidden units, it is forced to learn a compressed representation of the input, potentially uncovering an interesting structure of the data. Most often the encoding and decoding parts of an autoencoder are symmetrical, in that they both feature the same number of hidden layers with the same number of hidden units per layer. The output of the decoder is most commonly used to validate information loss, while the smallest hidden layer of the encoder (or code, at the bottleneck of the network) corresponds to the dimension-reduced data representation.

Let L denote the number of hidden layers and $K^{(l)}$ denote the number of hidden units in each layer, for $l = 1, \dots, L$, while the output of unit k in layer l is defined as the vector $z_k^{(l)}$ and the output of layer l as the matrix $Z^{(l)} = (z_1^{(l)}, \dots, z_{K^{(l)}}^{(l)})$. The original data, X_t , enters the network through the input layer ($l = 0$), while in each hidden layer inputs from the previous layer are transformed through nonlinear activation functions $h(\cdot)$ before being passed as inputs onto the next layer. The output of each hidden unit k in layer l is based on the function

$$z_k^{(l)} = h(Z^{(l-1)}W^{(l-1)} + b^{(l-1)}), \quad (7)$$

where $W^{(l-1)}$ is a $K^{(l-1)} \times K^{(l)}$ weight matrix and $b^{(l-1)}$ is a $1 \times K^{(l)}$ bias vector. For the first hidden layer the matrix of predictors is used as input, $Z^{(0)} = X_t$, such that $z_k^{(1)} = h(X_t W^{(0)} + b^{(0)})$. We use the hyperbolic tangent (tanh) activation function defined as $h(x) = 2/(1 + e^{-2x}) - 1$, which is a zero-centered function whose range lies between -1 to 1. The results from each hidden layer are aggregated in the output layer

$$\hat{X}_t = h(Z^{(L-1)}W^{(L-1)} + b^{(L-1)}). \quad (8)$$

Since an autoencoder tries to approximate X_t the dimensions of the input and the output layer are identical, $K^{(0)} = p = K^{(L)}$. We consider four different network architectures based on the depth of the

network. First, we construct a shallow autoencoder (AEN1) with a single hidden layer (the code). The other three models include additional hidden layers to the encoder and decoder representation, up to a maximum of three layers (AEN2, AEN3 and AEN4). The number of hidden nodes in each layer is selected according to the geometric pyramid rule by Masters (1993).

It can also be shown that linear autoencoders are equivalent to PCA (Baldi and Hornik, 1989; Karhunen and Joutsensalo 1995; Japkowicz, Hanson and Gluck, 2000). Specifically, when the autoencoder has a single hidden layer, so the network representation becomes $X_t \rightarrow Z^{(1)} \rightarrow \hat{X}_t$ and all activation functions are linear, it can be shown that the $K^{(1)}$ latent variables at the bottleneck correspond to the first K principal components of the data. Hinton and Salakhutdinov (2006) show that deep autoencoders outperform shallow or linear autoencoders in image recognition tasks, however, recent applications in finance (see Gu, Kelly and Xiu, 2020; Bianchi, Büchner and Tamoni, 2021) find that shallower networks generate better performance.

The weights and biases of the neural network are estimated by minimizing the square loss of the form

$$\operatorname{argmin}_{b, W} \mathcal{L}(X_t, \hat{X}_t) = \operatorname{argmin}_{b, W} \|X_t - \hat{X}_t\|^2, \quad \text{s.t.} \quad \|W^{(l)}\|^2 = i_{K^{(l)}}. \quad (9)$$

The estimates of the parameters of a neural network are solutions of a non-convex optimization problem. The neural network is trained using stochastic gradient descent (SGD). Unlike standard gradient descent that uses the entire training sample to evaluate the gradient at each iteration of the optimization, SGD evaluates the gradient from a random subset of the data and iteratively minimizes the objective function through back propagation. The version of SGD we implement is the adaptive moment estimation algorithm (Adam), introduced by Kingma and Ba (2015). Adam computes individual adaptive learning rates for the model parameters using estimates of first and second moments of the gradients.

Training a neural network can be challenging due to the large number of parameters to be estimated and the nonconvexity of the objective function. To alleviate those concerns we modify the loss function by adding a penalty on the output of the layers (activations), encouraging the activations of the nodes to be sparse (Goodfellow, Bengio and Courville, 2016). We consider activity regularization based on

the elastic net penalty that shrinks the output of the bottleneck layer. Following papers such as Gu, Kelly and Xiu (2020), we implement early stopping. In each iteration of the optimization algorithm the parameter estimates are updated to reduce prediction errors in the training sample and then the predictive performance of the model for that iteration is evaluated using data from the validation sample. Early stopping is implemented by stopping the training process prematurely when the validation error no longer decreases. Specifically, the optimization process halts when the maximum number of iterations is reached or if the validation error has not improved for a certain number of consecutive iterations. In both cases the parameter estimates of the best performing model are retrieved. The effects of early stopping are that it prevents overfitting and significantly speeds up the training process.

2.2 Factor-based Covariance Estimation

After the factor model is estimated from equation (1) or (2) the covariance matrix of returns, Σ_r , is obtained by its decomposition into two components: the first is based on the factor loadings and the factor covariance matrix, while the second is the covariance matrix of the errors. The time-invariant covariance matrix of the returns $R = (r_1, \dots, r_N)$ is given by:

$$\Sigma_r = B' \Sigma_f B + \Sigma_u, \quad (10)$$

where B is a $K \times N$ matrix with the i th column containing the vector of time-invariant factor loadings β_i and Σ_f and Σ_u denote the time-invariant covariance matrices of the factors and the errors respectively. We focus on exact factor models (Fan, Fan and Lv, 2008), where the covariance matrix of the residuals u_t is diagonal, $\Sigma_u \equiv \text{diag}(\Sigma_u)$, by assuming cross-sectional independence.

The models presented so far rely on a static specification. However, Sargent and Sims (1977) and Geweke (1977) introduce dynamic factor models (DFM) as an extension.⁴ There are various definitions of DFMs (see Stock and Watson, 2011), the one we follow in this study is a model that allows the factor loadings to be time varying (Avramov and Chordia, 2006; Engle, 2016 and Bali, Engle and Tang, 2017)

⁴ DFMs have been further developed by Forni, Hallin, Lippi and Reichlin (2000; 2005), while Boivin and Ng (2005) compare the forecasting performance of models that use static or dynamic factors.

or models in which either the factor or residual covariance matrix varies over time (Engle, Ng and Rothchild, 1990). In the description below, the factors can be observed quantities or latent factors.⁵

A dynamic factor model is one in which at least one of the following three generalizations holds true: (i) the intercept and factor loadings are time-varying, (ii) the covariance matrix of the factors is time-varying or (iii) the covariance matrix of the errors is time-varying.

When the intercepts a_i and factor loadings β_i are allowed to be time-varying the conditional dynamic factor model takes the following form

$$r_{i,t} = a_{i,t} + \beta_{i,t}F_t + u_{i,t}. \quad (11)$$

The estimates of the time-varying regression coefficients are then obtained by $\hat{\beta}_{i,t} = \Sigma_{f,t}^{-1} \sigma_{fr_{i,t}}$. The coefficients, $\hat{\beta}_{i,t}$, of this expression are the dynamic conditional betas and are based on time-varying estimates of the factor covariance matrix $\Sigma_{f,t}$ and the vector of covariances, $\sigma_{fr_{i,t}}$ between the returns of asset i , r_i and factor f_k , with $k = 1, \dots, K$. The intercept can be obtained by $\hat{a}_{i,t} = \bar{r}_i - \hat{\beta}_{i,t}\bar{F}$. The time-varying covariance matrix of R_t is given by:

$$\Sigma_{r,t} = B_t' \Sigma_f B_t + \Sigma_u, \quad (12)$$

where B_t is a $K \times N$ matrix with the i th column containing the vector of time-varying factor loadings $\beta_{i,t}$.

The unconditional dynamic factor model under generalization (ii) and (iii) takes a form similar to equations (1) or (2), but with time-varying conditional covariance matrices for f_t and u_t respectively.

If Σ_f is assumed to be time-varying, then the covariance matrix of R_t is given by

$$\Sigma_{r,t} = B' \Sigma_{f,t} B + \Sigma_u. \quad (13)$$

Otherwise, if Σ_u is assumed to be time-varying, then

$$\Sigma_{r,t} = B' \Sigma_f B + \Sigma_{u,t}. \quad (14)$$

⁵ The econometric properties of PCA in a static (Bai and Ng, 2002) or a dynamic (Stock and Watson, 2002a; 2002b) setting, have been examined when factor models are used for economic forecasting with many predictors.

The factor covariance matrix, $\Sigma_{f,t}$ is estimated by the dynamic conditional correlation (DCC) model (Engle, 2002) and the diagonal elements of $\Sigma_{u,t}$ are estimated by univariate GARCH models.

2.3. Minimum-Variance Portfolios

The sensitivity of portfolio weights to estimates of asset means is well documented (Best and Grauer, 1991; Chopra and Turner, 1993). To this end, we focus on minimum-variance portfolios, which have frequently been used in the portfolio optimization literature (see e.g. Carroll, Conlon, Cotter, Salvador, 2017; Moura, Santos and Ruiz, 2020), thus avoiding the issue of estimation error in expected returns.

Specifically, the different estimates of the covariance matrix, $\hat{\Sigma}_r$, from the factor models are evaluated through the minimum-variance framework with short-selling constraints, where the goal is to minimize portfolio risk. Assuming there are N assets in the investment universe and $r_t = (r_{1,t}, \dots, r_{N,t})$ is a vector of asset returns, the objective of the minimum-variance optimization problem is

$$\underset{\omega}{\operatorname{argmin}} \omega' \hat{\Sigma}_r \omega, \quad \text{s.t.} \quad \omega' \mathbf{i}_N = 1, \quad \omega_i \geq 0, \text{ for } i = 1, \dots, N, \quad (15)$$

where $\omega = (\omega_1, \dots, \omega_N)$ is the portfolio weight vector and $\mathbf{i}_N$ is a $N \times 1$ unit vector. The return of the portfolio can then be calculated as $r_{p,t+1} = \hat{\omega}' r_{t+1}$. All portfolios include short-selling and leverage constraints to avoid implausible positions, by imposing a lower bound of zero on all weights and that the sum of the weights does not exceed one. The additional non-negativity constraint on minimum variance portfolios has been shown (Jagannathan and Ma, 2003) to be equivalent to shrinking the elements of the covariance matrix.

3. Data, Sample Splitting and Hyperparameter Tuning

The data set consists of monthly total individual stock returns from the Center for Research in Security Prices (CRSP) starting on January 1960 to December 2019, for a period of 60 years (or $T = 720$ monthly observations). Our approach regarding the backtest and the restrictions we impose on the data set is similar to that of Engle, Ledoit and Wolf (2019) and De Nard, Ledoit and Wolf (2019), but adapted

to a monthly frequency. We restrict our data set to stocks listed on the NYSE, AMEX, and NASDAQ stock exchanges (exchange codes 1, 2 or 3) and to ordinary common shares (share codes 10 or 11).

We adopt a rolling window approach to examine the out-of-sample (OOS) performance of our models. The size of the rolling window is set to $T_0 = 240$ monthly observations (or 20 years), with the initial window spanning the period from January 1960 to December 1979. The rolling window moves across the full sample by one monthly observation at a time, leading to an out-of-sample size of $T_{OOS} = T - T_0 = 480$ monthly observations (or 40 years), from January 1980 to December 2019. The portfolios are constructed in each iteration of the rolling window, based on stocks that have at least 97.5% history of returns available over the past $T_0 = 240$ months and are also not missing the return observation for the following month after the end of the rolling window. This forward-looking restriction is commonly applied to allow for the out-of-sample evaluation of portfolios, which are based on in-sample estimates of the covariance matrix. Finally, we consider stocks whose price is greater than \$5 within each iteration. In the baseline case, the latent factors, covariance matrices and portfolio weights are estimated based on the $N = 100$ stocks with the highest market capitalization within each iteration of the rolling window, before we expand the analysis to larger portfolios.⁶

In each iteration of the rolling window, we cross-sectionally transform the predictors, X_t , before estimating the latent factors in a similar way to Freyberger, Neuhierl, and Weber (2020). Specifically, we calculate the rank of a stock based on the return and then divide the ranks by the number of observations. We follow Kelly, Pruitt, and Su (2019) and subtract 0.5, to map the features into the $[-0.5, 0.5]$ interval. This transformation focuses on the ordering of the data and is insensitive to outliers.

The machine learning models used to derive the latent factors rely on hyperparameter tuning. The choice of hyperparameters controls the amount of model complexity and is critical for the performance of the model. Specifically, we adopt the validation sample approach, in which the optimal set of values for the tuning parameters is selected in the validation sample. One of the advantages of using this approach

⁶ Figure A1 in the Appendix displays the number of stocks in the sample for each month for the full sample period, while the number of stocks in each iteration of the rolling window for the out-of-sample period is displayed in Figure A2.

over k -fold cross validation is that we maintain the temporal ordering of the data. For a detailed description of cross-validation see Friedman, Hastie and Tibshirani (2009). Specifically, in each iteration of the rolling window, the in-sample ($T_0 = 240$) is split into two disjointed periods, the training subsample, $T_0^{\mathcal{T}}$, consisting of 80% of the observations, with the remaining observations belonging to the validation subsample, $T_0^{\mathcal{V}}$. In the training subsample the model is estimated for several sets of values of the tuning parameters. The second subsample is used to select the optimal set of tuning parameters, by using the latent factor weight and loading estimates for each set of hyperparameters from the training sample, forecasts are constructed for the observations in the validation sample.

Specifically, the factor model is first estimated using information only from the training sample, from the following regression equation

$$r_{i,t} = a_i^{\mathcal{T}} + \beta_i^{\mathcal{T}}(X_t W^{\mathcal{T}}) + u_{i,t}, \text{ for } t = 1, \dots, T_0^{\mathcal{T}}, \quad (16)$$

where $W^{\mathcal{T}}$ is the factor weight matrix estimated by one of the dimensionality reduction methods using a specific set of hyperparameters, $\beta_i^{\mathcal{T}}$ is one of the N columns of the matrix of factor loadings, $B^{\mathcal{T}}$, estimated by OLS using data only from $\mathcal{T}$ and $a_i^{\mathcal{T}}$ is the intercept. The covariance over the training subsample is then calculated by

$$\Sigma_r^{\mathcal{T}} = B^{\mathcal{T}'} \Sigma_f^{\mathcal{T}} B^{\mathcal{T}} + \Sigma_u^{\mathcal{T}}, \quad (17)$$

where $\Sigma_f^{\mathcal{T}}$ is the covariance of the factors $X_t W^{\mathcal{T}}$ and $\Sigma_u^{\mathcal{T}}$ is the covariance of the errors u_t for $t = 1, \dots, T_0^{\mathcal{T}}$. The covariance over the training set is compared to that of the validation set, which is derived using the estimated matrices $W^{\mathcal{V}}$ and $B^{\mathcal{V}}$. The covariance using data from the validation subsample is estimated as

$$\Sigma_r^{\mathcal{V}} = B^{\mathcal{V}'} \Sigma_f^{\mathcal{V}} B^{\mathcal{V}} + \Sigma_u^{\mathcal{V}}, \quad (18)$$

where $\Sigma_f^{\mathcal{V}}$ is the covariance of the factors $X_t W^{\mathcal{V}}$, for $t = 1, \dots, T_0^{\mathcal{V}}$ and $\Sigma_u^{\mathcal{V}}$ is the covariance of the errors, derived by $u_{i,t} = r_{i,t} - (a_i^{\mathcal{T}} + \beta_i^{\mathcal{T}}(X_t W^{\mathcal{T}}))$, for $t = 1, \dots, T_0^{\mathcal{V}}$.

The covariance matrices based on different sets of hyperparameters are evaluated by employing a measure of economic performance related to the given portfolio application. Following Engle and

Colacito (2006) and Becker, Clements, Doolan and Hurn (2015), the optimal set of hyperparameters is then chosen as to minimize the portfolio variance. The loss function is

$$\mathcal{L}_{\text{MVP}}(\hat{\omega}, \hat{\Sigma}_r) = \hat{\omega}' \hat{\Sigma}_r \hat{\omega}. \quad (19)$$

The vector of portfolio weights, ω , is obtained by solving the minimum-variance portfolio problem from equation (15), based on the covariance over the training set, Σ_r^T . The portfolio variance, $\mathcal{L}_{\text{MVP}}$, is calculated using the weight estimates and setting $\hat{\Sigma}_r = \Sigma_r^V$, for each set of hyperparameters. The set of optimal tuning parameters is the one that yields the lowest portfolio variance over the validation sample. The weights used to construct the latent factors and the factor loadings are re-estimated using all observations in the rolling window for the optimal set of tuning parameters. Finally, the true out-of-sample performance is evaluated by constructing the return of the portfolio using the asset returns one month ahead from the end of each rolling window, which are not included in the validation procedure or parameter estimation.

4. Characteristics of a Machine Learning Portfolio

In this section we explore the links between the latent factors and popular factor models, examine the structure of the alternative covariance matrix specifications and analyze the properties of the portfolio weight vectors. Specifically, we investigate how the factors based on several dimensionality reduction approaches relate to those of the Fama and French (2015) five-factor model and the long-short anomaly portfolios constructed by Chen and Zimmermann (2020).⁷ Furthermore, we compare the covariance matrices for different factor model specifications to the sample covariance estimator (Sample) and factor models based on observed factors such as the single index (Market) model (Sharpe, 1963) or the three-factor model (FF3) by Fama and French (1993).⁸ The portfolio weights based on factor models

⁷ Data on the long-short anomaly portfolios are obtained from Andrew Y. Chen's website. We consider portfolios that have a full history of returns from January 1960 to December 2019, which reduces the initial number of available series from 205 to 110.

⁸ Data on the Fama-French factors were downloaded from Kenneth French's Data Library.

are also compared to the equal-weighted (EW) portfolio, a scheme which requires no parameter estimation, since the weights are $\omega_i = 1/N$, for $i = 1, \dots, N$.

Latent factors produced by PCA, PLS and their regularized versions better relate to various factor proxies, compared to their neural network counterparts from an analysis of the Fama-French five-factor model. This is illustrated by higher explanatory power for these models' factors compared to those from autoencoder models. The structure of covariance matrices based on unsupervised methods exhibit the greatest differences from the sample estimator. When comparing across covariance specifications, the differences become more apparent in the covariance structure when Σ_u varies over time. Finally, portfolios based on machine learning tend to produce weights which are smaller, less volatile throughout time and more diversified, than models based on observed factors. Covariance matrices based on unsupervised methods also lead to portfolios that require less frequent rebalancing relative to their supervised counterparts.

4.1 Links to Popular Factors

In this section we examine the links that the estimated factors have with popular characteristic-based factors from the literature. We investigate the links of the latent factors, first with the five-factors from Fama and French (2015) and then with a larger dataset which consists of the long-short anomaly portfolios constructed by Chen and Zimmermann (2020).

For each dimensionality reduction method, we regress each of the estimated latent factors $f_{t,k}$, for $k \in [1, 5]$, on the factors from the Fama and French (2015) five-factor model using OLS.⁹ To compare how well the Fama and French model explains the latent factors across dimensionality reduction approaches, we report the adjusted R^2 , averaged over the 438 estimation windows, in Figure 1. A higher average

⁹ Information on the five-factors by Fama and French (2015) is available from July 1963. Given a rolling window size of 240 observations, we have an out-of-sample size of 438 observations, from July 1983 to December 2019. The OLS regressions are re-estimated in each iteration of the rolling window for all combinations of dimensionality reduction techniques and fixed number of factors K . The OLS regressions include an intercept, but since we focus on the potential relationship between the latent factors with the observed factors, we omit the intercept from the Figures for the sake of brevity.

adjusted R^2 indicates that the Fama and French five-factor model is well suited to explain the variability of that specific latent factor throughout the out-of-sample period.

[Insert Figure 1 about here]

The factors based on supervised methods tend to have higher R^2 than those of unsupervised methods, while the R^2 of factors based on autoencoders varies less with K compared to the remaining approaches. Specifically, the results for PCA show that the Fama and French five-factor model is better at explaining the first and third principal components, with R^2 between 34% and 36%, while the R^2 for the remaining components is between 8% and 15%, decreasing as K increases. The pattern for sparse principal components is similar, but the average R^2 is smaller for the first and third factor, decreasing to about 33% and 27%, respectively, while for the remaining components it increases to between 14% and 22%. The Fama and French model explains almost 50% of the variability of the first factor by PLS or SPLS. For the remaining four factors the average R^2 varies between approximately 13% and 23%. The average R^2 of latent factors based on autoencoders remains relatively stable across different values of K and number of hidden layers, with values between 18% and 25%.

Following Gu, Kelly and Xiu (2020), we quantify the influence of the Fama-French factors as the change in R^2 from setting the observations of a factor proxy to zero within each estimation window. The values are averaged to obtain a single variable importance measure for each of the five Fama and French factors and then scaled to sum to 100. The variable importance is presented in Figure 2.

[Insert Figure 2 about here]

According to the change in R^2 , the results for PCA indicate that the market factor (MKTRF) is most influential for the first principal component, $K = 1$, while the value factor (HML) relates mainly to the third latent factor, $K = 3$. The profitability (RMW) and investment (CMA) factors are better at explaining the $K = 4$ and $K = 5$ components, respectively. In contrast, none of the proxies particularly dominates in terms of explaining the second component, $K = 2$. The characteristics that are influential to the corresponding sparse principal components are similar to those for PCA. However, their importance is decreased with the remaining proxies contributing more to the explanation of the latent

factors. In particular, the fourth sparse principal component poses an exception since it relates more to the size factor (SMB) than RMW.

The two PLS methods exhibit a very similar pattern in their relation to the Fama-French five factor model. The first latent factor of both approaches exhibits a connection to the market, which is much stronger than the one observed for the respective principal component. The value characteristic can explain the second, third and fourth latent factors of PLS and SPLS, while RMW and CMA equally explain the fifth factor. The results for the autoencoders remain relatively unchanged when comparing across different values of K and number of hidden layers. The value and, to a lesser extent, the market factors are those which exhibit the strongest relation to the latent factors.

We further examine the connection of the latent factors to the Fama and French five-factor model by aggregating the distributional properties of the t -statistics, estimated throughout the out-of-sample period, using boxplots. Boxplots of the t -statistics provide a simple five-number summary of their distribution, which consists of the median (marked by the line within the box), first and third quartiles (the edges of the box, with its length representing the interquartile range, IQR), and the minimum and maximum individual t -statistics (depicted by the two lines or whiskers, with the distance from the end of each line representing the range).

We consider that a latent factor of a specific dimensionality reduction technique is linked to one of the observed factor proxies, when the median t -statistic is non-zero and when the size of the box (IQR) or the distance between the ends of the two whiskers (range) is large, indicating that the majority of the t -statistics throughout the rolling window iterations are significantly different from zero. On the contrary, the relationship of a latent factor with a particular Fama-French factor will tend to be weak if the box falls within the lines depicting the Student's t critical values at the 5% level.

[Insert Figure 3 about here]

The results based on the five factors by Fama and French (2015) are presented in Figure 3 and corroborate those from the variable importance analysis. Overall, there is substantial significance for the latent factors based on PCA and PLS methods with some of the Fama-French factors, and especially

the market factor. When comparing PCA with SPCA, the relationship of the latent factors with the observed factors is diminished for the latter. In contrast, both PLS approaches generate latent factors that exhibit a similar pattern. Autoencoders do not display any strong links to a particular factor, indicating that latent factors based on neural networks cannot be adequately explained by the Fama-French factors.

Focusing on the results for PCA, the first principal component, $K = 1$, is primarily related to the market factor (MKTRF), with the middle 50% of the t -statistics exceeding 5 in absolute value and to the size factor (SMB) where most of the t -statistics are around three standard errors from zero. For both factors the distribution of t -statistics is right skewed since the median is located towards the left and the left whisker is shorter, indicating a negative relationship. The second factor, $K = 2$, based on PCA does not exhibit any strong links to any of the observed proxies, since the t -statistics are less dispersed, as evidenced by the interquartile range and values that are clustered within the 5% critical values bands. The third principal component, $K = 3$, has a strong relation to the value (HML) factor, with t -statistics in the range of -11 to 11. The fourth latent factor, $K = 4$, relates to the size and profitability (RMW) characteristics, while the fifth factor, $K = 5$, is primarily linked to the investment (CMA) factor. The pattern of the t -statistics for SPCA is similar to that of PCA, however the connection of the sparse principal components to the five factors is overall weaker, as evidenced by the smaller dispersion of the t -statistics.

Turning to the results for supervised methods, the patterns observed when comparing the boxplots of both PLS-type approaches appear similar. The first latent factor of PLS and SPLS is positively related to the market factor and, to a lesser extent, the size factor according to the median, with the middle 50% of the t -statistics lying approximately between -9 and 8 for MKTRF and from approximately -2 to 5 for SMB. The remaining latent factors are all positively related to the market, albeit to a lesser degree than the first factor. The second latent factor of both methods is also related to HML, according to the negatively skewed boxplot, with the lower quartile being approximately equal to -1 and the third quartile to 5. The third factor is also linked to the value factor, although the distribution of the t -statistics is more symmetrical. The fourth factor displays a weak relationship with the size and value factors, while

the fifth factor is weakly related to RMW and CMA. Finally, the results for the latent factors based on autoencoders do not reveal any significantly strong links with a particular proxy, when compared to the other dimensionality reduction approaches. For the majority of the autoencoder factors the boxes are symmetrical and smaller, with the middle 50% of the t -statistics being within the 5% critical value bands and concentrated around the median, however, the whiskers are longer indicating a greater range of potential values over the out-of-sample period.

We also examine the relationship of the latent factors with the Hou, Xue and Zhang (2015) q-factor model augmented by the expected growth factor (Hou, Mo, Xue and Zhang, 2021).¹⁰ For PCA and PLS methods the q-factor model is better at explaining the first latent factor, with an average R^2 between 35% and 50%, which decreases significantly for the remaining four factors (8% to 16%). The R^2 does not vary as much when comparing across factors based on autoencoders, with values in the range of 14% to 21%. The market factor is better at explaining the first factor of PCA and PLS approaches, while the investment, return-on-equity and expected growth factors exert greater influence on the remaining factors. For supervised methods the market also relates strongly to the fourth latent factor. The factors based on autoencoders are primarily related to the market and to a lesser extent the investment and return-on-equity characteristics.

We extend the variable importance analysis to a larger dataset, by examining the relationship between the latent factors and long-short anomaly portfolios constructed by Chen and Zimmermann (2020). The equity portfolios are based on a comprehensive reconstruction of firm-level characteristics, which have been replicated using the same data and methods as the original papers. Due to the large number of variables, the influence of each feature to the respective latent factor is derived based on the lasso (Tibshirani, 1996). The penalized model is estimated using a rolling window approach and the results are averaged throughout the out-of-sample period, from January 1980 to December 2019. The variable importance is estimated as the change in R^2 from setting the observations of a feature to zero within each iteration of the rolling window. The results are further aggregated by summing the variable

¹⁰ The results based on OLS regressions of each of the five latent factors on factors from the augmented q-factor model are reported in Figures A3, A4 and A5 in the Appendix.

importance of the characteristics-based portfolios belonging in the same group. We use the same categorization as Chen and Zimmermann (2020) to assign a variable to a group. Details on which anomaly portfolios belong in each of the six groups can be found in Table A1 in the Appendix. Figure 4 presents the results of the variable importance.

[Insert Figure 4 about here]

Across all different models the group comprised of portfolios sorted by accounting characteristics is the most influential according to the lasso. Important variables within this group include portfolios sorted on gross profits/total assets, dividend yield, book-to-market and leverage. The second most influential group is that of portfolios sorted on price characteristics. Individual portfolios with high variable importance values are based on characteristics such as CAPM beta, earnings-to-price ratio, tail risk beta, coskewness and idiosyncratic risk. Portfolios based on trading characteristics, such as liquidity and volume indicators, also relate significantly to the majority of the latent factors. The portfolios from the remaining categories contribute a relatively small part to the total variable importance. The variable importance pattern remains similar across autoencoders with different number of hidden layers, while the results for PCA and PLS methods are less homogenous.

As an alternative we consider the macroeconomic dataset by McCracken and Ng (2015).¹¹ The first latent factor of PCA and PLS methods is strongly related to interest rates and exchange rates. For the remaining factors the results vary based on the dimensionality reduction approach and the value of K . Factors constructed using autoencoders are better explained by interest and exchange rate variables, followed by economic indicators in the labor market or money and credit categories.

4.2 Structure of the Covariance Matrices

Given the dependence of the minimum-variance portfolio on the estimates of the covariance, it is informative to further investigate how machine learning factors affect the structure of the covariance

¹¹ The results based on the McCracken and Ng (2015) macroeconomic dataset can be found in Figure A6 in the Appendix, while details on the eight groups and the variables within each group are reported in Table A2.

matrix. The differences between factor-implied covariance matrices are examined by comparing the structure of the alternative covariance matrix, Σ , to that of the sample estimator, S .

Following the analysis of Moskowitz (2003) we consider three measures. The first measure examines the similarity between two matrices and is given by $Eig_t = \sqrt{\text{tr}(\Sigma'_t \Sigma_t) / \text{tr}(S'_t S_t)}$, where $\text{tr}(\cdot)$ denotes the trace of a matrix. This metric represents the sum of the eigenvalues of the factor-implied covariance matrix as a fraction of the sum of the eigenvalues of the sample covariance matrix. The matrices are squared to capture the absolute amount of covariation. The sum of the eigenvalues provides a measure of total covariation represented by the matrix. The second measure compares the factor-implied covariance matrix with the sample covariance in terms of magnitude and is calculated as $Mag_t = (i' |S_t - \Sigma_t| i) / (i' |S_t| i)$. This measure sums the absolute value of all the elements of the difference between the two matrices and scales this sum with the sum of the absolute value of all the elements of the sample covariance matrix. Finally, we compare the alternative covariance estimates with the benchmark in terms of the direction of the covariances, by determining the fraction of the covariances from the alternative models that have the same sign with those of the sample covariance matrix and is given by $Dir_t = (i' \text{sign}(S_t \circ \Sigma_t) i) / \text{rank}(S_t)^2$.

We report the average value of the measures across the out-of-sample period in Table 1. A high Eig ratio indicates that the alternative covariance matrix is close to the sample estimator. Additionally, if the factor-implied covariance matrix captures similar information to the sample covariance matrix then the magnitude, Mag , should be close to zero and the direction, Dir , should be close to unity. We also examine whether the difference from the sample estimator of a factor-implied covariance is the same as that of a covariance matrix based on the market factor. The two-sided bootstrapped p -value is adjusted for autocorrelation up to 12-month lags.

[Insert Table 1 about here]

Overall, covariance matrices based on latent factors and especially those where the factors are derived from unsupervised methods, exhibit the greatest differences from the sample estimator, for all three measures. The structure of the covariances based on SPCA and shallow autoencoders are the ones that

deviate most from the benchmark. In contrast, the structure of covariances using observed factors is closest to the sample estimator. When comparing across covariance specifications, the differences become more pronounced in the covariance structure when the residual covariance matrix is dynamic. Furthermore, the information captured by the covariance matrices based on latent factors relative to the sample estimator is significantly different at the 1% level to that of the corresponding matrix based on the market factor for all measures.

Specifically, according to the *Eig* measure, the covariance matrices based on latent factors differ considerably from the sample estimator, by a range of 40% to 52%. In contrast, covariance matrices based on the three Fama-French factors or the market factor, differ to the sample covariance matrix by only 1% to 7% or 7% to 9%, respectively. The results under the magnitude and direction measures, remain consistent, with covariances based on latent factors deviating considerably from the sample covariance, while those of observed factors remaining very close to the benchmark. In particular, the values of the latent factors for the *Mag* measure are closer to unity (0.60 to 0.77) further highlighting the difference from the sample estimator, while the results for the observed factors are much closer to zero (0.01 to 0.1). For *Dir* the results show that on average, 36% to 65% of the covariances have a different sign from those of the benchmark covariance, compared to observed factors where there is only a 2.5% to 5% difference. It is interesting to note that the values for the direction measure do not significantly change across covariance specifications.

4.3 Properties of Portfolio Weights

We now explore how the weighing structure of the portfolios based on machine learning differs compared to the naïve allocation, which assigns equal weight to all assets, and to that of the sample estimator and the observed factors.

We start by analyzing the properties of the portfolio weights, $\hat{\omega}$, using the maximum weight (MAX), the standard deviation of the portfolio weights (SD_{ω}) and the mean absolute deviation from the equal-weighted portfolio (MAD_{EW}) calculated as $1/N \sum_{i=1}^N |\hat{\omega}_i - 1/N|$. In line with DeMiguel, Garlappi and Uppal (2009), we report the average monthly portfolio turnover (TO) computed as the average absolute

change of the portfolio weights over the T_{OOS} rebalancing periods across the N assets. The turnover at time $t + 1$ is given by $\|\omega_{t+1} - \omega_t\|_1$, where ω_{t+1} is the vector of portfolio weights at time $t + 1$ and ω_t are the portfolio weights at the time before rebalancing. Table 2 reports the average value of each weight characteristic over the 480 out-of-sample periods.

[Insert Table 2 about here]

Overall, the portfolios based on machine learning methods tend to produce weights which are smaller, less volatile and closer to those of EW portfolio, than models based on observed factors. The maximum weight attained for a single asset over the out-of-sample period by the sample estimator is 38%. Portfolios based on observed factors also have high maximum weights, from 30% to 44.5% for the market and from 31% to 45% for FF3. On the contrary, the maximum weight for portfolios based on latent factor models is considerably lower, with values between 6.6% and 16.5%. Comparing across covariance specifications, when the error covariance is allowed to vary over time the portfolios produce higher maximum weights than the other specifications. Allocations based on latent factors and static covariance matrices tend to have smaller maximum weights than those based on observed factors or dynamic covariances, which is an indicator of less extreme portfolio positions.

Portfolios with the least volatile weights are those based on unsupervised methods, with a standard deviation that is lower by approximately a factor of three from that of the sample or observed factors. For latent factors, SD_ω is between 3.5% and 6% per annum, while observed factors have the most dispersed weights, with annualized values between 8.5% and 12% (monthly SD_ω is annualized by scaling with $\sqrt{12}$). The weights of allocations based on dynamic error covariance vary more over time, with an increase in standard deviation by over 20% to that of static covariance. Portfolios based on unsupervised learning methods tend to produce weights with the smallest deviation from the $1/N$ portfolio, with MAD_{EW} values between 2.7% to 4% per annum, indicating that these portfolios are better diversified. In contrast, observed factor models exhibit the highest deviation from the benchmark, having annualized MAD_{EW} in the range of 5.3% to 5.9%, potentially concentrating on a smaller number of assets. The weights of portfolios based on the dynamic error covariance deviate more from the

benchmark than the remaining specifications. Having weights that vary less throughout the out-of-sample period and closer to the $1/N$ strategy, is a positive indicator regarding the effects of transaction costs on portfolio performance. This is due to portfolios of unsupervised methods being more likely to exhibit lower turnover (and thus transaction costs) than those of observed factors or the sample estimator.

The results show that monthly turnover is considerably lower for portfolios using latent factors. Portfolios based on unsupervised methods for the static specification have low rebalancing requirements, producing monthly turnover that is up to 30% lower compared to the sample or observed factor portfolios. Lower turnover indicates that these portfolios would be less affected by transaction costs. Turnover for the equal-weighted portfolio is naturally very low, since the only cause for rebalancing are changes in the stock universe from one period to the next. The portfolio based on the sample estimator generates a monthly turnover of 41.269%, which is higher than most of the portfolios, except those based on observed factors or those that allow $\Sigma_{u,t}$ to vary over time. Comparing across different factor model specifications, portfolios based on dynamic covariance matrices exhibit higher turnover than those using the static factor covariance. Regardless of the specification, factor models with observed factors generate higher turnover than those with latent factor models. The lowest turnover is produced by portfolios with static and dynamic factor covariances based on PCA (29.737% and 30.017%) and SPCA (29.931% and 29.973%) factors, followed by portfolios using autoencoder factors with turnover varying from 31.060% to 49.029%, depending on factor specification. Neural networks with more hidden layers have consistently higher rebalancing requirements than shallower networks.

5. Asset Allocation

We now proceed to analyze the out-of-sample performance of the portfolios based on the factor and covariance specifications explored in Section 4 using a variety of performance measures. The buy-and-hold portfolio returns are calculated for the period of one month and the portfolio is rebalanced monthly until the end of the evaluation period (January 1980 to December 2019). In addition, we analyze the

behavior of the portfolios during different subperiods and investigate the effects of transaction costs on portfolio performance. We further evaluate the performance of portfolios for allocations without short-selling constraints or for a penalized minimum-variance objective function. Finally, we examine the effects on portfolio performance for a different number of assets or alternative data sets.

The results show that portfolios based on machine learning significantly outperform the equal-weighted benchmark and improve upon allocations based on observed factors or the sample covariance estimator. These economic benefits persist when performance is evaluated using alternative risk metrics. Furthermore, the risk-adjusted performance of the machine learning portfolios significantly improves relative to the benchmark as the investor's aversion to risk increases. The best performing models are those based on shallow autoencoders or sparse PCA and covariance specifications that allow B or Σ_u to be time-varying. The benefits of machine learning are amplified during periods of high volatility, while the economic value of factor-based optimization is also evident during different inflation and credit spread regimes.

All portfolios generate positive breakeven transaction costs, indicating they outperform the benchmark. However, an investor would realize greater economic gains over the EW portfolio by choosing machine learning portfolios. When the short-selling restriction is relaxed, performance of latent factor portfolios remains stable relative to those using observed factors, indicating a greater inherent error in covariance estimation by the latter. Likewise, penalizing portfolio turnover, is beneficial primarily to portfolios that were more sensitive to transaction costs, such as those of observed factors or latent factors based on supervised methods. The factor-based portfolios continue to outperform the benchmark even when the number of assets is increased. The turnover of the portfolios increases dramatically with the number of assets, while the breakeven transaction costs decrease, with machine learning models still generating the highest performance across different portfolio sizes. Finally, considering alternative datasets based on industry or book-to-market sorted portfolios, the results tend to favor observed factors or supervised methods.

5.1. Portfolio Performance

We focus on evaluating the performance of the portfolios based on measures of risk and risk-adjusted returns, since equation (15) is designed to minimize variance rather than maximize the expected return. Therefore, similar to Ledoit and Wolf (2017) and De Nard, Ledoit and Wolf (2019), we primarily compare the economic value of the alternative covariance matrix estimates using the standard deviation, followed by the Sharpe ratio. In Table 3 we report the monthly performance of the portfolios over the out-of-sample period, T_{OOS} , based on the standard deviation (SD) of the 480 out-of-sample portfolio returns in excess of the risk-free rate and the Sharpe ratio (SR) of the portfolio calculated as $(\bar{r}_p - \bar{r}_f)/SD$, where $\bar{r}_p$ is the average value of the 480 out-of-sample portfolio returns and $\bar{r}_f$ is the average value of the risk-free rate.

We also consider the question whether one portfolio delivers improved out-of-sample performance relative to another portfolio at a level that is statistically significant. Since we consider 42 portfolios, there would be 861 pairwise comparisons. DeMiguel, Garlappi, and Uppal (2009) provide persuasive evidence that the simple equal-weighted portfolio should serve as a natural benchmark to assess the performance of more sophisticated asset allocation strategies. To avoid a multiple-testing problem and since one of the major goals of this study is to outperform the $1/N$ rule, we restrict our focus to the comparison of the equal-weighted with the alternative portfolios. For each case, a two-sided p -value is obtained by the prewhitened HAC_{PW} test proposed by Ledoit and Wolf (2011) for the null hypothesis of equal standard deviations and by Ledoit and Wolf (2008) for the null hypothesis of equal Sharpe ratios.

[Insert Table 3 about here]

The results from Table 3 indicate that portfolios using machine learning consistently outperform the equal-weighted benchmark in terms of standard deviation and Sharpe ratio by a wide margin. Using a covariance matrix based on machine learning factors, can lead to a statistically significant decrease in out-of-sample standard deviation of up to 29% and a significant increase in Sharpe ratio of over 25% relative to the $1/N$ portfolio. Factors based on sparse principal component analysis and autoencoders

are found to yield the best performance, with shallower neural networks outperforming those with more hidden layers. Additionally, estimators that allow the loadings or error covariance to be time-varying outperform the static or dynamic covariance specifications. It is interesting to note that the factor models whose covariance structure diverges most from the sample covariance correspond to those that generate the highest portfolio performance. The latent factor models that tend to diverge the most from the sample estimator are based on PCA, sparse PCA and an autoencoder with one hidden layer.

Unsupervised learning approaches used to derive the latent factors tend to produce portfolios with lower standard deviation. The best performing model is AEN1, with monthly standard deviation of 3.216% and 3.318% depending on covariance specification, except for dynamic beta covariance where SPLS yields the lowest standard deviation with a value of 3.261%. The outperformance of the shallow autoencoder (AEN1) from the EW strategy is between 2.9% and 3.3% per annum, varying based on the covariance specification. The shallow autoencoder and SPCA have the highest Sharpe ratios, with values between 0.82 and 0.85 in annual terms. AEN1 has the highest ratio in the dynamic beta covariance specification, while SPCA is the best performing model for the remaining specifications. Overall, the best performing portfolios are based on the dynamic error covariance or dynamic beta covariance specifications, while portfolios based on static or dynamic factor covariance generate comparable performance.

The outperformance over $1/N$ portfolio across factors and model specifications in terms of standard deviation is statistically significant at the 1% level. The outperformance in terms of Sharpe ratio is, however, statistically significant only for latent factor models. For the Sharpe ratio, latent factor models generate statistically significant outperformance, compared to observed factors and the sample estimator that yield insignificant results. Specifically, SPCA consistently outperforms the $1/N$ portfolio at the 1% level, followed by AEN1 that for dynamic error covariance is statistically significant at the 5% level and at the 1% level for the remaining specifications. All other models are statistically significant at the 5% or 10% levels. The exception are portfolios based on PLS, for static or dynamic factor specifications, which are the only latent factor models that are not found to be statistically different from the $1/N$ portfolio.

The latent factor models consistently outperform the sample estimator across all covariance specifications, for both performance measures. For example, a portfolio based on autoencoders would result in a standard deviation and Sharpe ratio which improve by up to 8% and 17%, respectively, compared to the portfolio formed using the sample estimator. On the other hand, the observed factor models improve upon the sample estimator primarily in terms of Sharpe ratio, with the Fama-French three-factor model generating better results than the market. Latent factor models generate lower standard deviation and higher Sharpe ratio than models using the market or FF3 factors. The decrease in annualized standard deviation compared to portfolios based on the worst performing factor model (Market) can be up to 1.5%. Furthermore, choosing the machine learning factors can lead to an increase in Sharpe ratio of 10% to over 16% relative to the market factor, depending on the covariance specification. Latent factors tend to favor a time-varying error covariance, $\Sigma_{u,t}$, while the observed factors yield better results when the matrix B_t is time-varying.

The results throughout the paper are based on models using a validation window consisting of the last 20% of the observations in the rolling window, as described in Section 3. We also examine the performance of the portfolios when the size of the validation window is reduced to 10% or increased to 30% of the observations in the rolling window. The results are reported in Table A3 in the Appendix and are qualitatively similar to the baseline case, with methods such as SPCA and autoencoders favoring the longer validation window.

Furthermore, we examine the out-of-sample monthly portfolio performance using several alternative risk measures, including the mean absolute deviation (MAD), value-at-risk (VaR) and conditional value-at-risk (CVaR) of the 480 out-of-sample portfolio returns in excess of the risk-free rate. Tail risk is of particular importance for portfolios during periods of financial distress. The historical VaR at the $100(1 - a)\%$ level is computed as $-z_a \bar{\sigma}_p - \bar{r}_p$, where $z_a = \Phi^{-1}(1 - a)$ is the $a\%$ quantile of a standard normal distribution and Φ is the cumulative standard normal distribution function. The historical CVaR at the $100(1 - a)\%$ level calculated as $(1 - a)^{-1} \varphi(z_a) \bar{\sigma}_p - \bar{r}_p$, where φ is the probability density function of the standard normal distribution. The results for the MAD along with the VaR and CVaR calculated at the 95% confidence level are reported in Table 4.

[Insert Table 4 about here]

Results for the alternative risk criteria also point towards the benefits of using machine learning latent factors for covariance estimation, with all models outperforming the equal-weighted benchmark. Using a factor-implied covariance matrix based on machine learning can lead to an improvement of up to 27% in terms of MAD and between 23% to 26% in terms of VaR and CVaR compared to the $1/N$ portfolio, offering a measure of protection to investors concerned with tail risk.

The best performing models are based on autoencoders or SPCA, outperforming the EW benchmark by up to 2.9%, 1.26% and 1.57% per annum, in terms of MAD, VaR and CVaR respectively. Latent factor models outperform portfolios based on the sample covariance or observed factor models. In comparison to the market factor the best performing factors based on machine learning can lead to an over 17% reduction in terms of MAD and an improvement of over 13% and 12% in terms of VaR and CVaR respectively. Observed factors tend to favor covariance specifications that allow B_t to vary over time. For latent factors, the greatest improvement from the benchmark, according to MAD, occurs when the residual covariance is time-varying, while the results for VaR and CVaR appear more favorable in the static specification.

We further investigate the economic value of the covariance matrix estimates using the certainty equivalent return (CER), defined as: $CER = \bar{r}_p - 0.5\gamma\bar{\sigma}_p^2$, where $\bar{r}_p$ and $\bar{\sigma}_p^2$ are the mean and variance of the portfolio returns over the out-of-sample period. The CER can be interpreted as the risk-free return that a mean-variance investor with a coefficient of relative risk aversion γ is willing to accept instead of investing in the risky portfolio. We consider values of the risk aversion parameter γ , that roughly correspond to an aggressive ($\gamma = 2$), moderate ($\gamma = 5$) or conservative ($\gamma = 10$) investor.¹² Given that the optimization objective is to minimize variance, the portfolios would be of interest to investors more sensitive to risk, which makes CER with higher values of γ a more representative performance measure.

Following Neely, Rapach, Tu and Zhou (2014), we report the difference in monthly CER (ΔCER), which is equivalent to the percentage CER generated by the portfolio based on alternative covariance matrices

¹² Similar values for the parameter of risk aversion have been used in DeMiguel, Garlappi and Uppal (2009).

minus that of the EW benchmark. ΔCER can be interpreted as the performance fee that the investor would be willing to pay to use the information of each alternative covariance estimator instead of the benchmark. To test the statistical significance of the CER gains, relative to the EW allocation approach, we use the two-sided p -value obtained by the test developed by DeMiguel, Garlappi and Uppal (2009), for the null hypothesis of equal CER.

[Insert Table 5 about here]

The results reported in Table 5 show that portfolios based on machine learning outperform simpler benchmarks, leading to statistically significant utility gains that exceed those of the EW by 2.5% to 4.5% on an annual basis, for investors with moderate or conservative risk preferences, respectively. The CER for the $1/N$ strategy decreases as the parameter of risk aversion increases, turning negative when $\gamma = 10$. All portfolios generate positive ΔCER , with the outperformance from the EW portfolio increasing as the parameter of risk aversion, γ , increases in value. The best performing model in the case of the static factor covariance is SPCA with monthly ΔCER between 0.109% and 0.346% depending on the degree of risk aversion. In the case of dynamic estimators, autoencoders yield the highest ΔCER for $\gamma = 10$. For $\gamma = 5$, AEN1 outperforms the other models for dynamic beta covariance, while SPCA performs best when $\Sigma_{f,t}$ and $\Sigma_{u,t}$ are time varying.

Latent factor models generate statistically significant results at the 1% level across all covariance specifications for a conservative investor. In the case of a moderate investor, the portfolios that offer significant CER gains over the EW allocation at the levels of 5% or 10%, are those based on unsupervised methods, except for SPLS which generates statistically significant results when B_t varies over time. Compared to latent factor models, portfolios using the sample estimator or observed factors do not generate statistically significant CER relative to the naïve allocation when $\gamma = 5$. For $\gamma = 10$, the outperformance from the EW portfolio tends to be insignificant, with only a few exceptions. The sample covariance has statistically significant ΔCER at the 10% level. Portfolios based on observed factors yield statistically significant results only in the case of the dynamic beta covariance specification or at the 10% level for the Fama-French three-factor model based on the dynamic error covariance.

However, in the case of an aggressive investor ($\gamma = 2$) the outperformance of the alternative estimators of the covariance matrix is not statistically significant.

To summarize, we find that the performance of the models significantly improves relative to the benchmark as the coefficient of risk aversion increases. The comparison of the ΔCER across models confirms the conclusions reached from the analysis of the Sharpe ratios, with unsupervised methods that induce sparsity or introduce non-linearities to the factors yielding the highest significant performance. The factor models can earn a consistently higher out-of-sample CER return than the EW portfolio, especially when latent factors are derived by machine learning methods, with performance that remains statistically significant for investors with moderate or conservative risk appetite. Ideally the results would remain statistically significant across all values of γ . However, since the objective of this asset allocation exercise is to minimize variance, the portfolios would primarily be of interest to investors that are not willing to undertake a high degree of risk, while more aggressive investors would opt for an objective function that seeks to maximize return or Sharpe ratio.

5.2 Subperiod Analysis

In this section we examine portfolio performance during different subperiods as defined by market volatility, inflation and credit spread. The impact of different market regimes on asset allocation has been well documented in the literature (see Ang and Bekaert, 2002a; Guidolin and Timmermann, 2008; Pettenuzzo and Timmermann, 2011). To analyze the results under different market regimes, we derive the periods of high and low market volatility using a Markov Switching model.¹³

[Insert Table 6 about here]

During high volatility periods (Panel A) all portfolios outperform the equal-weighted benchmark in terms of both standard deviation and Sharpe ratio. Strategies based on machine learning models,

¹³ The regimes are determined based on the filtered probabilities of the following two-state Markov Switching model: $r_{m,t} = \mu_s + e_{t,s}$, with $e_{t,s} \sim N(0, \sigma_{t,s}^2)$, where r_m is the market factor return, s represents the latent state and μ_s and σ_s^2 denote the state dependent mean and variance. When the filtered probability of the low volatility state is lower than 0.5 the market is in a high-volatility period, while observations where the filtered probability of the low volatility state is higher than 0.5 are low-volatility periods.

especially unsupervised methods, generate the highest statistically significant results. Specifically, the best performing strategy is that of a shallow autoencoder with monthly Sharpe ratios ranging from 0.197 to 0.205 and significant at the 1% level. An exception is the case of covariances with loadings allowed to vary over time, which leads to the Fama-French three-factor model having the highest performance with a ratio of 0.213 that is significant at the 5% level. For low volatility periods (Panel B) machine learning portfolios are less risky by up to 6%, albeit statistically indistinguishable from the $1/N$ portfolio. Unsupervised methods outperform supervised methods, producing a standard deviation that is lower by 13% to 28% than portfolios based on observed factors. Furthermore, portfolios based on machine learning are statistically indistinguishable from the EW portfolio also in terms of Sharpe ratio. In contrast, portfolios based on observed factors are more negatively affected than those based on latent factors and exhibit significantly reduced performance from the benchmark.

Asset allocation is also affected by shifts in the economic environment due to inflation and credit spread, where the presence of separate regimes has been detected (see Ang and Bekaert, 2002b; Kritzman, Page and Turkington, 2010 for example). To this end, we also investigate changes in portfolio performance during subperiods of high and low inflation or credit spread.¹⁴

[Insert Table 7 about here]

Table 7 presents the results for different inflation subperiods. During periods of high inflation (Panel A) all strategies dramatically outperform the EW portfolio. Additionally, an investor interested more in minimizing volatility would prefer latent factor models. Specifically, portfolios based on SPCA or AEN have statistically significant standard deviations that are up to 24% lower than the benchmark. On the other hand, an investor opting to maximize the Sharpe ratio would choose observed factor models, which offer slightly higher ratios than latent factors, but are significant at a lower level. The results for observed factors tend to vary in terms of significance, while for latent factors they are consistently

¹⁴ The series used are based on the year-on-year change of Consumer Price Index for all urban consumers, for inflation and the difference between Moody's Baa corporate bond yield and the yield on a 10-year Treasury, for credit spread. Both series are retrieved from FRED. Periods of high (low) inflation or credit spread are defined by whether inflation or credit spread for a specific month is higher (lower) than the median over the out-of-sample period.

significant at the 1% level across specifications for both measures. In contrast, during low inflation periods, the Sharpe ratio of the EW allocation is high, with strategies based on unsupervised methods outperforming it, albeit insignificantly. In terms of standard deviation all strategies yield improved and significant performance, with unsupervised methods generating the lowest risk. An investor would realize a reduction of over 20% in standard deviation and an increase of up to 7% in Sharpe ratio by choosing strategies based on machine learning. Comparing across covariance specifications, those that have time-varying B and Σ_u tend to have higher performance.

[Insert Table 8 about here]

Turning to the results for the credit spread in Table 8, we observe that during the high credit spread regime (Panel A), all models outperform the EW. The standard deviation is significantly lower than the $1/N$ strategy at the 1% level for all portfolios, however, the Sharpe ratios tend to be insignificant even though they are higher than the benchmark. Some outliers include the latent factor models for strategies based on dynamic error covariance, the Fama-French factors for time-varying loadings and the shallow autoencoder for static factor models, which produce statistically significant values. In the low credit spread regime the differences across models become more apparent. Compared to periods with a high spread, the covariance specification that benefits most asset allocation is based on static factor models or a covariance with dynamic loadings or factors. The standard deviation of portfolios based on latent factors is lower by over 17% from the EW and statistically significant at the 1% level across specifications, as opposed to the observed factors that are significant only for dynamic loadings. Finally, the factor-based portfolios of unsupervised methods have significant Sharpe ratios, which can be 28% higher than those of the benchmark portfolio.

5.3 Effects of Transaction Costs

Following Han (2006), we consider the breakeven transaction costs in basis points that would cause an investor to be indifferent between a certain strategy and the benchmark strategy. The breakeven transaction costs in terms of Sharpe ratio of a portfolio relative to the equal-weighted portfolio, c_{ew} , are calculated as $\Delta SR / \Delta TO$, where $\Delta SR = SR_p - SR_{ew}$ is the difference in Sharpe ratios of the alternative

portfolio from the equal-weighted benchmark and $\Delta TO = TO_p - TO_{ew}$ is the difference in the respective average turnover. Breakeven transaction costs become important in the absence of reliable estimates of transaction costs, with positive values of c_{ew} indicating that the alternative model outperforms the benchmark.

[Insert Table 9 about here]

Table 9 shows that all models generate positive c_{ew} , indicating outperformance from the benchmark. However, an investor would realize greater economic benefits over the EW portfolio by choosing machine learning portfolios, since they exhibit higher breakeven transaction costs by a factor of three from portfolios based on observed factors. The breakeven transaction costs are higher for unsupervised methods than the sample estimator or the remaining factor models. Furthermore, static and dynamic factor covariance strategies produce the highest breakeven transaction costs across all specifications. Portfolios with factors estimated by SPCA have annualized breakeven transaction costs between 48 and 69 bps, while a shallow autoencoder would realize c_{ew} from 46 to 64 bps per annum. On the contrary, the worst performing factor-based portfolios are those of the market factor, which yield the lowest breakeven transaction costs (14 to 22 basis points) followed by that of the sample estimator with breakeven transaction costs of approximately 22 basis points. This concurs with the earlier evidence from portfolio turnover, which shows that these strategies have lower rebalancing requirements. The results for average turnover and breakeven transaction costs suggest that rebalancing frequency is a key contributor to portfolio performance. This is in keeping with the literature (see Han, 2006), that finds increasing breakeven transaction costs for reduced rebalancing frequency.

We also examine the performance of the portfolio for specific levels of transaction costs. In this setting transaction costs would arise from changes to the stock universe from one month to the next and from the change in weights of stocks that remain in the stock universe for multiple iterations. The portfolio's return is modified to account for transaction costs based on portfolio turnover. Given a transaction cost of c , the trading cost of the entire portfolio is $c\|\omega_{t+1} - \omega_t\|_1$. The return of the portfolio after

transaction costs becomes $r_{p,t+1}^{TC} = (1 + r_{p,t+1})(1 - c\|\omega_{t+1} - \omega_t\|_1) - 1$. We consider values for transaction costs of $c \in \{5, 20\}$ basis points.¹⁵

The results are reported in Table A4 in the Appendix, for transaction costs of $c = 5$ basis points (Panel A) and for $c = 20$ basis points (Panel B). Overall, the results remain consistent with those of Table 9. Models that exhibit higher average monthly turnover and lower breakeven transaction costs are more affected by transaction costs. In terms of standard deviation, the results remain virtually unchanged compared to the baseline case, regardless of the level of transaction costs. Unsurprisingly, the $1/N$ portfolio suffers the least when transaction costs are introduced. For transaction costs of 5 bps the values of the Sharpe ratios also remain relatively unaffected, but there is a change in their levels of significance. Specifically, sparse PCA and the shallow autoencoder are now significant at the 5% instead at the 1% level, while PLS-type methods and AEN4 are no longer statistically significant in most covariance specifications. When transaction costs are increased to 20 bps, portfolios based on the sample covariance and market factor either marginally or fail to outperform the EW benchmark. The latent factors that significantly outperform the benchmark at the 10% level are those of SPCA and AEN1, for all specifications except when the error covariance is dynamic.

5.4 Portfolios Allowing Short-Selling

In this section we consider the case of the minimum-variance portfolio in the absence of short-selling constraints. Imposing a restriction to short positions is known to limit extreme portfolio weights. It is interesting to investigate how removing short-selling constraints, thus potentially allowing for large short positions, affects portfolio performance. The objective becomes

$$\underset{\omega}{\operatorname{argmin}} \omega' \hat{\Sigma}_r \omega, \quad \text{s.t.} \quad \omega' \mathbf{i}_N = 1. \quad (20)$$

¹⁵ The value of 5 bps may be low by academic standards, where values as high as 50 bps (Kirby and Ostdiek, 2012) have been used. Other studies are less conservative and use a range of transaction costs. Ledoit and Wolf (2017) consider transaction costs of $c \in \{3, 50\}$ bps, pointing out that 3 bps is representative for liquid stocks, while Moura, Santos and Ruiz (2020), use $c \in \{5, 10\}$ bps.

Compared to the $1/N$ portfolio or the problem in (15), the asset weights, ω , can be negative. The results are reported in Table 10.

[Insert Table 10 about here]

Removing the short-selling constraint leads to an overall decrease in portfolio performance. Portfolios based on latent factors remain relatively unaffected, in terms of magnitude and significance of the various measures. On the contrary, portfolios based on the sample covariance and observed factors show a considerable increase in standard deviation. However, an increase in return results in Sharpe ratios of the same level as portfolios that allow short-sales. Furthermore, when loadings are dynamic, allocations based on observed factors generate standard deviations at a level closer to the baseline results. The increase in volatility for the observed factors is between 6% and 22% depending on the covariance specification. Allowing the portfolio weights to be negative has also led to a dramatic increase in turnover and a decrease in breakeven transaction costs for observed factors and the sample estimator. This indicates that imposing a short-selling constraint to portfolios based on the sample covariance or observed factors helps to reduce estimation error due to covariance misspecification. In contrast, even without restricting short positions, the risk of portfolios based on latent factors remains at a similar level to the portfolios with $\omega_i \geq 0$.

The results on the maximum weight, standard deviation of portfolio weights and deviation of the weights from those of the EW portfolio are reported in Table A5 in the Appendix. When short-selling is allowed, portfolios based on the sample estimator, followed by those of observed factors, have weights that are larger, more volatile and further away from those of the $1/N$ portfolio than portfolios based on latent factors. Equivalently, minimum-variance portfolios based on a time-varying residual covariance matrix exhibit higher values across the three measures than the remaining specifications.

5.5 Turnover-Constrained Portfolios

We follow Olivares-Nadal and DeMiguel (2018) who show that incorporating a l_p transaction cost term in the portfolio problem may help to reduce the impact of estimation error. Here we consider the case of proportional transaction costs and modify the minimum-variance optimization problem by adding a

l_1 transaction cost term, which is equivalent to assuming that transaction costs are proportional to the amount traded. The new constrained optimization problem becomes

$$\underset{\omega}{\operatorname{argmin}} \omega' \hat{\Sigma}_T \omega + \kappa \|\omega - \omega_0\|_1, \quad \text{s.t.} \quad \omega' \mathbf{1}_N = 1, \quad \omega_i \geq 0, \text{ for } i = 1, \dots, N, \quad (21)$$

where κ is the transaction cost parameter that controls for the degree to which portfolio turnover is penalized and ω_0 are the weights of the portfolio from the previous period before rebalancing. For the first period of the expanding window the weights ω_0 are initialized based on the original minimum-variance allocation. The transaction costs are set to $\kappa = 5$ bps for each asset. When $\kappa = 0$ the above optimization problem becomes equivalent to the one in (15). Table 11 presents the results for the penalized portfolios.

[Insert Table 11 about here]

Overall, including a turnover penalty has an adverse effect for portfolios based on unsupervised methods that exhibit low turnover, but is beneficial to portfolios with high turnover such as supervised methods and observed factors. This improved performance is due to an increase in portfolio return since standard deviation remains either at the same or at a higher level. The effects of regularization become more evident when looking at the results in Panel B. Turnover for observed factors and supervised methods is reduced by approximately 50%, while the breakeven transaction costs more than double compared to unpenalized portfolios.

The results for the remaining weight characteristics for the penalized portfolios can be found in Table A6 in the Appendix. The positive effects of regularization are apparent in the cases of the sample estimator and the observed factors, where the standard deviation of the weights is reduced by up to 7.5% and MAD_{EW} by over 6% compared to the baseline case.

5.6 Increased Number of Assets

Here we examine how portfolio performance is affected when the number of assets increases. Along with the baseline case for $N = 100$, the results for portfolios for $N = \{200, 300, 400, 500\}$ largest

stocks by market capitalization are presented in Figure 5 for the case of the static factor covariance specification.¹⁶

[Insert Figure 5 about here]

When the number of assets is increased, the $1/N$ portfolio is consistently outperformed in terms of standard deviation and Sharpe ratio by all portfolios. The standard deviation of the EW portfolio increases with the size of the portfolio, while the pattern for observed factors is mixed, first increasing for $N = \{200, 300\}$ and then decreasing slightly for $N = \{400, 500\}$. The volatility of latent factor models slightly increases for $N = 200$ and then remains relatively stable across different portfolio sizes. The Sharpe ratio for all factor models is highest for $N = 100$, decreases when $N = \{200, 300\}$ and then increases again for $N = \{400, 500\}$, with latent factor models producing higher ratios than observed factors. Average monthly turnover steadily increases with the number of stocks in the portfolio, with observed factor models generating higher turnover than latent factor models consistently across all portfolio sizes, indicating that portfolios based on the Market and FF3 factors are more sensitive to transaction costs. The results show that increasing the size of the portfolios has a considerable negative impact on breakeven transaction costs, with all models experiencing a sharp decrease in c_{ew} , indicating larger portfolios are more sensitive to transaction costs. Comparing across factor specifications, unsupervised methods generate the highest breakeven transaction costs for different values of N . In contrast, portfolios based on the market factor generate negative breakeven transaction costs for $N = \{200, 300\}$, failing to outperform the benchmark, and then positive but small values for larger portfolios.

5.7 Different Data Sets

In our analysis up to this point we have focused on individual stocks as assets, since this case tends to be of greater interest to investors and fund managers. On the other hand, a great number of academic

¹⁶ The results for the remaining covariance specifications exhibit a similar pattern to that of the static case and are presented in Figures A7, A8 and A9 in the Appendix, for the cases when B , Σ_f and Σ_u are dynamic, respectively. Additionally, the results for autoencoders with three and four hidden layers were similar to those of an autoencoder with 2 hidden layers and are not presented for the sake of brevity.

studies have been devoted to asset portfolios. To this end we consider two alternative data sets. The first is comprised of 49 portfolios based on industry classification and the other of 21 assets, including 20 portfolios sorted based on size and book-to-market (BM) in addition to the S&P 500.¹⁷ The size and splitting of the sample are the same as for individual stock data.

Looking at the results for the 49 industry portfolios (Table 12), all minimum-variance portfolios outperform the EW allocation, in terms of both standard deviation and Sharpe ratio. The best performing portfolios are those based on PLS methods and the market factor. The standard deviation is statistically significant at the 1% level for all models; however, latent factor methods generate significant outperformance at the 1% level in terms of Sharpe ratios, compared to the sample estimator or observed factors, which are significant at the 5% or 10% levels. The factors that generate the best performing portfolios in terms of breakeven transaction costs and average turnover depend on the covariance specification. Observed factors tend to favor the static covariance specification, while machine learning factors yield lower turnover and higher breakeven transaction costs relative to observed factors, for dynamic covariance specifications.¹⁸

[Insert Table 12 about here]

Specifically, using a covariance matrix based on SPLS or the market factor leads to Sharpe ratios between 0.78 and 0.83 per annum, which is an increase of 50% compared to the EW portfolio. The annual decrease in standard deviation from the $1/N$ portfolio is between 3.11% to 4.40% depending on the factor and covariance specification, with latent factors favoring allocations based on covariance matrices where Σ_u is dynamic. Turnover varies largely depending on the model. Observed factors generate monthly turnover between 2% (Market) and 2.5% (FF3) when the covariance is static, while in a dynamic specification average monthly turnover significantly increases to the range of 6.2% to 25.8% for the market and from 8.6% to 32.1% for the Fama-French three-factor model. In contrast,

¹⁷ Following Wang (2005) and DeMiguel, Garlappi and Uppal (2009), we exclude the five portfolios containing the largest firms because the market, SMB, and HML are almost a linear combination of the Fama and French portfolios.

¹⁸ Return observations are not available from the beginning of the sample period for all industry portfolios. This causes a small increase to monthly turnover as the portfolios that were missing observations in the beginning of the backtest enter the asset universe.

latent factor models generate lower turnover in a dynamic specification. Supervised approaches tend to require less frequent rebalancing than unsupervised methods, with the exception of the dynamic beta specification that leads to PCA and AEN portfolios with lower turnover (between 16% and 17.8%). The breakeven transaction costs indicate that observed factors yield higher monthly c_{ew} when the factor-based covariance is static (392 bps for the Market and 275 bps for FF3). In a dynamic specification PLS and SPLS yield the highest breakeven transaction costs ranging from 38.9 to 157.5 basis points per month, compared to observed factors (20.3 to 122 bps).

Turning to the results for the 20 book-to-market portfolios and the S&P 500 (Table 13) the standard deviation is significantly lower at the 1% level from the benchmark for all alternative allocations. Fama and French factors and the sample estimator yield standard deviations that are lower by approximately 2.7% from the $1/N$ portfolio per annum compared to a decrease from 1.4% to 1.9% for the latent factor models. The variation in Sharpe ratios across models is small, with the Market factor producing the highest ratio. Sharpe ratios are statistically higher than the benchmark at the 5% or 10% level. However, portfolios based on the sample estimator or observed factors, exhibit increased turnover compared to latent factor models, which has a considerable effect on breakeven transaction costs. For example, the monthly c_{ew} for PLS, SPLS and shallow autoencoders for the static covariance specification, is between 154.5 and 171.7 compared to 94.2 to 97.8 bps for the observed factors or 102.3 bps for the sample covariance.

[Insert Table 13 about here]

6. Conclusion

In this paper we explore the properties of machine learning factor-based portfolios. We then examine whether factor-implied covariance matrices based on machine learning dimensionality reduction techniques can benefit minimum-variance portfolios comprised of individual stocks. Overall, our findings indicate that machine learning can help improve factor-based portfolio optimization.

When exploring the characteristics of machine learning portfolios, we find that factors based on PCA and PLS exhibit a stronger relationship with commonly used factor proxies than autoencoders. Furthermore, the structure of covariance matrices that are dynamic or based on unsupervised methods diverges the most from that of the sample estimator. Unsupervised learning methods yield portfolios that require less frequent rebalancing, with weights that are less volatile and more diversified relative to their supervised counterparts or observed factors.

Our analysis also shows that methods which induce sparsity and autoencoder neural networks tend to be the best performing models. Specifically, we find that the proposed models can lead to a statistically significant reduction in portfolio volatility of up to 3.3% per annum and a significant increase in the Sharpe ratios of approximately 25%, relative to the $1/N$ portfolio. Furthermore, using a factor-implied covariance matrix based on machine learning can lead to an improvement of up to 27% in terms of MAD and between 23% to 26% in terms of tail risk measures compared to simpler benchmarks. These findings become more acute as an investor's sensitivity to risk increases. Investors with moderate or conservative risk preferences would realize statistically significant utility gains that exceed those of the equal-weighted allocation by 2.5% to 4.5% on an annual basis. The benefits of machine learning to factor-based allocations are increased during periods of high volatility and are also evident in different inflation and credit spread regimes. Furthermore, the results show that shallower neural networks outperform deeper architectures, which falls in line with conclusions reached by recent applications of machine learning in finance. Finally, when comparing various static and dynamic factor model specifications, the results indicate that models which allow the error component of the covariance matrix to vary over time can deliver increased performance but at the cost of higher portfolio turnover.

References

Ang, A., and G. Bekaert. 2002a. International asset allocation with regime shifts. *The Review of Financial Studies* 15:1137-87.

- . 2002b. Regime switches in interest rates. *Journal of Business & Economic Statistics* 20:163-82.
- Ardia, D., G. Bolliger, K. Boudt, and J. Gagnon-Fleury. 2017. The impact of covariance misspecification in risk-based portfolios. *Annals of Operations Research* 254:1-16.
- Avramov, D., and T. Chordia. 2006. Asset pricing models and financial market anomalies. *The Review of Financial Studies* 19:1001-40.
- Bai, J., and S. Ng. 2002. Determining the number of factors in approximate factor models. *Econometrica: Journal of the Econometric Society* 70:191-221.
- Baldi, P., and K. Hornik. 1989. Neural networks and principal component analysis: Learning from examples without local minima. *Neural Networks* 2:53-8.
- Bali, T. G., R. F. Engle, and Y. Tang. 2017. Dynamic conditional beta is alive and well in the cross section of daily stock returns. *Management Science* 63:3760-79.
- Becker, R., A. E. Clements, M. B. Doolan, and A. S. Hurn. 2015. Selecting volatility forecasting models for portfolio allocation purposes. *International Journal of Forecasting* 31:849-61.
- Best, M. J., and R. R. Grauer. 1991. On the sensitivity of mean-variance-efficient portfolios to changes in asset means: Some analytical and computational results. *The Review of Financial Studies* 4:315-42.
- Bianchi, D., M. Büchner, and A. Tamoni. 2021. Bond risk premiums with machine learning. *The Review of Financial Studies* 34:1046-89.
- Boivin, J., and S. Ng. 2005. Understanding and Comparing Factor-Based Forecasts. *International Journal of Central Banking* 3:117-151.
- Bourlard, H., and Y. Kamp. 1988. Auto-association by multilayer perceptrons and singular value decomposition. *Biological Cybernetics* 59:291-4.
- Branger, N., K. Lučivjanská, and A. Weissensteiner. 2019. Optimal granularity for portfolio choice. *Journal of Empirical Finance* 50:125-46.
- Broadie, M. 1993. Computing efficient frontiers using estimated parameters. *Annals of Operations Research* 45:21-58.

- Callot, L., M. Caner, A. Ö. Önder, and E. Ulaşan. 2019. A nodewise regression approach to estimating large portfolios. *Journal of Business & Economic Statistics* 39:520-531.
- Carroll, R., T. Conlon, J. Cotter, and E. Salvador. 2017. Asset allocation with correlation: A composite trade-off. *European Journal of Operational Research* 262:1164-80.
- Chamberlain, G., and M. Rothschild. 1983. Arbitrage, factor structure, and mean-variance analysis on large asset markets. *Econometrica: Journal of the Econometric Society* 51:1281-304.
- Chan, L. K., J. Karceski, and J. Lakonishok. 1999. On portfolio optimization: Forecasting covariances and choosing the risk model. *The Review of Financial Studies* 12:937-74.
- Chen, A. Y., and T. Zimmermann. 2020. Open source cross-sectional asset pricing. *Available at SSRN* 3604626.
- Chen, N., R. Roll, and A. Stephen. 1986. Ross, 1986, economic forces and the stock market. *Journal of Business* 59:383-403.
- Chopra, V. K., C. R. Hensel, and A. L. Turner. 1993. Massaging mean-variance inputs: Returns from alternative global investment strategies in the 1980s. *Management Science* 39:845-55.
- Chun, H., and S. Keleş. 2010. Sparse partial least squares regression for simultaneous dimension reduction and variable selection. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 72:3-25.
- Connor, G., and R. A. Korajczyk. 1988. Risk and return in an equilibrium APT: Application of a new test methodology. *Journal of Financial Economics* 21:255-289.
- Cybenko, G. 1989. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems* 2:303-14.
- D'Hondt, C., R. De Winne, E. Ghysels, and S. Raymond. 2020. Artificial intelligence alter egos: Who might benefit from robo-investing? *Journal of Empirical Finance* 59:278-99.
- De Jong, S. 1993. SIMPLS: An alternative approach to partial least squares regression. *Chemometrics and Intelligent Laboratory Systems* 18:251-63.
- De Nard, G., O. Ledoit, and M. Wolf. 2018. Factor models for portfolio selection in large dimensions: The good, the better and the ugly. *Journal of Financial Econometrics*.

- DeMiguel, V., L. Garlappi, F. J. Nogales, and R. Uppal. 2009. A generalized approach to portfolio optimization: Improving performance by constraining portfolio norms. *Management Science* 55:798-812.
- DeMiguel, V., L. Garlappi, and R. Uppal. 2009. Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy? *The Review of Financial Studies* 22:1915-53.
- Duchin, R., and H. Levy. 2009. Markowitz versus the Talmudic portfolio diversification strategies. *The Journal of Portfolio Management* 35:71-4.
- Engle, R. 2002. Dynamic conditional correlation: A simple class of multivariate generalized autoregressive conditional heteroskedasticity models. *Journal of Business & Economic Statistics* 20:339-50.
- Engle, R. F. 2016. Dynamic conditional beta. *Journal of Financial Econometrics* 14:643-67.
- Engle, R. F., O. Ledoit, and M. Wolf. 2019. Large dynamic covariance matrices. *Journal of Business & Economic Statistics* 37:363-75.
- Engle, R. F., V. K. Ng, and M. Rothschild. 1990. Asset pricing with a factor-ARCH covariance structure: Empirical estimates for treasury bills. *Journal of Econometrics* 45:213-37.
- Engle, R., and R. Colacito. 2006. Testing and valuing dynamic correlations for asset allocation. *Journal of Business & Economic Statistics* 24:238-53.
- Fama, E. F., and K. R. French. 2015. A five-factor asset pricing model. *Journal of Financial Economics* 116:1-22.
- . 1993. Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics* 33:3-56.
- Fan, J., Y. Fan, and J. Lv. 2008. High dimensional covariance matrix estimation using a factor model. *Journal of Econometrics* 147:186-97.
- Fan, J., A. Furger, and D. Xiu. 2016. Incorporating global industrial classification standard into portfolio allocation: A simple factor-based large covariance matrix estimator with high-frequency data. *Journal of Business & Economic Statistics* 34:489-503.

- Fan, J., Y. Liao, and M. Mincheva. 2013. Large covariance estimation by thresholding principal orthogonal complements. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 75: 603-680.
- . 2011. High dimensional covariance matrix estimation in approximate factor models. *Annals of Statistics* 39:3320-56.
- Feng, G., S. Giglio, and D. Xiu. 2020. Taming the factor zoo: A test of new factors. *The Journal of Finance* 75:1327-70.
- Feng, G., N. Polson, and J. Xu. 2020. Deep learning in characteristics-sorted factor models. *Available at SSRN* 3243683.
- Forni, M., M. Hallin, M. Lippi, and L. Reichlin. 2005. The generalized dynamic factor model: One-sided estimation and forecasting. *Journal of the American Statistical Association* 100:830-40.
- . 2000. The generalized dynamic-factor model: Identification and estimation. *Review of Economics and Statistics* 82:540-54.
- Frahm, G. 2010. Linear statistical inference for global and local minimum variance portfolios. *Statistical Papers* 51:789-812.
- Frahm, G., and C. Memmel. 2010. Dominating estimators for minimum-variance portfolios. *Journal of Econometrics* 159:289-302.
- Freyberger, J., A. Neuhierl, and M. Weber. 2020. Dissecting characteristics nonparametrically. *The Review of Financial Studies* 33:2326-77.
- Friedman, J., T. Hastie, and R. Tibshirani. 2008. Sparse inverse covariance estimation with the graphical lasso. *Biostatistics* 9:432-41.
- Geweke, J. 1977. The dynamic factor analysis of economic time series. *Latent Variables in Socio-Economic Models*.
- Goodfellow, I., Y. Bengio, and A. Courville. 2016. *Deep learning*. Cambridge: MIT Press.
- Green, R. C., and B. Hollifield. 1992. When will mean-variance efficient portfolios be well diversified? *The Journal of Finance* 47:1785-809.
- Gu, S., B. Kelly, and D. Xiu. 2020. Autoencoder asset pricing models. *Journal of Econometrics* 222:429-450.

- . 2020. Empirical asset pricing via machine learning. *The Review of Financial Studies* 33:2223-73.
- Guidolin, M., and A. Timmermann. 2008. International asset allocation under regime switching, skew, and kurtosis preferences. *The Review of Financial Studies* 21:889-935.
- Han, Y. 2006. Asset allocation with a high dimensional latent factor stochastic volatility model. *The Review of Financial Studies* 19:237-71.
- Hastie, T., R. Tibshirani, and J. Friedman. 2009. *The elements of statistical learning*. New York: Springer.
- Hinton, G. E., and R. S. Zemel. 1994. Autoencoders, minimum description length, and Helmholtz free energy. *Advances in Neural Information Processing Systems* 6:3-10.
- Hinton, G. E., and R. R. Salakhutdinov. 2006. Reducing the dimensionality of data with neural networks. *Science* 313:504-7.
- Hjalmarsson, E., and P. Manchev. 2012. Characteristic-based mean-variance portfolio choice. *Journal of Banking & Finance* 36:1392-401.
- Hornik, K., M. Stinchcombe, and H. White. 1989. Multilayer feedforward networks are universal approximators. *Neural Networks* 2:359-66.
- Hou, K., H. Mo, C. Xue, and L. Zhang. 2021. An augmented q-factor model with expected growth. *Review of Finance* 25:1-41.
- Hou, K., C. Xue, and L. Zhang. 2015. Digesting anomalies: An investment approach. *The Review of Financial Studies* 28:650-705.
- Huang, D., J. Li, and G. Zhou. 2019. Shrinking factor dimension: A reduced-rank approach. *Available at SSRN* 3205697.
- Hui, T., E. K. Kwan, and C. Lee. 1993. Optimal portfolio diversification: Empirical bayes versus classical approach. *Journal of the Operational Research Society* 44:1155-9.
- Jagannathan, R., and T. Ma. 2003. Risk reduction in large portfolios: Why imposing the wrong constraints helps. *The Journal of Finance* 58:1651-83.
- Japkowicz, N., S. J. Hanson, and M. A. Gluck. 2000. Nonlinear autoassociation is not equivalent to PCA. *Neural Computation* 12:531-45.

- Kan, R., and G. Zhou. 2007. Optimal portfolio choice with parameter uncertainty. *Journal of Financial and Quantitative Analysis* 42:621-56.
- Karhunen, J., and J. Joutsensalo. 1995. Generalizations of principal component analysis, optimization problems, and neural networks. *Neural Networks* 8:549-62.
- Kelly, B. T., S. Pruitt, and Y. Su. 2019. Characteristics are covariances: A unified model of risk and return. *Journal of Financial Economics* 134:501-24.
- Kempf, A., and C. Memmel. 2006. Estimating the global minimum variance portfolio. *Schmalenbach Business Review* 58:332-48.
- Kingma, D. P., and J. Ba. 2014. Adam: A method for stochastic optimization. *ArXiv Preprint arXiv:1412.6980*.
- Kirby, C., and B. Ostdiek. 2012. It's all in the timing: Simple active portfolio strategies that outperform naïve diversification. *Journal of Financial and Quantitative Analysis* 47:437-67.
- Kozak, S., S. Nagel, and S. Santosh. 2020. Shrinking the cross-section. *Journal of Financial Economics* 135:271-92.
- Kritzman, M., S. Page, and D. Turkington. 2010. In defense of optimization: The fallacy of $1/N$. *Financial Analysts Journal* 66:31-9.
- Lassance, N., V. DeMiguel, and F. D. Vrans. 2020. Optimal portfolio diversification via independent component analysis. *Available at SSRN 3285156*.
- LeCun, Y., B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. 1989. Backpropagation applied to handwritten zip code recognition. *Neural Computation* 1:541-51.
- Ledoit, O., and M. Wolf. 2008. Robust performance hypothesis testing with the Sharpe ratio. *Journal of Empirical Finance* 15:850-9.
- Ledoit, O., and M. Wolf. 2017. Nonlinear shrinkage of the covariance matrix for portfolio selection: Markowitz meets goldilocks. *The Review of Financial Studies* 30:4349-88.
- . 2012. Nonlinear shrinkage estimation of large-dimensional covariance matrices. *The Annals of Statistics* 40:1024-60.
- . 2011. Robust performances hypothesis testing with the variance. *Wilmott* 2011:86-9.

- . 2003. Improved estimation of the covariance matrix of stock returns with an application to portfolio selection. *Journal of Empirical Finance* 10:603-21.
- Lettau, M., and M. Pelger. 2020. Factors that fit the time series and cross-section of stock returns. *The Review of Financial Studies* 33:2274-325.
- Light, N., D. Maslov, and O. Rytchkov. 2017. Aggregation of information about the cross section of stock returns: A latent variable approach. *The Review of Financial Studies* 30:1339-81.
- Lintner, J. 1965. Security prices, risk, and maximal gains from diversification. *The Journal of Finance* 20:587-615.
- MacKinlay, C. A., and L. Pástor. 2000. Asset pricing models: Implications for expected returns and portfolio selection. *The Review of Financial Studies* 13:883-916.
- Markowitz, H. 1952. Portfolio selection. *The Journal of Finance* 7:77-91.
- McCracken, M. W., and S. Ng. 2016. FRED-MD: A monthly database for macroeconomic research. *Journal of Business & Economic Statistics* 34:574-89.
- Merton, R. C. 1980. On estimating the expected return on the market: An exploratory investigation. *Journal of Financial Economics* 8:323-61.
- . 1973. An intertemporal capital asset pricing model. *Econometrica: Journal of the Econometric Society* 41:867-87.
- Moskowitz, T. J. 2003. An analysis of covariance risk and pricing anomalies. *The Review of Financial Studies* 16:417-57.
- Moura, G. V., A. A. Santos, and E. Ruiz. 2020. Comparing high-dimensional conditional covariance matrices: Implications for portfolio selection. *Journal of Banking & Finance* 118:105882.
- Neely, C. J., D. E. Rapach, J. Tu, and G. Zhou. 2014. Forecasting the equity risk premium: The role of technical indicators. *Management Science* 60:1772-91.
- Newey, W. K., and K. D. West. 1987. A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica: Journal of the Econometric Society* 55:703-8.
- Olivares-Nadal, A. V., and V. DeMiguel. 2018. A robust perspective on transaction costs in portfolio optimization. *Operations Research* 66:733-9.

- Palczewski, A., and J. Palczewski. 2014. Theoretical and empirical estimates of mean–variance portfolio sensitivity. *European Journal of Operational Research* 234:402-10.
- Pettenuzzo, D., and A. Timmermann. 2011. Predictability of stock returns and asset allocation under structural breaks. *Journal of Econometrics* 164:60-78.
- Platanakis, E., C. Sutcliffe, and X. Ye. 2021. Horses for courses: Mean-variance for asset allocation and 1/N for stock selection. *European Journal of Operational Research* 288:302-17.
- Rapach, D. E., J. K. Strauss, and G. Zhou. 2013. International stock return predictability: What is the role of the United States? *The Journal of Finance* 68:1633-62.
- Ross, S. 1976. The arbitrage theory of capital asset pricing. *Journal of Economic Theory* 13:341-60.
- Sargent, T. J., and C. A. Sims. 1977. Business cycle modeling without pretending to have too much a priori economic theory. *New Methods in Business Cycle Research* 1:145-68.
- Sharpe, W. F. 1964. Capital asset prices: A theory of market equilibrium under conditions of risk. *The Journal of Finance* 19:425-42.
- . 1963. A simplified model for portfolio analysis. *Management Science* 9:277-93.
- Stock, J. H., and M. Watson. 2011. *Dynamic factor models*. Oxford Handbooks Online.
- Stock, J. H., and M. W. Watson. 2002. Forecasting using principal components from a large number of predictors. *Journal of the American Statistical Association* 97:1167-79.
- . 2002. Macroeconomic forecasting using diffusion indexes. *Journal of Business & Economic Statistics* 20:147-62.
- Tibshirani, R. 1996. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 58:267-88.
- Masters, T. 1993. *Practical neural network recipes in C++*. New York: Academic Press.
- Wang, Z. 2005. A shrinkage approach to model uncertainty and asset allocation. *Review of Financial Studies* 18:673-705.
- Wold, H. 1966. Estimation of principal components and related models by iterative least squares. *Multivariate Analysis* 391-420.
- Wu, W., J. Chen, Z. Yang, and M. L. Tindall. 2020. A cross-sectional machine learning approach for hedge fund return prediction and selection. *Management Science*.

- Zou, H., and T. Hastie. 2005. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 67:301-20.
- Zou, H., T. Hastie, and R. Tibshirani. 2006. Sparse principal component analysis. *Journal of Computational and Graphical Statistics* 15:265-86.

Tables

Table 1
Comparison of factor-based covariance matrices

	Static Factor Covariance			Dynamic Beta Covariance			Dynamic Factor Covariance			Dynamic Error Covariance		
	Eig	Mag	Dir	Eig	Mag	Dir	Eig	Mag	Dir	Eig	Mag	Dir
Market	0.913	0.076	0.973	0.933	0.105	0.954	0.926	0.070	0.973	0.908	0.082	0.973
FF3	0.932	0.041	0.976	0.990	0.062	0.951	0.963	0.015	0.973	0.926	0.047	0.976
PCA	0.501***	0.742***	0.393***	0.517***	0.754***	0.384***	0.523***	0.725***	0.394***	0.485***	0.750***	0.393***
PLS	0.584***	0.616***	0.619***	0.603***	0.634***	0.580***	0.597***	0.601***	0.620***	0.573***	0.624***	0.619***
SPCA	0.487***	0.763***	0.479***	0.513***	0.757***	0.462***	0.489***	0.764***	0.474***	0.469***	0.771***	0.479***
SPLS	0.574***	0.626***	0.648***	0.595***	0.637***	0.610***	0.577***	0.619***	0.648***	0.563***	0.634***	0.648***
AEN1	0.480***	0.761***	0.370***	0.501***	0.770***	0.363***	0.501***	0.745***	0.368***	0.461***	0.769***	0.370***
AEN2	0.498***	0.744***	0.389***	0.509***	0.76***	0.380***	0.532***	0.720***	0.388***	0.479***	0.753***	0.389***
AEN3	0.505***	0.735***	0.404***	0.506***	0.759***	0.397***	0.551***	0.701***	0.404***	0.487***	0.743***	0.404***
AEN4	0.507***	0.734***	0.405***	0.511***	0.757***	0.396***	0.551***	0.702***	0.406***	0.489***	0.742***	0.405***

This table reports monthly measures that compare the factor-implied covariance matrices to the sample estimator, based on total covariation (Eig), Magnitude (Mag) and direction (Dir), over the out-of-sample period from January 1980 to December 2019. The results are presented for four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). The significant deviation from the sample estimator of the alternative factor-implied covariance matrices from a covariance matrix based on the market factor is denoted by: *, **, and *** for significance at the 10%, 5%, and 1% level, respectively.

Table 2
Characteristics of the portfolio weight vectors

A. Average turnover and maximum weight

	TO	MAX
EW	1.081	0.010
Sample	41.269	0.383

	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	TO	MAX	TO	MAX	TO	MAX	TO	MAX
Market	41.297	0.302	60.745	0.296	43.957	0.433	56.045	0.445
FF3	42.785	0.311	71.744	0.312	47.423	0.450	57.039	0.421
PCA	29.737	0.070	35.064	0.078	30.017	0.073	46.098	0.139
PLS	33.408	0.095	39.693	0.092	33.496	0.094	49.269	0.157
SPCA	29.931	0.071	36.287	0.087	29.973	0.071	46.137	0.134
SPLS	33.372	0.100	40.552	0.108	33.332	0.098	49.203	0.194
AEN1	31.196	0.066	36.925	0.075	31.396	0.067	46.615	0.119
AEN2	31.060	0.066	35.694	0.079	31.368	0.068	46.855	0.123
AEN3	33.136	0.074	37.462	0.076	33.577	0.073	48.857	0.166
AEN4	33.346	0.072	37.276	0.076	33.885	0.078	49.029	0.132

B. Standard deviation of the weights and mean absolute deviation from the equal-weighted portfolio

	SD_{ω}	MAD_{EW}
EW	0.000	0.000
Sample	3.146	1.621

	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD_{ω}	MAD_{EW}	SD_{ω}	MAD_{EW}	SD_{ω}	MAD_{EW}	SD_{ω}	MAD_{EW}
Market	2.824	1.616	2.452	1.535	2.769	1.582	3.433	1.711
FF3	2.738	1.582	2.473	1.520	2.669	1.546	3.342	1.669
PCA	1.091	0.820	1.047	0.791	1.098	0.824	1.323	0.928
PLS	1.419	1.050	1.334	0.997	1.423	1.052	1.700	1.165
SPCA	1.067	0.772	1.059	0.777	1.064	0.771	1.288	0.882
SPLS	1.441	1.033	1.359	0.993	1.439	1.032	1.727	1.153
AEN1	1.053	0.802	1.012	0.771	1.059	0.806	1.281	0.909
AEN2	1.078	0.815	1.030	0.779	1.089	0.822	1.305	0.920
AEN3	1.107	0.835	1.048	0.793	1.121	0.845	1.349	0.943
AEN4	1.111	0.837	1.052	0.795	1.124	0.846	1.344	0.943

This table presents the monthly characteristics of the portfolio weight vectors. Panel A reports average turnover (TO) and maximum weight (MAX), whereas the standard deviation of the weights (SD_{ω}) and mean absolute deviation from the equal-weighted benchmark (MAD_{EW}) can be found in Panel B. The average value of each weight characteristic over the out-of-sample period from January 1980 to December 2019 is reported. TO, SD_{ω} and MAD_{EW} are reported as a percentage. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN).

Table 3
Portfolio performance based on standard deviation and Sharpe ratio

	SD	SR						
EW	4.159	0.183						
Sample	3.469***	0.209						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	3.630***	0.209	3.347***	0.218	3.640***	0.210	3.597***	0.205
FF3	3.522***	0.218	3.329***	0.241	3.538***	0.213	3.469***	0.223
PCA	3.361***	0.235**	3.290***	0.239**	3.360***	0.235**	3.268***	0.241**
PLS	3.330***	0.229	3.276***	0.232*	3.335***	0.227	3.217***	0.238*
SPCA	3.373***	0.241***	3.307***	0.243***	3.378***	0.240***	3.263***	0.246***
SPLS	3.339***	0.233*	3.261***	0.242**	3.347***	0.231*	3.225***	0.242*
AEN1	3.318***	0.239***	3.285***	0.243***	3.315***	0.239***	3.216***	0.244**
AEN2	3.352***	0.232**	3.297***	0.235**	3.350***	0.232**	3.259***	0.237**
AEN3	3.338***	0.238**	3.283***	0.240**	3.334***	0.238**	3.235***	0.246**
AEN4	3.357***	0.225*	3.301***	0.227**	3.356***	0.224*	3.256***	0.227*

This table documents monthly portfolio performance measured using the standard deviation (SD) and Sharpe ratio (SR), over the out-of-sample period from January 1980 to December 2019. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). The significant outperformance of the alternative strategies from the equal-weighted strategy is denoted by: *, **, and *** for significance at the 10%, 5%, and 1% level, respectively.

Table 4
Portfolio performance based on alternative risk measures

	MAD	VaR	CVaR									
EW	3.693	6.079	7.817									
Sample	3.111	4.981	6.431									
	Static Factor Covariance			Dynamic Beta Covariance			Dynamic Factor Covariance			Dynamic Error Covariance		
	MAD	VaR	CVaR	MAD	VaR	CVaR	MAD	VaR	CVaR	MAD	VaR	CVaR
Market	3.220	5.215	6.732	3.187	4.775	6.173	3.106	5.223	6.745	3.370	5.181	6.684
FF3	3.168	5.024	6.495	3.065	4.674	6.065	3.178	5.066	6.545	3.254	4.932	6.381
PCA	2.819	4.739	6.143	2.775	4.625	6.000	2.826	4.737	6.141	2.917	4.586	5.952
PLS	2.882	4.715	6.107	2.886	4.627	5.995	2.911	4.728	6.121	2.798	4.525	5.869
SPCA	2.773	4.736	6.146	2.696	4.637	6.019	2.788	4.745	6.157	2.767	4.565	5.929
SPLS	2.963	4.715	6.111	2.897	4.577	5.940	2.952	4.734	6.132	2.883	4.526	5.874
AEN1	2.774	4.664	6.050	2.699	4.605	5.978	2.780	4.661	6.046	2.785	4.504	5.848
AEN2	2.841	4.737	6.137	2.867	4.648	6.025	2.847	4.735	6.135	2.874	4.590	5.952
AEN3	2.772	4.697	6.092	2.818	4.613	5.984	2.806	4.691	6.084	2.831	4.526	5.878
AEN4	2.893	4.767	6.169	2.853	4.681	6.061	2.894	4.768	6.171	2.878	4.618	5.978

In this table, we present the monthly portfolio performance measured using the mean absolute deviation (MAD), value-at-risk (VaR) and conditional value-at-risk (CVaR), over the out-of-sample period from January 1980 to December 2019. The VaR and CVaR are calculated at the 95% confidence level. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN).

Table 5
Portfolio performance based on certainty equivalent return

	$\gamma = 2$	$\gamma = 5$	$\gamma = 10$
EW	0.589	0.330	-0.103
Sample	0.016	0.095	0.226*

	Static Factor Covariance			Dynamic Beta Covariance			Dynamic Factor Covariance			Dynamic Error Covariance		
	$\gamma = 2$	$\gamma = 5$	$\gamma = 10$	$\gamma = 2$	$\gamma = 5$	$\gamma = 10$	$\gamma = 2$	$\gamma = 5$	$\gamma = 10$	$\gamma = 2$	$\gamma = 5$	$\gamma = 10$
Market	0.036	0.098	0.201	0.029	0.120	0.273*	0.043	0.104	0.205	0.017	0.082	0.191
FF3	0.056	0.129	0.252	0.101	0.194	0.350***	0.040	0.111	0.231	0.065	0.144	0.276*
PCA	0.087	0.177**	0.327***	0.089	0.186**	0.348***	0.087	0.177**	0.327***	0.093	0.192*	0.358***
PLS	0.061	0.155	0.310***	0.065	0.163	0.327***	0.057	0.150	0.304***	0.074	0.178	0.352***
SPCA	0.109	0.198**	0.346***	0.104	0.200**	0.359***	0.108	0.196**	0.343***	0.106	0.206**	0.372***
SPLS	0.077	0.169	0.323***	0.092	0.192*	0.359***	0.071	0.162	0.315***	0.086	0.190	0.362***
AEN1	0.095	0.189**	0.346***	0.101	0.199**	0.362***	0.093	0.188**	0.346***	0.092	0.197**	0.371***
AEN2	0.075	0.166*	0.317***	0.077	0.173*	0.334***	0.074	0.165*	0.317***	0.075	0.176*	0.343***
AEN3	0.092	0.185**	0.339***	0.090	0.188**	0.351***	0.093	0.185**	0.340***	0.102	0.204**	0.375***
AEN4	0.052	0.143	0.294***	0.051	0.147	0.307***	0.050	0.140	0.291***	0.043	0.143	0.311***

This table reports monthly portfolio performance measured using the difference in certainty equivalent return (Δ CER) for various levels of risk aversion, γ , over the out-of-sample period from January 1980 to December 2019. For the EW portfolio the monthly CER is reported as a percentage, while for the remaining portfolios the percentage Δ CER is provided, which is calculated as the difference in monthly CER between the alternative strategies and the equal-weighted strategy. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). The significant outperformance of the alternative strategies from the equal-weighted strategy is denoted by: *, **, and *** for significance at the 10%, 5%, and 1% level, respectively.

Table 6
Portfolio performance during different volatility regimes

A. High volatility regime

	SD	SR						
EW	5.037	0.128						
Sample	3.994***	0.179						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	4.075***	0.177	3.732***	0.197	4.153***	0.173	3.958***	0.180
FF3	3.978***	0.192	3.792***	0.213**	4.048***	0.175	3.846***	0.205
PCA	3.968***	0.181**	3.874***	0.183**	3.965***	0.181**	3.820***	0.187**
PLS	3.882***	0.180	3.820***	0.179	3.888***	0.178	3.700***	0.196*
SPCA	3.988***	0.188**	3.893***	0.188**	3.994***	0.187**	3.832***	0.192**
SPLS	3.896***	0.186*	3.798***	0.191*	3.907***	0.183	3.714***	0.200*
AEN1	3.904***	0.198***	3.859***	0.198***	3.899***	0.197***	3.759***	0.205***
AEN2	3.952***	0.178*	3.878***	0.179*	3.949***	0.178*	3.819***	0.184*
AEN3	3.930***	0.189**	3.857***	0.188**	3.923***	0.189**	3.776***	0.197**
AEN4	3.956***	0.179**	3.882***	0.178*	3.953***	0.179*	3.801***	0.182*

B. Low volatility regime

	SD	SR						
EW	2.283	0.411						
Sample	2.493*	0.298*						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	2.846***	0.285	2.677***	0.269**	2.705***	0.307	2.986***	0.258*
FF3	2.705***	0.288	2.485*	0.319	2.602**	0.315	2.820***	0.267*
PCA	2.157	0.414	2.14*	0.423	2.160	0.414	2.199	0.410
PLS	2.268	0.377	2.229	0.393	2.271	0.377	2.319	0.357
SPCA	2.150*	0.423	2.151*	0.423	2.151*	0.423	2.148	0.420
SPLS	2.269	0.378	2.230	0.396	2.269	0.378	2.316	0.361
AEN1	2.171	0.381	2.163*	0.392	2.173	0.381	2.167	0.372
AEN2	2.163	0.409	2.154*	0.415	2.165	0.409	2.169	0.402
AEN3	2.174	0.399	2.158	0.408	2.177	0.399	2.195	0.398
AEN4	2.176	0.377	2.160	0.388	2.181	0.376	2.207	0.366

In this table, we document the monthly portfolio performance measured using the standard deviation (SD) and Sharpe ratio (SR), during high (Panel A) and low (Panel B) volatility periods based on the filtered probabilities of a Markov-switching model estimated using the market factor. Observations where the filtered probability of the low volatility regime is above 0.5 are considered low-volatility periods, and observations where the filtered probability of the low volatility regime is below 0.5 are considered high-volatility periods. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). The significant outperformance of the alternative strategies from the equal-weighted strategy is denoted by: *, **, and *** for significance at the 10%, 5%, and 1% level, respectively.

Table 7
Portfolio performance during different inflation regimes

A. High inflation regime

	SD	SR						
EW	4.515	0.072						
Sample	3.800***	0.178**						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	3.696**	0.224**	3.385***	0.261***	3.730**	0.217*	3.521***	0.258**
FF3	3.657***	0.222**	3.412***	0.261***	3.693**	0.209**	3.548***	0.258***
PCA	3.529***	0.186***	3.488***	0.200***	3.519***	0.187***	3.420***	0.206***
PLS	3.552***	0.193***	3.510***	0.203***	3.558***	0.191***	3.335***	0.237***
SPCA	3.570***	0.184***	3.502***	0.201***	3.574***	0.182***	3.450***	0.206***
SPLS	3.564***	0.199***	3.488***	0.223***	3.572***	0.195***	3.347***	0.240***
AEN1	3.552***	0.185***	3.513***	0.198***	3.547***	0.186***	3.417***	0.210***
AEN2	3.539***	0.180***	3.510***	0.193***	3.530***	0.181***	3.438***	0.204***
AEN3	3.513***	0.190***	3.498***	0.201***	3.500***	0.191***	3.404***	0.212***
AEN4	3.547***	0.175***	3.518***	0.183***	3.543***	0.175***	3.417***	0.195***

B. Low inflation regime

	SD	SR						
EW	3.926	0.259						
Sample	3.271***	0.231						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	3.598*	0.199	3.327***	0.193	3.594*	0.206	3.642	0.175
FF3	3.448**	0.216	3.285***	0.228	3.453**	0.215	3.426***	0.202
PCA	3.264***	0.265	3.176***	0.264	3.268***	0.264	3.182***	0.263
PLS	3.200***	0.252	3.140***	0.252	3.205***	0.250	3.153***	0.239
SPCA	3.258***	0.277	3.193***	0.270	3.263***	0.277	3.155***	0.271
SPLS	3.209***	0.255	3.129***	0.254	3.216***	0.253	3.159***	0.243
AEN1	3.179***	0.275	3.151***	0.272	3.178***	0.273	3.099***	0.266
AEN2	3.243***	0.264	3.173***	0.262	3.246***	0.263	3.157***	0.257
AEN3	3.237***	0.267	3.157***	0.264	3.239***	0.267	3.140***	0.267
AEN4	3.246***	0.256	3.174***	0.255	3.248***	0.255	3.165***	0.246

In this table, we document the monthly portfolio performance measured using the standard deviation (SD) and Sharpe ratio (SR), during periods of high (Panel A) and low (Panel B) inflation. Periods of high (low) inflation are those when inflation for the specific month is higher (lower) than the median over the out-of-sample period. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). The significant outperformance of the alternative strategies from the equal-weighted strategy is denoted by: *, **, and *** for significance at the 10%, 5%, and 1% level, respectively.

Table 8
Portfolio performance during different credit spread regimes

A. High credit spread regime

	SD	SR						
EW	5.343	0.127						
Sample	3.843***	0.176						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	3.966***	0.178	3.580***	0.223	4.112***	0.167	3.850***	0.195
FF3	3.782***	0.192	3.555***	0.247**	3.964***	0.168	3.662***	0.232
PCA	4.050***	0.177	3.773***	0.192	4.053***	0.176	3.774***	0.201*
PLS	3.868***	0.185	3.701***	0.198	3.883***	0.181	3.624***	0.224*
SPCA	4.090***	0.171	3.844***	0.188	4.106***	0.169	3.769***	0.204*
SPLS	3.901***	0.186	3.674***	0.211	3.926***	0.181	3.621***	0.228*
AEN1	3.931***	0.188*	3.781***	0.195	3.923***	0.186	3.702***	0.214**
AEN2	3.988***	0.170	3.736***	0.184	3.989***	0.169	3.759***	0.196
AEN3	4.009***	0.180	3.770***	0.189	4.004***	0.181	3.737***	0.206*
AEN4	4.002***	0.184	3.764***	0.193	4.002***	0.183	3.755***	0.205*

B. Low credit spread regime

	SD	SR						
EW	3.701	0.213						
Sample	3.344**	0.222						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	3.520	0.220	3.272**	0.216	3.479	0.227	3.516	0.208
FF3	3.438	0.228	3.256***	0.238	3.394	0.230	3.409	0.220
PCA	3.109***	0.261*	3.122***	0.258	3.106***	0.262*	3.091***	0.259
PLS	3.140***	0.247	3.131***	0.246	3.141***	0.247	3.078***	0.244
SPCA	3.110***	0.273**	3.117***	0.266*	3.110***	0.273**	3.085***	0.263
SPLS	3.141***	0.253	3.120***	0.254	3.141***	0.252	3.091***	0.247
AEN1	3.098***	0.262*	3.112***	0.263**	3.098***	0.262*	3.046***	0.257
AEN2	3.122***	0.259*	3.146***	0.256	3.119***	0.259*	3.084***	0.254
AEN3	3.094***	0.264*	3.113***	0.261*	3.090***	0.264*	3.060***	0.263
AEN4	3.124***	0.243	3.141***	0.241	3.123***	0.243	3.082***	0.236

In this table, we document the monthly portfolio performance measured using the standard deviation (SD) and Sharpe ratio (SR), during periods of high (Panel A) and low (Panel B) credit spread. Periods of high (low) credit spread are those when the spread for the specific month is higher (lower) than the median over the out-of-sample period. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). The significant outperformance of the alternative strategies from the equal-weighted strategy is denoted by: *, **, and *** for significance at the 10%, 5%, and 1% level, respectively.

Table 9**Portfolio performance based on breakeven transaction costs**

Sample	6.470			
	Static Factor Covariance	Dynamic Beta Covariance	Dynamic Factor Covariance	Dynamic Error Covariance
Market	6.465	5.866	6.297	4.003
FF3	8.392	8.208	6.474	7.148
PCA	18.146	16.479	17.971	12.884
PLS	14.230	12.690	13.574	11.414
SPCA	20.104	17.043	19.729	13.983
SPLS	15.484	14.948	14.883	12.261
AEN1	18.595	16.739	18.473	13.397
AEN2	16.345	15.023	16.179	11.797
AEN3	17.158	15.668	16.925	13.187
AEN4	13.017	12.156	12.498	9.177

This table presents monthly portfolio performance measured breakeven transaction costs (c_{ew}) with respect to the equal-weighted portfolio (EW), over the out-of-sample period from January 1980 to December 2019. The results are presented for the minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). The breakeven transaction costs are reported in basis points and a positive value indicates that the alternative portfolio outperforms the EW.

Table 10
Portfolio performance when short-selling is allowed

A. Standard deviation and Sharpe ratio

	SD	SR						
EW	4.159	0.183						
Sample	4.380	0.163						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	4.109	0.211	3.639**	0.231	4.136	0.207	4.332	0.214
FF3	4.120	0.214	3.515***	0.242	4.163	0.202	4.441	0.206
PCA	3.387***	0.231**	3.305***	0.238**	3.386***	0.231**	3.292***	0.240**
PLS	3.41***	0.224	3.289***	0.234*	3.411***	0.223	3.308***	0.237
SPCA	3.425***	0.235**	3.342***	0.241***	3.426***	0.235**	3.305***	0.241**
SPLS	3.432***	0.223	3.279***	0.239*	3.435***	0.222	3.326***	0.234
AEN1	3.328***	0.239***	3.283***	0.244***	3.324***	0.239***	3.226***	0.245**
AEN2	3.375***	0.229**	3.309***	0.235**	3.373***	0.229**	3.28***	0.234**
AEN3	3.362***	0.237**	3.298***	0.241**	3.359***	0.237**	3.258***	0.246**
AEN4	3.374***	0.225*	3.316***	0.226*	3.373***	0.225*	3.278***	0.228*

B. Average turnover and breakeven transaction costs

	TO	c_{ew}						
EW	1.081	NA						
Sample	220.992	-0.909						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	TO	c_{ew}	TO	c_{ew}	TO	c_{ew}	TO	c_{ew}
Market	78.725	3.606	96.923	5.008	79.291	3.069	117.430	2.664
FF3	93.821	3.343	121.381	4.904	96.111	1.999	144.917	1.599
PCA	30.215	16.476	35.686	15.894	30.437	16.351	47.820	11.981
PLS	35.856	11.790	42.400	12.343	35.929	11.478	55.090	9.998
SPCA	29.738	18.146	36.120	16.553	29.798	18.108	47.116	12.599
SPLS	35.123	11.750	42.530	13.511	35.075	11.473	54.284	9.586
AEN1	31.497	18.411	37.252	16.864	31.683	18.299	47.918	13.237
AEN2	31.597	15.074	36.269	14.778	31.860	14.945	48.511	10.964
AEN3	33.660	16.575	38.092	15.671	33.954	16.427	50.529	12.741
AEN4	34.001	12.758	37.986	11.652	34.410	12.602	50.839	9.044

This table reports monthly portfolio performance based on minimum-variance portfolios without short-selling constraints. Panel A reports the standard deviation (SD) and Sharpe ratio (SR), whereas the average turnover (TO) and breakeven transaction costs (c_{ew}) with respect to the EW portfolio can be found in Panel B. The out-of-sample period is from January 1980 to December 2019. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). NA indicates that the value is not available. The significant outperformance of the alternative strategies from the equal-weighted strategy is denoted by: *, **, and *** for significance at the 10%, 5%, and 1% level, respectively.

Table 11
Portfolio performance using a penalized minimum-variance objective function

A. Standard deviation and Sharpe ratio

	SD	SR						
EW	4.159	0.183						
Sample	3.626***	0.206						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	3.522***	0.228	3.331***	0.243*	3.59***	0.221	3.456***	0.225
FF3	3.554***	0.232	3.33***	0.242*	3.591***	0.217	3.517***	0.229
PCA	3.645***	0.207	3.527***	0.222**	3.623***	0.209	3.593***	0.214*
PLS	3.547***	0.231**	3.444***	0.225**	3.553***	0.230**	3.458***	0.238***
SPCA	3.648***	0.216**	3.539***	0.212*	3.656***	0.215**	3.606***	0.218**
SPLS	3.551***	0.227**	3.426***	0.221*	3.563***	0.223**	3.484***	0.231**
AEN1	3.647***	0.203	3.532***	0.215*	3.626***	0.206	3.607***	0.208
AEN2	3.655***	0.204	3.538***	0.219*	3.633***	0.206	3.615***	0.213*
AEN3	3.606***	0.210*	3.514***	0.224**	3.564***	0.213*	3.565***	0.219**
AEN4	3.621***	0.206	3.524***	0.215*	3.599***	0.206	3.567***	0.218**

B. Average turnover and breakeven transaction costs

	TO	c_{ew}						
EW	1.081	NA						
Sample	19.863	12.246						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	TO	c_{ew}	TO	c_{ew}	TO	c_{ew}	TO	c_{ew}
Market	24.111	19.540	30.242	20.575	25.009	15.881	25.955	16.885
FF3	22.065	23.351	33.574	18.158	23.964	14.858	24.013	20.495
PCA	13.251	19.721	16.390	25.475	14.117	19.945	14.634	22.873
PLS	15.204	33.987	18.307	24.382	15.553	32.477	16.914	34.738
SPCA	14.039	25.467	16.826	18.419	14.307	24.195	15.759	23.845
SPLS	15.539	30.433	18.815	21.428	15.527	27.689	17.000	30.153
AEN1	13.546	16.045	17.031	20.063	14.057	17.725	15.416	17.440
AEN2	13.954	16.313	17.089	22.489	14.812	17.479	15.079	21.432
AEN3	13.809	21.213	16.851	25.999	14.661	22.091	15.326	25.974
AEN4	14.254	17.460	16.692	20.498	15.241	16.243	15.577	24.145

This table reports monthly portfolio performance based on minimum-variance portfolios with a turnover penalty for a parameter $c = 5$ bps. Panel A reports the standard deviation (SD) and Sharpe ratio (SR), whereas the average turnover (TO) and breakeven transaction costs (c_{ew}) with respect to the EW portfolio can be found in Panel B. The out-of-sample period is from January 1980 to December 2019. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). NA indicates that the value is not available. The significant outperformance of the alternative strategies from the equal-weighted strategy is denoted by: *, **, and *** for significance at the 10%, 5%, and 1% level, respectively.

Table 12**Portfolio performance for the 49 Fama and French portfolios based on industry classification***A. Standard deviation and Sharpe ratio*

	SD	SR						
EW	4.606	0.157						
Sample	3.481***	0.209*						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	3.443***	0.236**	3.335***	0.224*	3.47***	0.232**	3.411***	0.239**
FF3	3.435***	0.225**	3.371***	0.222**	3.433***	0.221*	3.416***	0.23**
PCA	3.692***	0.219***	3.654***	0.215***	3.697***	0.22***	3.57***	0.226***
PLS	3.528***	0.23***	3.442***	0.23***	3.554***	0.227***	3.39***	0.237***
SPCA	3.655***	0.226***	3.598***	0.221***	3.667***	0.227***	3.539***	0.233***
SPLS	3.531***	0.229***	3.444***	0.23***	3.554***	0.225***	3.397***	0.238***
AEN1	3.708***	0.218***	3.704***	0.212***	3.703***	0.218***	3.597***	0.231***
AEN2	3.71***	0.215***	3.709***	0.206***	3.702***	0.217***	3.603***	0.221***
AEN3	3.697***	0.218***	3.681***	0.211***	3.693***	0.22***	3.581***	0.226***
AEN4	3.702***	0.216***	3.708***	0.204**	3.695***	0.218***	3.583***	0.222***

B. Average turnover and breakeven transaction costs

	TO	c_{ew}						
EW	0.053	NA						
Sample	3.431	153.937						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	TO	c_{ew}	TO	c_{ew}	TO	c_{ew}	TO	c_{ew}
Market	2.094	391.965	25.793	26.030	6.201	121.991	15.446	53.271
FF3	2.525	275.081	32.103	20.281	8.599	74.889	17.385	42.119
PCA	2.871	220.014	16.023	36.318	3.983	160.305	18.698	37.007
PLS	2.919	254.710	19.276	37.975	4.565	155.142	18.117	44.287
SPCA	9.092	76.336	20.967	30.602	10.231	68.776	24.621	30.935
SPLS	3.246	225.493	18.838	38.861	4.370	157.517	18.968	42.823
AEN1	5.024	122.712	17.391	31.722	5.730	107.451	21.012	35.307
AEN2	4.098	143.387	16.133	30.473	5.038	120.361	20.651	31.556
AEN3	4.062	152.158	16.833	32.181	4.919	129.470	20.337	34.017
AEN4	5.244	111.732	17.750	26.558	6.149	100.066	20.187	32.284

In this table, we report the monthly performance for portfolios constructed using the 49 Fama and French portfolios based on industry classification. Panel A reports the standard deviation (SD) and Sharpe ratio (SR), whereas the average turnover (TO) and breakeven transaction costs (c_{ew}) with respect to the EW portfolio can be found in Panel B. The out-of-sample period is from January 1980 to December 2019. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). NA indicates that the value is not available. The significant outperformance of the alternative strategies from the equal-weighted strategy is denoted by: *, **, and *** for significance at the 10%, 5%, and 1% level, respectively.

Table 13**Portfolio performance for the S&P 500 and 20 Fama and French portfolios formed on size and book-to-market***A. Standard deviation and Sharpe ratio*

	SD	SR						
EW	5.106	0.156						
Sample	4.305***	0.185*						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	4.534***	0.188**	4.548***	0.184*	4.553***	0.188**	4.542***	0.19**
FF3	4.32***	0.183	4.454***	0.174	4.288***	0.178	4.307***	0.182
PCA	4.656***	0.181*	4.676***	0.18*	4.659***	0.183**	4.599***	0.183*
PLS	4.633***	0.182*	4.639***	0.18*	4.642***	0.183*	4.576***	0.184*
SPCA	4.65***	0.18*	4.662***	0.181*	4.655***	0.18*	4.598***	0.18*
SPLS	4.616***	0.181*	4.626***	0.179	4.625***	0.182*	4.561***	0.183*
AEN1	4.649***	0.181*	4.663***	0.179*	4.652***	0.182*	4.59***	0.183**
AEN2	4.663***	0.181*	4.684***	0.181**	4.661***	0.182*	4.602***	0.182*
AEN3	4.66***	0.18*	4.69***	0.178*	4.66***	0.181*	4.601***	0.181*
AEN4	4.65***	0.181*	4.664***	0.183**	4.652***	0.182**	4.598***	0.183*

B. Average turnover and breakeven transaction costs

	TO	c_{ew}						
EW	0.000	NA						
Sample	2.835	102.293						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	TO	c_{ew}	TO	c_{ew}	TO	c_{ew}	TO	c_{ew}
Market	3.398	94.173	30.734	9.110	5.067	63.154	11.727	28.993
FF3	2.762	97.755	35.149	5.121	11.270	19.521	6.109	40.923
PCA	2.216	112.816	17.862	13.436	4.109	65.709	19.594	13.780
PLS	1.514	171.731	16.169	14.843	2.575	104.854	18.302	15.299
SPCA	4.605	52.117	19.709	12.685	5.599	42.865	22.320	10.753
SPLS	1.411	177.179	15.964	14.407	2.550	101.961	18.924	14.268
AEN1	1.618	154.512	16.887	13.620	2.695	96.475	19.181	14.076
AEN2	2.128	112.782	18.770	13.319	3.598	69.483	20.025	12.984
AEN3	2.882	83.276	21.064	10.444	4.217	59.284	20.615	12.127
AEN4	2.421	103.263	18.233	14.808	3.571	72.809	20.186	13.376

In this table, we report the monthly performance for portfolios constructed using the S&P 500 and 20 Fama and French portfolios formed on size and book-to-market. Panel A reports the standard deviation (SD) and Sharpe ratio (SR), whereas the average turnover (TO) and breakeven transaction costs (c_{ew}) with respect to the EW portfolio can be found in Panel B. The out-of-sample period is from January 1980 to December 2019. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). NA indicates that the value is not available. The significant outperformance of the alternative strategies from the equal-weighted strategy is denoted by: *, **, and *** for significance at the 10%, 5%, and 1% level, respectively.

Figures

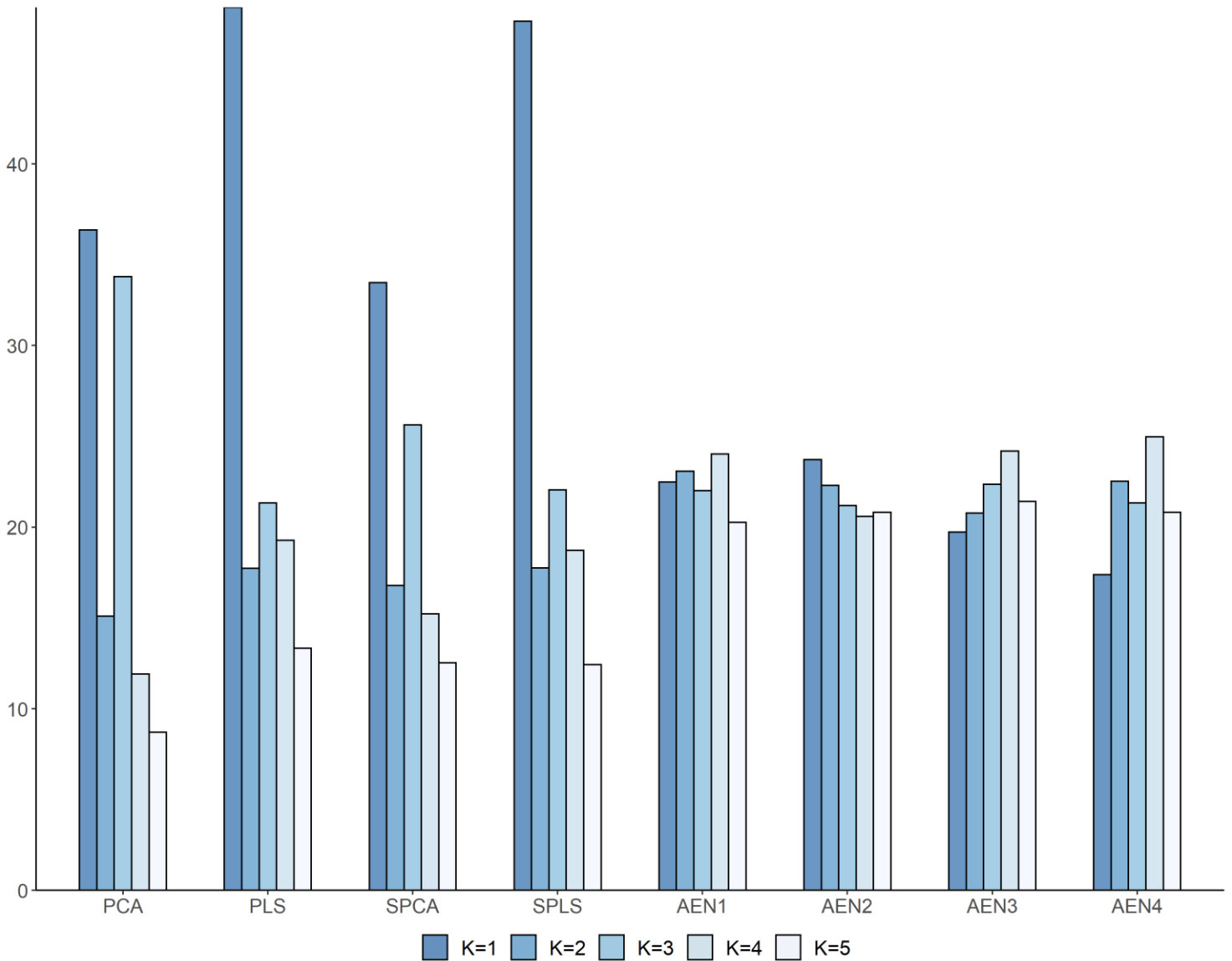


Figure 1

Average R^2_{adj} of the regressions of the latent factors on the Fama and French (2015) factors

This figure shows the R^2_{adj} as a percentage based on OLS estimation results for regressions of the latent factors on factors from the Fama and French (2015) five-factor model. The average over the out-of-sample period from July 1983 to December 2019 is given. The latent factor specifications are based on principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN).

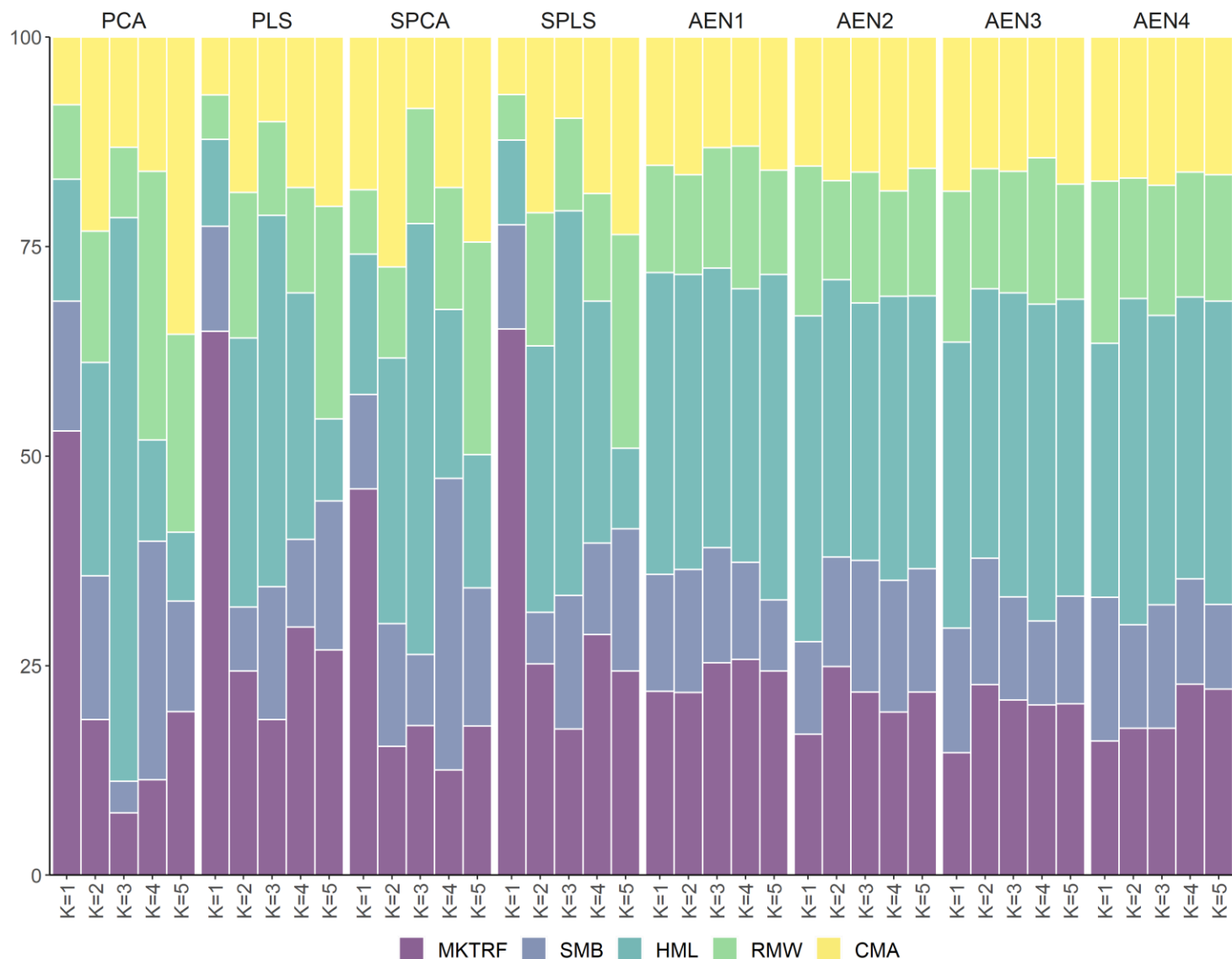


Figure 2

Variable importance of the Fama and French (2015) factors

This figure shows the variable importance based on OLS estimation results for regressions of the latent factors on factors from the Fama and French (2015) five-factor model. The measure of variable importance is calculated as the change in R^2 from setting the observations of a factor proxy to zero within each estimation window. The average over the out-of-sample period from July 1983 to December 2019 is given. The variable importance measures for each latent factor are scaled to sum to 100. The latent factor specifications are based on principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). MKTRF, SMB, HML, RMW, and CMA are the Fama and French (2015) excess returns of the market from the risk-free rate, size, value, profitability, and investment factors, respectively.

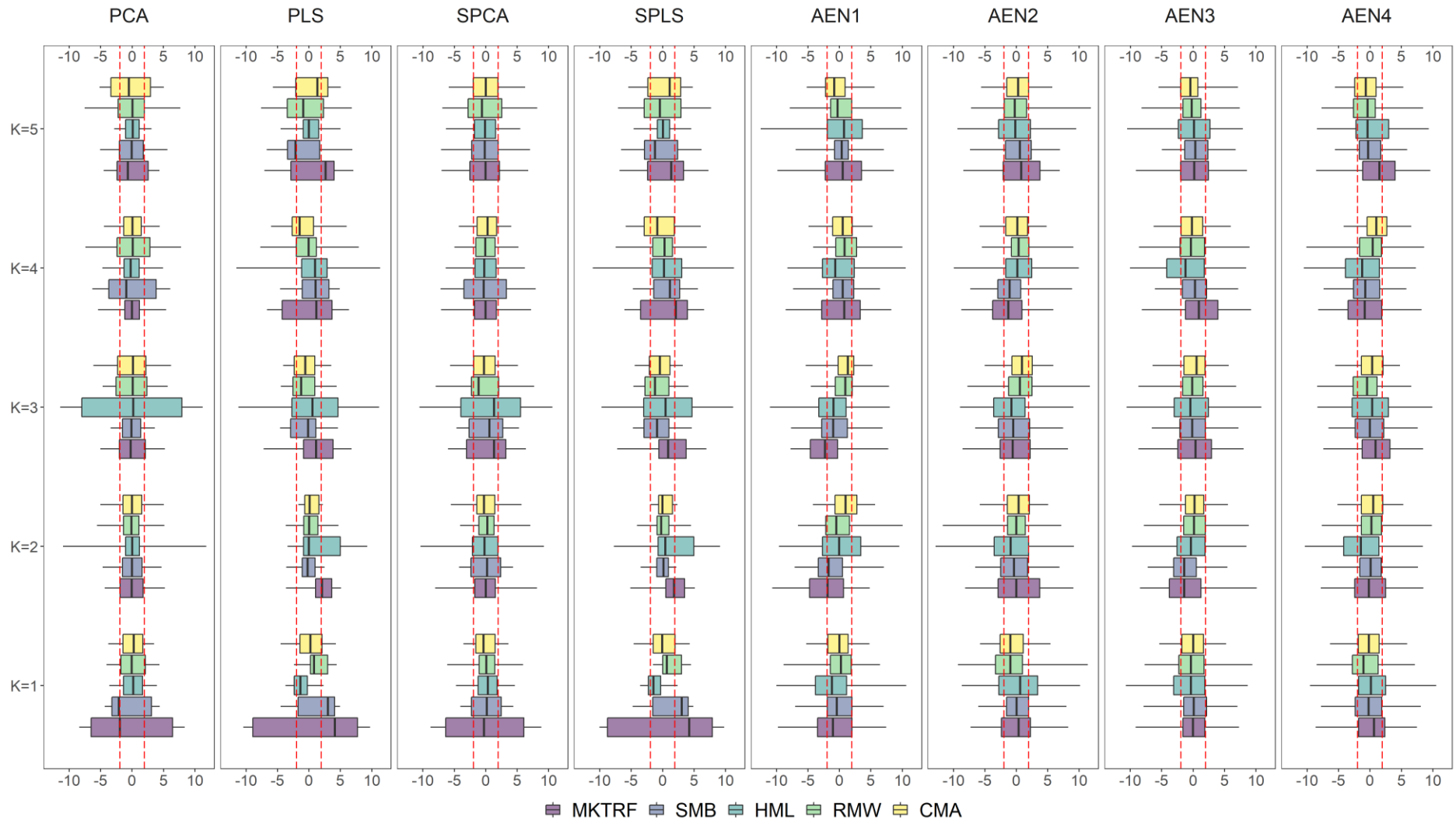


Figure 3
Explaining the latent factors based on the Fama and French (2015) five-factor model

This figure shows boxplots of the t -statistics based on OLS regressions of each of the five latent factors on factors from the Fama and French (2015) five-factor model. The horizontal axis reports t -statistics values ranging from -10 to 10, whereas the vertical axis reports the latent factors, $K = 1, \dots, 5$. The sample period is from July 1963 to December 2019. The median is marked by the line within the box, the edges of the box denote the first and third quartiles, while the minimum and maximum t -statistics are depicted by the end of the lines outside the box. The latent factor specifications are based on principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). MKTRF, SMB, HML, RMW, and CMA are the Fama and French (2015) excess returns of the market from the risk-free rate, size, value, profitability, and investment factors, respectively. The t -statistics are computed using heteroskedasticity and autocorrelation-robust standard errors (Newey and West, 1987). The red lines depict the Student's t critical values at the 5% level.

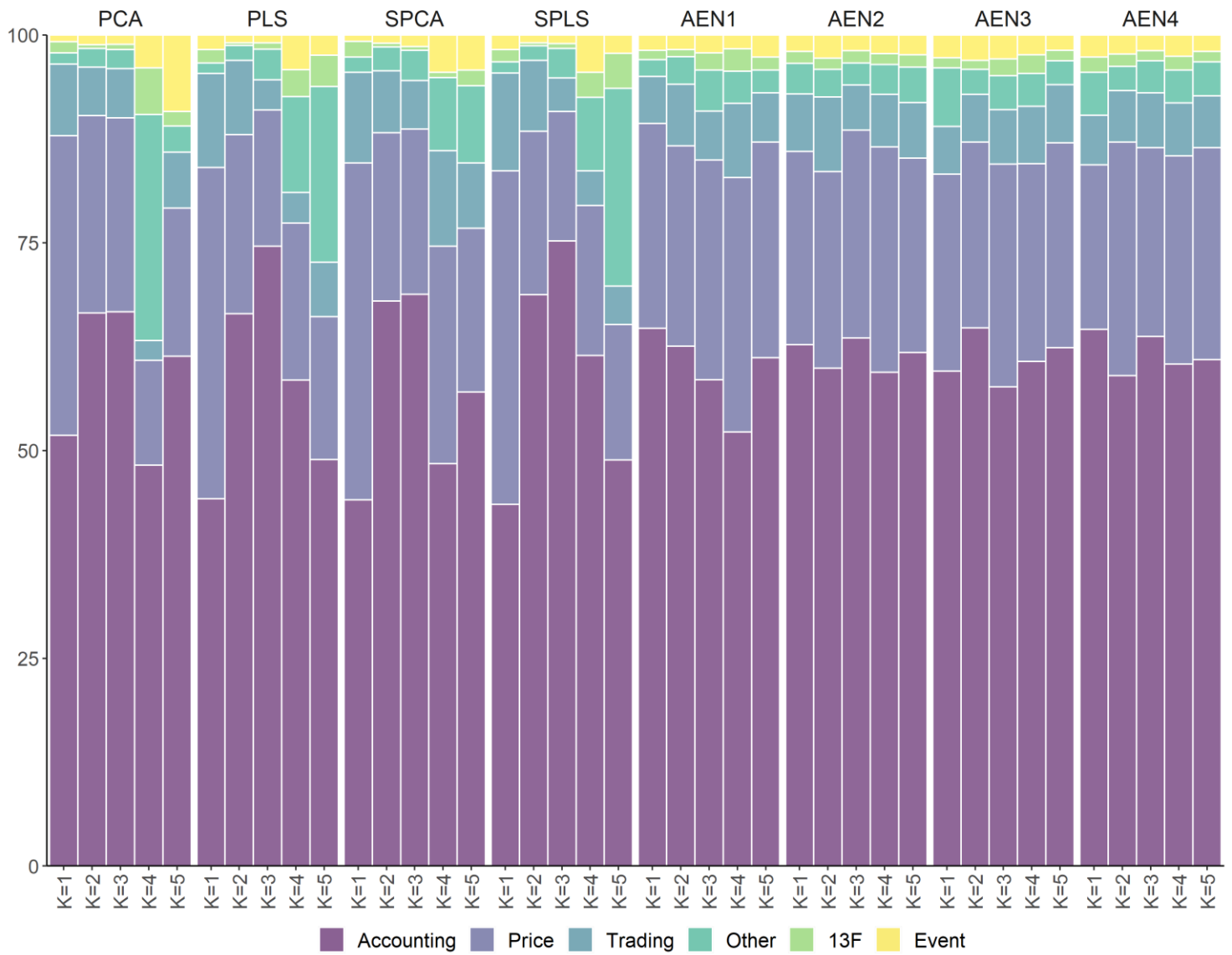


Figure 4
Variable importance of the long-short anomaly portfolio returns from the Chen and Zimmermann (2020) dataset
 This figure shows the variable importance based on lasso regressions of the latent factors on long-short anomaly portfolio returns. The measure of variable importance is calculated as the change in R^2 from setting the observations of a feature to zero within each estimation window. The results are aggregated by summing the variable importance of the characteristics-based portfolios belonging in the same group. Details on the portfolios within each group can be found in Table A1 in the Appendix. The average over the out-of-sample period from January 1980 to December 2019 is given. The variable importance measures for each group are scaled to sum to 100. The latent factor specifications are based on principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). The explanatory variables are 110 anomaly portfolios from the open-source asset pricing dataset by Chen and Zimmermann (2020), that have no missing values over the full sample period, from January 1960 to December 2019.

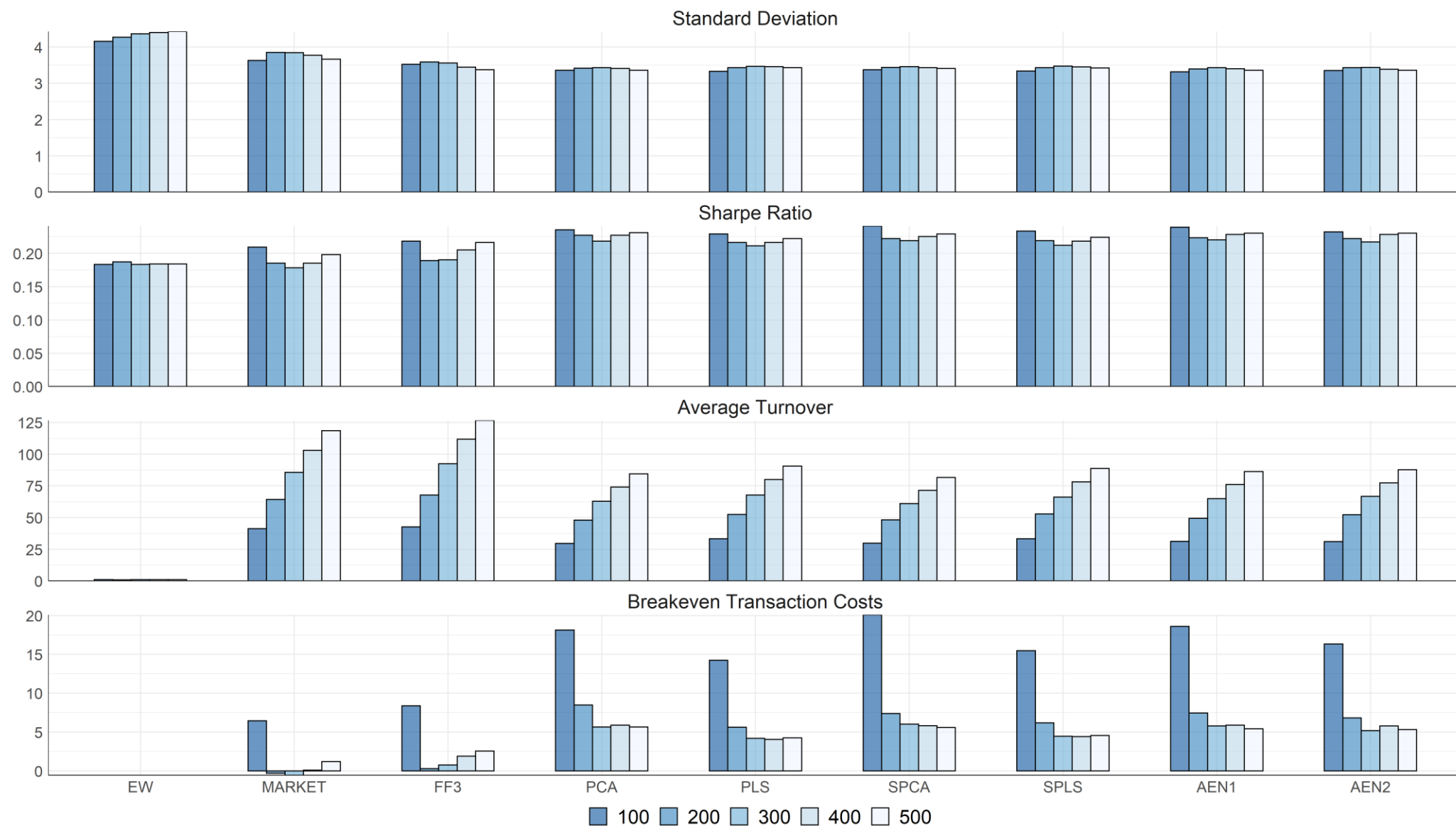


Figure 5
Portfolio performance for a different number of stocks: Static Factor Covariance

This figure shows the monthly portfolio performance for a varying number of assets. Performance is based on the standard deviation, Sharpe ratio, average turnover and breakeven transaction costs with respect to the EW portfolio. The out-of-sample period is from January 1980 to December 2019. The results are presented for the equal-weighted portfolio (EW) and for the static factor covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1 and 2 hidden layers (AEN). The standard deviation and average turnover are reported as a percentage. The breakeven transaction costs are reported in basis points and a positive value indicates that the alternative portfolio outperforms the EW.

Appendix

Tables

Table A1
Constituents of each group for the Chen and Zimmermann (2020) dataset

Group	# of Variables	Variables
13F	2	RIO_Turnover, RIO_Volatility
Accounting	46	Accruals, AM, AssetGrowth, BMdec, BookLeverage, CashProd, CF, ChAssetTurnover, ChInv, ChInvIA, ChNNCOA, ChNWC, CompEquIss, CompositeDebtIssuance, DelCOA, DelCOL, DelFINL, DelLTI, DelNetFin, DivYieldST, EarningsConsistency, EntMult, GP, grcapx, grcapx3y, GrLTNOA, GrSaleToGrInv, GrSaleToGrOverhead, IntanCFP, IntanEP, IntanSP, Investment, InvestPPEInv, InvGrowth, MeanRankRevGrowth, OPLeverage, OrgCap, RD, RDAbility, ShareIss1Y, ShareIss5Y, SP, tang, Tax, TotalAccruals, VarCF
Event	5	ConvDebt, DivInit, DivSeason, Spinoff, SurpriseRD
Other	5	FirmAge, Herf, HerfAsset, HerfBE, sinAlgo
Price	41	Beta, BetaFP, BetaTailRisk, Coskewness, EP, High52, IdioRisk, IdioVol3F, IdioVolAHT, IndMom, IndRetBig, IntMom, Leverage, LRreversal, MaxRet, Mom12m, Mom12mOffSeason, Mom6m, MomOffSeason, MomOffSeason06YrPlus, MomOffSeason11YrPlus, MomOffSeason16YrPlus, MomRev, MomSeason, MomSeason06YrPlus, MomSeason11YrPlus, MomSeason16YrPlus, MomSeasonShort, MomVol, MRreversal, NetPayoutYield, PayoutYield, Price, PriceDelayRsqr, PriceDelaySlope, PriceDelayTstat, ResidualMomentum, ReturnSkew, ReturnSkew3F, Size, STreversal
Trading	11	BidAskSpread, DoIVol, Illiquidity, ShareVol, std_turn, VolMkt, VolSD, VolumeTrend, zerotrade, zerotradeAlt1, zerotradeAlt2

This table presents information on the six groups of variables for the Chen and Zimmermann (2020) dataset.

Table A2
Constituents of each group for the McCracken and Ng (2015) dataset

Group	# of Variables	Variables
Consumption, orders, and inventories	7	DPCERA3M086SBEA, CMRMTSPLx, RETAILx, AMDMNOx, AMDMUOx, BUSINVx, ISRATIOx
Housing	5	HOUST, HOUSTNE, HOUSTMW, HOUSTS, HOUSTW
Interest and exchange rates	19	FEDFUNDS, TB3MS, TB6MS, GS1, GS5, GS10, AAA, BAA, TB3SMFFM, TB6SMFFM, T1YFFM, T5YFFM, T10YFFM, AAAFFM, BAAFFM, EXSZUSx, EXJPUSx, EXUSUKx, EXCAUSx
Labor market	31	HWI, HWIURATIO, CLF16OV, CE16OV, UNRATE, UEMPMEAN, UEMPLT5, UEMP5TO14, UEMP15OV, UEMP15T26, UEMP27OV, CLAIMSx, PAYEMS, USGOOD, CES1021000001, USCONS, MANEMP, DMANEMP, NDMANEMP, SRVPRD, USTPU, USWTRADE, USTRATE, USFIRE, USGOVT, CES0600000007, AWOTMAN, AWHMAN, CES0600000008, CES2000000008, CES3000000008
Money and credit	14	M1SL, M2SL, M2REAL, BOGMBASE, TOTRESNS, NONBORRES, BUSLOANS, REALLN, NONREVSL, CONSPI, MZMSL, DTCOLNVHFN, DTCTHFN, INVEST
Output and income	16	RPI, W875RX1, INDPRO, IPFPNSS, IPFINAL, IPCONGD, IPDCONGD, IPNCONGD, IPBUSEQ, IPMAT, IPDMAT, IPNMAT, IPMANSICS, IPB51222S, IPFUELS, CUMFNS
Prices	20	WPSFD49207, WPSFD49502, WPSID61, WPSID62, OILPRICEx, PPICMM, CPIAUCSL, CPIAPPSL, CPITRNSL, CPIMEDSL, CUSR0000SAC, CUSR0000SAD, CUSR0000SAS, CPIULFSL, CUSR0000SA0L2, CUSR0000SA0L5, PCEPI, DDURRG3M086SBEA, DNDGRG3M086SBEA, DSERRG3M086SBEA
Stock market	4	SP500, SPindust, SPdivyield, SPPEratio

This table presents information on the eight groups of variables for the McCracken and Ng (2015) dataset.

Table A3
Portfolio performance for different validation window size

A. Validation subsample set to 10%

	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
PCA	3.361***	0.235**	3.29***	0.239**	3.36***	0.235**	3.268***	0.241**
PLS	3.33***	0.229	3.276***	0.233*	3.335***	0.227	3.217***	0.238*
SPCA	3.387***	0.235**	3.322***	0.236**	3.392***	0.234**	3.273***	0.244***
SPLS	3.328***	0.232*	3.264***	0.236*	3.337***	0.23*	3.21***	0.241*
AEN1	3.333***	0.232**	3.288***	0.237**	3.331***	0.232**	3.235***	0.239**
AEN2	3.356***	0.23**	3.294***	0.236**	3.357***	0.229**	3.271***	0.235**
AEN3	3.353***	0.234**	3.305***	0.238**	3.351***	0.234**	3.245***	0.244**
AEN4	3.361***	0.225*	3.304***	0.228**	3.361***	0.225*	3.265***	0.234**

B. Validation subsample set to 30%

	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
PCA	3.367***	0.236**	3.312***	0.237**	3.368***	0.235**	3.265***	0.242**
PLS	3.349***	0.226	3.296***	0.231*	3.354***	0.225	3.231***	0.237*
SPCA	3.386***	0.237**	3.309***	0.238**	3.395***	0.235**	3.292***	0.243**
SPLS	3.338***	0.232*	3.262***	0.24**	3.346***	0.230	3.218***	0.242*
AEN1	3.34***	0.238***	3.297***	0.24***	3.338***	0.239***	3.23***	0.243**
AEN2	3.36***	0.23**	3.316***	0.229**	3.36***	0.23**	3.268***	0.234**
AEN3	3.338***	0.226*	3.302***	0.231**	3.336***	0.226*	3.241***	0.236**
AEN4	3.363***	0.228**	3.322***	0.229**	3.366***	0.229*	3.257***	0.236**

This table documents monthly portfolio performance measured using the standard deviation (SD) and Sharpe ratio (SR), over the out-of-sample period from January 1980 to December 2019. The hyperparameters are selected using two alternative validation window sizes to the 20% of the baseline results: 10% (Panel A) and 30% (Panel B). The results are presented for the four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). The significant outperformance of the alternative strategies from the equal-weighted strategy is denoted by: *, **, and *** for significance at the 10%, 5%, and 1% level, respectively.

Table A4
Portfolio performance for different levels of transaction costs

A. Transaction costs of 5 bps

	SD	SR						
EW	4.159	0.183						
Sample	3.470***	0.203						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	3.630***	0.203	3.346***	0.209	3.640***	0.204	3.596***	0.197
FF3	3.521***	0.212	3.327***	0.23	3.538***	0.206	3.468***	0.215
PCA	3.361***	0.230**	3.290***	0.234**	3.359***	0.230**	3.267***	0.234**
PLS	3.329***	0.224	3.276***	0.226	3.334***	0.222	3.216***	0.231
SPCA	3.373***	0.236**	3.306***	0.237**	3.377***	0.236**	3.262***	0.239**
SPLS	3.339***	0.228	3.260***	0.236*	3.346***	0.226	3.224***	0.234
AEN1	3.317***	0.235**	3.284***	0.238***	3.314***	0.234**	3.215***	0.237**
AEN2	3.351***	0.227**	3.296***	0.230**	3.349***	0.227**	3.258***	0.229*
AEN3	3.337***	0.233**	3.282***	0.234**	3.333***	0.233**	3.234***	0.238**
AEN4	3.356***	0.22	3.300***	0.221*	3.355***	0.219	3.255***	0.219

B. Transaction costs of 20 bps

	SD	SR						
EW	4.159	0.183						
Sample	3.471***	0.185						
	Static Factor Covariance		Dynamic Beta Covariance		Dynamic Factor Covariance		Dynamic Error Covariance	
	SD	SR	SD	SR	SD	SR	SD	SR
Market	3.629***	0.186	3.344***	0.182	3.641***	0.186	3.595***	0.173
FF3	3.520***	0.194	3.323***	0.197	3.537***	0.186	3.464***	0.19
PCA	3.361***	0.217	3.291***	0.218	3.359***	0.217	3.266***	0.213
PLS	3.328***	0.209	3.276***	0.208	3.333***	0.207	3.213***	0.208
SPCA	3.371***	0.223*	3.305***	0.221*	3.376***	0.222*	3.26***	0.217
SPLS	3.337***	0.213	3.260***	0.217	3.345***	0.211	3.222***	0.211
AEN1	3.315***	0.220*	3.282***	0.221*	3.312***	0.220*	3.213***	0.215
AEN2	3.349***	0.213	3.295***	0.213	3.347***	0.213	3.256***	0.208
AEN3	3.335***	0.218	3.281***	0.217	3.331***	0.218	3.232***	0.216
AEN4	3.355***	0.205	3.299***	0.204	3.354***	0.204	3.253***	0.196

This table presents monthly portfolio performance measured using the standard deviation (SD) and Sharpe ratio (SR), after transaction costs are taken into account. Panel A reports the results for transaction costs of 5 bps, while Panel B presents the results for transaction costs of 20 bps. The out-of-sample period is from January 1980 to December 2019. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). The significant outperformance of the alternative strategies from the equal-weighted strategy is denoted by: *, **, and *** for significance at the 10%, 5%, and 1% level, respectively.

Table A5**Characteristics of the portfolio weight vectors when short-selling is allowed**

	MAX	SD _ω	MAD _{EW}									
EW	0.010	0.000	0.000									
Sample	0.378	6.317	4.873									
	Static Factor Covariance			Dynamic Beta Covariance			Dynamic Factor Covariance			Dynamic Error Covariance		
	MAX	SD _ω	MAD _{EW}	MAX	SD _ω	MAD _{EW}	MAX	SD _ω	MAD _{EW}	MAX	SD _ω	MAD _{EW}
Market	0.226	2.855	2.122	0.176	2.412	1.864	0.253	2.803	2.082	0.297	3.319	2.327
FF3	0.237	3.242	2.383	0.201	2.628	2.001	0.278	3.132	2.313	0.309	3.929	2.765
PCA	0.064	1.051	0.785	0.065	1.014	0.763	0.063	1.056	0.789	0.132	1.262	0.894
PLS	0.078	1.320	0.979	0.072	1.255	0.940	0.078	1.322	0.981	0.140	1.556	1.095
SPCA	0.060	1.010	0.727	0.073	1.014	0.743	0.060	1.009	0.726	0.128	1.216	0.839
SPLS	0.082	1.317	0.947	0.077	1.266	0.927	0.081	1.316	0.947	0.163	1.554	1.069
AEN1	0.062	1.024	0.776	0.064	0.984	0.748	0.062	1.028	0.780	0.114	1.234	0.883
AEN2	0.058	1.042	0.783	0.068	0.997	0.752	0.059	1.049	0.788	0.112	1.251	0.888
AEN3	0.063	1.070	0.804	0.066	1.016	0.765	0.063	1.078	0.810	0.130	1.288	0.911
AEN4	0.067	1.071	0.804	0.069	1.018	0.766	0.067	1.080	0.810	0.128	1.283	0.911

This table presents monthly characteristics of the portfolio weight vectors based on the maximum weight (MAX), standard deviation of the weights (SD_ω) and mean absolute deviation from the equal-weighted benchmark (MAD_{EW}). The results are based on minimum-variance portfolios without short-selling constraints. The average value of each weight characteristic over the out-of-sample period from January 1980 to December 2019 is reported. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN).

Table A6**Characteristics of the portfolio weight vectors when using a penalized minimum-variance objective function**

	MAX	SD _ω	MAD _{EW}									
EW	0.010	0.000	0.000									
Sample	0.330	2.912	1.514									
	Static Factor Covariance			Dynamic Beta Covariance			Dynamic Factor Covariance			Dynamic Error Covariance		
	MAX	SD _ω	MAD _{EW}	MAX	SD _ω	MAD _{EW}	MAX	SD _ω	MAD _{EW}	MAX	SD _ω	MAD _{EW}
Market	0.258	2.647	1.547	0.297	2.466	1.522	0.392	2.873	1.572	0.351	3.305	1.624
FF3	0.271	2.659	1.513	0.289	2.456	1.501	0.354	2.819	1.539	0.336	3.228	1.602
PCA	0.073	1.209	0.982	0.085	1.298	1.042	0.079	1.259	1.017	0.145	1.446	1.071
PLS	0.081	1.447	1.148	0.092	1.453	1.144	0.101	1.487	1.168	0.150	1.726	1.238
SPCA	0.072	1.188	0.984	0.087	1.290	1.051	0.076	1.212	0.990	0.168	1.447	1.054
SPLS	0.085	1.465	1.169	0.107	1.503	1.173	0.095	1.476	1.172	0.194	1.726	1.227
AEN1	0.071	1.144	0.931	0.090	1.259	1.005	0.076	1.183	0.950	0.128	1.390	1.014
AEN2	0.070	1.202	0.981	0.085	1.304	1.050	0.083	1.262	1.020	0.137	1.456	1.077
AEN3	0.074	1.225	0.999	0.080	1.299	1.048	0.085	1.286	1.039	0.139	1.478	1.097
AEN4	0.075	1.246	1.001	0.083	1.309	1.048	0.082	1.299	1.037	0.147	1.476	1.072

This table reports monthly characteristics of the portfolio weight vectors based on the maximum weight (MAX), standard deviation of the weights (SD_ω) and mean absolute deviation from the equal-weighted benchmark (MAD_{EW}). The results are based on minimum-variance portfolios with a turnover penalty for a parameter $c = 5$ bps. The average value of each weight characteristic over the out-of-sample period from January 1980 to December 2019 is reported. The results are presented for the equal-weighted portfolio (EW) and minimum-variance portfolios based on the sample estimator (Sample) and four factor-implied covariance specifications: static factor covariance, dynamic beta covariance, dynamic factor covariance and dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN).

Figures

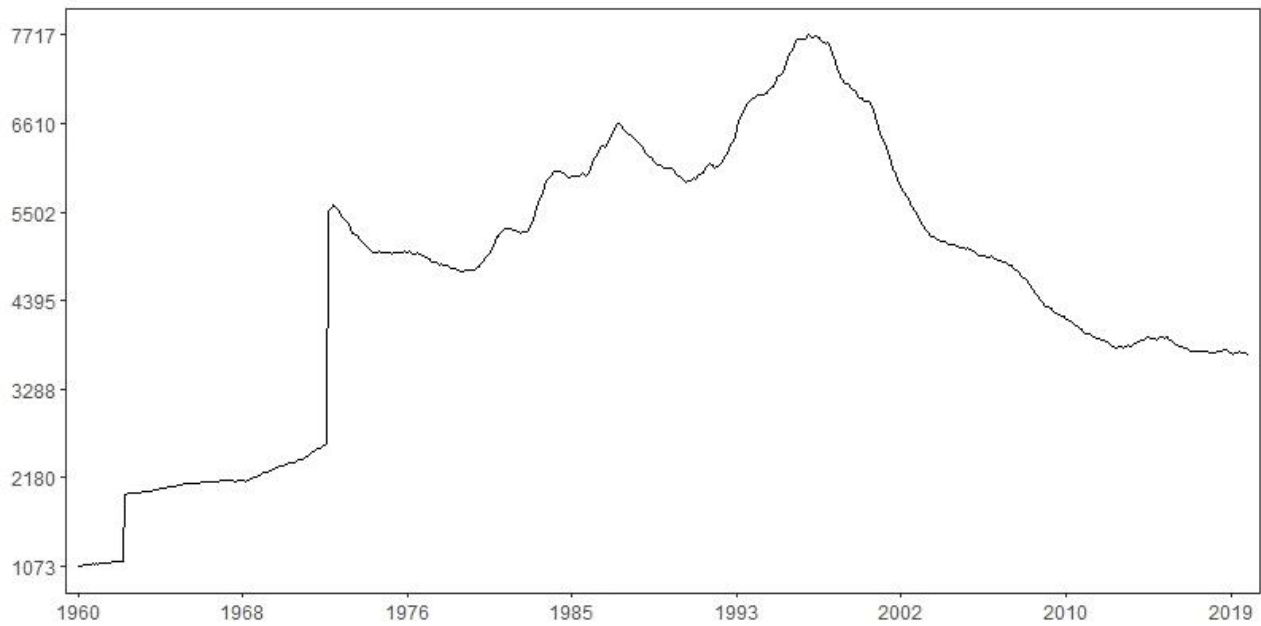


Figure A1

Number of stocks per month after the filters have been applied to the CRSP dataset

This figure shows the monthly number of stocks that are listed to the NYSE, AMEX, and NASDAQ stock exchanges (exchange codes 1, 2 or 3) and are ordinary common shares (share codes 10 or 11), over the full sample period from January 1960 to December 2019 (720 observations).

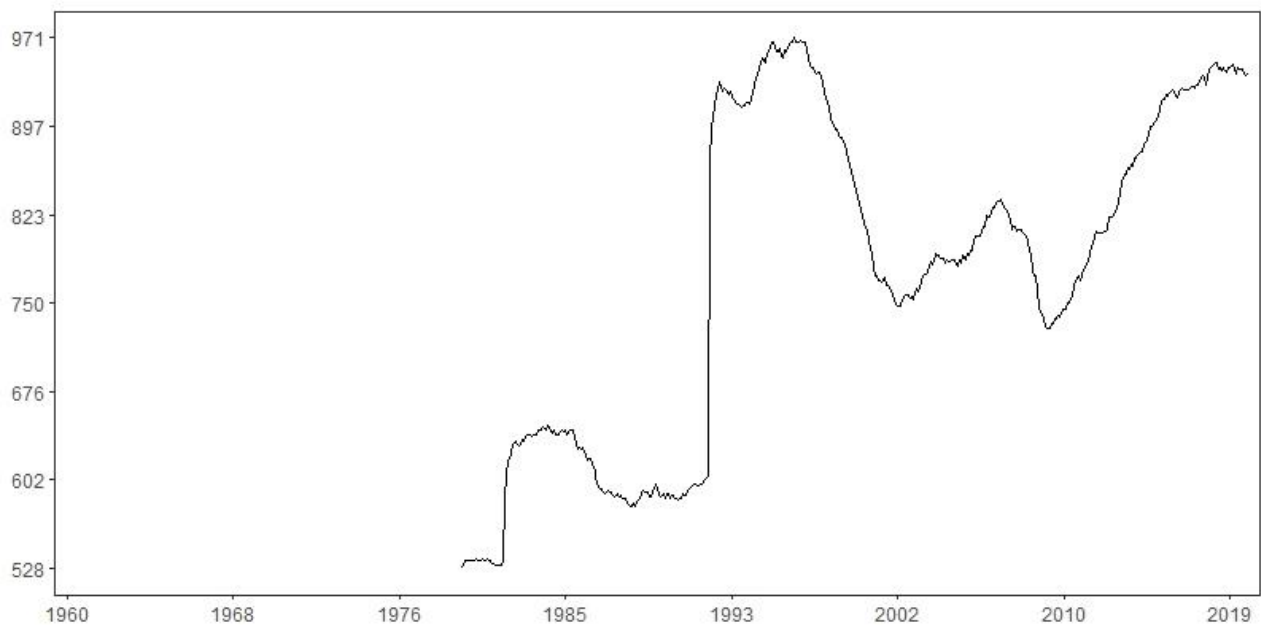


Figure A2

Number of stocks per month that fulfil the conditions of the rolling window

This figure shows the number of stocks in each iteration of the rolling window. Stocks are considered if they have at least 97.5% of the in-sample observations available, if they are not missing a return observation for the next month after the end of the rolling window and have a price greater than \$5. The out-of-sample period is from December 1980 to December 2019 (480 observations).

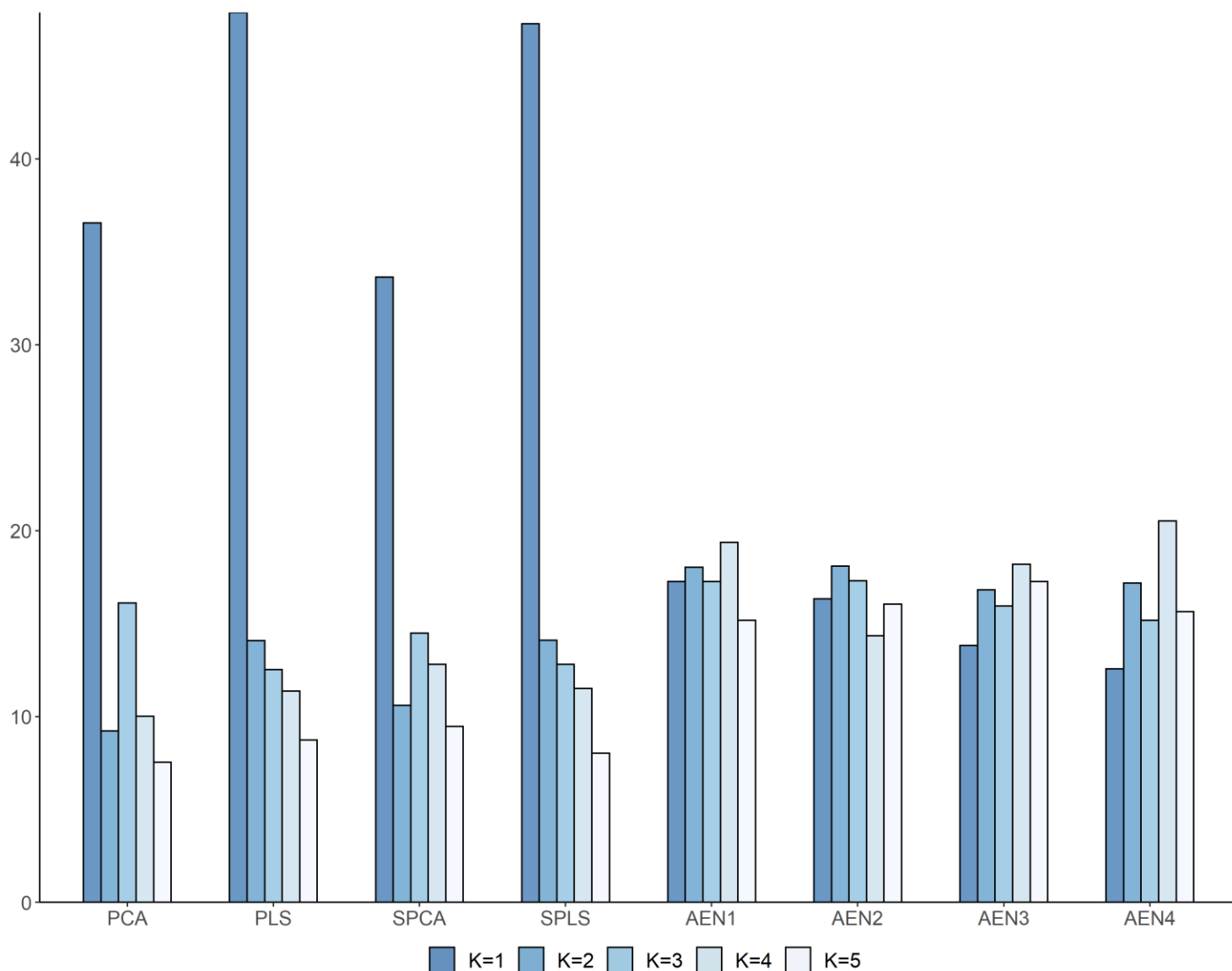


Figure A3

Average R^2_{adj} of the regressions of the latent factors on factors from the augmented q-factor model

This figure shows the R^2_{adj} as a percentage based on OLS estimation results for regressions of the latent factors on factors from the augmented q-factor model. The average over the out-of-sample period from January 1987 to December 2019 is given. The latent factor specifications are based on principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN).

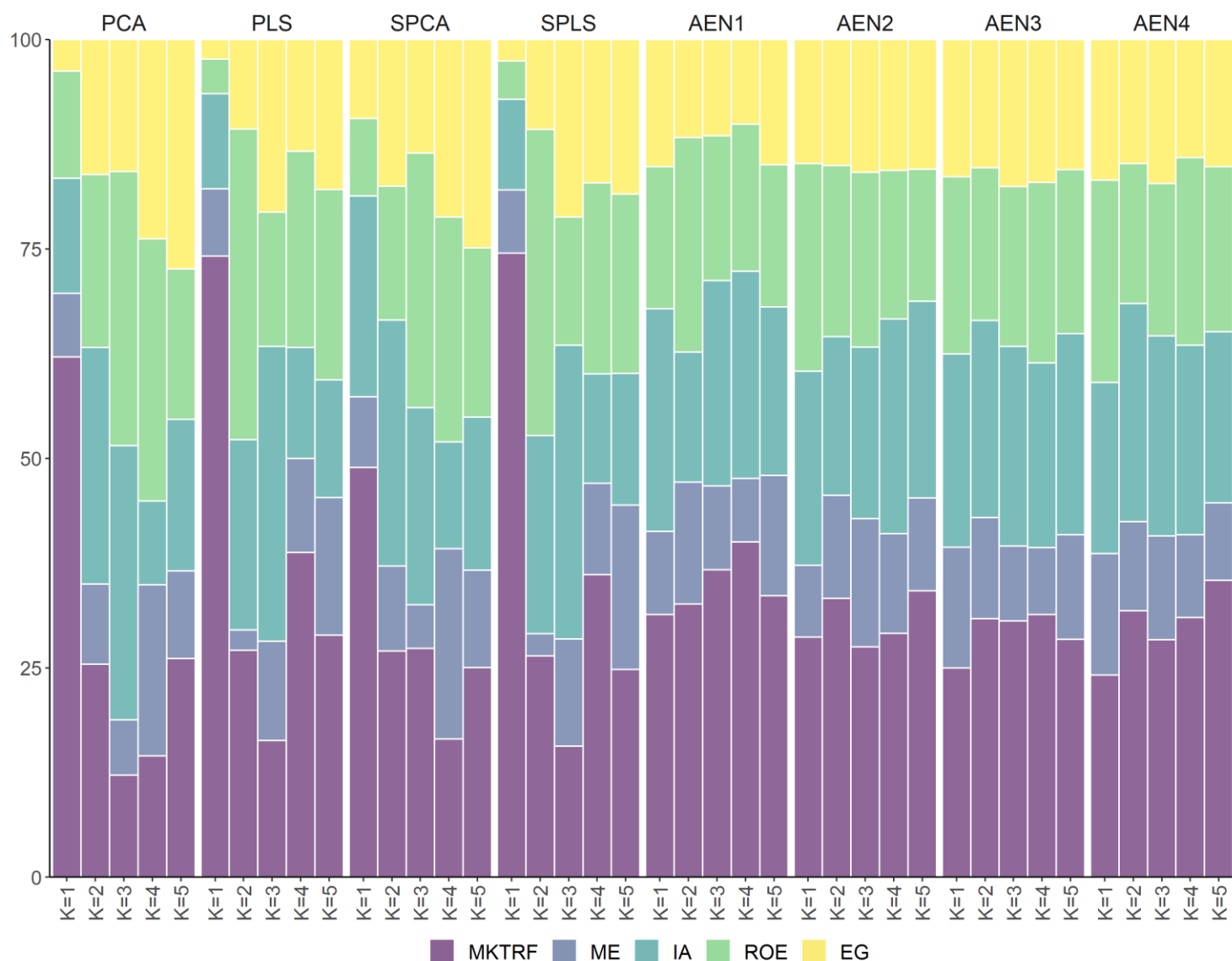


Figure A4

Variable importance based on the augmented q-factor model

This figure shows the variable importance based on OLS estimation results for regressions of the latent factors on factors from the augmented q-factor model. The measure of variable importance is calculated as the change in R^2 from setting the observations of a factor proxy to zero within each estimation window. The average over the out-of-sample period from January 1987 to December 2019 is given. The variable importance measures for each latent factor are scaled to sum to 100. The latent factor specifications are based on principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). MKTRF, ME, IA, ROE and EG are the Fama and French excess returns of the market from the risk-free rate, the Hou, Xue and Zhang (2015) size, investment, return-on-equity factors and the Hou, Mo, Xue and Zhang (2021) expected growth factor, respectively.

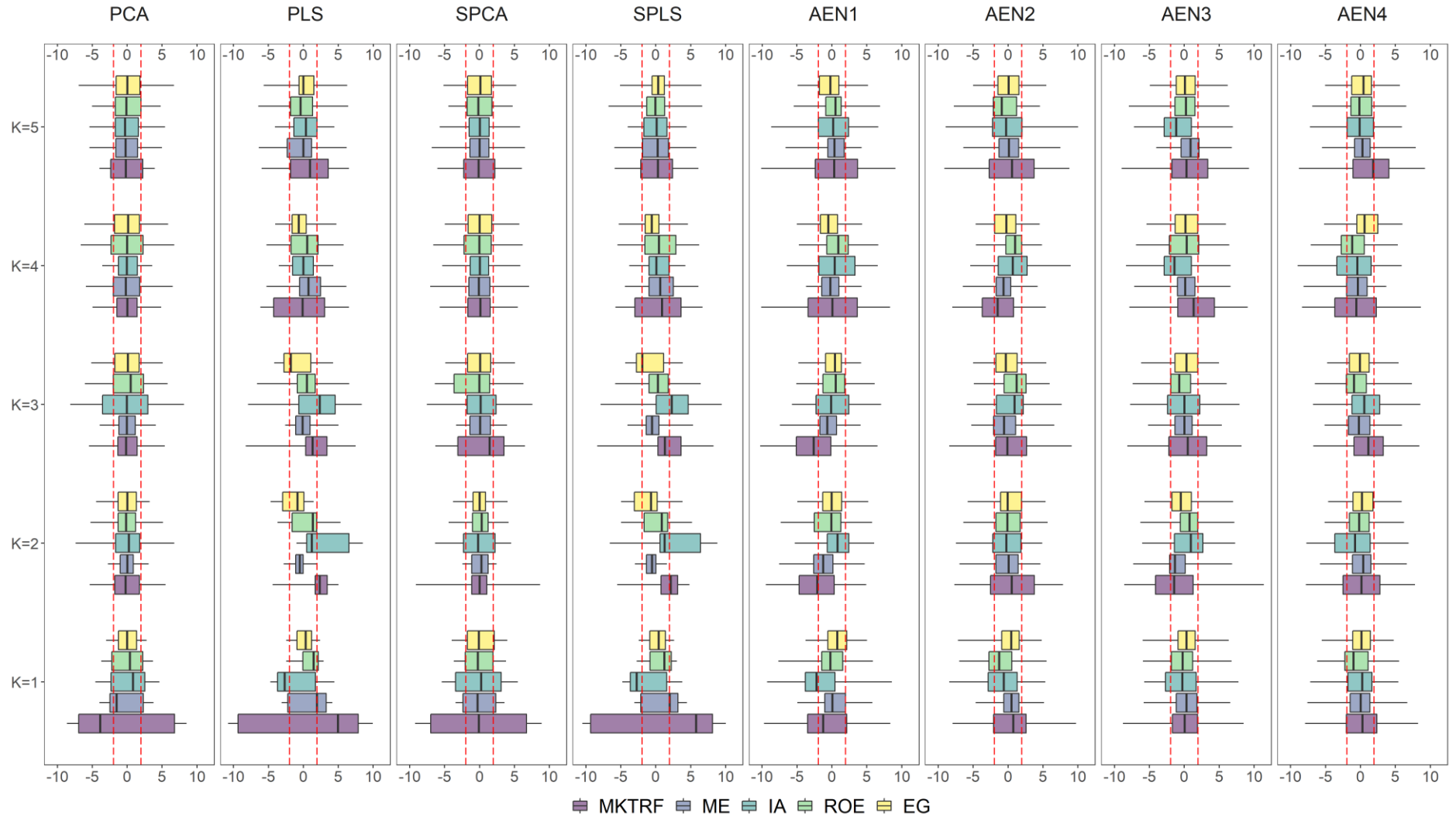


Figure A5

Explaining the latent factors based on the augmented q-factor model

This figure shows boxplots of the t -statistics based on OLS regressions of each of the five latent factors on factors from the augmented q-factor model. The horizontal axis reports t -statistics values ranging from -10 to 10 whereas, the vertical axis reports the latent factors, $K = 1, \dots, 5$. The sample period is from January 1967 to December 2019. The median is marked by the line within the box, the edges of the box denote the first and third quartiles, while the minimum and maximum t -statistics are depicted by the end of the lines outside the box. The latent factor specifications are based on principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). MKTRF, ME, IA, ROE and EG are the Fama and French excess returns of the market from the risk-free rate, the Hou, Xue and Zhang (2015) size, investment, return-on-equity factors and the Hou, Mo, Xue and Zhang (2021) expected growth factor, respectively. The t -statistics are computed using heteroskedasticity and autocorrelation-robust standard errors (Newey and West, 1987). The red lines depict the Student's t critical values at the 5% level.

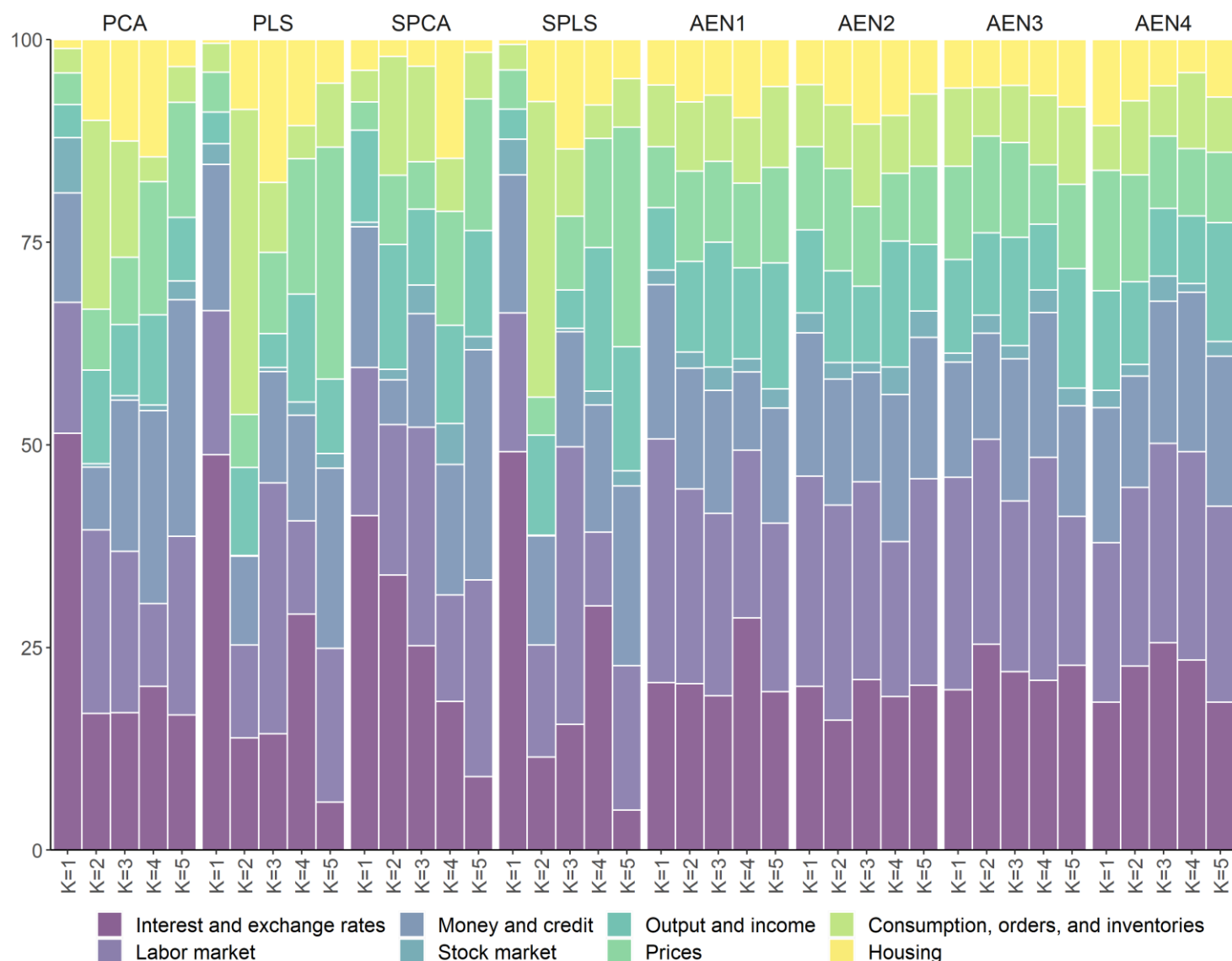


Figure A6

Variable importance of the economic indicators from the McCracken and Ng (2015) dataset

This figure shows the variable importance based on lasso regressions of the latent factors on economic indicators. The measure of variable importance is calculated as the change in R^2 from setting the observations of a feature to zero within each estimation window. The results are aggregated by summing the variable importance of the economic indicators belonging in the same group. Details on the variables within each group can be found in Table A2 in the Appendix. The average over the out-of-sample period from January 1980 to December 2019 is given. The variable importance measures for each group are scaled to sum to 100. The latent factor specifications are based on principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1, 2, 3 and 4 hidden layers (AEN). The explanatory variables are the 116 lagged economic indicators from the FRED-MD dataset by McCracken and Ng (2015) that have no missing values over the full sample period, from January 1960 to December 2019.

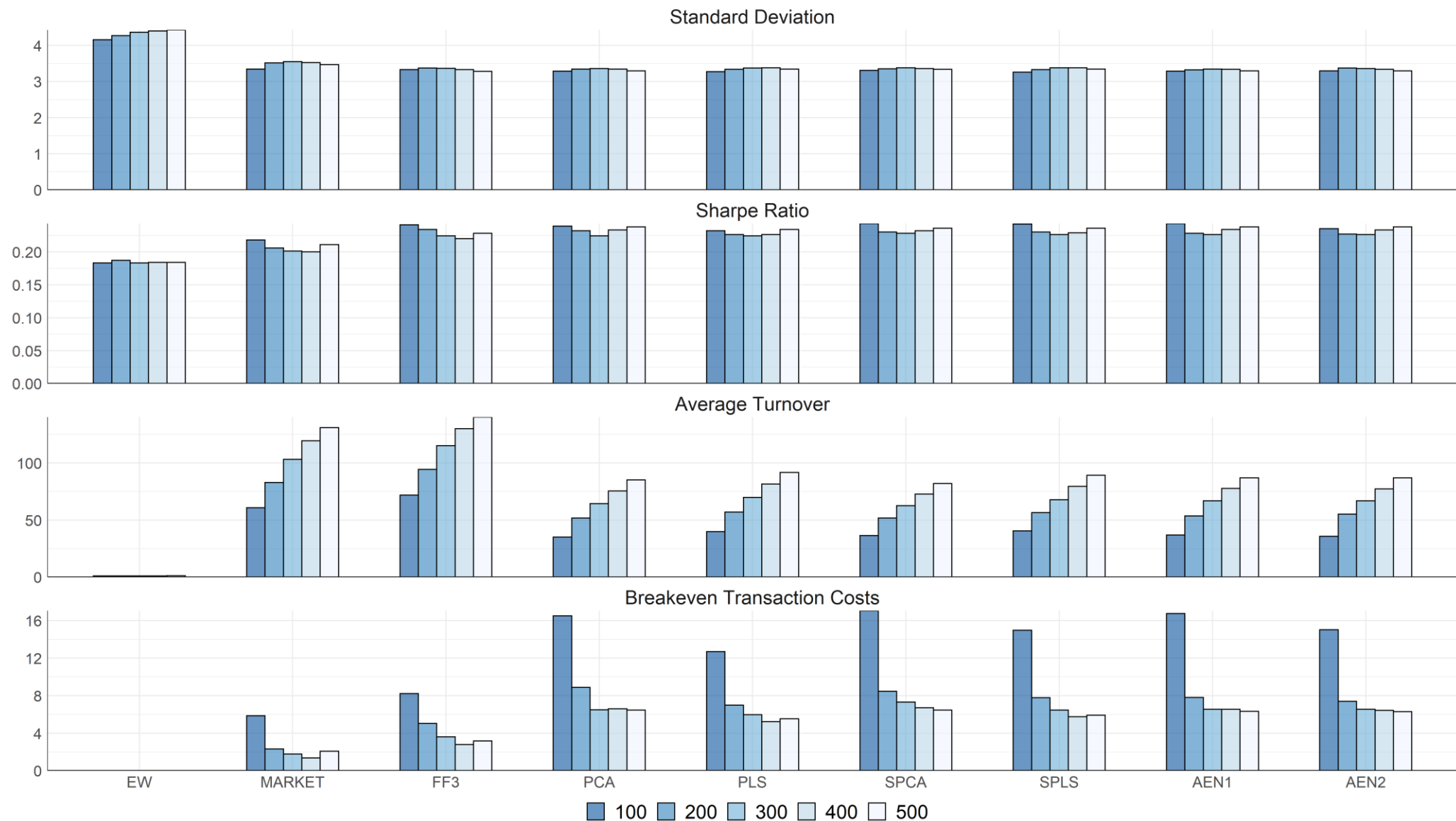


Figure A7
Portfolio performance for a different number of stocks: Dynamic Beta Covariance

This figure shows the monthly portfolio performance for a varying number of assets. Performance is based on the standard deviation, Sharpe ratio, average turnover and breakeven transaction costs with respect to the EW portfolio. The out-of-sample period is from January 1980 to December 2019. The results are presented for the equal-weighted portfolio (EW) and for the dynamic beta covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1 and 2 hidden layers (AEN). The standard deviation and average turnover are reported as a percentage. The breakeven transaction costs are reported in basis points and a positive value indicates that the alternative portfolio outperforms the EW.

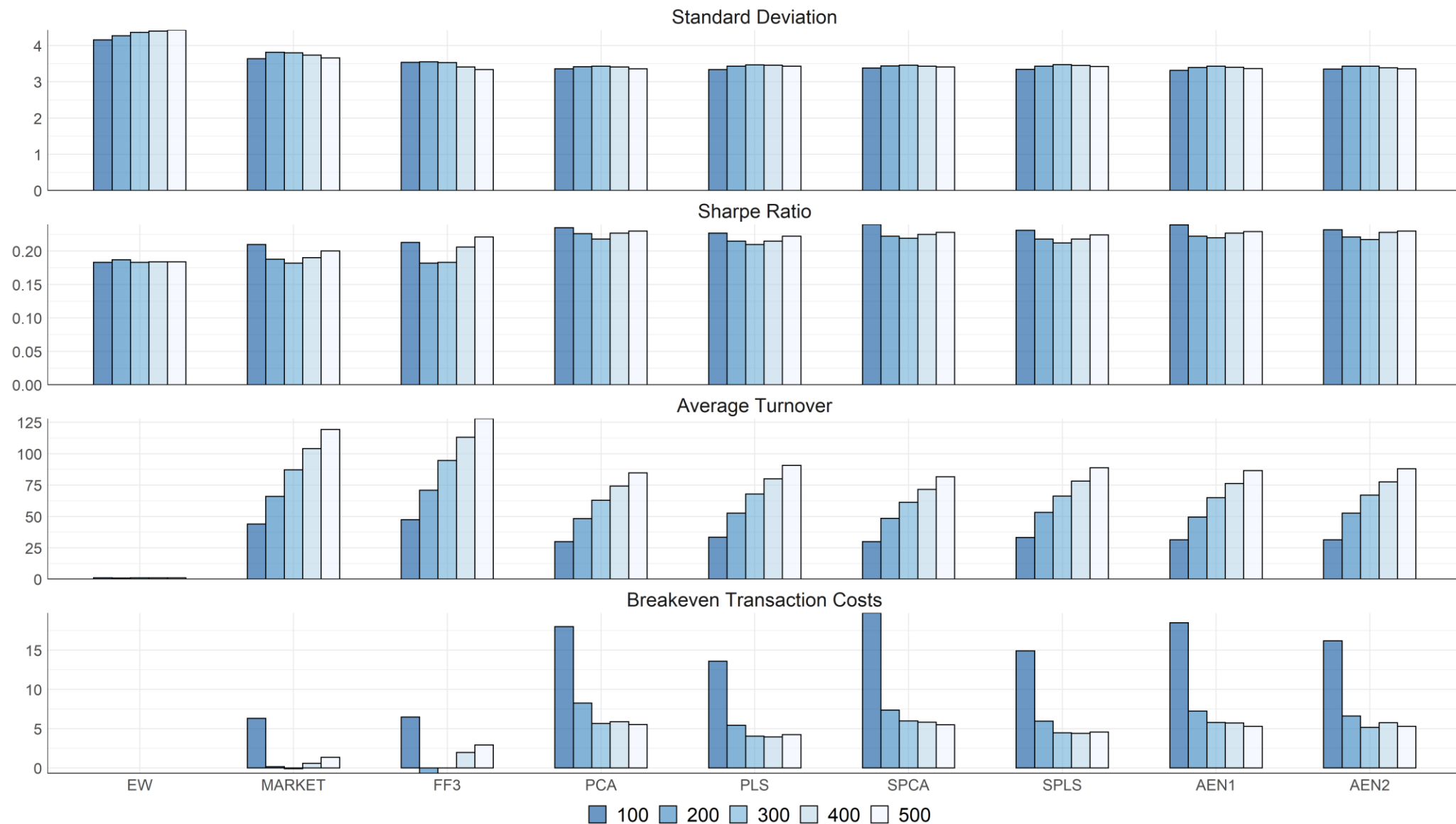


Figure A8
Portfolio performance for a different number of stocks: Dynamic Factor Covariance

This figure shows the monthly performance for a varying number of assets. Performance is based on the standard deviation, Sharpe ratio, average turnover and breakeven transaction costs with respect to the EW portfolio. The out-of-sample period is from January 1980 to December 2019. The results are presented for the equal-weighted portfolio (EW) and for the dynamic factor covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1 and 2 hidden layers (AEN). The standard deviation and average turnover are reported as a percentage. The breakeven transaction costs are reported in basis points and a positive value indicates that the alternative portfolio outperforms the EW.

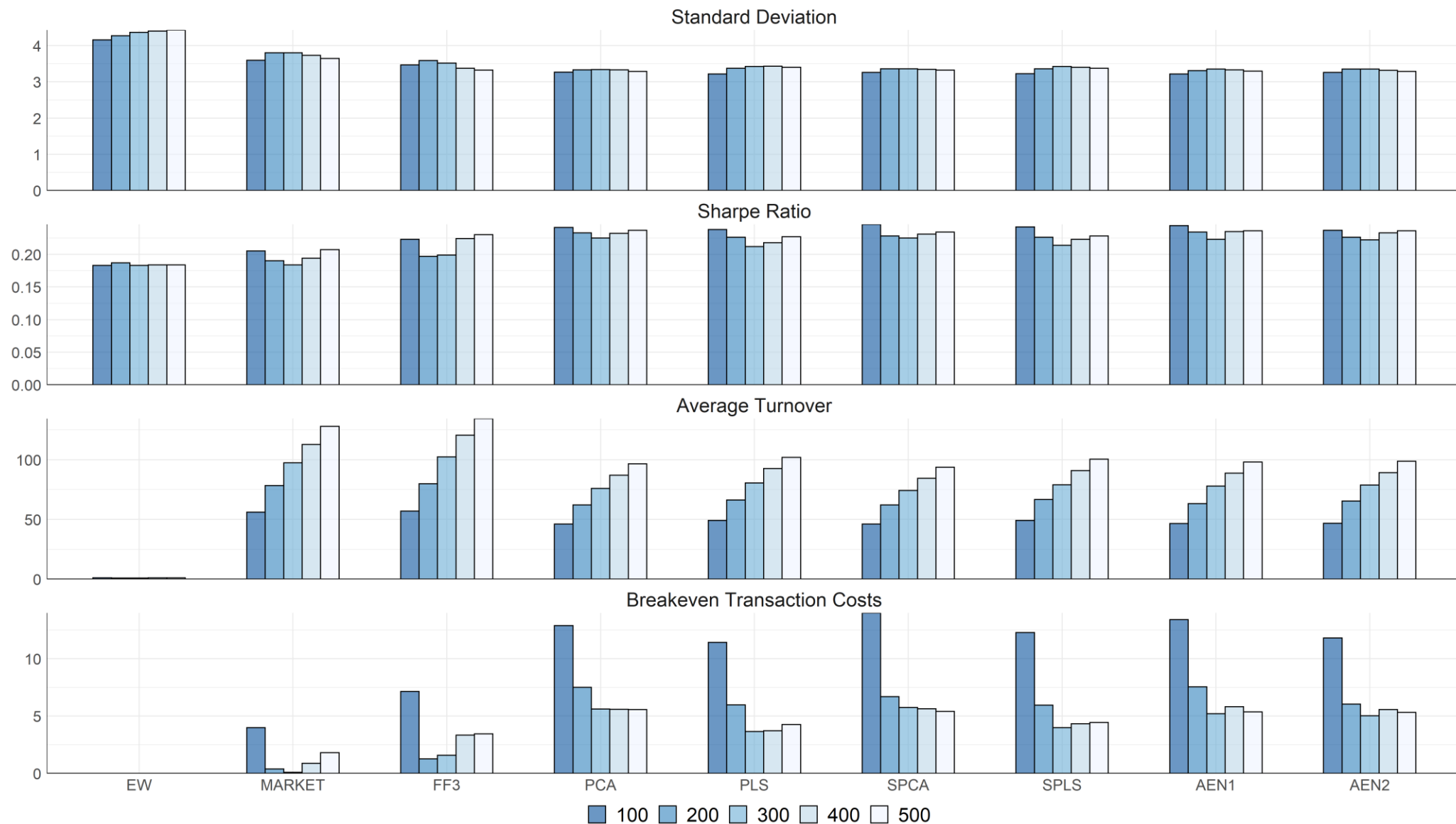


Figure A9

Portfolio performance for a different number of stocks: Dynamic Error Covariance

This figure shows the monthly performance for a varying number of assets. Performance is based on the standard deviation, Sharpe ratio, average turnover and breakeven transaction costs with respect to the EW portfolio. The out-of-sample period is from January 1980 to December 2019. The results are presented for the equal-weighted portfolio (EW) and for the dynamic error covariance. The factor specifications are based on the single factor model (Market), the Fama-French 3-factor model (FF3), principal component analysis (PCA), partial least squares (PLS), sparse principal component analysis (SPCA), sparse partial least squares (SPLS) and autoencoders with 1 and 2 hidden layers (AEN). The standard deviation and average turnover are reported as a percentage. The breakeven transaction costs are reported in basis points and a positive value indicates that the alternative portfolio outperforms the EW.



A Gentle Introduction to Ensemble Learning Algorithms

by Jason Brownlee on April 27, 2021 in Ensemble Learning  17



Ensemble learning is a general meta approach to machine learning that seeks better predictive performance by combining the predictions from multiple models.

Although there are a seemingly unlimited number of ensembles that you can develop for your predictive modeling problem, there are three methods that dominate the field of ensemble learning. So much so, that rather than algorithms per se, each is a field of study that has spawned many more specialized methods.

The three main classes of ensemble learning methods are **bagging**, **stacking**, and **boosting**, and it is important to both have a detailed understanding of each method and to consider them on your predictive modeling project.

But, before that, you need a gentle introduction to these approaches and the key ideas behind each method prior to layering on math and code.

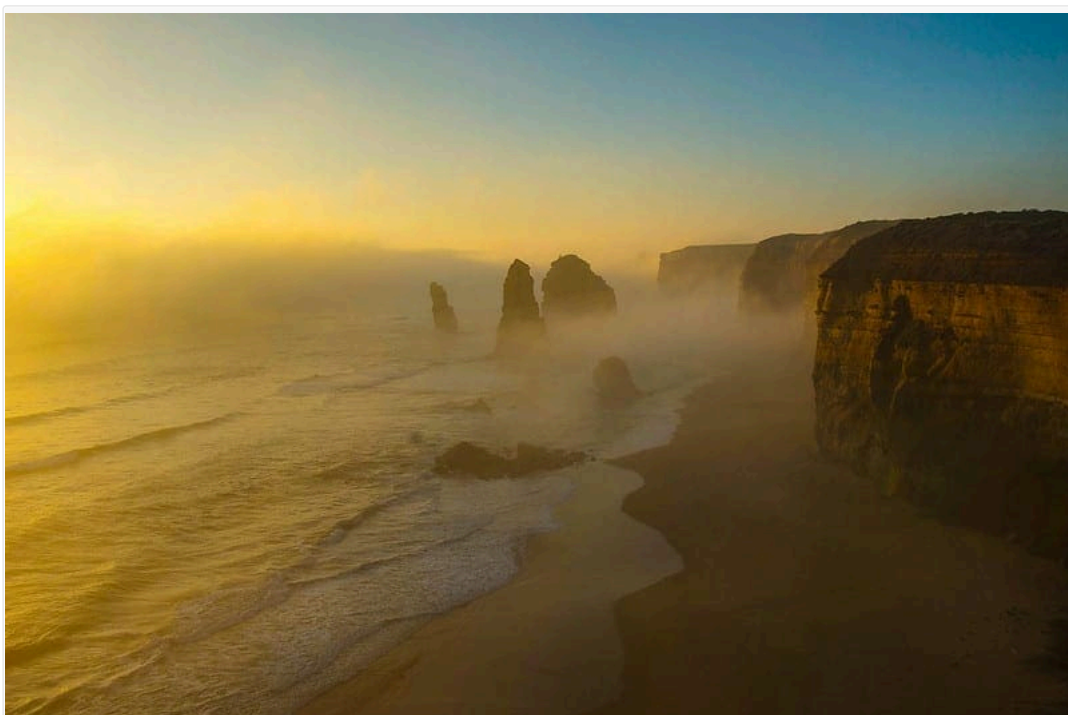
In this tutorial, you will discover the three standard ensemble learning techniques for machine learning.

After completing this tutorial, you will know:

- Bagging involves fitting many decision trees on different samples of the same dataset and averaging the predictions.
- Stacking involves fitting many different models types on the same data and using another model to learn how to best combine the predictions.
- Boosting involves adding ensemble members sequentially that correct the predictions made by prior models and outputs a weighted average of the predictions.

Kick-start your project with my new book [Ensemble Learning Algorithms With Python](#), including *step-by-step tutorials* and the *Python source code* files for all examples.

Let's get started.



A Gentle Introduction to Ensemble Learning Algorithms
Photo by [Rajiv Bhuttan](#), some rights reserved.

Tutorial Overview

This tutorial is divided into four parts; they are:

1. Standard Ensemble Learning Strategies
2. Bagging Ensemble Learning
3. Stacking Ensemble Learning
4. Boosting Ensemble Learning

Standard Ensemble Learning Strategies

Ensemble learning refers to algorithms that combine the predictions from two or more models.

Although there is nearly an unlimited number of ways that this can be achieved, there are perhaps three classes of ensemble learning techniques that are most commonly discussed and used in practice. Their popularity is due in large part to their ease of implementation and success on a wide range of predictive modeling problems.



A rich collection of ensemble-based classifiers have been developed over the last several years. However, many of these are some variation of the select few well-established algorithms whose capabilities have also been extensively tested and widely reported.

— Page 11, [Ensemble Machine Learning](#), 2012.

Given their wide use, we can refer to them as “standard” ensemble learning strategies; they are:

1. Bagging.
2. Stacking.
3. Boosting.

There is an algorithm that describes each approach, although more importantly, the success of each approach has spawned a myriad of extensions and related techniques. As such, it is more useful to describe each as a class of techniques or standard approaches to ensemble learning.

Rather than dive into the specifics of each method, it is useful to step through, summarize, and contrast each approach. It is also important to remember that although discussion and use of these methods are pervasive, these three methods alone do not define the extent of ensemble learning.

Next, let’s take a closer look at bagging.

Bagging Ensemble Learning

[Bootstrap aggregation](#), or bagging for short, is an ensemble learning method that seeks a diverse group of ensemble members by varying the training data.



The name Bagging came from the abbreviation of Bootstrap AGGREGatING. As the name implies, the two key ingredients of Bagging are bootstrap and aggregation.

— Page 48, [Ensemble Methods](#), 2012.

This typically involves using a single machine learning algorithm, almost always an unpruned decision tree, and training each model on a different sample of the same training dataset. The predictions made by the ensemble members are then combined using simple statistics, such as voting or averaging.



The diversity in the ensemble is ensured by the variations within the bootstrapped replicas on which each classifier is trained, as well as by using a relatively weak classifier whose decision boundaries measurably vary with respect to relatively small perturbations in the training data.

— Page 11, [Ensemble Machine Learning](#), 2012.

Key to the method is the manner in which each sample of the dataset is prepared to train ensemble members. Each model gets its own unique sample of the dataset.

Examples (rows) are drawn from the dataset at random, although with replacement.



Bagging adopts the bootstrap distribution for generating different base learners. In other words, it applies bootstrap sampling to obtain the data subsets for training the base learners.

Replacement means that if a row is selected, it is returned to the training dataset for potential re-selection in the same training dataset. This means that a row of data may be selected zero, one, or multiple times for a given training dataset.

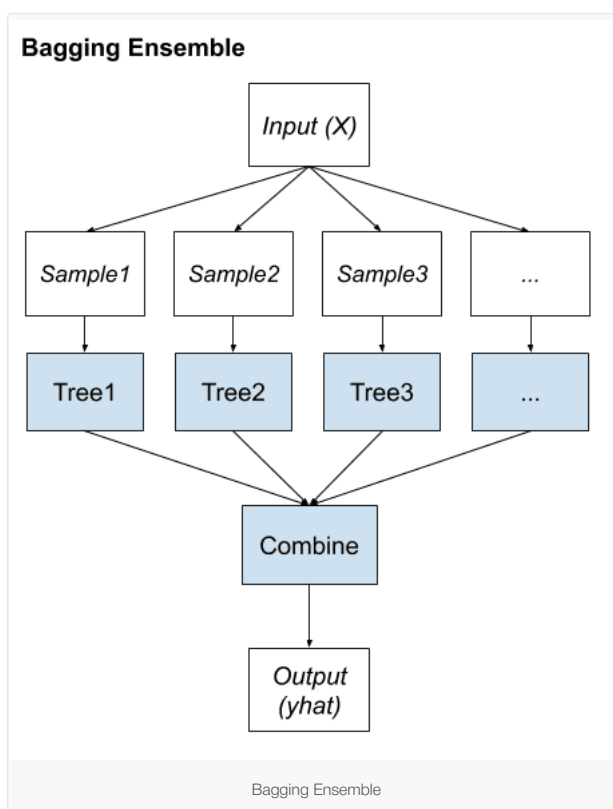
This is called a [bootstrap sample](#). It is a technique often used in statistics with small datasets to estimate the statistical value of a data sample. By preparing multiple different bootstrap samples and estimating a statistical quantity and calculating the mean of the estimates, a better overall estimate of the desired quantity can be achieved than simply estimating from the dataset directly.

In the same manner, multiple different training datasets can be prepared, used to estimate a predictive model, and make predictions. Averaging the predictions across the models typically results in better predictions than a single model fit on the training dataset directly.

We can summarize the key elements of bagging as follows:

- Bootstrap samples of the training dataset.
- Unpruned decision trees fit on each sample.
- Simple voting or averaging of predictions.

In summary, the contribution of bagging is in the varying of the training data used to fit each ensemble member, which, in turn, results in skillful but different models.



It is a general approach and easily extended. For example, more changes to the training dataset can be introduced, the algorithm fit on the training data can be replaced, and the mechanism used to combine predictions can be modified.

Many popular ensemble algorithms are based on this approach, including:

- Bagged Decision Trees (canonical bagging)
- Random Forest
- Extra Trees

Next, let's take a closer look at stacking.

Want to Get Started With Ensemble Learning?

Take my free 7-day email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

[Download Your FREE Mini-Course](#)

Stacking Ensemble Learning

[Stacked Generalization](#), or stacking for short, is an ensemble method that seeks a diverse group of members by varying the model types fit on the training data and using a model to combine predictions.

“Stacking is a general procedure where a learner is trained to combine the individual learners. Here, the individual learners are called the first-level learners, while the combiner is called the second-level learner, or meta-learner.

— Page 83, [Ensemble Methods](#), 2012.

Stacking has its own nomenclature where ensemble members are referred to as level-0 models and the model that is used to combine the predictions is referred to as a level-1 model.

The two-level hierarchy of models is the most common approach, although more layers of models can be used. For example, instead of a single level-1 model, we might have 3 or 5 level-1 models and a single level-2 model that combines the predictions of level-1 models in order to make a prediction.

“Stacking is probably the most-popular meta-learning technique. By using a meta-learner, this method tries to induce which classifiers are reliable and which are not.

— Page 82, [Pattern Classification Using Ensemble Methods](#), 2010.

Any machine learning model can be used to aggregate the predictions, although it is common to use a linear model, such as linear regression for regression and logistic regression for binary classification. This encourages the complexity of the model to reside at the lower-level ensemble member models and simple models to learn how to harness the variety of predictions made.

“Using trainable combiners, it is possible to determine which classifiers are likely to be successful in which part of the feature space and combine them accordingly.

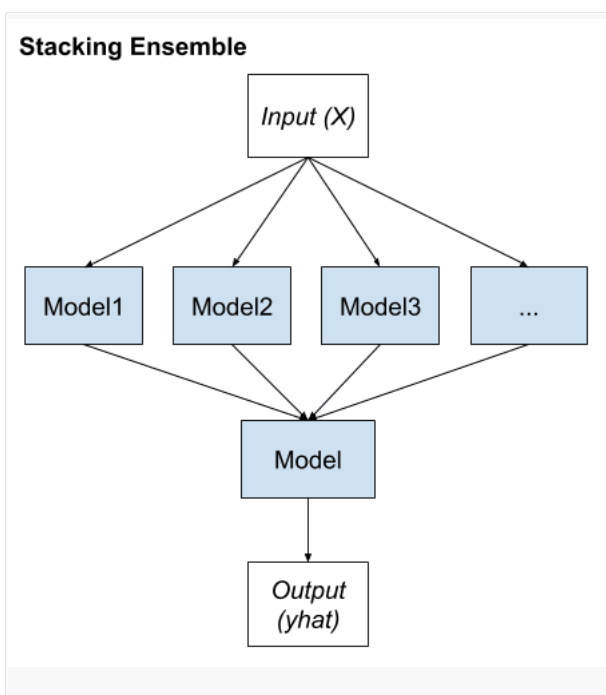
— Page 15, [Ensemble Machine Learning](#), 2012.

We can summarize the key elements of stacking as follows:

- Unchanged training dataset.
- Different machine learning algorithms for each ensemble member.
- Machine learning model to learn how to best combine predictions.

Diversity comes from the different machine learning models used as ensemble members.

As such, it is desirable to use a suite of models that are learned or constructed in very different ways, ensuring that they make different assumptions and, in turn, have less correlated prediction errors.



Many popular ensemble algorithms are based on this approach, including:

- Stacked Models (canonical stacking)
- Blending
- Super Ensemble

Next, let's take a closer look at boosting.

Boosting Ensemble Learning

Boosting is an ensemble method that seeks to change the training data to focus attention on examples that previous fit models on the training dataset have gotten wrong.



In boosting, [...] the training dataset for each subsequent classifier increasingly focuses on instances misclassified by previously generated classifiers.

— Page 13, [Ensemble Machine Learning](#), 2012.

The key property of boosting ensembles is the idea of correcting prediction errors. The models are fit and added to the ensemble sequentially such that the second model attempts to correct the predictions of the first model, the third corrects the second model, and so on.

This typically involves the use of very simple decision trees that only make a single or a few decisions, referred to in boosting as weak learners. The predictions of the weak learners are combined using simple voting or averaging, although the contributions are weighed proportional to their performance or capability. The objective is to develop a so-called “*strong-learner*” from many purpose-built “*weak-learners*.”



... an iterative approach for generating a strong classifier, one that is capable of achieving arbitrarily low training error, from an ensemble of weak classifiers, each of which can barely do better than random guessing.

— Page 13, [Ensemble Machine Learning](#), 2012.

Typically, the training dataset is left unchanged and instead, the learning algorithm is modified to pay more or less attention to specific examples (rows of data) based on whether they have been predicted correctly or incorrectly by previously added ensemble members. For example, the rows of data can be weighed to indicate the amount of focus a learning algorithm must give while learning the model.

We can summarize the key elements of boosting as follows:

- Bias training data toward those examples that are hard to predict.
- Iteratively add ensemble members to correct predictions of prior models.
- Combine predictions using a weighted average of models.

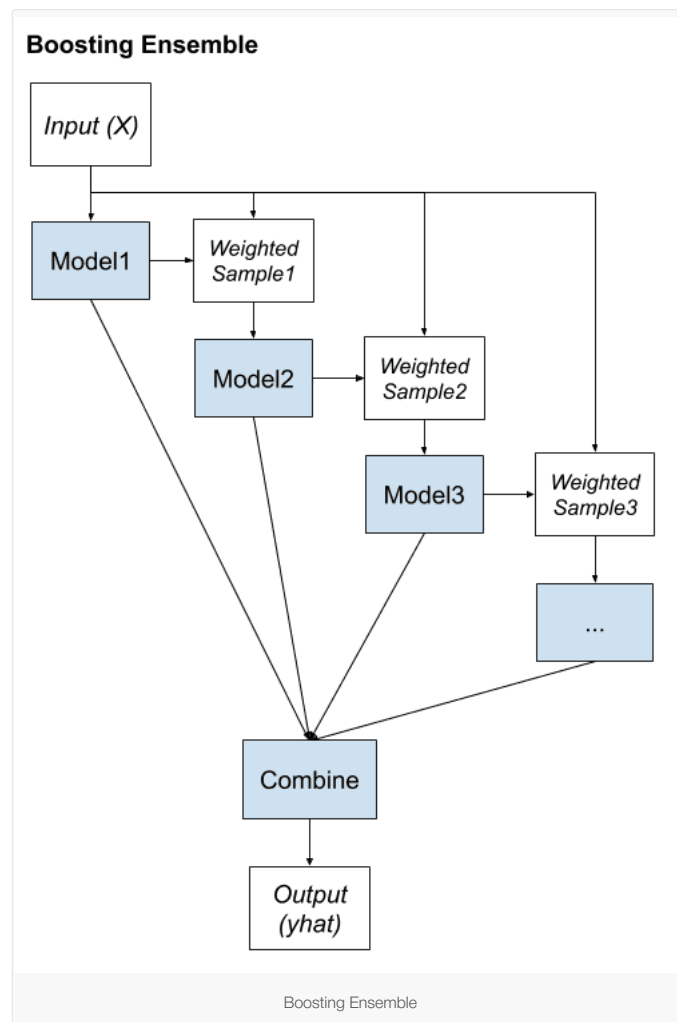
The idea of combining many weak learners into strong learners was first proposed theoretically and many algorithms were proposed with little success. It was not until the [Adaptive Boosting \(AdaBoost\)](#) algorithm was developed that boosting was demonstrated as an effective ensemble method.



The term boosting refers to a family of algorithms that are able to convert weak learners to strong learners.

— Page 23, [Ensemble Methods](#), 2012.

Since AdaBoost, many boosting methods have been developed and some, like stochastic gradient boosting, may be among the most effective techniques for classification and regression on tabular (structured) data.



To summarize, many popular ensemble algorithms are based on this approach, including:

- AdaBoost (canonical boosting)
- Gradient Boosting Machines
- Stochastic Gradient Boosting (XGBoost and similar)

This completes our tour of the standard ensemble learning techniques.

Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Books

- [Pattern Classification Using Ensemble Methods](#), 2010.
- [Ensemble Methods](#), 2012.
- [Ensemble Machine Learning](#), 2012.
- [Ensemble Methods in Data Mining](#), 2010.

Articles

- [Ensemble learning](#), Wikipedia.
- [Bootstrap aggregating](#), Wikipedia.
- [Boosting \(machine learning\)](#), Wikipedia.

Summary

In this tutorial, you discovered the three standard ensemble learning techniques for machine learning.

Specifically, you learned:

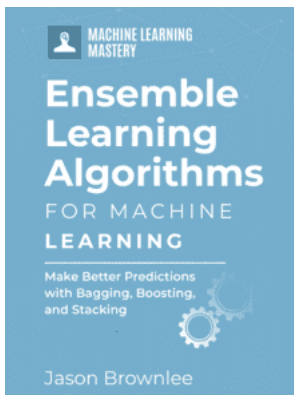
- Bagging involves fitting many decision trees on different samples of the same dataset and averaging the predictions.
- Stacking involves fitting many different models types on the same data and using another model to learn how to best combine the predictions.

- Boosting involves adding ensemble members sequentially that correct the predictions made by prior models and outputs a weighted average of the predictions.

Do you have any questions?

Ask your questions in the comments below and I will do my best to answer.

Get a Handle on Modern Ensemble Learning!



Improve Your Predictions in Minutes

...with just a few lines of python code

Discover how in my new Ebook:

[Ensemble Learning Algorithms With Python](#)

It provides **self-study tutorials** with **full working code** on:

Stacking, Voting, Boosting, Bagging, Blending, Super Learner, and much more...

Bring Modern Ensemble Learning Techniques to Your Machine Learning Projects

SEE WHAT'S INSIDE

[f Share](#) [X Post](#) [in Share](#)

More On This Topic



[How to Use Ensemble Machine Learning Algorithms in Weka](#)



[Ensemble Machine Learning Algorithms in Python with...](#)



[Bagging and Random Forest Ensemble Algorithms for...](#)



[How to Build an Ensemble Of Machine Learning Algorithms in R](#)



[Spot Check Machine Learning Algorithms in R...](#)



[A Gentle Introduction to Ensemble Diversity for...](#)



About Jason Brownlee

Jason Brownlee, PhD is a machine learning specialist who teaches developers how to get results with modern machine learning methods via hands-on tutorials.

[View all posts by Jason Brownlee →](#)

17 Responses to *A Gentle Introduction to Ensemble Learning Algorithms***Meharunnisa M** April 23, 2021 at 5:24 am #

REPLY ↩

Sir, can you show the code for same in R?

**Jason Brownlee** April 24, 2021 at 5:12 am #

REPLY ↩

Perhaps this will help:

<https://machinelearningmastery.com/machine-learning-ensembles-with-r/>

**Haleh** October 26, 2021 at 3:22 am #

REPLY ↩

Hi, Thanks for the explanation of each method. It was very helpful. I am a bit confused about weighted samples in boosting ensemble learning. Wouldn't giving weights to features based on their importance introduce noise to the system? It is like changing the training data. Thanks.

**Adrian Tam** October 27, 2021 at 2:44 am #

REPLY ↩

Yes, but instead of the noise, we are more concerned about the accuracy to the rare cases when there are imbalanced samples.

**Anas** December 1, 2021 at 4:13 am #

REPLY ↩

Great explanation as usual.. All methods talk about weak ensemble members. What about making having an ensemble learning of weak and strong algorithms. For instance, for a problem of image classification, a decision tree (weak) model to learn from meta data of the images and a CNN (strong) model to learn from the image dataset itself. Do you think these two could work together to give better results? And how do you think this could be implemented?

Thanks

**Adrian Tam** December 2, 2021 at 2:08 am #

REPLY ↩

I think you can try that. For implementation, I believe the existing ensemble library can be used, or otherwise you can write your own function for ensemble.

**Najam-ul-Saher** February 10, 2022 at 2:10 am #

REPLY ↩

what is the best approach in bagging or boosting?
bagging is better or boosting is better please tell me from exam point of view? which technique is better?

**James Carmichael** February 11, 2022 at 12:06 pm #

REPLY ↩

Hi Najum-ul-Saher...Thank you for your questions! While I cannot speak to your exam questions, I am most able to help you with specific questions regarding our materials and code listings.

**Julius** June 3, 2022 at 9:55 pm #

REPLY ↩

Thank you very much for this tutorial.
All the content is helpfull for me.
Many thanks again.

**James Carmichael** June 4, 2022 at 10:19 am #

REPLY ↩

You are very welcome Julius! We appreciate the support and feedback!



gndkup November 7, 2022 at 9:16 am #

REPLY ↩

Hello,
Which of these ensemble methods has majority voting?



James Carmichael November 7, 2022 at 10:07 am #

REPLY ↩

Hi gndkup...You may find the following of interest:
[https://machinelearningmastery.com/voting-ensembles-with-python/#:~:text=A%20voting%20ensemble%20\(or%20a,model%20used%20in%20the%20ensemble.](https://machinelearningmastery.com/voting-ensembles-with-python/#:~:text=A%20voting%20ensemble%20(or%20a,model%20used%20in%20the%20ensemble.)



Arowona Nurudeen March 6, 2023 at 9:57 pm #

REPLY ↩

Please how can I use the ensemble learning in R. Code needed please. Thanks



James Carmichael March 7, 2023 at 8:06 am #

REPLY ↩

Hi Arowona...Our content is primarily devoted to Python. The following resource is however available for R:
<https://machinelearningmastery.com/machine-learning-with-r/>



Noor April 2, 2023 at 1:46 am #

REPLY ↩

Hello thank you for the explanation.
In what class do Model Averaging and Classifier Voting fall into?
I'm quite puzzled as they are mentioned in other sites as ensemble strategies/methods but always out of the 3 categories you explained.
Thanx



James Carmichael April 2, 2023 at 6:21 am #

REPLY ↩

Hi Noor...You are very welcome! We consider them ensemble learning algorithms.
The following is a great starting point for this topic:
<https://machinelearningmastery.com/voting-ensembles-with-python/>



sabit_ayzal February 11, 2024 at 6:31 am #

REPLY ↩

weighted ensemble and weighted avg ensemble are not the same thing right? im little confused. i made a ensemble model combining 3 different cnn models, each multiplied by optimized weights that i fine-tuned by doing a grid search of weights. Then i sum the whole thing($model1*w1+model2*w2...$) and made the final ensemble model, not doing any averaging,, is this a dumb idea? ahaha ,,i wanted some models to contribute differently from the others.

Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT



Welcome!

I'm *Jason Brownlee* PhD

and I **help developers** get results with **machine learning**.

[Read more](#)

Never miss a tutorial:



Picked for you:



[How to Develop Multi-Output Regression Models with Python](#)



[How to Develop Super Learner Ensembles in Python](#)



[Stacking Ensemble Machine Learning With Python](#)



[How to Develop Voting Ensembles With Python](#)



[One-vs-Rest and One-vs-One for Multi-Class Classification](#)

Loving the Tutorials?

The [Ensemble Learning With Python](#) EBook
is where you'll find the ***Really Good*** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved

DEEP LEARNING FOR FINANCE

MODULE 6 | LESSON 2

APPLICATION OF AUTOENCODERS: EXTRACTING FEATURES FROM STOCK RETURNS

Reading Time	50 minutes
Prior Knowledge	Neural Networks, machine learning
Keywords	Autoencoder, Principal Components, Factors

1. INTRODUCTION

In this notebook, we will describe how to build a basic autoencoder model with TensorFlow using stock returns data. The exercise here will illustrate the tools that we described in Lesson 1 and the typical outputs generated by autoencoders to understand the variation that drives security prices. In Lesson 4, we will extend the analysis allowing an autoencoder model to become a tool for portfolio choice.

Let's first load some libraries needed in this notebook altogether with the data. We are going to analyze 20 tech stocks and their evolution over the 2017-2022 period. Using autoencoders, we will extract the common encodings, or latent "factors", that determine the variation of stock returns and how each stock is exposed to those factors.

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: # List of tickers (same order)
tickers = [
    "AAPL", "MSFT", "GOOG", "AMZN", "TSLA",
    "TCEHY", "NVDA", "TSM", "META", "BABA",
    "ASML", "AVGO", "ORCL", "CSCO", "ADBE",
    "CRM", "NFLX", "TXN", "QCOM", "SAP"
]

# Read CSV directly with Date as index
print("Loading data from CSV...")
data = pd.read_csv('adjusted_prices.csv', index_col='Date', parse_dates=True)

print(f"Data loaded: {data.shape}")
print(f"Complete range: {data.index[0].date()} to {data.index[-1].date()}")

# Calculate daily returns (pct_change) for all tickers
```

```
print("Calculating daily returns...")
returns_data = data[tickers].pct_change()

# Remove first row (NaN from pct_change)
df = returns_data.dropna()

df.head()
```

Loading data from CSV...

Data loaded: (1509, 20)

Complete range: 2019-01-02 to 2024-12-30

Calculating daily returns...

Out[2]:

	AAPL	MSFT	GOOG	AMZN	TSLA	TCEHY	NVDA	
Date								
2019-01-03 00:00:00-05:00	-0.099607	-0.036788	-0.028484	-0.025241	-0.031472	-0.028347	-0.060417	-0.010000
2019-01-04 00:00:00-05:00	0.042689	0.046509	0.053786	0.050064	0.057697	0.058088	0.064068	0.010000
2019-01-07 00:00:00-05:00	-0.002226	0.001275	-0.002167	0.034353	0.054361	0.006893	0.052940	0.010000
2019-01-08 00:00:00-05:00	0.019063	0.007251	0.007385	0.016612	0.001164	-0.003423	-0.024895	-0.010000
2019-01-09 00:00:00-05:00	0.016981	0.014300	-0.001505	0.001714	0.009483	0.038518	0.019667	0.010000

Use 80% of the data as a training sample (below we will use an additional 20% of the training data for validation purposes) and the remaining 20% belongs to the test part.

In [3]:

```
print(f"Final DataFrame: {df.shape}")
print("Tickers included:", df.columns.tolist())

# Train/test split (80/20)
train_size = int(0.8 * df.shape[0])
Y_train = df.iloc[:train_size + 1].to_numpy()
Y_test = df.iloc[train_size + 1:].to_numpy()
```

Final DataFrame: (1508, 20)

Tickers included: ['AAPL', 'MSFT', 'GOOG', 'AMZN', 'TSLA', 'TCEHY', 'NVDA', 'TSM', 'META', 'BABA', 'ASML', 'AVGO', 'ORCL', 'CSCO', 'ADBE', 'CRM', 'NFLX', 'TXN', 'QCOM', 'SAP']

In [4]:

```
print(f"\nBeginning of training sample: {df.index[0].date()}")
print(f"End of training sample: {df.index[train_size].date()}")
```

```
print(f"End of test sample: {df.index[-1].date()}")
print(f"Training samples: {Y_train.shape}")
print(f"Test samples: {Y_test.shape}")
```

Beginning of training sample: 2019-01-03

End of training sample: 2023-10-18

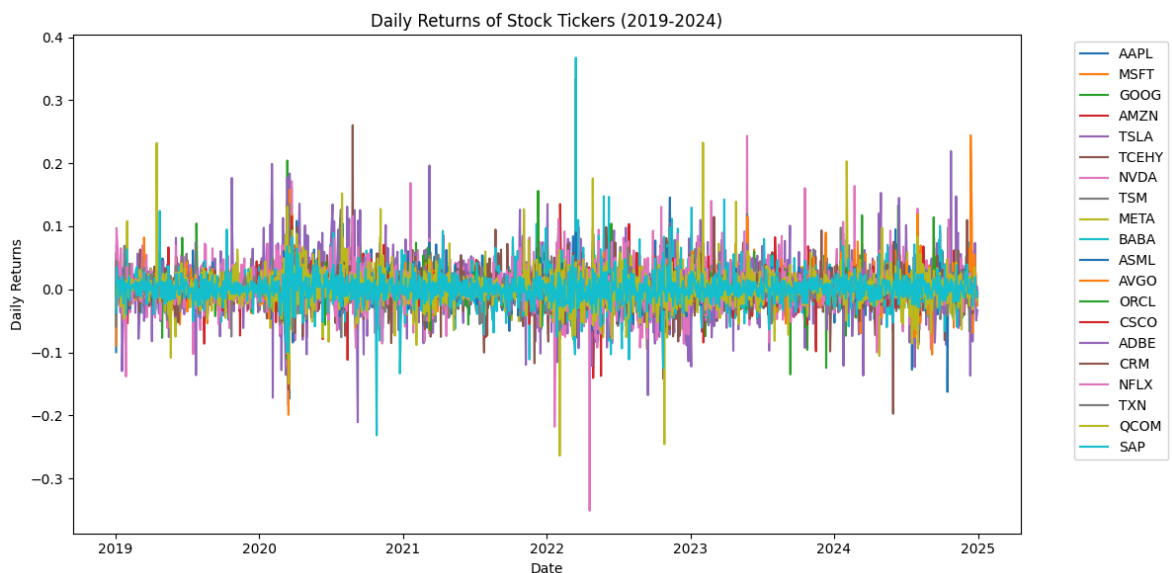
End of test sample: 2024-12-30

Training samples: (1207, 20)

Test samples: (301, 20)

In [5]: `import matplotlib.pyplot as plt`

```
plt.figure(figsize=(12, 6))
plt.plot(df.index, df)
plt.title(f'Daily Returns of Stock Tickers ({df.index[0].year}-{df.index[-1].year})')
plt.xlabel('Date')
plt.ylabel('Daily Returns')
plt.legend(df.columns, bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



2. APPLYING PRINCIPAL COMPONENT ANALYSIS (PCA)

Before building the autoencoder, we are going to perform principal component analysis (PCA) on the data. This analysis serves as a good benchmark when we later train autoencoders to learn the latent factors.

Remember that PCA consists on a linear transformation of the data to a low-dimensional space of factors that are orthogonal between each other. Below, we perform PCA on the data, extracting the first 3 principal components. The principal components are the columns of the matrix of eigenvectors from diagonalizing the variance-covariance matrix of the data. We have chosen the first 3 principal components to explain the stock return variation of 20 stocks, meaning that we obtain three 20x1 vectors of loadings. The loadings capture how much each single stock is "exposed" to the principal component.

We can also extract the proportion of the variation of the data explained by the first 3 principal components. See how the first principal component explains more than half of the variation in the data. The second and third components explain, respectively, roughly 10 percent and 5 percent of the variation.

The factors that drive stock returns and that are extracted from PCA represent latent variables that are sometimes hard to interpret or to give them a "name," particularly as we consider the second, third, and further components. However, when we analyze stock returns, the first principal component can be most of the time attributed to a "market factor" that generates positive comovement in returns across stocks. As we show, the loadings on this first factor are all negative across stocks (remember that the variance of $\$Z$ is the same as the variance of $\$-Z$). This confirms that the stock price of a single stock tends to move in the same direction as all the remaining stocks, and this explains more than 50% of the variation of returns.

```
In [6]: from sklearn.decomposition import PCA

pca = PCA(n_components=3)
pca.fit(Y_train)
print(pca.components_.T)
# Cumulative share of the variation of the data predicted the factors
print(np.cumsum(pca.explained_variance_ratio_))
```

```
[[-0.20332611 -0.08862559 -0.00286622]
 [-0.1996453  -0.13252331  0.01765695]
 [-0.19036866 -0.11370978  0.03904352]
 [-0.20425213 -0.06864972  0.00102689]
 [-0.3344536   0.47144555 -0.79461501]
 [-0.1901963   0.487918   0.37368002]
 [-0.3408636  -0.15417502 -0.02877177]
 [-0.20345294 -0.04641713  0.04437633]
 [-0.24227164 -0.11326835  0.10667999]
 [-0.21053143  0.57238145  0.44041048]
 [-0.26026589 -0.09684741  0.04523965]
 [-0.22736039 -0.12364523  0.00800747]
 [-0.13365969 -0.1512764   0.07292133]
 [-0.13219045 -0.14066919  0.05780975]
 [-0.23419577 -0.12478373  0.02581529]
 [-0.21978161 -0.08489683 -0.00770036]
 [-0.22835938  0.03082332  0.00750172]
 [-0.19340585 -0.13229862  0.05043233]
 [-0.24460667 -0.12143996  0.04582046]
 [-0.15779398 -0.0831969   0.05263593]]
[0.52145945 0.60819791 0.69199829]
```

Notice that the factors generated by PCA are indeed uncorrelated and that the loadings are orthonormal.

```
In [7]: # Factors (encodings) generated by PCA
pca_factors = np.dot(pca.components_, Y_train.T)
# Show that factors are uncorrelated
print(np.round(np.cov(pca_factors, rowvar=True), 5))
# Show that Loadings are orthonormal
print(np.round(np.dot(pca.components_, pca.components_.T), 5))
```

```
[[ 0.00672 -0.      -0.      ]
 [-0.      0.00112  0.      ]
 [-0.      0.      0.00108]]
[[ 1.  0. -0.]
 [ 0.  1. -0.]
 [-0. -0.  1.]]
```

An important remark is that the loadings and factors from PCA are not unique. Any linear, orthogonal transformation of the eigenvectors also maximizes the variance generated by the projections. Different software packages may generate different solutions. However, all the solutions should explain the same amount of variation in a given dataset.

Below we perform PCA extracting the first 6 principal components.

```
In [8]: pca = PCA(n_components=6)
pca.fit(Y_train)
print(pca.components_.T)
# Cumulative share of the variation of the data predicted the factors
print(np.cumsum(pca.explained_variance_ratio_))
```

```
[[-2.03326113e-01 -8.86305513e-02 -2.86892556e-03 -2.18050311e-03
  1.39588981e-01 1.28823724e-02]
 [-1.99645303e-01 -1.32516970e-01 1.76537624e-02 -4.81170301e-02
  1.71106411e-01 5.93128190e-02]
 [-1.90368657e-01 -1.13710192e-01 3.90304093e-02 -1.16007207e-01
  1.85887877e-01 -1.35038202e-01]
 [-2.04252134e-01 -6.86576446e-02 1.03641457e-03 -2.39601579e-01
  4.00280187e-02 -1.01450360e-01]
 [-3.34453600e-01 4.71445859e-01 -7.94615233e-01 4.42999102e-02
  1.25302888e-01 4.66791209e-04]
 [-1.90196303e-01 4.87912683e-01 3.73670019e-01 1.13861766e-01
  1.42086030e-02 5.28128873e-02]
 [-3.40863598e-01 -1.54182844e-01 -2.87800844e-02 1.77631264e-01
  -3.02636192e-01 -2.48178199e-01]
 [-2.03452942e-01 -4.64344566e-02 4.43868884e-02 2.80096037e-01
  -2.01393272e-01 -7.76834049e-02]
 [-2.42271644e-01 -1.13266798e-01 1.06681564e-01 -3.87717440e-01
  2.42011811e-01 -6.54256789e-01]
 [-2.10531430e-01 5.72387453e-01 4.40417989e-01 -6.95735330e-03
  9.04838690e-02 -1.43220865e-02]
 [-2.60265891e-01 -9.68224394e-02 4.52474096e-02 2.35928622e-01
  -1.61806236e-01 -2.68969329e-02]
 [-2.27360388e-01 -1.23630789e-01 8.01110306e-03 2.37829369e-01
  -5.97119217e-02 8.00429060e-02]
 [-1.33659691e-01 -1.51271071e-01 7.29213556e-02 1.08568379e-02
  2.54283356e-01 3.95803673e-01]
 [-1.32190446e-01 -1.40679185e-01 5.78071144e-02 7.91530861e-02
  2.17840845e-01 2.46379970e-01]
 [-2.34195770e-01 -1.24760700e-01 2.58375904e-02 -1.10667433e-01
  1.49822631e-01 1.56191581e-01]
 [-2.19781612e-01 -8.49051109e-02 -7.70505031e-03 -2.04613672e-01
  2.21465526e-01 2.42335076e-01]
 [-2.28359384e-01 3.08221802e-02 7.49818855e-03 -6.11911575e-01
  -6.34343821e-01 3.21201283e-01]
 [-1.93405853e-01 -1.32295528e-01 5.04350341e-02 1.88358479e-01
  -4.28439101e-02 6.10902382e-02]
 [-2.44606667e-01 -1.21448248e-01 4.58119833e-02 2.59948448e-01
  -2.13862509e-01 -8.99871562e-02]
 [-1.57793978e-01 -8.32232783e-02 5.26238628e-02 3.60058548e-02
  1.92232203e-01 2.15636056e-01]]
[0.52145945 0.60819791 0.69199829 0.7478485 0.78188589 0.81024463]
```

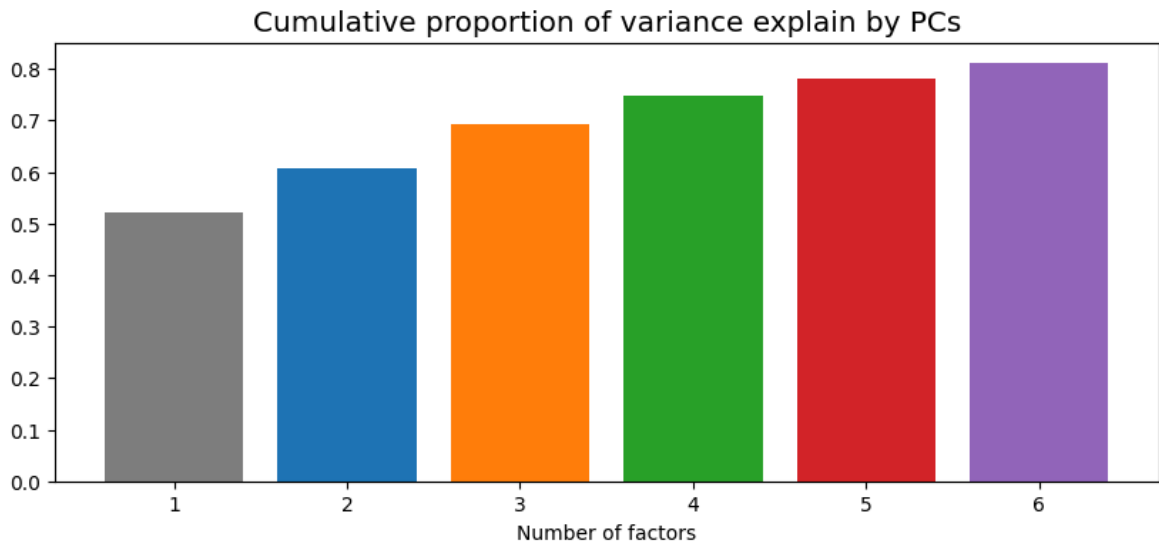
You can verify that the solution with 3 principal components is nested by the solution when we extract additional principal components. That is, the loadings on the first three components and the variation explained by them is the same as we obtained above.

```
In [9]: # plotting the coefficient score
fig, ax = plt.subplots(figsize=(10, 4))
color = [
    "tab:gray",
    "tab:blue",
    "tab:orange",
    "tab:green",
    "tab:red",
    "tab:purple",
    "tab:brown",
    "tab:pink",
    "tab:gray",
    "tab:olive",
```

```

    "tab:cyan",
    "tab:orange",
    "tab:green",
    "tab:blue",
    "tab:olive",
]
ax.bar(list(range(1, 7)), np.cumsum(pca.explained_variance_ratio_), color=color)
plt.style.use("ggplot")
plt.title("Cumulative proportion of variance explain by PCs")
plt.xlabel("Number of factors")
plt.show()

```



3. BUILDING A LINEAR AUTOENCODER WITH TENSORFLOW

Below we build a single-layer linear autoencoder with TensorFlow via [subclassing](#). The autoencoder has two Dense (fully connected) layers: an encoder, which compresses the data into a latent vector with a dimension set by "bottleneck_dim," and a decoder, which reconstructs the original input from the latent space.

In [10]: `import tensorflow as tf`

```

2025-05-26 10:46:01.846294: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-26 10:46:01.890793: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-05-26 10:46:03.404539: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.

```

In [11]: `class autoencoder(tf.keras.Model):`
 `def __init__(self, output_dim, bottleneck_dim=1):`
 `super(autoencoder, self).__init__()`

```

self.output_dim = output_dim
self.bottleneck_dim = bottleneck_dim
initializer = tf.keras.initializers.GlorotUniform()
# Build the encoder layer
self.encoder = tf.keras.Sequential(
    [
        tf.keras.layers.Dense(
            bottleneck_dim,
            kernel_initializer=initializer,
            bias_initializer=tf.keras.initializers.Zeros(),
        )
    ]
)
# Build the decoder layer
self.decoder = tf.keras.Sequential(
    [
        tf.keras.layers.Dense(
            output_dim,
            kernel_initializer=initializer,
            bias_initializer=tf.keras.initializers.Zeros(),
        )
    ]
)

def call(self, x):
    encoded = self.encoder(x)
    decoded = self.decoder(encoded)
    return decoded

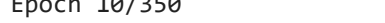
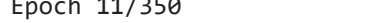
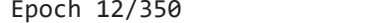
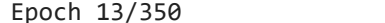
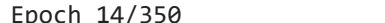
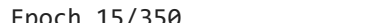
```

Below we compile and train the model using our training data. We choose a "bottleneck" with 3 latent factors. We train the model for at most 1,000 epochs, setting an early stopping criterion using a validation sample of 20% of all the training observations. The early stopping criterion may not bind, depending on the particular seed used in the training algorithm.

```

In [12]: # Choose the number of latent factors
nlatent = 3
# Choose the proportion of training data used for validation
val_split = 0.2
# Determine early stopping criterion
early_stop = tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=50)
# Choose optimizer
adam = tf.keras.optimizers.Adam(learning_rate=1e-4)
# Build and compile the model
model = autoencoder(Y_train.shape[1], nlatent)
model.compile(optimizer=adam, loss="mse")
# Train the model
model.fit(
    x=Y_train,
    y=Y_train,
    batch_size=32,
    epochs=350,
    validation_split=val_split,
    callbacks=[early_stop],
    verbose=1,
)

```

```
Epoch 1/350
31/31  1s 7ms/step - loss: 9.2919e-04 - val_loss: 7.5377e-04
Epoch 2/350
31/31  0s 4ms/step - loss: 9.0401e-04 - val_loss: 7.3931e-04
Epoch 3/350
31/31  0s 4ms/step - loss: 8.4248e-04 - val_loss: 7.2670e-04
Epoch 4/350
31/31  0s 4ms/step - loss: 8.6337e-04 - val_loss: 7.1399e-04
Epoch 5/350
31/31  0s 4ms/step - loss: 9.2274e-04 - val_loss: 7.0303e-04
Epoch 6/350
31/31  0s 4ms/step - loss: 8.8731e-04 - val_loss: 6.9276e-04
Epoch 7/350
31/31  0s 4ms/step - loss: 7.6780e-04 - val_loss: 6.8293e-04
Epoch 8/350
31/31  0s 3ms/step - loss: 8.1681e-04 - val_loss: 6.7424e-04
Epoch 9/350
31/31  0s 3ms/step - loss: 8.1545e-04 - val_loss: 6.6614e-04
Epoch 10/350
31/31  0s 3ms/step - loss: 8.1747e-04 - val_loss: 6.5837e-04
Epoch 11/350
31/31  0s 4ms/step - loss: 8.1108e-04 - val_loss: 6.5109e-04
Epoch 12/350
31/31  0s 4ms/step - loss: 8.1877e-04 - val_loss: 6.4440e-04
Epoch 13/350
31/31  0s 4ms/step - loss: 7.9767e-04 - val_loss: 6.3781e-04
Epoch 14/350
31/31  0s 4ms/step - loss: 7.8676e-04 - val_loss: 6.3212e-04
Epoch 15/350
31/31  0s 4ms/step - loss: 7.2650e-04 - val_loss: 6.2640e-04
Epoch 16/350
31/31  0s 4ms/step - loss: 8.4396e-04 - val_loss: 6.2102e-04
Epoch 17/350
31/31  0s 4ms/step - loss: 7.5019e-04 - val_loss: 6.1572e-04
Epoch 18/350
31/31  0s 3ms/step - loss: 7.1734e-04 - val_loss: 6.1096e-04
Epoch 19/350
31/31  0s 3ms/step - loss: 6.8568e-04 - val_loss: 6.0638e-04
Epoch 20/350
31/31  0s 4ms/step - loss: 7.2140e-04 - val_loss: 6.0150e-04
Epoch 21/350
31/31  0s 4ms/step - loss: 7.4381e-04 - val_loss: 5.9690e-04
Epoch 22/350
31/31  0s 4ms/step - loss: 6.6287e-04 - val_loss: 5.9267e-04
Epoch 23/350
31/31  0s 4ms/step - loss: 6.9261e-04 - val_loss: 5.8892e-04
Epoch 24/350
31/31  0s 4ms/step - loss: 6.9214e-04 - val_loss: 5.8469e-04
Epoch 25/350
31/31  0s 4ms/step - loss: 6.7579e-04 - val_loss: 5.8076e-04
Epoch 26/350
31/31  0s 4ms/step - loss: 6.8817e-04 - val_loss: 5.7696e-04
Epoch 27/350
31/31  0s 3ms/step - loss: 6.8490e-04 - val_loss: 5.7286e-04
Epoch 28/350
31/31  0s 3ms/step - loss: 6.1713e-04 - val_loss: 5.6921e-04
Epoch 29/350
31/31  0s 4ms/step - loss: 6.7855e-04 - val_loss: 5.6550e-04
Epoch 30/350
31/31  0s 4ms/step - loss: 6.6751e-04 - val_loss: 5.6142e-04
```

```
Epoch 31/350
31/31  0s 4ms/step - loss: 6.4237e-04 - val_loss: 5.5770e-04
Epoch 32/350
31/31  0s 4ms/step - loss: 6.4744e-04 - val_loss: 5.5383e-04
Epoch 33/350
31/31  0s 4ms/step - loss: 6.6856e-04 - val_loss: 5.5003e-04
Epoch 34/350
31/31  0s 4ms/step - loss: 6.3021e-04 - val_loss: 5.4650e-04
Epoch 35/350
31/31  0s 4ms/step - loss: 6.3122e-04 - val_loss: 5.4242e-04
Epoch 36/350
31/31  0s 4ms/step - loss: 6.4134e-04 - val_loss: 5.3859e-04
Epoch 37/350
31/31  0s 4ms/step - loss: 6.0461e-04 - val_loss: 5.3535e-04
Epoch 38/350
31/31  0s 4ms/step - loss: 6.1082e-04 - val_loss: 5.3173e-04
Epoch 39/350
31/31  0s 4ms/step - loss: 6.8435e-04 - val_loss: 5.2781e-04
Epoch 40/350
31/31  0s 4ms/step - loss: 5.9851e-04 - val_loss: 5.2407e-04
Epoch 41/350
31/31  0s 4ms/step - loss: 5.9480e-04 - val_loss: 5.2038e-04
Epoch 42/350
31/31  0s 4ms/step - loss: 6.1261e-04 - val_loss: 5.1671e-04
Epoch 43/350
31/31  0s 4ms/step - loss: 6.3842e-04 - val_loss: 5.1291e-04
Epoch 44/350
31/31  0s 4ms/step - loss: 5.7130e-04 - val_loss: 5.0945e-04
Epoch 45/350
31/31  0s 4ms/step - loss: 5.7881e-04 - val_loss: 5.0573e-04
Epoch 46/350
31/31  0s 3ms/step - loss: 6.1606e-04 - val_loss: 5.0204e-04
Epoch 47/350
31/31  0s 4ms/step - loss: 5.4292e-04 - val_loss: 4.9840e-04
Epoch 48/350
31/31  0s 4ms/step - loss: 5.8006e-04 - val_loss: 4.9462e-04
Epoch 49/350
31/31  0s 3ms/step - loss: 5.7763e-04 - val_loss: 4.9101e-04
Epoch 50/350
31/31  0s 3ms/step - loss: 5.4772e-04 - val_loss: 4.8748e-04
Epoch 51/350
31/31  0s 3ms/step - loss: 5.7466e-04 - val_loss: 4.8358e-04
Epoch 52/350
31/31  0s 4ms/step - loss: 5.4748e-04 - val_loss: 4.8021e-04
Epoch 53/350
31/31  0s 4ms/step - loss: 5.4614e-04 - val_loss: 4.7637e-04
Epoch 54/350
31/31  0s 4ms/step - loss: 5.3049e-04 - val_loss: 4.7280e-04
Epoch 55/350
31/31  0s 4ms/step - loss: 5.3070e-04 - val_loss: 4.6932e-04
Epoch 56/350
31/31  0s 4ms/step - loss: 5.0853e-04 - val_loss: 4.6543e-04
Epoch 57/350
31/31  0s 4ms/step - loss: 5.1897e-04 - val_loss: 4.6169e-04
Epoch 58/350
31/31  0s 4ms/step - loss: 5.4831e-04 - val_loss: 4.5808e-04
Epoch 59/350
31/31  0s 4ms/step - loss: 4.6257e-04 - val_loss: 4.5483e-04
Epoch 60/350
31/31  0s 4ms/step - loss: 5.2082e-04 - val_loss: 4.5117e-04
```

```
Epoch 61/350
31/31  0s 4ms/step - loss: 4.8605e-04 - val_loss: 4.4763e-04
Epoch 62/350
31/31  0s 4ms/step - loss: 5.0857e-04 - val_loss: 4.4419e-04
Epoch 63/350
31/31  0s 4ms/step - loss: 4.9395e-04 - val_loss: 4.4064e-04
Epoch 64/350
31/31  0s 5ms/step - loss: 5.0147e-04 - val_loss: 4.3713e-04
Epoch 65/350
31/31  0s 4ms/step - loss: 4.7816e-04 - val_loss: 4.3351e-04
Epoch 66/350
31/31  0s 4ms/step - loss: 5.0089e-04 - val_loss: 4.3027e-04
Epoch 67/350
31/31  0s 4ms/step - loss: 4.9325e-04 - val_loss: 4.2655e-04
Epoch 68/350
31/31  0s 4ms/step - loss: 4.9347e-04 - val_loss: 4.2344e-04
Epoch 69/350
31/31  0s 4ms/step - loss: 4.6400e-04 - val_loss: 4.1986e-04
Epoch 70/350
31/31  0s 4ms/step - loss: 4.6083e-04 - val_loss: 4.1635e-04
Epoch 71/350
31/31  0s 4ms/step - loss: 4.6817e-04 - val_loss: 4.1292e-04
Epoch 72/350
31/31  0s 4ms/step - loss: 4.4257e-04 - val_loss: 4.0943e-04
Epoch 73/350
31/31  0s 4ms/step - loss: 4.6016e-04 - val_loss: 4.0576e-04
Epoch 74/350
31/31  0s 4ms/step - loss: 4.4633e-04 - val_loss: 4.0281e-04
Epoch 75/350
31/31  0s 4ms/step - loss: 4.8629e-04 - val_loss: 3.9924e-04
Epoch 76/350
31/31  0s 4ms/step - loss: 4.8398e-04 - val_loss: 3.9635e-04
Epoch 77/350
31/31  0s 4ms/step - loss: 3.9727e-04 - val_loss: 3.9289e-04
Epoch 78/350
31/31  0s 3ms/step - loss: 4.1785e-04 - val_loss: 3.8952e-04
Epoch 79/350
31/31  0s 4ms/step - loss: 4.4408e-04 - val_loss: 3.8617e-04
Epoch 80/350
31/31  0s 4ms/step - loss: 4.3119e-04 - val_loss: 3.8283e-04
Epoch 81/350
31/31  0s 4ms/step - loss: 4.2323e-04 - val_loss: 3.7961e-04
Epoch 82/350
31/31  0s 4ms/step - loss: 4.1451e-04 - val_loss: 3.7675e-04
Epoch 83/350
31/31  0s 4ms/step - loss: 4.1517e-04 - val_loss: 3.7340e-04
Epoch 84/350
31/31  0s 4ms/step - loss: 4.0943e-04 - val_loss: 3.7014e-04
Epoch 85/350
31/31  0s 4ms/step - loss: 4.1974e-04 - val_loss: 3.6712e-04
Epoch 86/350
31/31  0s 4ms/step - loss: 4.2334e-04 - val_loss: 3.6435e-04
Epoch 87/350
31/31  0s 4ms/step - loss: 4.1801e-04 - val_loss: 3.6121e-04
Epoch 88/350
31/31  0s 4ms/step - loss: 3.9369e-04 - val_loss: 3.5826e-04
Epoch 89/350
31/31  0s 4ms/step - loss: 4.1828e-04 - val_loss: 3.5541e-04
Epoch 90/350
31/31  0s 4ms/step - loss: 4.2806e-04 - val_loss: 3.5233e-04
```

```
Epoch 91/350
31/31  0s 4ms/step - loss: 4.1618e-04 - val_loss: 3.4964e-04
Epoch 92/350
31/31  0s 4ms/step - loss: 3.9535e-04 - val_loss: 3.4657e-04
Epoch 93/350
31/31  0s 4ms/step - loss: 3.7432e-04 - val_loss: 3.4396e-04
Epoch 94/350
31/31  0s 4ms/step - loss: 3.7788e-04 - val_loss: 3.4136e-04
Epoch 95/350
31/31  0s 4ms/step - loss: 3.4525e-04 - val_loss: 3.3849e-04
Epoch 96/350
31/31  0s 4ms/step - loss: 3.8217e-04 - val_loss: 3.3589e-04
Epoch 97/350
31/31  0s 4ms/step - loss: 3.8639e-04 - val_loss: 3.3339e-04
Epoch 98/350
31/31  0s 4ms/step - loss: 3.7903e-04 - val_loss: 3.3091e-04
Epoch 99/350
31/31  0s 4ms/step - loss: 3.6542e-04 - val_loss: 3.2864e-04
Epoch 100/350
31/31  0s 4ms/step - loss: 3.3699e-04 - val_loss: 3.2649e-04
Epoch 101/350
31/31  0s 4ms/step - loss: 3.5413e-04 - val_loss: 3.2395e-04
Epoch 102/350
31/31  0s 4ms/step - loss: 3.5483e-04 - val_loss: 3.2177e-04
Epoch 103/350
31/31  0s 5ms/step - loss: 3.4037e-04 - val_loss: 3.1961e-04
Epoch 104/350
31/31  0s 4ms/step - loss: 3.2986e-04 - val_loss: 3.1754e-04
Epoch 105/350
31/31  0s 4ms/step - loss: 3.3308e-04 - val_loss: 3.1543e-04
Epoch 106/350
31/31  0s 4ms/step - loss: 3.4204e-04 - val_loss: 3.1346e-04
Epoch 107/350
31/31  0s 4ms/step - loss: 3.4981e-04 - val_loss: 3.1108e-04
Epoch 108/350
31/31  0s 4ms/step - loss: 3.3718e-04 - val_loss: 3.0934e-04
Epoch 109/350
31/31  0s 4ms/step - loss: 3.4511e-04 - val_loss: 3.0761e-04
Epoch 110/350
31/31  0s 4ms/step - loss: 3.1178e-04 - val_loss: 3.0590e-04
Epoch 111/350
31/31  0s 4ms/step - loss: 3.1879e-04 - val_loss: 3.0442e-04
Epoch 112/350
31/31  0s 3ms/step - loss: 3.2512e-04 - val_loss: 3.0249e-04
Epoch 113/350
31/31  0s 3ms/step - loss: 3.2146e-04 - val_loss: 3.0099e-04
Epoch 114/350
31/31  0s 4ms/step - loss: 3.2598e-04 - val_loss: 2.9937e-04
Epoch 115/350
31/31  0s 4ms/step - loss: 3.2396e-04 - val_loss: 2.9784e-04
Epoch 116/350
31/31  0s 4ms/step - loss: 3.1402e-04 - val_loss: 2.9641e-04
Epoch 117/350
31/31  0s 3ms/step - loss: 3.0830e-04 - val_loss: 2.9478e-04
Epoch 118/350
31/31  0s 4ms/step - loss: 2.9192e-04 - val_loss: 2.9323e-04
Epoch 119/350
31/31  0s 4ms/step - loss: 3.0044e-04 - val_loss: 2.9223e-04
Epoch 120/350
31/31  0s 4ms/step - loss: 3.2041e-04 - val_loss: 2.9080e-04
```

```
Epoch 121/350
31/31  0s 4ms/step - loss: 2.9978e-04 - val_loss: 2.8943e-04
Epoch 122/350
31/31  0s 4ms/step - loss: 3.0148e-04 - val_loss: 2.8807e-04
Epoch 123/350
31/31  0s 4ms/step - loss: 3.0802e-04 - val_loss: 2.8688e-04
Epoch 124/350
31/31  0s 4ms/step - loss: 3.3759e-04 - val_loss: 2.8565e-04
Epoch 125/350
31/31  0s 4ms/step - loss: 3.0873e-04 - val_loss: 2.8480e-04
Epoch 126/350
31/31  0s 4ms/step - loss: 3.0583e-04 - val_loss: 2.8332e-04
Epoch 127/350
31/31  0s 4ms/step - loss: 2.9842e-04 - val_loss: 2.8229e-04
Epoch 128/350
31/31  0s 4ms/step - loss: 3.1516e-04 - val_loss: 2.8133e-04
Epoch 129/350
31/31  0s 4ms/step - loss: 2.9283e-04 - val_loss: 2.8027e-04
Epoch 130/350
31/31  0s 3ms/step - loss: 2.9061e-04 - val_loss: 2.7938e-04
Epoch 131/350
31/31  0s 4ms/step - loss: 3.0848e-04 - val_loss: 2.7841e-04
Epoch 132/350
31/31  0s 4ms/step - loss: 2.9066e-04 - val_loss: 2.7748e-04
Epoch 133/350
31/31  0s 3ms/step - loss: 2.8932e-04 - val_loss: 2.7647e-04
Epoch 134/350
31/31  0s 4ms/step - loss: 2.9497e-04 - val_loss: 2.7555e-04
Epoch 135/350
31/31  0s 4ms/step - loss: 2.9395e-04 - val_loss: 2.7474e-04
Epoch 136/350
31/31  0s 4ms/step - loss: 3.1819e-04 - val_loss: 2.7409e-04
Epoch 137/350
31/31  0s 4ms/step - loss: 2.8042e-04 - val_loss: 2.7303e-04
Epoch 138/350
31/31  0s 5ms/step - loss: 2.9634e-04 - val_loss: 2.7220e-04
Epoch 139/350
31/31  0s 3ms/step - loss: 2.7763e-04 - val_loss: 2.7167e-04
Epoch 140/350
31/31  0s 4ms/step - loss: 2.9456e-04 - val_loss: 2.7072e-04
Epoch 141/350
31/31  0s 4ms/step - loss: 3.1669e-04 - val_loss: 2.6984e-04
Epoch 142/350
31/31  0s 4ms/step - loss: 3.0520e-04 - val_loss: 2.6929e-04
Epoch 143/350
31/31  0s 4ms/step - loss: 2.7530e-04 - val_loss: 2.6838e-04
Epoch 144/350
31/31  0s 4ms/step - loss: 2.6872e-04 - val_loss: 2.6786e-04
Epoch 145/350
31/31  0s 3ms/step - loss: 2.6615e-04 - val_loss: 2.6707e-04
Epoch 146/350
31/31  0s 4ms/step - loss: 2.7653e-04 - val_loss: 2.6634e-04
Epoch 147/350
31/31  0s 3ms/step - loss: 2.7918e-04 - val_loss: 2.6569e-04
Epoch 148/350
31/31  0s 4ms/step - loss: 2.8134e-04 - val_loss: 2.6496e-04
Epoch 149/350
31/31  0s 4ms/step - loss: 2.7893e-04 - val_loss: 2.6437e-04
Epoch 150/350
31/31  0s 3ms/step - loss: 2.9071e-04 - val_loss: 2.6380e-04
```

```
Epoch 151/350
31/31  0s 4ms/step - loss: 2.7412e-04 - val_loss: 2.6298e-04
Epoch 152/350
31/31  0s 4ms/step - loss: 2.8317e-04 - val_loss: 2.6229e-04
Epoch 153/350
31/31  0s 4ms/step - loss: 2.7402e-04 - val_loss: 2.6170e-04
Epoch 154/350
31/31  0s 4ms/step - loss: 2.9113e-04 - val_loss: 2.6110e-04
Epoch 155/350
31/31  0s 4ms/step - loss: 2.8000e-04 - val_loss: 2.6029e-04
Epoch 156/350
31/31  0s 4ms/step - loss: 2.7153e-04 - val_loss: 2.5981e-04
Epoch 157/350
31/31  0s 4ms/step - loss: 2.6143e-04 - val_loss: 2.5924e-04
Epoch 158/350
31/31  0s 4ms/step - loss: 2.7795e-04 - val_loss: 2.5843e-04
Epoch 159/350
31/31  0s 3ms/step - loss: 2.9790e-04 - val_loss: 2.5786e-04
Epoch 160/350
31/31  0s 4ms/step - loss: 2.7053e-04 - val_loss: 2.5743e-04
Epoch 161/350
31/31  0s 3ms/step - loss: 2.7532e-04 - val_loss: 2.5680e-04
Epoch 162/350
31/31  0s 3ms/step - loss: 2.6738e-04 - val_loss: 2.5611e-04
Epoch 163/350
31/31  0s 4ms/step - loss: 2.9914e-04 - val_loss: 2.5548e-04
Epoch 164/350
31/31  0s 4ms/step - loss: 2.7534e-04 - val_loss: 2.5510e-04
Epoch 165/350
31/31  0s 4ms/step - loss: 2.6316e-04 - val_loss: 2.5431e-04
Epoch 166/350
31/31  0s 4ms/step - loss: 2.7122e-04 - val_loss: 2.5388e-04
Epoch 167/350
31/31  0s 3ms/step - loss: 2.6221e-04 - val_loss: 2.5332e-04
Epoch 168/350
31/31  0s 4ms/step - loss: 2.5142e-04 - val_loss: 2.5279e-04
Epoch 169/350
31/31  0s 4ms/step - loss: 2.6817e-04 - val_loss: 2.5208e-04
Epoch 170/350
31/31  0s 4ms/step - loss: 2.7834e-04 - val_loss: 2.5152e-04
Epoch 171/350
31/31  0s 4ms/step - loss: 2.5123e-04 - val_loss: 2.5122e-04
Epoch 172/350
31/31  0s 3ms/step - loss: 2.6014e-04 - val_loss: 2.5049e-04
Epoch 173/350
31/31  0s 4ms/step - loss: 2.5680e-04 - val_loss: 2.4998e-04
Epoch 174/350
31/31  0s 4ms/step - loss: 2.8767e-04 - val_loss: 2.4915e-04
Epoch 175/350
31/31  0s 4ms/step - loss: 2.5986e-04 - val_loss: 2.4899e-04
Epoch 176/350
31/31  0s 4ms/step - loss: 2.6431e-04 - val_loss: 2.4853e-04
Epoch 177/350
31/31  0s 3ms/step - loss: 2.6315e-04 - val_loss: 2.4809e-04
Epoch 178/350
31/31  0s 4ms/step - loss: 2.6235e-04 - val_loss: 2.4774e-04
Epoch 179/350
31/31  0s 4ms/step - loss: 2.7654e-04 - val_loss: 2.4686e-04
Epoch 180/350
31/31  0s 4ms/step - loss: 2.3677e-04 - val_loss: 2.4625e-04
```

```
Epoch 181/350
31/31  0s 4ms/step - loss: 2.4118e-04 - val_loss: 2.4573e-04
Epoch 182/350
31/31  0s 4ms/step - loss: 2.6377e-04 - val_loss: 2.4525e-04
Epoch 183/350
31/31  0s 4ms/step - loss: 2.6628e-04 - val_loss: 2.4480e-04
Epoch 184/350
31/31  0s 4ms/step - loss: 2.7813e-04 - val_loss: 2.4446e-04
Epoch 185/350
31/31  0s 4ms/step - loss: 2.5880e-04 - val_loss: 2.4380e-04
Epoch 186/350
31/31  0s 3ms/step - loss: 2.5003e-04 - val_loss: 2.4333e-04
Epoch 187/350
31/31  0s 3ms/step - loss: 2.6732e-04 - val_loss: 2.4270e-04
Epoch 188/350
31/31  0s 4ms/step - loss: 2.5005e-04 - val_loss: 2.4226e-04
Epoch 189/350
31/31  0s 3ms/step - loss: 2.6316e-04 - val_loss: 2.4163e-04
Epoch 190/350
31/31  0s 4ms/step - loss: 2.5228e-04 - val_loss: 2.4113e-04
Epoch 191/350
31/31  0s 4ms/step - loss: 2.4807e-04 - val_loss: 2.4077e-04
Epoch 192/350
31/31  0s 4ms/step - loss: 2.5585e-04 - val_loss: 2.4019e-04
Epoch 193/350
31/31  0s 4ms/step - loss: 2.6429e-04 - val_loss: 2.3985e-04
Epoch 194/350
31/31  0s 4ms/step - loss: 2.3686e-04 - val_loss: 2.3929e-04
Epoch 195/350
31/31  0s 4ms/step - loss: 2.5530e-04 - val_loss: 2.3867e-04
Epoch 196/350
31/31  0s 4ms/step - loss: 2.5364e-04 - val_loss: 2.3842e-04
Epoch 197/350
31/31  0s 4ms/step - loss: 2.4861e-04 - val_loss: 2.3746e-04
Epoch 198/350
31/31  0s 4ms/step - loss: 2.4805e-04 - val_loss: 2.3743e-04
Epoch 199/350
31/31  0s 4ms/step - loss: 2.5497e-04 - val_loss: 2.3679e-04
Epoch 200/350
31/31  0s 4ms/step - loss: 2.4203e-04 - val_loss: 2.3681e-04
Epoch 201/350
31/31  0s 3ms/step - loss: 2.5512e-04 - val_loss: 2.3597e-04
Epoch 202/350
31/31  0s 5ms/step - loss: 2.4466e-04 - val_loss: 2.3555e-04
Epoch 203/350
31/31  0s 3ms/step - loss: 2.6862e-04 - val_loss: 2.3504e-04
Epoch 204/350
31/31  0s 3ms/step - loss: 2.4463e-04 - val_loss: 2.3458e-04
Epoch 205/350
31/31  0s 3ms/step - loss: 2.4368e-04 - val_loss: 2.3417e-04
Epoch 206/350
31/31  0s 4ms/step - loss: 2.5015e-04 - val_loss: 2.3349e-04
Epoch 207/350
31/31  0s 4ms/step - loss: 2.5718e-04 - val_loss: 2.3324e-04
Epoch 208/350
31/31  0s 4ms/step - loss: 2.6617e-04 - val_loss: 2.3274e-04
Epoch 209/350
31/31  0s 4ms/step - loss: 2.3474e-04 - val_loss: 2.3237e-04
Epoch 210/350
31/31  0s 4ms/step - loss: 2.2944e-04 - val_loss: 2.3184e-04
```

```
Epoch 211/350
31/31  0s 4ms/step - loss: 2.4875e-04 - val_loss: 2.3159e-04
Epoch 212/350
31/31  0s 4ms/step - loss: 2.2187e-04 - val_loss: 2.3111e-04
Epoch 213/350
31/31  0s 4ms/step - loss: 2.5539e-04 - val_loss: 2.3026e-04
Epoch 214/350
31/31  0s 3ms/step - loss: 2.3744e-04 - val_loss: 2.3036e-04
Epoch 215/350
31/31  0s 4ms/step - loss: 2.3950e-04 - val_loss: 2.2968e-04
Epoch 216/350
31/31  0s 4ms/step - loss: 2.2768e-04 - val_loss: 2.2915e-04
Epoch 217/350
31/31  0s 4ms/step - loss: 2.4342e-04 - val_loss: 2.2868e-04
Epoch 218/350
31/31  0s 4ms/step - loss: 2.5912e-04 - val_loss: 2.2843e-04
Epoch 219/350
31/31  0s 4ms/step - loss: 2.3217e-04 - val_loss: 2.2813e-04
Epoch 220/350
31/31  0s 4ms/step - loss: 2.3110e-04 - val_loss: 2.2767e-04
Epoch 221/350
31/31  0s 4ms/step - loss: 2.4011e-04 - val_loss: 2.2723e-04
Epoch 222/350
31/31  0s 4ms/step - loss: 2.5601e-04 - val_loss: 2.2669e-04
Epoch 223/350
31/31  0s 5ms/step - loss: 2.2960e-04 - val_loss: 2.2627e-04
Epoch 224/350
31/31  0s 5ms/step - loss: 2.2889e-04 - val_loss: 2.2597e-04
Epoch 225/350
31/31  0s 5ms/step - loss: 2.3126e-04 - val_loss: 2.2532e-04
Epoch 226/350
31/31  0s 4ms/step - loss: 2.4700e-04 - val_loss: 2.2498e-04
Epoch 227/350
31/31  0s 3ms/step - loss: 2.4021e-04 - val_loss: 2.2449e-04
Epoch 228/350
31/31  0s 3ms/step - loss: 2.4193e-04 - val_loss: 2.2434e-04
Epoch 229/350
31/31  0s 3ms/step - loss: 2.3008e-04 - val_loss: 2.2408e-04
Epoch 230/350
31/31  0s 5ms/step - loss: 2.2903e-04 - val_loss: 2.2374e-04
Epoch 231/350
31/31  0s 3ms/step - loss: 2.1749e-04 - val_loss: 2.2329e-04
Epoch 232/350
31/31  0s 4ms/step - loss: 2.1405e-04 - val_loss: 2.2272e-04
Epoch 233/350
31/31  0s 4ms/step - loss: 2.2114e-04 - val_loss: 2.2250e-04
Epoch 234/350
31/31  0s 4ms/step - loss: 2.1744e-04 - val_loss: 2.2224e-04
Epoch 235/350
31/31  0s 4ms/step - loss: 2.1687e-04 - val_loss: 2.2182e-04
Epoch 236/350
31/31  0s 4ms/step - loss: 2.3701e-04 - val_loss: 2.2141e-04
Epoch 237/350
31/31  0s 3ms/step - loss: 2.4453e-04 - val_loss: 2.2126e-04
Epoch 238/350
31/31  0s 4ms/step - loss: 2.3995e-04 - val_loss: 2.2090e-04
Epoch 239/350
31/31  0s 4ms/step - loss: 2.2271e-04 - val_loss: 2.2052e-04
Epoch 240/350
31/31  0s 4ms/step - loss: 2.2968e-04 - val_loss: 2.2010e-04
```

```
Epoch 241/350
31/31  0s 4ms/step - loss: 2.3475e-04 - val_loss: 2.1975e-04
Epoch 242/350
31/31  0s 4ms/step - loss: 2.2368e-04 - val_loss: 2.1946e-04
Epoch 243/350
31/31  0s 4ms/step - loss: 2.2646e-04 - val_loss: 2.1929e-04
Epoch 244/350
31/31  0s 4ms/step - loss: 2.3054e-04 - val_loss: 2.1867e-04
Epoch 245/350
31/31  0s 4ms/step - loss: 2.1360e-04 - val_loss: 2.1846e-04
Epoch 246/350
31/31  0s 4ms/step - loss: 2.2103e-04 - val_loss: 2.1804e-04
Epoch 247/350
31/31  0s 4ms/step - loss: 2.3626e-04 - val_loss: 2.1771e-04
Epoch 248/350
31/31  0s 4ms/step - loss: 2.2420e-04 - val_loss: 2.1752e-04
Epoch 249/350
31/31  0s 4ms/step - loss: 2.2959e-04 - val_loss: 2.1734e-04
Epoch 250/350
31/31  0s 4ms/step - loss: 2.2242e-04 - val_loss: 2.1683e-04
Epoch 251/350
31/31  0s 4ms/step - loss: 2.2956e-04 - val_loss: 2.1673e-04
Epoch 252/350
31/31  0s 4ms/step - loss: 2.3320e-04 - val_loss: 2.1610e-04
Epoch 253/350
31/31  0s 4ms/step - loss: 2.2083e-04 - val_loss: 2.1599e-04
Epoch 254/350
31/31  0s 3ms/step - loss: 2.2383e-04 - val_loss: 2.1567e-04
Epoch 255/350
31/31  0s 4ms/step - loss: 2.3467e-04 - val_loss: 2.1551e-04
Epoch 256/350
31/31  0s 4ms/step - loss: 2.2593e-04 - val_loss: 2.1503e-04
Epoch 257/350
31/31  0s 4ms/step - loss: 2.1063e-04 - val_loss: 2.1464e-04
Epoch 258/350
31/31  0s 4ms/step - loss: 2.2670e-04 - val_loss: 2.1439e-04
Epoch 259/350
31/31  0s 3ms/step - loss: 2.1563e-04 - val_loss: 2.1422e-04
Epoch 260/350
31/31  0s 4ms/step - loss: 2.1897e-04 - val_loss: 2.1398e-04
Epoch 261/350
31/31  0s 4ms/step - loss: 2.4046e-04 - val_loss: 2.1374e-04
Epoch 262/350
31/31  0s 4ms/step - loss: 2.2263e-04 - val_loss: 2.1334e-04
Epoch 263/350
31/31  0s 4ms/step - loss: 2.1588e-04 - val_loss: 2.1305e-04
Epoch 264/350
31/31  0s 4ms/step - loss: 2.3484e-04 - val_loss: 2.1292e-04
Epoch 265/350
31/31  0s 4ms/step - loss: 2.1303e-04 - val_loss: 2.1268e-04
Epoch 266/350
31/31  0s 3ms/step - loss: 2.2309e-04 - val_loss: 2.1264e-04
Epoch 267/350
31/31  0s 5ms/step - loss: 2.1479e-04 - val_loss: 2.1212e-04
Epoch 268/350
31/31  0s 4ms/step - loss: 2.3818e-04 - val_loss: 2.1205e-04
Epoch 269/350
31/31  0s 4ms/step - loss: 2.2981e-04 - val_loss: 2.1164e-04
Epoch 270/350
31/31  0s 4ms/step - loss: 2.3156e-04 - val_loss: 2.1150e-04
```

```
Epoch 271/350
31/31  0s 4ms/step - loss: 2.3433e-04 - val_loss: 2.1120e-04
Epoch 272/350
31/31  0s 4ms/step - loss: 2.1463e-04 - val_loss: 2.1084e-04
Epoch 273/350
31/31  0s 4ms/step - loss: 2.1969e-04 - val_loss: 2.1084e-04
Epoch 274/350
31/31  0s 3ms/step - loss: 2.2105e-04 - val_loss: 2.1052e-04
Epoch 275/350
31/31  0s 4ms/step - loss: 2.0029e-04 - val_loss: 2.1020e-04
Epoch 276/350
31/31  0s 4ms/step - loss: 2.2881e-04 - val_loss: 2.1013e-04
Epoch 277/350
31/31  0s 4ms/step - loss: 2.2436e-04 - val_loss: 2.0973e-04
Epoch 278/350
31/31  0s 4ms/step - loss: 2.0763e-04 - val_loss: 2.0952e-04
Epoch 279/350
31/31  0s 4ms/step - loss: 2.1689e-04 - val_loss: 2.0933e-04
Epoch 280/350
31/31  0s 5ms/step - loss: 2.0525e-04 - val_loss: 2.0914e-04
Epoch 281/350
31/31  0s 3ms/step - loss: 2.0969e-04 - val_loss: 2.0886e-04
Epoch 282/350
31/31  0s 4ms/step - loss: 2.2953e-04 - val_loss: 2.0877e-04
Epoch 283/350
31/31  0s 4ms/step - loss: 2.1767e-04 - val_loss: 2.0853e-04
Epoch 284/350
31/31  0s 4ms/step - loss: 2.1847e-04 - val_loss: 2.0819e-04
Epoch 285/350
31/31  0s 4ms/step - loss: 2.1546e-04 - val_loss: 2.0811e-04
Epoch 286/350
31/31  0s 4ms/step - loss: 2.1574e-04 - val_loss: 2.0793e-04
Epoch 287/350
31/31  0s 4ms/step - loss: 2.0963e-04 - val_loss: 2.0764e-04
Epoch 288/350
31/31  0s 4ms/step - loss: 2.2068e-04 - val_loss: 2.0757e-04
Epoch 289/350
31/31  0s 4ms/step - loss: 1.9897e-04 - val_loss: 2.0738e-04
Epoch 290/350
31/31  0s 4ms/step - loss: 2.1572e-04 - val_loss: 2.0719e-04
Epoch 291/350
31/31  0s 4ms/step - loss: 2.1309e-04 - val_loss: 2.0697e-04
Epoch 292/350
31/31  0s 4ms/step - loss: 2.1309e-04 - val_loss: 2.0680e-04
Epoch 293/350
31/31  0s 3ms/step - loss: 2.2720e-04 - val_loss: 2.0671e-04
Epoch 294/350
31/31  0s 3ms/step - loss: 2.0990e-04 - val_loss: 2.0659e-04
Epoch 295/350
31/31  0s 4ms/step - loss: 2.0501e-04 - val_loss: 2.0619e-04
Epoch 296/350
31/31  0s 4ms/step - loss: 2.1179e-04 - val_loss: 2.0606e-04
Epoch 297/350
31/31  0s 4ms/step - loss: 2.1826e-04 - val_loss: 2.0585e-04
Epoch 298/350
31/31  0s 4ms/step - loss: 1.9870e-04 - val_loss: 2.0593e-04
Epoch 299/350
31/31  0s 4ms/step - loss: 2.2780e-04 - val_loss: 2.0575e-04
Epoch 300/350
31/31  0s 4ms/step - loss: 2.0097e-04 - val_loss: 2.0546e-04
```

```
Epoch 301/350
31/31  0s 4ms/step - loss: 2.2071e-04 - val_loss: 2.0521e-04
Epoch 302/350
31/31  0s 3ms/step - loss: 1.9862e-04 - val_loss: 2.0519e-04
Epoch 303/350
31/31  0s 4ms/step - loss: 2.0336e-04 - val_loss: 2.0494e-04
Epoch 304/350
31/31  0s 4ms/step - loss: 2.1712e-04 - val_loss: 2.0492e-04
Epoch 305/350
31/31  0s 4ms/step - loss: 2.0353e-04 - val_loss: 2.0463e-04
Epoch 306/350
31/31  0s 3ms/step - loss: 2.1409e-04 - val_loss: 2.0450e-04
Epoch 307/350
31/31  0s 4ms/step - loss: 2.1249e-04 - val_loss: 2.0449e-04
Epoch 308/350
31/31  0s 4ms/step - loss: 2.1514e-04 - val_loss: 2.0421e-04
Epoch 309/350
31/31  0s 4ms/step - loss: 2.1128e-04 - val_loss: 2.0394e-04
Epoch 310/350
31/31  0s 4ms/step - loss: 1.9921e-04 - val_loss: 2.0393e-04
Epoch 311/350
31/31  0s 4ms/step - loss: 2.2008e-04 - val_loss: 2.0366e-04
Epoch 312/350
31/31  0s 3ms/step - loss: 2.0216e-04 - val_loss: 2.0357e-04
Epoch 313/350
31/31  0s 4ms/step - loss: 2.0661e-04 - val_loss: 2.0337e-04
Epoch 314/350
31/31  0s 4ms/step - loss: 2.2240e-04 - val_loss: 2.0343e-04
Epoch 315/350
31/31  0s 3ms/step - loss: 2.0416e-04 - val_loss: 2.0314e-04
Epoch 316/350
31/31  0s 3ms/step - loss: 2.1321e-04 - val_loss: 2.0311e-04
Epoch 317/350
31/31  0s 4ms/step - loss: 2.1170e-04 - val_loss: 2.0304e-04
Epoch 318/350
31/31  0s 4ms/step - loss: 2.0606e-04 - val_loss: 2.0284e-04
Epoch 319/350
31/31  0s 4ms/step - loss: 2.1507e-04 - val_loss: 2.0258e-04
Epoch 320/350
31/31  0s 4ms/step - loss: 1.9794e-04 - val_loss: 2.0248e-04
Epoch 321/350
31/31  0s 4ms/step - loss: 2.0619e-04 - val_loss: 2.0238e-04
Epoch 322/350
31/31  0s 3ms/step - loss: 2.2175e-04 - val_loss: 2.0225e-04
Epoch 323/350
31/31  0s 4ms/step - loss: 2.1865e-04 - val_loss: 2.0215e-04
Epoch 324/350
31/31  0s 4ms/step - loss: 2.0982e-04 - val_loss: 2.0201e-04
Epoch 325/350
31/31  0s 4ms/step - loss: 2.1024e-04 - val_loss: 2.0201e-04
Epoch 326/350
31/31  0s 3ms/step - loss: 2.0268e-04 - val_loss: 2.0183e-04
Epoch 327/350
31/31  0s 4ms/step - loss: 1.9721e-04 - val_loss: 2.0176e-04
Epoch 328/350
31/31  0s 3ms/step - loss: 1.9514e-04 - val_loss: 2.0172e-04
Epoch 329/350
31/31  0s 4ms/step - loss: 2.0991e-04 - val_loss: 2.0149e-04
Epoch 330/350
31/31  0s 4ms/step - loss: 2.0969e-04 - val_loss: 2.0131e-04
```

```

Epoch 331/350
31/31  0s 3ms/step - loss: 2.0704e-04 - val_loss: 2.0117e-04
Epoch 332/350
31/31  0s 4ms/step - loss: 1.9152e-04 - val_loss: 2.0106e-04
Epoch 333/350
31/31  0s 4ms/step - loss: 2.0893e-04 - val_loss: 2.0106e-04
Epoch 334/350
31/31  0s 4ms/step - loss: 1.9503e-04 - val_loss: 2.0088e-04
Epoch 335/350
31/31  0s 3ms/step - loss: 2.0499e-04 - val_loss: 2.0079e-04
Epoch 336/350
31/31  0s 4ms/step - loss: 2.0441e-04 - val_loss: 2.0082e-04
Epoch 337/350
31/31  0s 4ms/step - loss: 2.1289e-04 - val_loss: 2.0060e-04
Epoch 338/350
31/31  0s 4ms/step - loss: 2.1388e-04 - val_loss: 2.0054e-04
Epoch 339/350
31/31  0s 4ms/step - loss: 2.0253e-04 - val_loss: 2.0033e-04
Epoch 340/350
31/31  0s 4ms/step - loss: 2.2594e-04 - val_loss: 2.0024e-04
Epoch 341/350
31/31  0s 4ms/step - loss: 1.9862e-04 - val_loss: 2.0020e-04
Epoch 342/350
31/31  0s 4ms/step - loss: 1.9603e-04 - val_loss: 2.0020e-04
Epoch 343/350
31/31  0s 4ms/step - loss: 2.0171e-04 - val_loss: 2.0010e-04
Epoch 344/350
31/31  0s 4ms/step - loss: 2.1142e-04 - val_loss: 1.9984e-04
Epoch 345/350
31/31  0s 4ms/step - loss: 2.0991e-04 - val_loss: 1.9982e-04
Epoch 346/350
31/31  0s 4ms/step - loss: 1.9801e-04 - val_loss: 1.9965e-04
Epoch 347/350
31/31  0s 4ms/step - loss: 2.0323e-04 - val_loss: 1.9969e-04
Epoch 348/350
31/31  0s 4ms/step - loss: 2.1097e-04 - val_loss: 1.9958e-04
Epoch 349/350
31/31  0s 3ms/step - loss: 1.8777e-04 - val_loss: 1.9949e-04
Epoch 350/350
31/31  0s 4ms/step - loss: 1.9985e-04 - val_loss: 1.9914e-04

```

```
Out[12]: <keras.src.callbacks.history.History at 0x7968caeea010>
```

We can analyze the performance of the linear autoencoder by extracting the reconstruction error of the model in the training and test samples. We compute, similar to computing R^2 in regression settings, the proportion of the variance of the inputs that is left unexplained by the model.

Remember that the lower bound on the in-sample reconstruction error, using a linear autoencoder and an MSE loss, is set by the sum of the principal eigenvalues $k+1, \dots, n$. From above, we obtained that the first three principal components explain roughly 67% of the variation in the data. The trained autoencoder has a reconstruction error of 32%, meaning that the error is close to the minimum possible when using a three-dimensional encoding.

Importantly, notice that we are using a validation sample with an early stopping criterion to avoid overfitting. This means that training may stop before the lower bound is

achieved, so we should not necessarily reach the PCA's minimum reconstruction error using the trained autoencoder.

Below, we report the reconstruction error of the model in the training set and validation set separately. The results depend on the random seed that initializes the training algorithm. However, after several runs, the model should display a relatively good out-of-sample performance in terms of reconstruction error.

```
In [13]: Y_pred = model.predict(Y_train, verbose=0)
error = np.sum((Y_train - Y_pred) ** 2) / np.sum((Y_train - np.mean(Y_train)) ** 2)
print("Reconstruction error in training sample", error)
Y_pred = model.predict(Y_test, verbose=0)
error_oos = np.sum((Y_test - Y_pred) ** 2) / np.sum((Y_test - np.mean(Y_test)) ** 2)
print("Reconstruction error in test sample", error_oos)
```

Reconstruction error in training sample 0.3179451109621889

Reconstruction error in test sample 0.4889370196068262

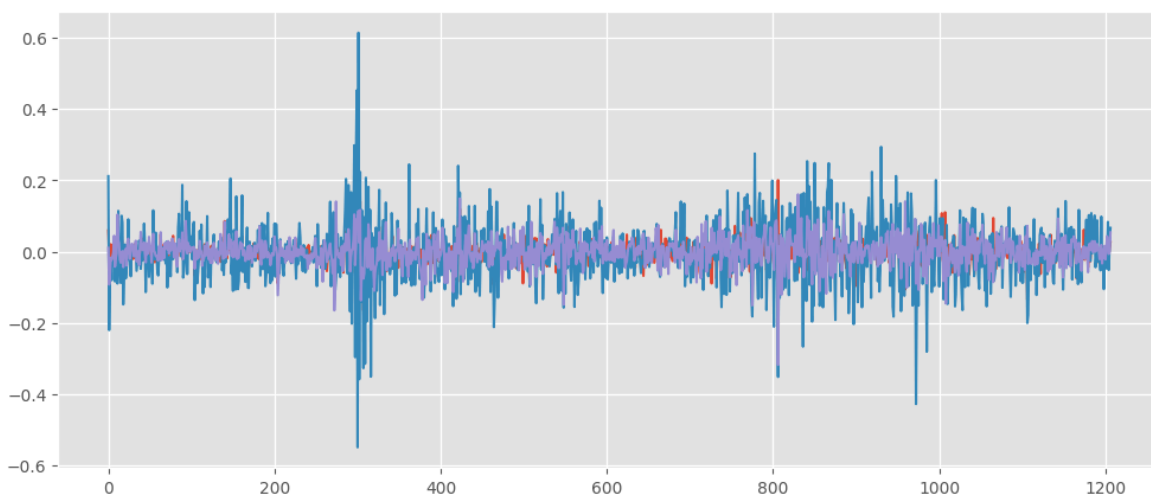
We can recover the time series of each latent factor generated by the autoencoder.

Notice that the latent factors recovered by the autoencoder are correlated, as opposed to the PCA factors. Besides, the autoencoder factors are not ordered in terms of total variation.

```
In [14]: factors = model.encoder(Y_train).numpy()
factors_test = model.encoder(Y_test).numpy()
factors_cov = np.cov(factors, rowvar=False)
print(factors_cov)
```

```
[[0.00104645 0.00133243 0.0003107 ]
 [0.00133243 0.00750614 0.00162256]
 [0.0003107  0.00162256 0.00160836]]
```

```
In [15]: # Plot the evolution of latent factors in the training period
plt.figure(figsize=(12, 5))
plt.plot(factors)
plt.show()
```

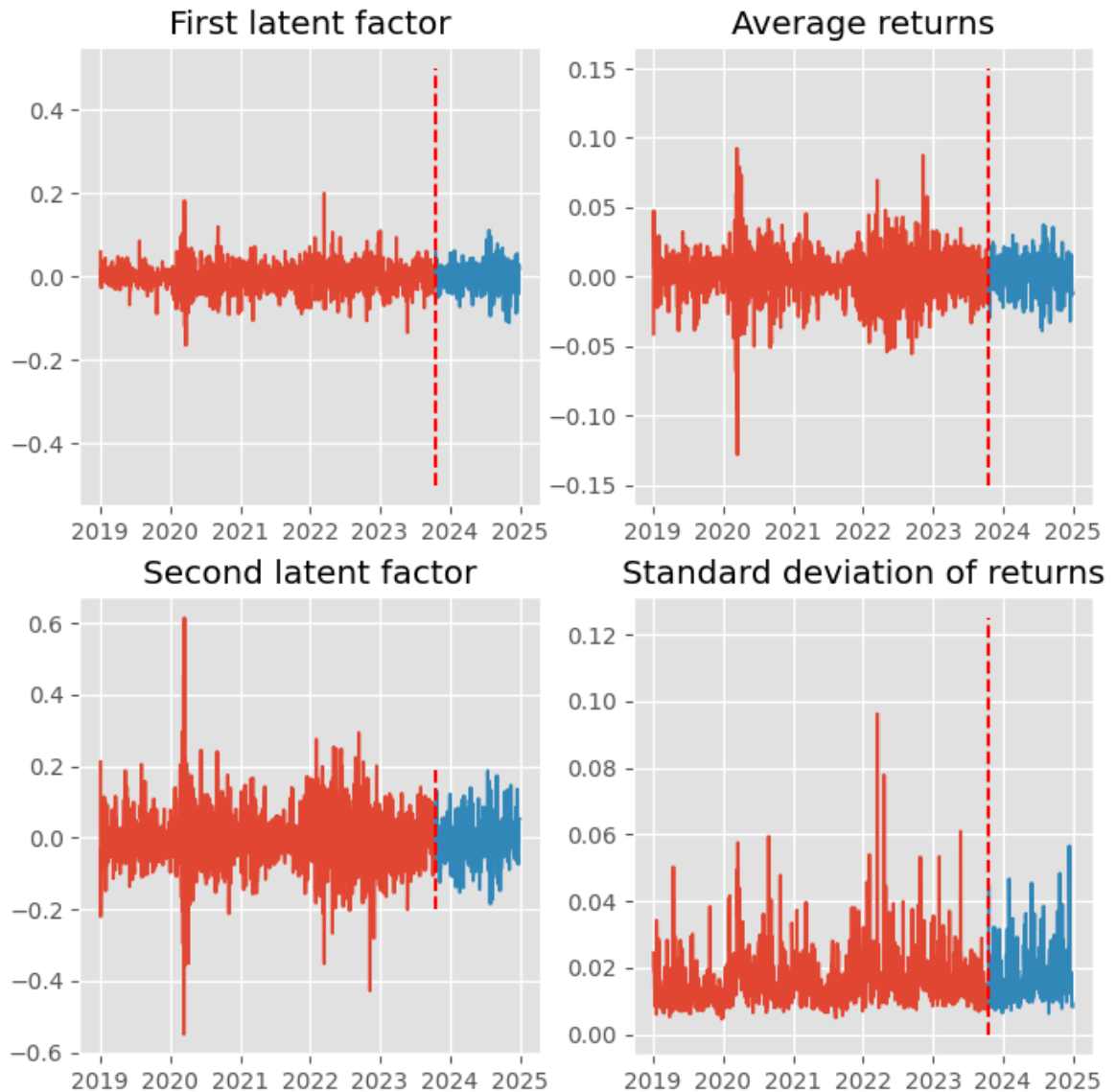


Although the results may vary depending on the seed, the highest-variance latent factor recovered by the autoencoder tends to capture the average level of returns across the stocks considered. The second latent factor in terms of variance seems to capture

variations in the cross-sectional standard deviation of returns. That is, when the second factor takes a high value it seems that returns across stocks are more disperse.

Note: The red dashed line denotes the end of the training sample.

```
In [16]: fig, axs = plt.subplots(2, 2, figsize=(8, 8))
# First factor
axs[0, 0].plot(df.index[: train_size + 1], factors[:, 0])
axs[0, 0].plot(df.index[train_size + 1 :], factors_test[:, 0])
axs[0, 0].plot([df.index[train_size + 1], df.index[train_size + 1]], [-0.5, 0.5])
axs[0, 0].title.set_text("First latent factor")
# Average of returns
axs[0, 1].plot(df.index[: train_size + 1], np.mean(Y_train, axis=1))
axs[0, 1].plot(df.index[train_size + 1 :], np.mean(Y_test, axis=1))
axs[0, 1].plot(
    [df.index[train_size + 1], df.index[train_size + 1]], [-0.15, 0.15], "r--"
)
axs[0, 1].title.set_text("Average returns")
# Second factor
axs[1, 0].plot(df.index[: train_size + 1], factors[:, 1])
axs[1, 0].plot(df.index[train_size + 1 :], factors_test[:, 1])
axs[1, 0].plot([df.index[train_size + 1], df.index[train_size + 1]], [-0.2, 0.2])
axs[1, 0].title.set_text("Second latent factor")
# Standard deviation of returns
axs[1, 1].plot(df.index[: train_size + 1], np.std(Y_train, axis=1))
axs[1, 1].plot(df.index[train_size + 1 :], np.std(Y_test, axis=1))
axs[1, 1].plot(
    [df.index[train_size + 1], df.index[train_size + 1]], [0.0, 0.125], "r--"
)
axs[1, 1].title.set_text("Standard deviation of returns")
```



How is each stock in our sample exposed to each of the latent factors extracted by the autoencoder? This exposure is measured by each stock's loadings (the trainable weights in the output layer). Below, we generate scatter plots that display the relationship between the loadings of each stock on each pair of factors.

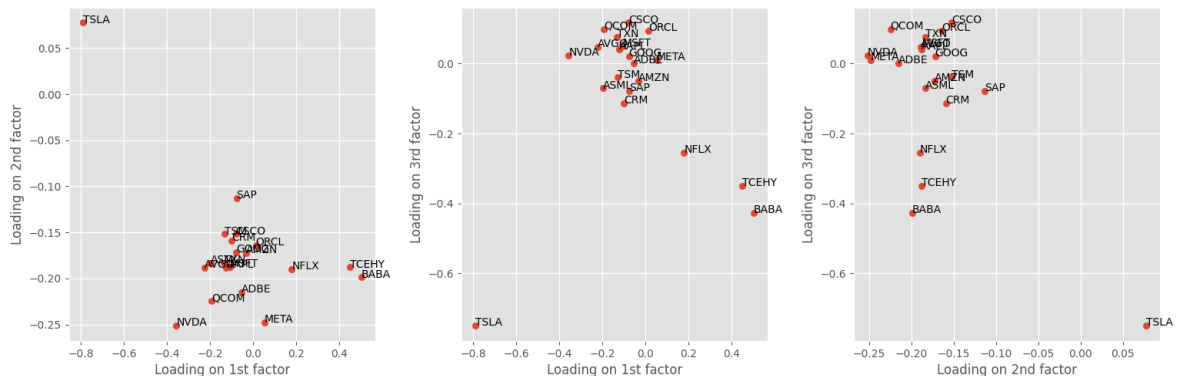
Noticeably, Tesla represents a clear outlier, a results that tends to appear regardless of the random seed that is used in the training algorithm. This suggests that a large portion of the cross-sectional variation in stock returns is actually driven by the behavior of Tesla's stock price. In other words, Tesla's stock price displays a behavior of its own that is separate from other stocks.

```
In [17]: # Extract Loadings generated by the autoencoder
w_decoder = model.decoder.layers[0].get_weights()[0]
fig, axs = plt.subplots(1, 3, figsize=(16, 6))
fig.tight_layout(pad=5.0)
axs[0].scatter(w_decoder[0], w_decoder[1])
axs[0].set_ylabel("Loading on 2nd factor")
axs[0].set_xlabel("Loading on 1st factor")
axs[1].scatter(w_decoder[0], w_decoder[2])
axs[1].set_ylabel("Loading on 3rd factor")
axs[1].set_xlabel("Loading on 1st factor")
axs[2].scatter(w_decoder[1], w_decoder[2])
```

```

axs[2].set_ylabel("Loading on 3rd factor")
axs[2].set_xlabel("Loading on 2nd factor")
for i, txt in enumerate(tickers):
    axs[0].annotate(txt, (w_decoder[0, i], w_decoder[1, i]))
    axs[1].annotate(txt, (w_decoder[0, i], w_decoder[2, i]))
    axs[2].annotate(txt, (w_decoder[1, i], w_decoder[2, i]))

```



4. AN AUTOENCODER WITH NON-LINEAR ACTIVATIONS

In the blocks of code below we build and train an autoencoder that includes two additional hidden layers, one on each side of the bottleneck, with non-linear activations. The trained model displays slightly less reconstruction error than the linear benchmark in the training sample, although the error in the test sample is approximately the same.

The non-linear autoencoder may be able to capture richer features than the linear version. However, it becomes harder to grasp the type of variation that each latent factor is recovering from the data. As we show below in the figures, the factors are not so tightly related to average returns or dispersion, while the presence of an intermediate hidden layer makes it hard to grasp the exposure of each stock to the latent factors. Thus, adding non-linearities and hidden layers may enhance the explanatory ability of our models, but we likely lose a great deal of interpretability.

```

In [18]: class autoencoder_nonlinear(tf.keras.Model):
    def __init__(self, output_dim, bottleneck_dim=1):
        super(autoencoder_nonlinear, self).__init__()
        self.output_dim = output_dim
        self.bottleneck_dim = bottleneck_dim
        act_fun = "relu"
        self.encoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(12, activation=act_fun),
                tf.keras.layers.Dense(bottleneck_dim),
            ]
        )
        self.decoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(12, activation=act_fun),
                tf.keras.layers.Dense(output_dim),
            ]
        )

    def call(self, x):

```

```
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
    return decoded
```

```
In [19]: nlatent = 3
# to make this notebook's output stable
# np.random.seed(42)
# tf.random.set_seed(42)
val_split = 0.2
early_stop = tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=50)
model_nonlinear = autoencoder_nonlinear(Y_train.shape[1], nlatent)
adam = tf.keras.optimizers.Adam(learning_rate=1e-4)
model_nonlinear.compile(optimizer=adam, loss="mse")
model_nonlinear.fit(
    x=Y_train,
    y=Y_train,
    batch_size=32,
    epochs=350,
    validation_split=val_split,
    callbacks=[early_stop],
    verbose=1,
)
```

```
Epoch 1/350
31/31  1s 8ms/step - loss: 7.6684e-04 - val_loss: 5.6887e-04
Epoch 2/350
31/31  0s 4ms/step - loss: 6.7629e-04 - val_loss: 5.6141e-04
Epoch 3/350
31/31  0s 4ms/step - loss: 6.2947e-04 - val_loss: 5.5522e-04
Epoch 4/350
31/31  0s 4ms/step - loss: 6.3512e-04 - val_loss: 5.4929e-04
Epoch 5/350
31/31  0s 4ms/step - loss: 7.1331e-04 - val_loss: 5.4184e-04
Epoch 6/350
31/31  0s 4ms/step - loss: 6.2695e-04 - val_loss: 5.3452e-04
Epoch 7/350
31/31  0s 4ms/step - loss: 6.1891e-04 - val_loss: 5.2715e-04
Epoch 8/350
31/31  0s 4ms/step - loss: 5.8189e-04 - val_loss: 5.1989e-04
Epoch 9/350
31/31  0s 3ms/step - loss: 6.3155e-04 - val_loss: 5.1103e-04
Epoch 10/350
31/31  0s 4ms/step - loss: 6.0388e-04 - val_loss: 5.0307e-04
Epoch 11/350
31/31  0s 4ms/step - loss: 6.0035e-04 - val_loss: 4.9376e-04
Epoch 12/350
31/31  0s 4ms/step - loss: 6.0533e-04 - val_loss: 4.8527e-04
Epoch 13/350
31/31  0s 4ms/step - loss: 5.7830e-04 - val_loss: 4.7579e-04
Epoch 14/350
31/31  0s 4ms/step - loss: 5.5804e-04 - val_loss: 4.6718e-04
Epoch 15/350
31/31  0s 4ms/step - loss: 5.6468e-04 - val_loss: 4.5852e-04
Epoch 16/350
31/31  0s 3ms/step - loss: 5.5596e-04 - val_loss: 4.5038e-04
Epoch 17/350
31/31  0s 4ms/step - loss: 5.2570e-04 - val_loss: 4.4298e-04
Epoch 18/350
31/31  0s 4ms/step - loss: 5.4704e-04 - val_loss: 4.3462e-04
Epoch 19/350
31/31  0s 4ms/step - loss: 5.0911e-04 - val_loss: 4.2834e-04
Epoch 20/350
31/31  0s 4ms/step - loss: 4.6692e-04 - val_loss: 4.2099e-04
Epoch 21/350
31/31  0s 4ms/step - loss: 4.7658e-04 - val_loss: 4.1451e-04
Epoch 22/350
31/31  0s 3ms/step - loss: 5.0085e-04 - val_loss: 4.0858e-04
Epoch 23/350
31/31  0s 4ms/step - loss: 4.9184e-04 - val_loss: 4.0117e-04
Epoch 24/350
31/31  0s 4ms/step - loss: 4.7263e-04 - val_loss: 3.9622e-04
Epoch 25/350
31/31  0s 4ms/step - loss: 4.4502e-04 - val_loss: 3.9005e-04
Epoch 26/350
31/31  0s 4ms/step - loss: 4.5332e-04 - val_loss: 3.8408e-04
Epoch 27/350
31/31  0s 4ms/step - loss: 4.4688e-04 - val_loss: 3.7942e-04
Epoch 28/350
31/31  0s 4ms/step - loss: 4.1738e-04 - val_loss: 3.7461e-04
Epoch 29/350
31/31  0s 3ms/step - loss: 4.2692e-04 - val_loss: 3.7044e-04
Epoch 30/350
31/31  0s 4ms/step - loss: 4.4853e-04 - val_loss: 3.6541e-04
```

```
Epoch 31/350
31/31  0s 4ms/step - loss: 3.8681e-04 - val_loss: 3.6099e-04
Epoch 32/350
31/31  0s 4ms/step - loss: 3.9449e-04 - val_loss: 3.5782e-04
Epoch 33/350
31/31  0s 4ms/step - loss: 3.8867e-04 - val_loss: 3.5331e-04
Epoch 34/350
31/31  0s 4ms/step - loss: 3.9347e-04 - val_loss: 3.4975e-04
Epoch 35/350
31/31  0s 4ms/step - loss: 3.8846e-04 - val_loss: 3.4636e-04
Epoch 36/350
31/31  0s 4ms/step - loss: 3.9032e-04 - val_loss: 3.4285e-04
Epoch 37/350
31/31  0s 4ms/step - loss: 3.8907e-04 - val_loss: 3.4014e-04
Epoch 38/350
31/31  0s 4ms/step - loss: 3.9369e-04 - val_loss: 3.3724e-04
Epoch 39/350
31/31  0s 4ms/step - loss: 3.5860e-04 - val_loss: 3.3363e-04
Epoch 40/350
31/31  0s 4ms/step - loss: 3.4960e-04 - val_loss: 3.3139e-04
Epoch 41/350
31/31  0s 6ms/step - loss: 3.9924e-04 - val_loss: 3.2752e-04
Epoch 42/350
31/31  0s 6ms/step - loss: 3.5631e-04 - val_loss: 3.2494e-04
Epoch 43/350
31/31  0s 6ms/step - loss: 3.3630e-04 - val_loss: 3.2365e-04
Epoch 44/350
31/31  0s 6ms/step - loss: 3.5517e-04 - val_loss: 3.2041e-04
Epoch 45/350
31/31  0s 6ms/step - loss: 3.2983e-04 - val_loss: 3.1884e-04
Epoch 46/350
31/31  0s 5ms/step - loss: 3.5246e-04 - val_loss: 3.1634e-04
Epoch 47/350
31/31  0s 6ms/step - loss: 3.5616e-04 - val_loss: 3.1338e-04
Epoch 48/350
31/31  0s 3ms/step - loss: 3.4471e-04 - val_loss: 3.1196e-04
Epoch 49/350
31/31  0s 4ms/step - loss: 3.4331e-04 - val_loss: 3.1018e-04
Epoch 50/350
31/31  0s 4ms/step - loss: 3.3593e-04 - val_loss: 3.0859e-04
Epoch 51/350
31/31  0s 3ms/step - loss: 3.2970e-04 - val_loss: 3.0693e-04
Epoch 52/350
31/31  0s 5ms/step - loss: 3.3326e-04 - val_loss: 3.0488e-04
Epoch 53/350
31/31  0s 5ms/step - loss: 3.2406e-04 - val_loss: 3.0272e-04
Epoch 54/350
31/31  0s 5ms/step - loss: 3.2708e-04 - val_loss: 3.0145e-04
Epoch 55/350
31/31  0s 5ms/step - loss: 3.2104e-04 - val_loss: 3.0047e-04
Epoch 56/350
31/31  0s 3ms/step - loss: 3.3432e-04 - val_loss: 2.9948e-04
Epoch 57/350
31/31  0s 4ms/step - loss: 3.3435e-04 - val_loss: 2.9625e-04
Epoch 58/350
31/31  0s 4ms/step - loss: 3.3143e-04 - val_loss: 2.9559e-04
Epoch 59/350
31/31  0s 4ms/step - loss: 3.5125e-04 - val_loss: 2.9447e-04
Epoch 60/350
31/31  0s 4ms/step - loss: 3.0695e-04 - val_loss: 2.9338e-04
```

```
Epoch 61/350
31/31  0s 4ms/step - loss: 3.0815e-04 - val_loss: 2.9146e-04
Epoch 62/350
31/31  0s 4ms/step - loss: 3.0087e-04 - val_loss: 2.9084e-04
Epoch 63/350
31/31  0s 4ms/step - loss: 3.2577e-04 - val_loss: 2.8957e-04
Epoch 64/350
31/31  0s 4ms/step - loss: 3.2133e-04 - val_loss: 2.8841e-04
Epoch 65/350
31/31  0s 6ms/step - loss: 3.0689e-04 - val_loss: 2.8729e-04
Epoch 66/350
31/31  0s 4ms/step - loss: 2.9215e-04 - val_loss: 2.8597e-04
Epoch 67/350
31/31  0s 4ms/step - loss: 3.2651e-04 - val_loss: 2.8513e-04
Epoch 68/350
31/31  0s 4ms/step - loss: 3.0124e-04 - val_loss: 2.8415e-04
Epoch 69/350
31/31  0s 4ms/step - loss: 3.1816e-04 - val_loss: 2.8278e-04
Epoch 70/350
31/31  0s 4ms/step - loss: 2.8879e-04 - val_loss: 2.8250e-04
Epoch 71/350
31/31  0s 4ms/step - loss: 3.0999e-04 - val_loss: 2.8091e-04
Epoch 72/350
31/31  0s 4ms/step - loss: 2.8722e-04 - val_loss: 2.7955e-04
Epoch 73/350
31/31  0s 4ms/step - loss: 2.9748e-04 - val_loss: 2.7961e-04
Epoch 74/350
31/31  0s 4ms/step - loss: 3.1717e-04 - val_loss: 2.7781e-04
Epoch 75/350
31/31  0s 4ms/step - loss: 3.1093e-04 - val_loss: 2.7685e-04
Epoch 76/350
31/31  0s 4ms/step - loss: 2.8151e-04 - val_loss: 2.7589e-04
Epoch 77/350
31/31  0s 4ms/step - loss: 2.6536e-04 - val_loss: 2.7607e-04
Epoch 78/350
31/31  0s 4ms/step - loss: 2.6969e-04 - val_loss: 2.7446e-04
Epoch 79/350
31/31  0s 4ms/step - loss: 3.0753e-04 - val_loss: 2.7360e-04
Epoch 80/350
31/31  0s 4ms/step - loss: 2.9730e-04 - val_loss: 2.7245e-04
Epoch 81/350
31/31  0s 4ms/step - loss: 3.1307e-04 - val_loss: 2.7207e-04
Epoch 82/350
31/31  0s 4ms/step - loss: 3.1436e-04 - val_loss: 2.7078e-04
Epoch 83/350
31/31  0s 3ms/step - loss: 2.6424e-04 - val_loss: 2.7041e-04
Epoch 84/350
31/31  0s 4ms/step - loss: 2.8041e-04 - val_loss: 2.6943e-04
Epoch 85/350
31/31  0s 5ms/step - loss: 3.0792e-04 - val_loss: 2.6865e-04
Epoch 86/350
31/31  0s 6ms/step - loss: 2.8270e-04 - val_loss: 2.6755e-04
Epoch 87/350
31/31  0s 5ms/step - loss: 2.8223e-04 - val_loss: 2.6747e-04
Epoch 88/350
31/31  0s 4ms/step - loss: 2.6932e-04 - val_loss: 2.6609e-04
Epoch 89/350
31/31  0s 4ms/step - loss: 2.6900e-04 - val_loss: 2.6555e-04
Epoch 90/350
31/31  0s 4ms/step - loss: 2.9153e-04 - val_loss: 2.6506e-04
```

```
Epoch 91/350
31/31  0s 4ms/step - loss: 2.7369e-04 - val_loss: 2.6403e-04
Epoch 92/350
31/31  0s 4ms/step - loss: 2.8406e-04 - val_loss: 2.6378e-04
Epoch 93/350
31/31  0s 4ms/step - loss: 2.8459e-04 - val_loss: 2.6255e-04
Epoch 94/350
31/31  0s 4ms/step - loss: 2.6245e-04 - val_loss: 2.6210e-04
Epoch 95/350
31/31  0s 4ms/step - loss: 2.7069e-04 - val_loss: 2.6169e-04
Epoch 96/350
31/31  0s 4ms/step - loss: 2.7910e-04 - val_loss: 2.6110e-04
Epoch 97/350
31/31  0s 4ms/step - loss: 2.6897e-04 - val_loss: 2.5982e-04
Epoch 98/350
31/31  0s 3ms/step - loss: 2.7645e-04 - val_loss: 2.5920e-04
Epoch 99/350
31/31  0s 5ms/step - loss: 2.8929e-04 - val_loss: 2.5857e-04
Epoch 100/350
31/31  0s 4ms/step - loss: 2.6989e-04 - val_loss: 2.5787e-04
Epoch 101/350
31/31  0s 3ms/step - loss: 2.7571e-04 - val_loss: 2.5751e-04
Epoch 102/350
31/31  0s 4ms/step - loss: 2.8700e-04 - val_loss: 2.5659e-04
Epoch 103/350
31/31  0s 4ms/step - loss: 2.8108e-04 - val_loss: 2.5594e-04
Epoch 104/350
31/31  0s 4ms/step - loss: 2.5291e-04 - val_loss: 2.5524e-04
Epoch 105/350
31/31  0s 4ms/step - loss: 2.8573e-04 - val_loss: 2.5398e-04
Epoch 106/350
31/31  0s 4ms/step - loss: 2.4930e-04 - val_loss: 2.5386e-04
Epoch 107/350
31/31  0s 4ms/step - loss: 2.7561e-04 - val_loss: 2.5356e-04
Epoch 108/350
31/31  0s 4ms/step - loss: 2.8838e-04 - val_loss: 2.5242e-04
Epoch 109/350
31/31  0s 4ms/step - loss: 2.5744e-04 - val_loss: 2.5138e-04
Epoch 110/350
31/31  0s 4ms/step - loss: 2.7010e-04 - val_loss: 2.5086e-04
Epoch 111/350
31/31  0s 4ms/step - loss: 2.7774e-04 - val_loss: 2.4990e-04
Epoch 112/350
31/31  0s 4ms/step - loss: 2.5348e-04 - val_loss: 2.4928e-04
Epoch 113/350
31/31  0s 3ms/step - loss: 2.6336e-04 - val_loss: 2.4857e-04
Epoch 114/350
31/31  0s 4ms/step - loss: 2.4283e-04 - val_loss: 2.4824e-04
Epoch 115/350
31/31  0s 4ms/step - loss: 2.7206e-04 - val_loss: 2.4728e-04
Epoch 116/350
31/31  0s 4ms/step - loss: 2.6939e-04 - val_loss: 2.4638e-04
Epoch 117/350
31/31  0s 4ms/step - loss: 2.5857e-04 - val_loss: 2.4606e-04
Epoch 118/350
31/31  0s 4ms/step - loss: 2.7000e-04 - val_loss: 2.4530e-04
Epoch 119/350
31/31  0s 3ms/step - loss: 2.3783e-04 - val_loss: 2.4441e-04
Epoch 120/350
31/31  0s 4ms/step - loss: 2.6705e-04 - val_loss: 2.4402e-04
```

```
Epoch 121/350
31/31  0s 4ms/step - loss: 2.6339e-04 - val_loss: 2.4299e-04
Epoch 122/350
31/31  0s 4ms/step - loss: 2.6074e-04 - val_loss: 2.4203e-04
Epoch 123/350
31/31  0s 4ms/step - loss: 2.5216e-04 - val_loss: 2.4129e-04
Epoch 124/350
31/31  0s 4ms/step - loss: 2.3659e-04 - val_loss: 2.4069e-04
Epoch 125/350
31/31  0s 4ms/step - loss: 2.5897e-04 - val_loss: 2.3959e-04
Epoch 126/350
31/31  0s 4ms/step - loss: 2.5349e-04 - val_loss: 2.3883e-04
Epoch 127/350
31/31  0s 4ms/step - loss: 2.4613e-04 - val_loss: 2.3795e-04
Epoch 128/350
31/31  0s 4ms/step - loss: 2.6000e-04 - val_loss: 2.3687e-04
Epoch 129/350
31/31  0s 4ms/step - loss: 2.3346e-04 - val_loss: 2.3625e-04
Epoch 130/350
31/31  0s 4ms/step - loss: 2.5746e-04 - val_loss: 2.3553e-04
Epoch 131/350
31/31  0s 4ms/step - loss: 2.5393e-04 - val_loss: 2.3473e-04
Epoch 132/350
31/31  0s 4ms/step - loss: 2.6801e-04 - val_loss: 2.3370e-04
Epoch 133/350
31/31  0s 3ms/step - loss: 2.4719e-04 - val_loss: 2.3242e-04
Epoch 134/350
31/31  0s 4ms/step - loss: 2.3875e-04 - val_loss: 2.3209e-04
Epoch 135/350
31/31  0s 4ms/step - loss: 2.2582e-04 - val_loss: 2.3093e-04
Epoch 136/350
31/31  0s 4ms/step - loss: 2.4557e-04 - val_loss: 2.3052e-04
Epoch 137/350
31/31  0s 5ms/step - loss: 2.5404e-04 - val_loss: 2.2927e-04
Epoch 138/350
31/31  0s 3ms/step - loss: 2.7054e-04 - val_loss: 2.2855e-04
Epoch 139/350
31/31  0s 3ms/step - loss: 2.5051e-04 - val_loss: 2.2799e-04
Epoch 140/350
31/31  0s 4ms/step - loss: 2.4993e-04 - val_loss: 2.2683e-04
Epoch 141/350
31/31  0s 4ms/step - loss: 2.5848e-04 - val_loss: 2.2593e-04
Epoch 142/350
31/31  0s 4ms/step - loss: 2.2620e-04 - val_loss: 2.2545e-04
Epoch 143/350
31/31  0s 4ms/step - loss: 2.4727e-04 - val_loss: 2.2433e-04
Epoch 144/350
31/31  0s 4ms/step - loss: 2.3988e-04 - val_loss: 2.2371e-04
Epoch 145/350
31/31  0s 4ms/step - loss: 2.6667e-04 - val_loss: 2.2333e-04
Epoch 146/350
31/31  0s 5ms/step - loss: 2.3766e-04 - val_loss: 2.2209e-04
Epoch 147/350
31/31  0s 4ms/step - loss: 2.3444e-04 - val_loss: 2.2155e-04
Epoch 148/350
31/31  0s 4ms/step - loss: 2.1763e-04 - val_loss: 2.2096e-04
Epoch 149/350
31/31  0s 4ms/step - loss: 2.3864e-04 - val_loss: 2.1989e-04
Epoch 150/350
31/31  0s 4ms/step - loss: 2.3049e-04 - val_loss: 2.1913e-04
```

```
Epoch 151/350
31/31  0s 4ms/step - loss: 2.2448e-04 - val_loss: 2.1825e-04
Epoch 152/350
31/31  0s 4ms/step - loss: 2.5715e-04 - val_loss: 2.1774e-04
Epoch 153/350
31/31  0s 4ms/step - loss: 2.2150e-04 - val_loss: 2.1705e-04
Epoch 154/350
31/31  0s 4ms/step - loss: 2.2644e-04 - val_loss: 2.1627e-04
Epoch 155/350
31/31  0s 4ms/step - loss: 2.2553e-04 - val_loss: 2.1573e-04
Epoch 156/350
31/31  0s 4ms/step - loss: 2.3170e-04 - val_loss: 2.1511e-04
Epoch 157/350
31/31  0s 4ms/step - loss: 2.4861e-04 - val_loss: 2.1423e-04
Epoch 158/350
31/31  0s 4ms/step - loss: 2.2992e-04 - val_loss: 2.1336e-04
Epoch 159/350
31/31  0s 3ms/step - loss: 2.3305e-04 - val_loss: 2.1322e-04
Epoch 160/350
31/31  0s 4ms/step - loss: 2.3245e-04 - val_loss: 2.1255e-04
Epoch 161/350
31/31  0s 4ms/step - loss: 2.3323e-04 - val_loss: 2.1236e-04
Epoch 162/350
31/31  0s 4ms/step - loss: 2.1822e-04 - val_loss: 2.1148e-04
Epoch 163/350
31/31  0s 4ms/step - loss: 2.1644e-04 - val_loss: 2.1089e-04
Epoch 164/350
31/31  0s 4ms/step - loss: 2.0697e-04 - val_loss: 2.1060e-04
Epoch 165/350
31/31  0s 4ms/step - loss: 2.1217e-04 - val_loss: 2.0971e-04
Epoch 166/350
31/31  0s 4ms/step - loss: 2.2394e-04 - val_loss: 2.0958e-04
Epoch 167/350
31/31  0s 4ms/step - loss: 2.1272e-04 - val_loss: 2.0917e-04
Epoch 168/350
31/31  0s 4ms/step - loss: 2.1337e-04 - val_loss: 2.0839e-04
Epoch 169/350
31/31  0s 4ms/step - loss: 2.2021e-04 - val_loss: 2.0805e-04
Epoch 170/350
31/31  0s 4ms/step - loss: 2.2408e-04 - val_loss: 2.0739e-04
Epoch 171/350
31/31  0s 4ms/step - loss: 2.0560e-04 - val_loss: 2.0678e-04
Epoch 172/350
31/31  0s 4ms/step - loss: 2.3028e-04 - val_loss: 2.0696e-04
Epoch 173/350
31/31  0s 4ms/step - loss: 2.2943e-04 - val_loss: 2.0662e-04
Epoch 174/350
31/31  0s 4ms/step - loss: 2.0895e-04 - val_loss: 2.0614e-04
Epoch 175/350
31/31  0s 4ms/step - loss: 2.3674e-04 - val_loss: 2.0590e-04
Epoch 176/350
31/31  0s 4ms/step - loss: 2.3880e-04 - val_loss: 2.0478e-04
Epoch 177/350
31/31  0s 4ms/step - loss: 2.0375e-04 - val_loss: 2.0523e-04
Epoch 178/350
31/31  0s 4ms/step - loss: 2.5151e-04 - val_loss: 2.0448e-04
Epoch 179/350
31/31  0s 4ms/step - loss: 2.2004e-04 - val_loss: 2.0424e-04
Epoch 180/350
31/31  0s 4ms/step - loss: 2.1041e-04 - val_loss: 2.0353e-04
```

```
Epoch 181/350
31/31  0s 3ms/step - loss: 2.1078e-04 - val_loss: 2.0353e-04
Epoch 182/350
31/31  0s 5ms/step - loss: 2.0853e-04 - val_loss: 2.0333e-04
Epoch 183/350
31/31  0s 3ms/step - loss: 2.1251e-04 - val_loss: 2.0316e-04
Epoch 184/350
31/31  0s 4ms/step - loss: 2.2410e-04 - val_loss: 2.0262e-04
Epoch 185/350
31/31  0s 4ms/step - loss: 2.1171e-04 - val_loss: 2.0160e-04
Epoch 186/350
31/31  0s 4ms/step - loss: 2.1867e-04 - val_loss: 2.0206e-04
Epoch 187/350
31/31  0s 3ms/step - loss: 2.4135e-04 - val_loss: 2.0151e-04
Epoch 188/350
31/31  0s 4ms/step - loss: 2.1081e-04 - val_loss: 2.0120e-04
Epoch 189/350
31/31  0s 3ms/step - loss: 2.3482e-04 - val_loss: 2.0101e-04
Epoch 190/350
31/31  0s 4ms/step - loss: 2.4107e-04 - val_loss: 2.0070e-04
Epoch 191/350
31/31  0s 4ms/step - loss: 2.0674e-04 - val_loss: 2.0036e-04
Epoch 192/350
31/31  0s 4ms/step - loss: 2.0915e-04 - val_loss: 2.0060e-04
Epoch 193/350
31/31  0s 4ms/step - loss: 2.1824e-04 - val_loss: 2.0027e-04
Epoch 194/350
31/31  0s 4ms/step - loss: 2.0515e-04 - val_loss: 1.9937e-04
Epoch 195/350
31/31  0s 4ms/step - loss: 1.9874e-04 - val_loss: 1.9950e-04
Epoch 196/350
31/31  0s 4ms/step - loss: 2.0570e-04 - val_loss: 1.9918e-04
Epoch 197/350
31/31  0s 4ms/step - loss: 2.1135e-04 - val_loss: 1.9897e-04
Epoch 198/350
31/31  0s 4ms/step - loss: 2.0553e-04 - val_loss: 1.9887e-04
Epoch 199/350
31/31  0s 4ms/step - loss: 2.2948e-04 - val_loss: 1.9851e-04
Epoch 200/350
31/31  0s 4ms/step - loss: 2.1294e-04 - val_loss: 1.9835e-04
Epoch 201/350
31/31  0s 5ms/step - loss: 2.1422e-04 - val_loss: 1.9769e-04
Epoch 202/350
31/31  0s 4ms/step - loss: 2.1610e-04 - val_loss: 1.9801e-04
Epoch 203/350
31/31  0s 4ms/step - loss: 2.1306e-04 - val_loss: 1.9771e-04
Epoch 204/350
31/31  0s 4ms/step - loss: 2.1151e-04 - val_loss: 1.9732e-04
Epoch 205/350
31/31  0s 3ms/step - loss: 2.1114e-04 - val_loss: 1.9735e-04
Epoch 206/350
31/31  0s 4ms/step - loss: 2.1810e-04 - val_loss: 1.9716e-04
Epoch 207/350
31/31  0s 4ms/step - loss: 2.3451e-04 - val_loss: 1.9690e-04
Epoch 208/350
31/31  0s 4ms/step - loss: 2.0838e-04 - val_loss: 1.9668e-04
Epoch 209/350
31/31  0s 4ms/step - loss: 2.0958e-04 - val_loss: 1.9666e-04
Epoch 210/350
31/31  0s 3ms/step - loss: 2.1117e-04 - val_loss: 1.9639e-04
```

```
Epoch 211/350
31/31  0s 4ms/step - loss: 2.0318e-04 - val_loss: 1.9612e-04
Epoch 212/350
31/31  0s 4ms/step - loss: 2.1623e-04 - val_loss: 1.9563e-04
Epoch 213/350
31/31  0s 5ms/step - loss: 2.1122e-04 - val_loss: 1.9611e-04
Epoch 214/350
31/31  0s 4ms/step - loss: 2.0964e-04 - val_loss: 1.9558e-04
Epoch 215/350
31/31  0s 4ms/step - loss: 2.0865e-04 - val_loss: 1.9563e-04
Epoch 216/350
31/31  0s 3ms/step - loss: 1.9419e-04 - val_loss: 1.9535e-04
Epoch 217/350
31/31  0s 4ms/step - loss: 2.1475e-04 - val_loss: 1.9530e-04
Epoch 218/350
31/31  0s 4ms/step - loss: 2.1312e-04 - val_loss: 1.9541e-04
Epoch 219/350
31/31  0s 4ms/step - loss: 2.1069e-04 - val_loss: 1.9475e-04
Epoch 220/350
31/31  0s 3ms/step - loss: 2.1244e-04 - val_loss: 1.9465e-04
Epoch 221/350
31/31  0s 4ms/step - loss: 2.0752e-04 - val_loss: 1.9488e-04
Epoch 222/350
31/31  0s 4ms/step - loss: 2.2567e-04 - val_loss: 1.9507e-04
Epoch 223/350
31/31  0s 3ms/step - loss: 2.1963e-04 - val_loss: 1.9451e-04
Epoch 224/350
31/31  0s 4ms/step - loss: 2.0757e-04 - val_loss: 1.9438e-04
Epoch 225/350
31/31  0s 3ms/step - loss: 2.0296e-04 - val_loss: 1.9452e-04
Epoch 226/350
31/31  0s 4ms/step - loss: 2.0718e-04 - val_loss: 1.9388e-04
Epoch 227/350
31/31  0s 3ms/step - loss: 2.2052e-04 - val_loss: 1.9403e-04
Epoch 228/350
31/31  0s 4ms/step - loss: 2.1412e-04 - val_loss: 1.9360e-04
Epoch 229/350
31/31  0s 4ms/step - loss: 2.0990e-04 - val_loss: 1.9378e-04
Epoch 230/350
31/31  0s 4ms/step - loss: 1.9940e-04 - val_loss: 1.9390e-04
Epoch 231/350
31/31  0s 4ms/step - loss: 2.0434e-04 - val_loss: 1.9331e-04
Epoch 232/350
31/31  0s 3ms/step - loss: 2.2831e-04 - val_loss: 1.9321e-04
Epoch 233/350
31/31  0s 4ms/step - loss: 2.0902e-04 - val_loss: 1.9321e-04
Epoch 234/350
31/31  0s 4ms/step - loss: 2.0122e-04 - val_loss: 1.9331e-04
Epoch 235/350
31/31  0s 4ms/step - loss: 2.1467e-04 - val_loss: 1.9304e-04
Epoch 236/350
31/31  0s 4ms/step - loss: 2.0322e-04 - val_loss: 1.9314e-04
Epoch 237/350
31/31  0s 4ms/step - loss: 1.9376e-04 - val_loss: 1.9283e-04
Epoch 238/350
31/31  0s 4ms/step - loss: 2.0861e-04 - val_loss: 1.9266e-04
Epoch 239/350
31/31  0s 4ms/step - loss: 1.8477e-04 - val_loss: 1.9247e-04
Epoch 240/350
31/31  0s 4ms/step - loss: 2.1122e-04 - val_loss: 1.9269e-04
```

```
Epoch 241/350
31/31  0s 4ms/step - loss: 2.1183e-04 - val_loss: 1.9264e-04
Epoch 242/350
31/31  0s 4ms/step - loss: 2.0603e-04 - val_loss: 1.9244e-04
Epoch 243/350
31/31  0s 4ms/step - loss: 2.0928e-04 - val_loss: 1.9231e-04
Epoch 244/350
31/31  0s 4ms/step - loss: 2.2061e-04 - val_loss: 1.9208e-04
Epoch 245/350
31/31  0s 3ms/step - loss: 2.1300e-04 - val_loss: 1.9226e-04
Epoch 246/350
31/31  0s 4ms/step - loss: 2.0945e-04 - val_loss: 1.9201e-04
Epoch 247/350
31/31  0s 4ms/step - loss: 2.0921e-04 - val_loss: 1.9215e-04
Epoch 248/350
31/31  0s 4ms/step - loss: 2.0047e-04 - val_loss: 1.9198e-04
Epoch 249/350
31/31  0s 4ms/step - loss: 2.0061e-04 - val_loss: 1.9182e-04
Epoch 250/350
31/31  0s 4ms/step - loss: 1.9471e-04 - val_loss: 1.9159e-04
Epoch 251/350
31/31  0s 4ms/step - loss: 2.2025e-04 - val_loss: 1.9179e-04
Epoch 252/350
31/31  0s 4ms/step - loss: 2.1785e-04 - val_loss: 1.9166e-04
Epoch 253/350
31/31  0s 4ms/step - loss: 1.9310e-04 - val_loss: 1.9155e-04
Epoch 254/350
31/31  0s 4ms/step - loss: 1.9552e-04 - val_loss: 1.9153e-04
Epoch 255/350
31/31  0s 4ms/step - loss: 2.0099e-04 - val_loss: 1.9164e-04
Epoch 256/350
31/31  0s 3ms/step - loss: 1.9817e-04 - val_loss: 1.9139e-04
Epoch 257/350
31/31  0s 4ms/step - loss: 2.2082e-04 - val_loss: 1.9148e-04
Epoch 258/350
31/31  0s 3ms/step - loss: 2.1275e-04 - val_loss: 1.9137e-04
Epoch 259/350
31/31  0s 4ms/step - loss: 2.1050e-04 - val_loss: 1.9125e-04
Epoch 260/350
31/31  0s 3ms/step - loss: 2.0023e-04 - val_loss: 1.9105e-04
Epoch 261/350
31/31  0s 5ms/step - loss: 2.0978e-04 - val_loss: 1.9092e-04
Epoch 262/350
31/31  0s 4ms/step - loss: 2.1119e-04 - val_loss: 1.9122e-04
Epoch 263/350
31/31  0s 4ms/step - loss: 1.9421e-04 - val_loss: 1.9110e-04
Epoch 264/350
31/31  0s 4ms/step - loss: 2.2247e-04 - val_loss: 1.9086e-04
Epoch 265/350
31/31  0s 4ms/step - loss: 2.0417e-04 - val_loss: 1.9094e-04
Epoch 266/350
31/31  0s 4ms/step - loss: 2.1107e-04 - val_loss: 1.9087e-04
Epoch 267/350
31/31  0s 4ms/step - loss: 2.0984e-04 - val_loss: 1.9075e-04
Epoch 268/350
31/31  0s 4ms/step - loss: 2.0120e-04 - val_loss: 1.9068e-04
Epoch 269/350
31/31  0s 4ms/step - loss: 2.0953e-04 - val_loss: 1.9071e-04
Epoch 270/350
31/31  0s 4ms/step - loss: 2.1178e-04 - val_loss: 1.9047e-04
```

```
Epoch 271/350
31/31  0s 4ms/step - loss: 1.9452e-04 - val_loss: 1.9066e-04
Epoch 272/350
31/31  0s 4ms/step - loss: 2.1369e-04 - val_loss: 1.9071e-04
Epoch 273/350
31/31  0s 4ms/step - loss: 2.0784e-04 - val_loss: 1.9045e-04
Epoch 274/350
31/31  0s 4ms/step - loss: 2.0072e-04 - val_loss: 1.9014e-04
Epoch 275/350
31/31  0s 4ms/step - loss: 2.0140e-04 - val_loss: 1.9037e-04
Epoch 276/350
31/31  0s 4ms/step - loss: 2.0593e-04 - val_loss: 1.9065e-04
Epoch 277/350
31/31  0s 4ms/step - loss: 2.0505e-04 - val_loss: 1.9022e-04
Epoch 278/350
31/31  0s 4ms/step - loss: 1.9466e-04 - val_loss: 1.9002e-04
Epoch 279/350
31/31  0s 4ms/step - loss: 2.0033e-04 - val_loss: 1.9028e-04
Epoch 280/350
31/31  0s 4ms/step - loss: 1.8371e-04 - val_loss: 1.9030e-04
Epoch 281/350
31/31  0s 3ms/step - loss: 2.1231e-04 - val_loss: 1.8999e-04
Epoch 282/350
31/31  0s 4ms/step - loss: 1.9531e-04 - val_loss: 1.9046e-04
Epoch 283/350
31/31  0s 4ms/step - loss: 2.1411e-04 - val_loss: 1.9029e-04
Epoch 284/350
31/31  0s 4ms/step - loss: 2.1142e-04 - val_loss: 1.9007e-04
Epoch 285/350
31/31  0s 3ms/step - loss: 2.0217e-04 - val_loss: 1.9016e-04
Epoch 286/350
31/31  0s 4ms/step - loss: 2.1213e-04 - val_loss: 1.8995e-04
Epoch 287/350
31/31  0s 4ms/step - loss: 1.8802e-04 - val_loss: 1.8999e-04
Epoch 288/350
31/31  0s 4ms/step - loss: 2.0642e-04 - val_loss: 1.9021e-04
Epoch 289/350
31/31  0s 4ms/step - loss: 2.0092e-04 - val_loss: 1.9008e-04
Epoch 290/350
31/31  0s 4ms/step - loss: 1.9661e-04 - val_loss: 1.8975e-04
Epoch 291/350
31/31  0s 4ms/step - loss: 1.9533e-04 - val_loss: 1.8989e-04
Epoch 292/350
31/31  0s 4ms/step - loss: 2.1424e-04 - val_loss: 1.8968e-04
Epoch 293/350
31/31  0s 4ms/step - loss: 2.0287e-04 - val_loss: 1.8986e-04
Epoch 294/350
31/31  0s 4ms/step - loss: 2.0000e-04 - val_loss: 1.9010e-04
Epoch 295/350
31/31  0s 4ms/step - loss: 2.0037e-04 - val_loss: 1.8980e-04
Epoch 296/350
31/31  0s 4ms/step - loss: 2.0847e-04 - val_loss: 1.8980e-04
Epoch 297/350
31/31  0s 4ms/step - loss: 1.9677e-04 - val_loss: 1.8966e-04
Epoch 298/350
31/31  0s 4ms/step - loss: 1.9978e-04 - val_loss: 1.8975e-04
Epoch 299/350
31/31  0s 4ms/step - loss: 2.0915e-04 - val_loss: 1.8989e-04
Epoch 300/350
31/31  0s 4ms/step - loss: 1.9817e-04 - val_loss: 1.8949e-04
```

```
Epoch 301/350
31/31  0s 4ms/step - loss: 2.0183e-04 - val_loss: 1.8987e-04
Epoch 302/350
31/31  0s 4ms/step - loss: 2.1049e-04 - val_loss: 1.8973e-04
Epoch 303/350
31/31  0s 4ms/step - loss: 2.0544e-04 - val_loss: 1.8961e-04
Epoch 304/350
31/31  0s 4ms/step - loss: 1.9764e-04 - val_loss: 1.8942e-04
Epoch 305/350
31/31  0s 3ms/step - loss: 1.9379e-04 - val_loss: 1.8957e-04
Epoch 306/350
31/31  0s 4ms/step - loss: 2.0660e-04 - val_loss: 1.8942e-04
Epoch 307/350
31/31  0s 4ms/step - loss: 2.0000e-04 - val_loss: 1.8958e-04
Epoch 308/350
31/31  0s 3ms/step - loss: 1.9812e-04 - val_loss: 1.8942e-04
Epoch 309/350
31/31  0s 4ms/step - loss: 1.9726e-04 - val_loss: 1.8940e-04
Epoch 310/350
31/31  0s 6ms/step - loss: 2.2123e-04 - val_loss: 1.8947e-04
Epoch 311/350
31/31  0s 5ms/step - loss: 2.1080e-04 - val_loss: 1.8898e-04
Epoch 312/350
31/31  0s 4ms/step - loss: 2.1011e-04 - val_loss: 1.8994e-04
Epoch 313/350
31/31  0s 4ms/step - loss: 2.1423e-04 - val_loss: 1.8910e-04
Epoch 314/350
31/31  0s 4ms/step - loss: 1.9890e-04 - val_loss: 1.8919e-04
Epoch 315/350
31/31  0s 5ms/step - loss: 1.9970e-04 - val_loss: 1.8936e-04
Epoch 316/350
31/31  0s 4ms/step - loss: 2.1570e-04 - val_loss: 1.8908e-04
Epoch 317/350
31/31  0s 4ms/step - loss: 2.1581e-04 - val_loss: 1.8916e-04
Epoch 318/350
31/31  0s 3ms/step - loss: 1.9793e-04 - val_loss: 1.8928e-04
Epoch 319/350
31/31  0s 4ms/step - loss: 1.9354e-04 - val_loss: 1.8905e-04
Epoch 320/350
31/31  0s 4ms/step - loss: 2.0648e-04 - val_loss: 1.8890e-04
Epoch 321/350
31/31  0s 4ms/step - loss: 2.0039e-04 - val_loss: 1.8914e-04
Epoch 322/350
31/31  0s 4ms/step - loss: 2.0062e-04 - val_loss: 1.8893e-04
Epoch 323/350
31/31  0s 4ms/step - loss: 2.0052e-04 - val_loss: 1.8900e-04
Epoch 324/350
31/31  0s 4ms/step - loss: 2.0428e-04 - val_loss: 1.8899e-04
Epoch 325/350
31/31  0s 4ms/step - loss: 1.8988e-04 - val_loss: 1.8915e-04
Epoch 326/350
31/31  0s 4ms/step - loss: 2.1172e-04 - val_loss: 1.8897e-04
Epoch 327/350
31/31  0s 4ms/step - loss: 1.9617e-04 - val_loss: 1.8871e-04
Epoch 328/350
31/31  0s 4ms/step - loss: 2.0471e-04 - val_loss: 1.8886e-04
Epoch 329/350
31/31  0s 5ms/step - loss: 2.0391e-04 - val_loss: 1.8920e-04
Epoch 330/350
31/31  0s 4ms/step - loss: 2.1750e-04 - val_loss: 1.8889e-04
```

```

Epoch 331/350
31/31  0s 4ms/step - loss: 1.8783e-04 - val_loss: 1.8890e-04
Epoch 332/350
31/31  0s 4ms/step - loss: 1.9765e-04 - val_loss: 1.8906e-04
Epoch 333/350
31/31  0s 3ms/step - loss: 2.0759e-04 - val_loss: 1.8882e-04
Epoch 334/350
31/31  0s 4ms/step - loss: 1.9244e-04 - val_loss: 1.8888e-04
Epoch 335/350
31/31  0s 4ms/step - loss: 2.1368e-04 - val_loss: 1.8897e-04
Epoch 336/350
31/31  0s 3ms/step - loss: 2.1242e-04 - val_loss: 1.8894e-04
Epoch 337/350
31/31  0s 4ms/step - loss: 2.0768e-04 - val_loss: 1.8857e-04
Epoch 338/350
31/31  0s 4ms/step - loss: 2.1237e-04 - val_loss: 1.8893e-04
Epoch 339/350
31/31  0s 4ms/step - loss: 2.0179e-04 - val_loss: 1.8875e-04
Epoch 340/350
31/31  0s 4ms/step - loss: 1.9195e-04 - val_loss: 1.8893e-04
Epoch 341/350
31/31  0s 4ms/step - loss: 1.8425e-04 - val_loss: 1.8900e-04
Epoch 342/350
31/31  0s 4ms/step - loss: 1.9478e-04 - val_loss: 1.8862e-04
Epoch 343/350
31/31  0s 4ms/step - loss: 1.9228e-04 - val_loss: 1.8914e-04
Epoch 344/350
31/31  0s 4ms/step - loss: 2.0520e-04 - val_loss: 1.8873e-04
Epoch 345/350
31/31  0s 4ms/step - loss: 2.1454e-04 - val_loss: 1.8882e-04
Epoch 346/350
31/31  0s 4ms/step - loss: 2.1062e-04 - val_loss: 1.8889e-04
Epoch 347/350
31/31  0s 4ms/step - loss: 1.8721e-04 - val_loss: 1.8845e-04
Epoch 348/350
31/31  0s 4ms/step - loss: 2.0844e-04 - val_loss: 1.8903e-04
Epoch 349/350
31/31  0s 4ms/step - loss: 2.0617e-04 - val_loss: 1.8870e-04
Epoch 350/350
31/31  0s 4ms/step - loss: 1.9899e-04 - val_loss: 1.8859e-04

```

Out[19]: <keras.src.callbacks.history.History at 0x7968c842fe50>

```

In [20]: Y_pred = model_nonlinear.predict(Y_train, verbose=0)
error_non = np.sum((Y_train - Y_pred) ** 2) / np.sum((Y_train - np.mean(Y_train)
print("Reconstruction error in training sample", error_non)
Y_pred = model_nonlinear.predict(Y_test, verbose=0)
error_non_oos = np.sum((Y_test - Y_pred) ** 2) / np.sum((Y_test - np.mean(Y_test)
print("Reconstruction error in test sample", error_non_oos)

```

```

Reconstruction error in training sample 0.3094527914957498
Reconstruction error in test sample 0.4788719014902

```

```

In [21]: factors = model_nonlinear.encoder(Y_train).numpy()
factors_test = model_nonlinear.encoder(Y_test).numpy()
print(np.cov(factors, rowvar=False))

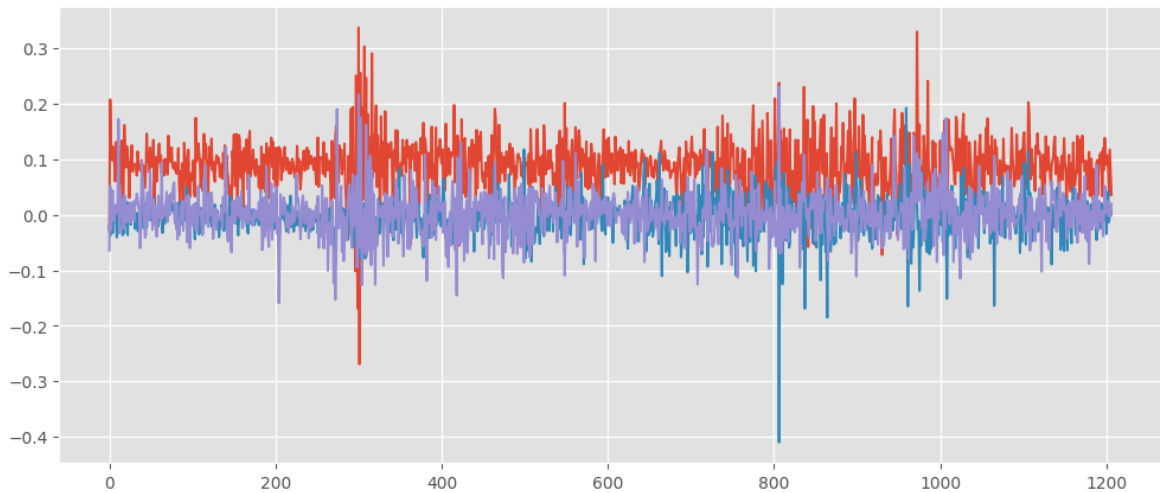
```

```

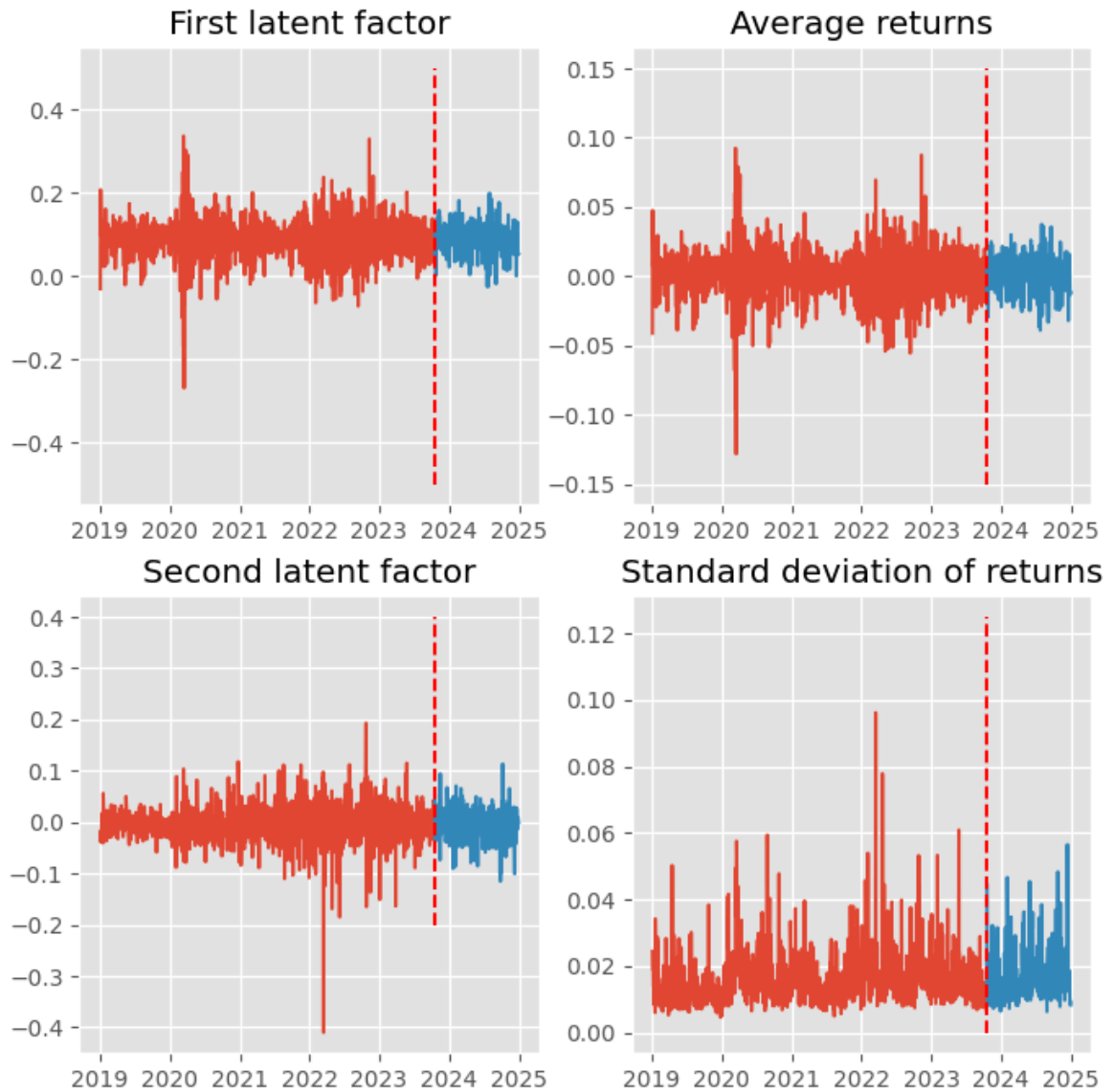
[[ 2.41902423e-03  1.45583215e-04  5.84779443e-04]
 [ 1.45583215e-04  1.35614948e-03 -5.56242140e-06]
 [ 5.84779443e-04 -5.56242140e-06  1.74229230e-03]]

```

```
In [22]: # Plot the evolution of latent factors in the training period
plt.figure(figsize=(12, 5))
plt.plot(factors)
plt.show()
```

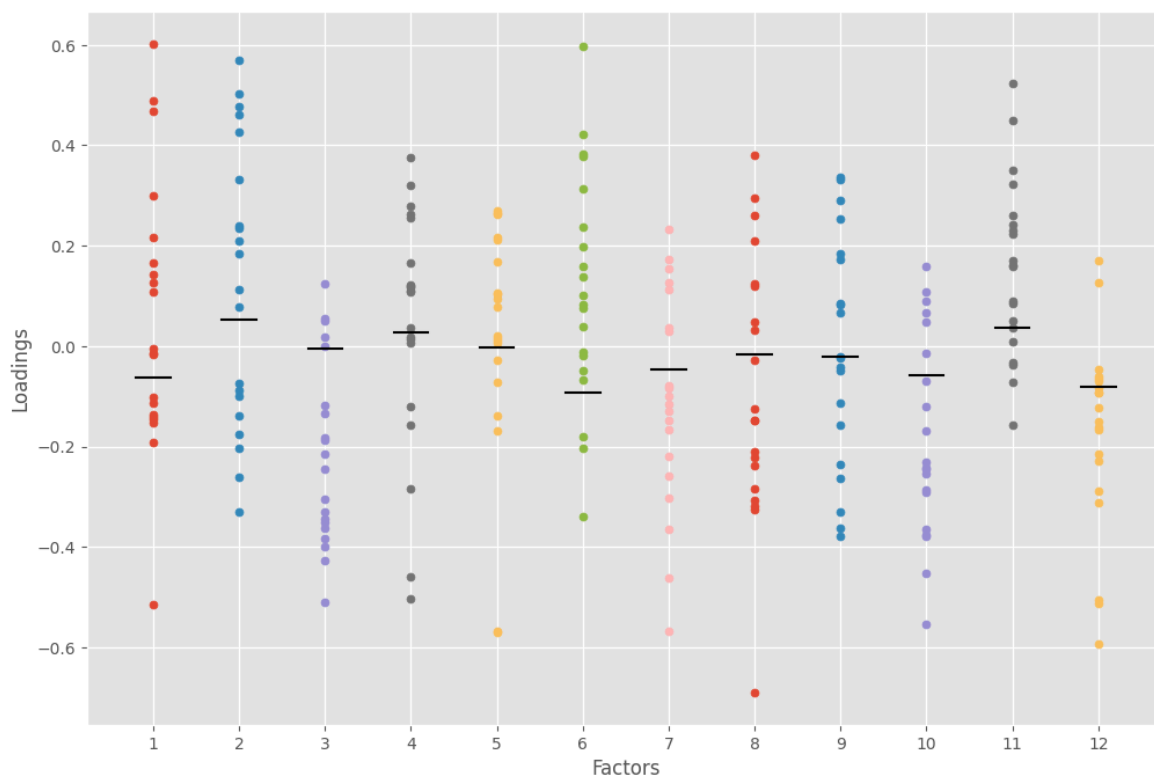


```
In [23]: fig, axs = plt.subplots(2, 2, figsize=(8, 8))
# First factor
axs[0, 0].plot(df.index[: train_size + 1], factors[:, 0])
axs[0, 0].plot(df.index[train_size + 1 :], factors_test[:, 0])
axs[0, 0].plot([df.index[train_size + 1], df.index[train_size + 1]], [-0.5, 0.5])
axs[0, 0].title.set_text("First latent factor")
# Average of returns
axs[0, 1].plot(df.index[: train_size + 1], np.mean(Y_train, axis=1))
axs[0, 1].plot(df.index[train_size + 1 :], np.mean(Y_test, axis=1))
axs[0, 1].plot(
    [df.index[train_size + 1], df.index[train_size + 1]], [-0.15, 0.15], "r--"
)
axs[0, 1].title.set_text("Average returns")
# Second factor
axs[1, 0].plot(df.index[: train_size + 1], factors[:, 1])
axs[1, 0].plot(df.index[train_size + 1 :], factors_test[:, 1])
axs[1, 0].plot([df.index[train_size + 1], df.index[train_size + 1]], [-0.2, 0.4])
axs[1, 0].title.set_text("Second latent factor")
# Standard deviation of returns
axs[1, 1].plot(df.index[: train_size + 1], np.std(Y_train, axis=1))
axs[1, 1].plot(df.index[train_size + 1 :], np.std(Y_test, axis=1))
axs[1, 1].plot(
    [df.index[train_size + 1], df.index[train_size + 1]], [0.0, 0.125], "r--"
)
axs[1, 1].title.set_text("Standard deviation of returns")
```



```
In [24]: # Extract Loadings generated by the autoencoder in the output layer
w_decoder = model_nonlinear.decoder.layers[1].get_weights()[0]
data = w_decoder
labels = list(range(1, 13))
width = 0.4
fig, ax = plt.subplots(figsize=(12, 8))
for i, l in enumerate(labels):
    x = np.ones(data.shape[1]) * i
    ax.scatter(x, data[i, :], s=25)
    mean = data[:, i].mean()
    ax.plot([i - width / 2.0, i + width / 2.0], [mean, mean], color="k")
    ax.set_xlabel("Factors")
    ax.set_ylabel("Loadings")
ax.set_xticks(range(len(labels)))
ax.set_xticklabels(labels)

plt.show()
```



5. CONCLUSION

In this notebook, we have learned to design and train autoencoders with TensorFlow as a tool to extract the main latent factors that drive the variation of financial time series.

In the next lesson, we will explore tools to refine autoencoders and describe other types of architectures.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

DEEP LEARNING FOR FINANCE

MODULE 6 | LESSON 4

PORTFOLIO CHOICE WITH A SPARSE AUTOENCODER

Reading Time 90 minutes

Prior Knowledge Neural Networks, machine learning, PCA, linear algebra

Keywords Autoencoder, portfolio choice

1. INTRODUCTION

In this notebook, we will build and train autoencoder models using the same data that we exploited in Lesson 2's notebook and some of the tools that we discussed in Lesson 3.

Additionally, we will implement the predictions of the autoencoder model to design a portfolio of stocks that maximizes the expected risk-weighted return in sample and compare its performance with other, more naïve strategies.

First of all, we load the libraries and data that we need for this notebook.

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: import tensorflow as tf
```

```
2025-05-26 12:36:29.001036: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-26 12:36:29.052234: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-05-26 12:36:30.581926: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

```
In [3]: # List of tickers (same order)
tickers = [
    "AAPL", "MSFT", "GOOG", "AMZN", "TSLA",
    "TCEHY", "NVDA", "TSM", "META", "BABA",
```

```

    "ASML", "AVGO", "ORCL", "CSCO", "ADBE",
    "CRM", "NFLX", "TXN", "QCOM", "SAP"
]

# Read CSV directly with Date as index
print("Loading data from CSV...")
data = pd.read_csv('adjusted_prices.csv', index_col='Date', parse_dates=True)

print(f"Data loaded: {data.shape}")
print(f"Complete range: {data.index[0].date()} to {data.index[-1].date()}")

# Calculate daily returns (pct_change) for all tickers
print("Calculating daily returns...")
returns_data = data[tickers].pct_change()

# Remove first row (NaN from pct_change)
df = returns_data.dropna()

df.head()

```

Loading data from CSV...
 Data loaded: (1509, 20)
 Complete range: 2019-01-02 to 2024-12-30
 Calculating daily returns...

Out[3]:

	AAPL	MSFT	GOOG	AMZN	TSLA	TCEHY	NVDA	
Date								
2019-01-03 00:00:05:00	-0.099607	-0.036788	-0.028484	-0.025241	-0.031472	-0.028347	-0.060417	-0.010000
2019-01-04 00:00:05:00	0.042689	0.046509	0.053786	0.050064	0.057697	0.058088	0.064068	0.010000
2019-01-07 00:00:05:00	-0.002226	0.001275	-0.002167	0.034353	0.054361	0.006893	0.052940	0.010000
2019-01-08 00:00:05:00	0.019063	0.007251	0.007385	0.016612	0.001164	-0.003423	-0.024895	-0.010000
2019-01-09 00:00:05:00	0.016981	0.014300	-0.001505	0.001714	0.009483	0.038518	0.019667	0.010000

In [4]:

```

print(f"Final DataFrame: {df.shape}")
print("Tickers included:", df.columns.tolist())

# Train/test split (80/20)
train_size = int(0.8 * df.shape[0])

```

```
Y_train = df.iloc[:train_size + 1].to_numpy()
Y_test = df.iloc[train_size + 1:].to_numpy()
```

Final DataFrame: (1508, 20)

Tickers included: ['AAPL', 'MSFT', 'GOOG', 'AMZN', 'TSLA', 'TCEHY', 'NVDA', 'TSM', 'META', 'BABA', 'ASML', 'AVGO', 'ORCL', 'CSCO', 'ADBE', 'CRM', 'NFLX', 'TXN', 'QCOM', 'SAP']

```
In [5]: print(f"\nBeginning of training sample: {df.index[0].date()}")
print(f"End of training sample: {df.index[train_size].date()}")
print(f"End of test sample: {df.index[-1].date()}")
print(f"Training samples: {Y_train.shape}")
print(f"Test samples: {Y_test.shape}")
```

Beginning of training sample: 2019-01-03

End of training sample: 2023-10-18

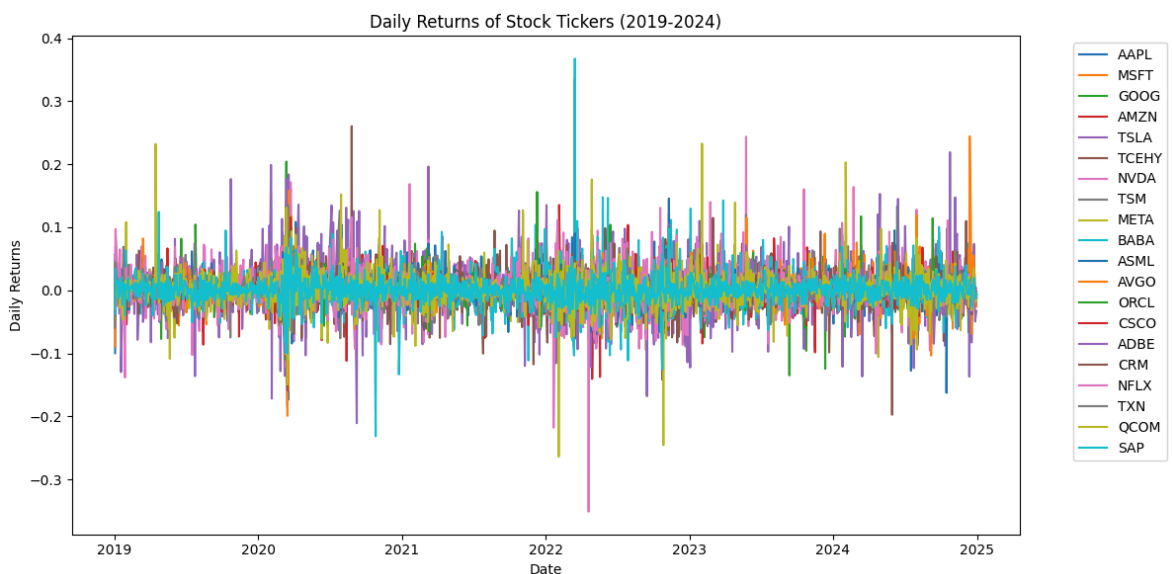
End of test sample: 2024-12-30

Training samples: (1207, 20)

Test samples: (301, 20)

```
In [6]: import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))
plt.plot(df.index, df)
plt.title(f'Daily Returns of Stock Tickers ({df.index[0].year}-{df.index[-1].year})')
plt.xlabel('Date')
plt.ylabel('Daily Returns')
plt.legend(df.columns, bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



2. BUILDING AND TRAINING A SPARSE AUTOENCODER

We will design a portfolio based on the predictions of an autoencoder model with a loss function that includes a regularization term on the value of the factors generated by the model.

In TensorFlow, we add an L1-regularizer in the bottleneck

"tf.keras.regularizers.l1(l1weight)", where "l1weight" is a hyperparameter that governs the

degree of regularization on the factor space. This parameter will determine the actual dimension of the bottleneck, without us having to impose it beforehand.

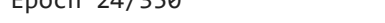
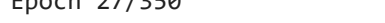
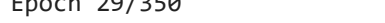
```
In [7]: class sparse_autoencoder(tf.keras.Model):
    def __init__(self, output_dim, bottleneck_dim=1, l1weight=0.0):
        super(sparse_autoencoder, self).__init__()
        self.output_dim = output_dim
        self.bottleneck_dim = bottleneck_dim
        self.l1weight = l1weight
        initializer = tf.keras.initializers.GlorotUniform()
        # Set the activity regularizer that will generate the sparsity of factor
        activity_regularizer = tf.keras.regularizers.l1(l1weight)
        # Build the encoding layer, including the activity regularizer
        self.encoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(
                    bottleneck_dim,
                    activity_regularizer=activity_regularizer,
                    kernel_initializer=initializer,
                    bias_initializer=tf.keras.initializers.Zeros(),
                )
            ]
        )
        # Build the decoding layer
        self.decoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(
                    output_dim,
                    kernel_initializer=initializer,
                    bias_initializer=tf.keras.initializers.Zeros(),
                )
            ]
        )

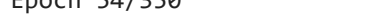
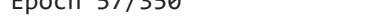
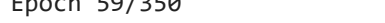
    def call(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded
```

Let's compile and train the model. We set the L1-regularization parameter to $\lambda_1=0.0025$.

```
In [8]: # Set the bottleneck size equal to the input size
nlatent = Y_train.shape[1]
# Set the parameter on the L1 activity-regularization term
l1weight = 0.0025
# Choose the proportion of training data used for validation
val_split = 0.2
# Determine early stopping criterion
early_stop = tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=50)
# Choose optimizer
adam = tf.keras.optimizers.Adam(learning_rate=1e-4)
# Build and compile the model
model = sparse_autoencoder(Y_train.shape[1], nlatent, l1weight)
model.compile(optimizer=adam, loss="mse")
# Train the model
model.fit(
    x=Y_train,
```

```
y=Y_train,  
batch_size=32,  
epochs=350,  
validation_split=val_split,  
callbacks=[early_stop],  
verbose=1,  
)
```

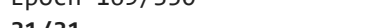
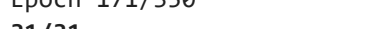
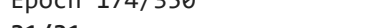
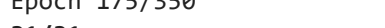
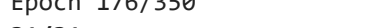
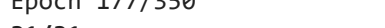
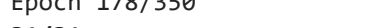
```
Epoch 1/350
31/31  1s 8ms/step - loss: 0.0273 - val_loss: 0.0245
Epoch 2/350
31/31  0s 4ms/step - loss: 0.0267 - val_loss: 0.0239
Epoch 3/350
31/31  0s 4ms/step - loss: 0.0268 - val_loss: 0.0234
Epoch 4/350
31/31  0s 5ms/step - loss: 0.0252 - val_loss: 0.0229
Epoch 5/350
31/31  0s 4ms/step - loss: 0.0241 - val_loss: 0.0225
Epoch 6/350
31/31  0s 4ms/step - loss: 0.0243 - val_loss: 0.0220
Epoch 7/350
31/31  0s 4ms/step - loss: 0.0238 - val_loss: 0.0216
Epoch 8/350
31/31  0s 4ms/step - loss: 0.0231 - val_loss: 0.0213
Epoch 9/350
31/31  0s 4ms/step - loss: 0.0232 - val_loss: 0.0209
Epoch 10/350
31/31  0s 4ms/step - loss: 0.0224 - val_loss: 0.0206
Epoch 11/350
31/31  0s 3ms/step - loss: 0.0214 - val_loss: 0.0203
Epoch 12/350
31/31  0s 4ms/step - loss: 0.0210 - val_loss: 0.0200
Epoch 13/350
31/31  0s 4ms/step - loss: 0.0214 - val_loss: 0.0197
Epoch 14/350
31/31  0s 4ms/step - loss: 0.0216 - val_loss: 0.0195
Epoch 15/350
31/31  0s 4ms/step - loss: 0.0200 - val_loss: 0.0192
Epoch 16/350
31/31  0s 4ms/step - loss: 0.0202 - val_loss: 0.0190
Epoch 17/350
31/31  0s 3ms/step - loss: 0.0204 - val_loss: 0.0188
Epoch 18/350
31/31  0s 4ms/step - loss: 0.0198 - val_loss: 0.0186
Epoch 19/350
31/31  0s 4ms/step - loss: 0.0196 - val_loss: 0.0184
Epoch 20/350
31/31  0s 4ms/step - loss: 0.0188 - val_loss: 0.0182
Epoch 21/350
31/31  0s 4ms/step - loss: 0.0183 - val_loss: 0.0180
Epoch 22/350
31/31  0s 4ms/step - loss: 0.0185 - val_loss: 0.0179
Epoch 23/350
31/31  0s 4ms/step - loss: 0.0186 - val_loss: 0.0177
Epoch 24/350
31/31  0s 4ms/step - loss: 0.0184 - val_loss: 0.0175
Epoch 25/350
31/31  0s 3ms/step - loss: 0.0181 - val_loss: 0.0174
Epoch 26/350
31/31  0s 4ms/step - loss: 0.0177 - val_loss: 0.0172
Epoch 27/350
31/31  0s 4ms/step - loss: 0.0181 - val_loss: 0.0171
Epoch 28/350
31/31  0s 4ms/step - loss: 0.0176 - val_loss: 0.0169
Epoch 29/350
31/31  0s 4ms/step - loss: 0.0171 - val_loss: 0.0168
Epoch 30/350
31/31  0s 3ms/step - loss: 0.0170 - val_loss: 0.0166
```

```
Epoch 31/350
31/31  0s 3ms/step - loss: 0.0167 - val_loss: 0.0165
Epoch 32/350
31/31  0s 3ms/step - loss: 0.0171 - val_loss: 0.0163
Epoch 33/350
31/31  0s 4ms/step - loss: 0.0171 - val_loss: 0.0162
Epoch 34/350
31/31  0s 4ms/step - loss: 0.0164 - val_loss: 0.0161
Epoch 35/350
31/31  0s 4ms/step - loss: 0.0165 - val_loss: 0.0159
Epoch 36/350
31/31  0s 4ms/step - loss: 0.0160 - val_loss: 0.0158
Epoch 37/350
31/31  0s 4ms/step - loss: 0.0165 - val_loss: 0.0156
Epoch 38/350
31/31  0s 4ms/step - loss: 0.0163 - val_loss: 0.0155
Epoch 39/350
31/31  0s 3ms/step - loss: 0.0161 - val_loss: 0.0154
Epoch 40/350
31/31  0s 4ms/step - loss: 0.0154 - val_loss: 0.0152
Epoch 41/350
31/31  0s 4ms/step - loss: 0.0158 - val_loss: 0.0151
Epoch 42/350
31/31  0s 4ms/step - loss: 0.0150 - val_loss: 0.0150
Epoch 43/350
31/31  0s 3ms/step - loss: 0.0156 - val_loss: 0.0148
Epoch 44/350
31/31  0s 4ms/step - loss: 0.0151 - val_loss: 0.0147
Epoch 45/350
31/31  0s 4ms/step - loss: 0.0148 - val_loss: 0.0146
Epoch 46/350
31/31  0s 4ms/step - loss: 0.0147 - val_loss: 0.0144
Epoch 47/350
31/31  0s 5ms/step - loss: 0.0147 - val_loss: 0.0143
Epoch 48/350
31/31  0s 4ms/step - loss: 0.0147 - val_loss: 0.0142
Epoch 49/350
31/31  0s 4ms/step - loss: 0.0144 - val_loss: 0.0140
Epoch 50/350
31/31  0s 4ms/step - loss: 0.0146 - val_loss: 0.0139
Epoch 51/350
31/31  0s 3ms/step - loss: 0.0141 - val_loss: 0.0138
Epoch 52/350
31/31  0s 3ms/step - loss: 0.0140 - val_loss: 0.0136
Epoch 53/350
31/31  0s 4ms/step - loss: 0.0139 - val_loss: 0.0135
Epoch 54/350
31/31  0s 3ms/step - loss: 0.0134 - val_loss: 0.0134
Epoch 55/350
31/31  0s 4ms/step - loss: 0.0136 - val_loss: 0.0133
Epoch 56/350
31/31  0s 4ms/step - loss: 0.0135 - val_loss: 0.0131
Epoch 57/350
31/31  0s 4ms/step - loss: 0.0133 - val_loss: 0.0130
Epoch 58/350
31/31  0s 4ms/step - loss: 0.0132 - val_loss: 0.0129
Epoch 59/350
31/31  0s 4ms/step - loss: 0.0130 - val_loss: 0.0127
Epoch 60/350
31/31  0s 4ms/step - loss: 0.0128 - val_loss: 0.0126
```

```
Epoch 61/350
31/31  0s 4ms/step - loss: 0.0126 - val_loss: 0.0125
Epoch 62/350
31/31  0s 4ms/step - loss: 0.0127 - val_loss: 0.0124
Epoch 63/350
31/31  0s 4ms/step - loss: 0.0123 - val_loss: 0.0122
Epoch 64/350
31/31  0s 3ms/step - loss: 0.0124 - val_loss: 0.0121
Epoch 65/350
31/31  0s 4ms/step - loss: 0.0121 - val_loss: 0.0120
Epoch 66/350
31/31  0s 4ms/step - loss: 0.0122 - val_loss: 0.0119
Epoch 67/350
31/31  0s 4ms/step - loss: 0.0117 - val_loss: 0.0118
Epoch 68/350
31/31  0s 4ms/step - loss: 0.0118 - val_loss: 0.0116
Epoch 69/350
31/31  0s 3ms/step - loss: 0.0118 - val_loss: 0.0115
Epoch 70/350
31/31  0s 4ms/step - loss: 0.0118 - val_loss: 0.0114
Epoch 71/350
31/31  0s 4ms/step - loss: 0.0114 - val_loss: 0.0113
Epoch 72/350
31/31  0s 4ms/step - loss: 0.0114 - val_loss: 0.0112
Epoch 73/350
31/31  0s 4ms/step - loss: 0.0113 - val_loss: 0.0110
Epoch 74/350
31/31  0s 4ms/step - loss: 0.0109 - val_loss: 0.0109
Epoch 75/350
31/31  0s 4ms/step - loss: 0.0106 - val_loss: 0.0108
Epoch 76/350
31/31  0s 4ms/step - loss: 0.0106 - val_loss: 0.0107
Epoch 77/350
31/31  0s 3ms/step - loss: 0.0106 - val_loss: 0.0106
Epoch 78/350
31/31  0s 3ms/step - loss: 0.0107 - val_loss: 0.0104
Epoch 79/350
31/31  0s 4ms/step - loss: 0.0103 - val_loss: 0.0103
Epoch 80/350
31/31  0s 4ms/step - loss: 0.0104 - val_loss: 0.0102
Epoch 81/350
31/31  0s 4ms/step - loss: 0.0100 - val_loss: 0.0101
Epoch 82/350
31/31  0s 3ms/step - loss: 0.0102 - val_loss: 0.0100
Epoch 83/350
31/31  0s 4ms/step - loss: 0.0099 - val_loss: 0.0099
Epoch 84/350
31/31  0s 4ms/step - loss: 0.0099 - val_loss: 0.0097
Epoch 85/350
31/31  0s 4ms/step - loss: 0.0097 - val_loss: 0.0096
Epoch 86/350
31/31  0s 4ms/step - loss: 0.0096 - val_loss: 0.0095
Epoch 87/350
31/31  0s 4ms/step - loss: 0.0094 - val_loss: 0.0094
Epoch 88/350
31/31  0s 4ms/step - loss: 0.0094 - val_loss: 0.0093
Epoch 89/350
31/31  0s 4ms/step - loss: 0.0094 - val_loss: 0.0092
Epoch 90/350
31/31  0s 4ms/step - loss: 0.0091 - val_loss: 0.0091
```

```
Epoch 91/350
31/31  0s 3ms/step - loss: 0.0091 - val_loss: 0.0089
Epoch 92/350
31/31  0s 4ms/step - loss: 0.0090 - val_loss: 0.0088
Epoch 93/350
31/31  0s 4ms/step - loss: 0.0084 - val_loss: 0.0087
Epoch 94/350
31/31  0s 4ms/step - loss: 0.0087 - val_loss: 0.0086
Epoch 95/350
31/31  0s 4ms/step - loss: 0.0086 - val_loss: 0.0085
Epoch 96/350
31/31  0s 4ms/step - loss: 0.0084 - val_loss: 0.0084
Epoch 97/350
31/31  0s 3ms/step - loss: 0.0084 - val_loss: 0.0083
Epoch 98/350
31/31  0s 4ms/step - loss: 0.0082 - val_loss: 0.0082
Epoch 99/350
31/31  0s 4ms/step - loss: 0.0082 - val_loss: 0.0081
Epoch 100/350
31/31  0s 4ms/step - loss: 0.0078 - val_loss: 0.0080
Epoch 101/350
31/31  0s 3ms/step - loss: 0.0078 - val_loss: 0.0079
Epoch 102/350
31/31  0s 3ms/step - loss: 0.0079 - val_loss: 0.0078
Epoch 103/350
31/31  0s 4ms/step - loss: 0.0078 - val_loss: 0.0077
Epoch 104/350
31/31  0s 4ms/step - loss: 0.0075 - val_loss: 0.0076
Epoch 105/350
31/31  0s 4ms/step - loss: 0.0074 - val_loss: 0.0075
Epoch 106/350
31/31  0s 3ms/step - loss: 0.0075 - val_loss: 0.0074
Epoch 107/350
31/31  0s 4ms/step - loss: 0.0071 - val_loss: 0.0073
Epoch 108/350
31/31  0s 4ms/step - loss: 0.0072 - val_loss: 0.0072
Epoch 109/350
31/31  0s 4ms/step - loss: 0.0073 - val_loss: 0.0071
Epoch 110/350
31/31  0s 6ms/step - loss: 0.0069 - val_loss: 0.0070
Epoch 111/350
31/31  0s 5ms/step - loss: 0.0071 - val_loss: 0.0069
Epoch 112/350
31/31  0s 4ms/step - loss: 0.0067 - val_loss: 0.0068
Epoch 113/350
31/31  0s 4ms/step - loss: 0.0066 - val_loss: 0.0067
Epoch 114/350
31/31  0s 4ms/step - loss: 0.0067 - val_loss: 0.0066
Epoch 115/350
31/31  0s 4ms/step - loss: 0.0064 - val_loss: 0.0065
Epoch 116/350
31/31  0s 4ms/step - loss: 0.0065 - val_loss: 0.0064
Epoch 117/350
31/31  0s 4ms/step - loss: 0.0063 - val_loss: 0.0063
Epoch 118/350
31/31  0s 5ms/step - loss: 0.0063 - val_loss: 0.0062
Epoch 119/350
31/31  0s 4ms/step - loss: 0.0060 - val_loss: 0.0061
Epoch 120/350
31/31  0s 4ms/step - loss: 0.0061 - val_loss: 0.0060
```

```
Epoch 121/350
31/31  0s 4ms/step - loss: 0.0059 - val_loss: 0.0060
Epoch 122/350
31/31  0s 4ms/step - loss: 0.0057 - val_loss: 0.0059
Epoch 123/350
31/31  0s 4ms/step - loss: 0.0056 - val_loss: 0.0058
Epoch 124/350
31/31  0s 3ms/step - loss: 0.0056 - val_loss: 0.0057
Epoch 125/350
31/31  0s 4ms/step - loss: 0.0057 - val_loss: 0.0056
Epoch 126/350
31/31  0s 4ms/step - loss: 0.0054 - val_loss: 0.0055
Epoch 127/350
31/31  0s 4ms/step - loss: 0.0055 - val_loss: 0.0054
Epoch 128/350
31/31  0s 3ms/step - loss: 0.0052 - val_loss: 0.0053
Epoch 129/350
31/31  0s 4ms/step - loss: 0.0053 - val_loss: 0.0052
Epoch 130/350
31/31  0s 4ms/step - loss: 0.0050 - val_loss: 0.0052
Epoch 131/350
31/31  0s 4ms/step - loss: 0.0050 - val_loss: 0.0051
Epoch 132/350
31/31  0s 4ms/step - loss: 0.0049 - val_loss: 0.0050
Epoch 133/350
31/31  0s 4ms/step - loss: 0.0050 - val_loss: 0.0049
Epoch 134/350
31/31  0s 4ms/step - loss: 0.0048 - val_loss: 0.0048
Epoch 135/350
31/31  0s 4ms/step - loss: 0.0046 - val_loss: 0.0048
Epoch 136/350
31/31  0s 4ms/step - loss: 0.0046 - val_loss: 0.0047
Epoch 137/350
31/31  0s 4ms/step - loss: 0.0045 - val_loss: 0.0046
Epoch 138/350
31/31  0s 3ms/step - loss: 0.0046 - val_loss: 0.0045
Epoch 139/350
31/31  0s 4ms/step - loss: 0.0044 - val_loss: 0.0044
Epoch 140/350
31/31  0s 3ms/step - loss: 0.0044 - val_loss: 0.0043
Epoch 141/350
31/31  0s 4ms/step - loss: 0.0042 - val_loss: 0.0043
Epoch 142/350
31/31  0s 4ms/step - loss: 0.0042 - val_loss: 0.0042
Epoch 143/350
31/31  0s 3ms/step - loss: 0.0040 - val_loss: 0.0041
Epoch 144/350
31/31  0s 4ms/step - loss: 0.0039 - val_loss: 0.0040
Epoch 145/350
31/31  0s 4ms/step - loss: 0.0041 - val_loss: 0.0039
Epoch 146/350
31/31  0s 4ms/step - loss: 0.0039 - val_loss: 0.0039
Epoch 147/350
31/31  0s 4ms/step - loss: 0.0037 - val_loss: 0.0038
Epoch 148/350
31/31  0s 4ms/step - loss: 0.0035 - val_loss: 0.0037
Epoch 149/350
31/31  0s 3ms/step - loss: 0.0036 - val_loss: 0.0037
Epoch 150/350
31/31  0s 4ms/step - loss: 0.0035 - val_loss: 0.0036
```

```
Epoch 151/350
31/31  0s 4ms/step - loss: 0.0035 - val_loss: 0.0035
Epoch 152/350
31/31  0s 4ms/step - loss: 0.0033 - val_loss: 0.0034
Epoch 153/350
31/31  0s 4ms/step - loss: 0.0034 - val_loss: 0.0034
Epoch 154/350
31/31  0s 4ms/step - loss: 0.0033 - val_loss: 0.0033
Epoch 155/350
31/31  0s 4ms/step - loss: 0.0032 - val_loss: 0.0032
Epoch 156/350
31/31  0s 3ms/step - loss: 0.0032 - val_loss: 0.0032
Epoch 157/350
31/31  0s 5ms/step - loss: 0.0030 - val_loss: 0.0031
Epoch 158/350
31/31  0s 4ms/step - loss: 0.0029 - val_loss: 0.0030
Epoch 159/350
31/31  0s 4ms/step - loss: 0.0029 - val_loss: 0.0030
Epoch 160/350
31/31  0s 3ms/step - loss: 0.0028 - val_loss: 0.0029
Epoch 161/350
31/31  0s 4ms/step - loss: 0.0029 - val_loss: 0.0028
Epoch 162/350
31/31  0s 4ms/step - loss: 0.0027 - val_loss: 0.0027
Epoch 163/350
31/31  0s 4ms/step - loss: 0.0027 - val_loss: 0.0027
Epoch 164/350
31/31  0s 4ms/step - loss: 0.0026 - val_loss: 0.0026
Epoch 165/350
31/31  0s 4ms/step - loss: 0.0025 - val_loss: 0.0025
Epoch 166/350
31/31  0s 4ms/step - loss: 0.0024 - val_loss: 0.0025
Epoch 167/350
31/31  0s 4ms/step - loss: 0.0024 - val_loss: 0.0024
Epoch 168/350
31/31  0s 3ms/step - loss: 0.0024 - val_loss: 0.0024
Epoch 169/350
31/31  0s 4ms/step - loss: 0.0024 - val_loss: 0.0023
Epoch 170/350
31/31  0s 3ms/step - loss: 0.0022 - val_loss: 0.0022
Epoch 171/350
31/31  0s 3ms/step - loss: 0.0022 - val_loss: 0.0022
Epoch 172/350
31/31  0s 4ms/step - loss: 0.0021 - val_loss: 0.0021
Epoch 173/350
31/31  0s 4ms/step - loss: 0.0021 - val_loss: 0.0021
Epoch 174/350
31/31  0s 3ms/step - loss: 0.0020 - val_loss: 0.0020
Epoch 175/350
31/31  0s 4ms/step - loss: 0.0020 - val_loss: 0.0020
Epoch 176/350
31/31  0s 4ms/step - loss: 0.0019 - val_loss: 0.0019
Epoch 177/350
31/31  0s 4ms/step - loss: 0.0019 - val_loss: 0.0019
Epoch 178/350
31/31  0s 4ms/step - loss: 0.0018 - val_loss: 0.0018
Epoch 179/350
31/31  0s 4ms/step - loss: 0.0018 - val_loss: 0.0018
Epoch 180/350
31/31  0s 4ms/step - loss: 0.0018 - val_loss: 0.0017
```

```
Epoch 181/350
31/31  0s 3ms/step - loss: 0.0017 - val_loss: 0.0017
Epoch 182/350
31/31  0s 4ms/step - loss: 0.0017 - val_loss: 0.0016
Epoch 183/350
31/31  0s 4ms/step - loss: 0.0017 - val_loss: 0.0016
Epoch 184/350
31/31  0s 4ms/step - loss: 0.0017 - val_loss: 0.0016
Epoch 185/350
31/31  0s 4ms/step - loss: 0.0016 - val_loss: 0.0015
Epoch 186/350
31/31  0s 3ms/step - loss: 0.0015 - val_loss: 0.0015
Epoch 187/350
31/31  0s 3ms/step - loss: 0.0015 - val_loss: 0.0014
Epoch 188/350
31/31  0s 4ms/step - loss: 0.0014 - val_loss: 0.0014
Epoch 189/350
31/31  0s 4ms/step - loss: 0.0014 - val_loss: 0.0014
Epoch 190/350
31/31  0s 4ms/step - loss: 0.0014 - val_loss: 0.0013
Epoch 191/350
31/31  0s 3ms/step - loss: 0.0013 - val_loss: 0.0013
Epoch 192/350
31/31  0s 4ms/step - loss: 0.0013 - val_loss: 0.0012
Epoch 193/350
31/31  0s 3ms/step - loss: 0.0012 - val_loss: 0.0012
Epoch 194/350
31/31  0s 4ms/step - loss: 0.0012 - val_loss: 0.0012
Epoch 195/350
31/31  0s 4ms/step - loss: 0.0012 - val_loss: 0.0011
Epoch 196/350
31/31  0s 4ms/step - loss: 0.0011 - val_loss: 0.0011
Epoch 197/350
31/31  0s 4ms/step - loss: 0.0011 - val_loss: 0.0011
Epoch 198/350
31/31  0s 4ms/step - loss: 0.0012 - val_loss: 0.0010
Epoch 199/350
31/31  0s 4ms/step - loss: 0.0011 - val_loss: 0.0010
Epoch 200/350
31/31  0s 4ms/step - loss: 0.0011 - val_loss: 9.9598e-04
Epoch 201/350
31/31  0s 4ms/step - loss: 0.0010 - val_loss: 9.6657e-04
Epoch 202/350
31/31  0s 4ms/step - loss: 0.0010 - val_loss: 9.4100e-04
Epoch 203/350
31/31  0s 4ms/step - loss: 0.0010 - val_loss: 9.2630e-04
Epoch 204/350
31/31  0s 4ms/step - loss: 9.4837e-04 - val_loss: 9.0591e-04
Epoch 205/350
31/31  0s 3ms/step - loss: 0.0010 - val_loss: 8.8512e-04
Epoch 206/350
31/31  0s 4ms/step - loss: 0.0010 - val_loss: 8.5939e-04
Epoch 207/350
31/31  0s 4ms/step - loss: 9.4026e-04 - val_loss: 8.5006e-04
Epoch 208/350
31/31  0s 4ms/step - loss: 8.9723e-04 - val_loss: 8.3320e-04
Epoch 209/350
31/31  0s 4ms/step - loss: 9.0880e-04 - val_loss: 8.0905e-04
Epoch 210/350
31/31  0s 4ms/step - loss: 8.8811e-04 - val_loss: 8.0341e-04
```

```
Epoch 211/350
31/31  0s 4ms/step - loss: 8.6462e-04 - val_loss: 7.8496e-04
Epoch 212/350
31/31  0s 4ms/step - loss: 9.6199e-04 - val_loss: 7.6977e-04
Epoch 213/350
31/31  0s 4ms/step - loss: 8.7442e-04 - val_loss: 7.5621e-04
Epoch 214/350
31/31  0s 4ms/step - loss: 7.9466e-04 - val_loss: 7.4551e-04
Epoch 215/350
31/31  0s 4ms/step - loss: 8.4721e-04 - val_loss: 7.3255e-04
Epoch 216/350
31/31  0s 4ms/step - loss: 7.9239e-04 - val_loss: 7.2596e-04
Epoch 217/350
31/31  0s 4ms/step - loss: 8.6110e-04 - val_loss: 7.1625e-04
Epoch 218/350
31/31  0s 4ms/step - loss: 7.5705e-04 - val_loss: 7.0012e-04
Epoch 219/350
31/31  0s 4ms/step - loss: 7.4169e-04 - val_loss: 6.9393e-04
Epoch 220/350
31/31  0s 4ms/step - loss: 7.7664e-04 - val_loss: 6.9053e-04
Epoch 221/350
31/31  0s 4ms/step - loss: 7.9245e-04 - val_loss: 6.7629e-04
Epoch 222/350
31/31  0s 4ms/step - loss: 7.8312e-04 - val_loss: 6.7416e-04
Epoch 223/350
31/31  0s 4ms/step - loss: 7.2096e-04 - val_loss: 6.6117e-04
Epoch 224/350
31/31  0s 4ms/step - loss: 7.1062e-04 - val_loss: 6.5456e-04
Epoch 225/350
31/31  0s 4ms/step - loss: 7.7285e-04 - val_loss: 6.4517e-04
Epoch 226/350
31/31  0s 4ms/step - loss: 7.5388e-04 - val_loss: 6.4444e-04
Epoch 227/350
31/31  0s 3ms/step - loss: 7.4369e-04 - val_loss: 6.3181e-04
Epoch 228/350
31/31  0s 3ms/step - loss: 6.8822e-04 - val_loss: 6.2895e-04
Epoch 229/350
31/31  0s 4ms/step - loss: 6.6178e-04 - val_loss: 6.2198e-04
Epoch 230/350
31/31  0s 3ms/step - loss: 7.3060e-04 - val_loss: 6.1930e-04
Epoch 231/350
31/31  0s 4ms/step - loss: 6.6227e-04 - val_loss: 6.1389e-04
Epoch 232/350
31/31  0s 3ms/step - loss: 7.1834e-04 - val_loss: 6.0468e-04
Epoch 233/350
31/31  0s 4ms/step - loss: 6.9119e-04 - val_loss: 6.0104e-04
Epoch 234/350
31/31  0s 4ms/step - loss: 7.3072e-04 - val_loss: 5.9892e-04
Epoch 235/350
31/31  0s 4ms/step - loss: 7.1064e-04 - val_loss: 5.9482e-04
Epoch 236/350
31/31  0s 4ms/step - loss: 6.8085e-04 - val_loss: 5.9023e-04
Epoch 237/350
31/31  0s 4ms/step - loss: 7.3317e-04 - val_loss: 5.8188e-04
Epoch 238/350
31/31  0s 4ms/step - loss: 7.2154e-04 - val_loss: 5.8049e-04
Epoch 239/350
31/31  0s 4ms/step - loss: 7.2158e-04 - val_loss: 5.8148e-04
Epoch 240/350
31/31  0s 4ms/step - loss: 7.0546e-04 - val_loss: 5.7321e-04
```

```
Epoch 241/350
31/31  0s 4ms/step - loss: 7.0606e-04 - val_loss: 5.8069e-04
Epoch 242/350
31/31  0s 3ms/step - loss: 6.5722e-04 - val_loss: 5.7367e-04
Epoch 243/350
31/31  0s 4ms/step - loss: 6.5207e-04 - val_loss: 5.7832e-04
Epoch 244/350
31/31  0s 4ms/step - loss: 6.5185e-04 - val_loss: 5.7417e-04
Epoch 245/350
31/31  0s 4ms/step - loss: 6.8971e-04 - val_loss: 5.7500e-04
Epoch 246/350
31/31  0s 4ms/step - loss: 6.9838e-04 - val_loss: 5.7275e-04
Epoch 247/350
31/31  0s 4ms/step - loss: 6.6729e-04 - val_loss: 5.7409e-04
Epoch 248/350
31/31  0s 4ms/step - loss: 6.7206e-04 - val_loss: 5.7674e-04
Epoch 249/350
31/31  0s 3ms/step - loss: 7.1314e-04 - val_loss: 5.7268e-04
Epoch 250/350
31/31  0s 4ms/step - loss: 6.7272e-04 - val_loss: 5.7438e-04
Epoch 251/350
31/31  0s 4ms/step - loss: 6.4517e-04 - val_loss: 5.7017e-04
Epoch 252/350
31/31  0s 4ms/step - loss: 6.8427e-04 - val_loss: 5.7634e-04
Epoch 253/350
31/31  0s 4ms/step - loss: 7.2278e-04 - val_loss: 5.7780e-04
Epoch 254/350
31/31  0s 4ms/step - loss: 6.2318e-04 - val_loss: 5.7251e-04
Epoch 255/350
31/31  0s 5ms/step - loss: 7.1412e-04 - val_loss: 5.7501e-04
Epoch 256/350
31/31  0s 4ms/step - loss: 7.0545e-04 - val_loss: 5.7014e-04
Epoch 257/350
31/31  0s 4ms/step - loss: 6.5516e-04 - val_loss: 5.7730e-04
Epoch 258/350
31/31  0s 4ms/step - loss: 6.9275e-04 - val_loss: 5.7832e-04
Epoch 259/350
31/31  0s 4ms/step - loss: 6.2398e-04 - val_loss: 5.7191e-04
Epoch 260/350
31/31  0s 4ms/step - loss: 6.8853e-04 - val_loss: 5.6961e-04
Epoch 261/350
31/31  0s 3ms/step - loss: 7.0878e-04 - val_loss: 5.7236e-04
Epoch 262/350
31/31  0s 3ms/step - loss: 7.1000e-04 - val_loss: 5.7069e-04
Epoch 263/350
31/31  0s 4ms/step - loss: 6.3854e-04 - val_loss: 5.7092e-04
Epoch 264/350
31/31  0s 3ms/step - loss: 6.2705e-04 - val_loss: 5.6668e-04
Epoch 265/350
31/31  0s 3ms/step - loss: 6.5795e-04 - val_loss: 5.7109e-04
Epoch 266/350
31/31  0s 4ms/step - loss: 6.3558e-04 - val_loss: 5.7240e-04
Epoch 267/350
31/31  0s 4ms/step - loss: 6.6518e-04 - val_loss: 5.7040e-04
Epoch 268/350
31/31  0s 4ms/step - loss: 6.9295e-04 - val_loss: 5.6849e-04
Epoch 269/350
31/31  0s 3ms/step - loss: 6.7890e-04 - val_loss: 5.7241e-04
Epoch 270/350
31/31  0s 4ms/step - loss: 6.5882e-04 - val_loss: 5.6973e-04
```

```
Epoch 271/350
31/31  0s 4ms/step - loss: 6.8025e-04 - val_loss: 5.7501e-04
Epoch 272/350
31/31  0s 4ms/step - loss: 6.9398e-04 - val_loss: 5.7323e-04
Epoch 273/350
31/31  0s 3ms/step - loss: 6.7430e-04 - val_loss: 5.7056e-04
Epoch 274/350
31/31  0s 4ms/step - loss: 6.5795e-04 - val_loss: 5.6806e-04
Epoch 275/350
31/31  0s 4ms/step - loss: 6.8659e-04 - val_loss: 5.7478e-04
Epoch 276/350
31/31  0s 4ms/step - loss: 6.4343e-04 - val_loss: 5.6935e-04
Epoch 277/350
31/31  0s 4ms/step - loss: 6.3373e-04 - val_loss: 5.7445e-04
Epoch 278/350
31/31  0s 4ms/step - loss: 7.3480e-04 - val_loss: 5.7532e-04
Epoch 279/350
31/31  0s 4ms/step - loss: 7.5003e-04 - val_loss: 5.7371e-04
Epoch 280/350
31/31  0s 4ms/step - loss: 7.1944e-04 - val_loss: 5.7226e-04
Epoch 281/350
31/31  0s 4ms/step - loss: 6.6910e-04 - val_loss: 5.7201e-04
Epoch 282/350
31/31  0s 4ms/step - loss: 6.4400e-04 - val_loss: 5.7376e-04
Epoch 283/350
31/31  0s 3ms/step - loss: 6.4126e-04 - val_loss: 5.7532e-04
Epoch 284/350
31/31  0s 4ms/step - loss: 6.5850e-04 - val_loss: 5.7647e-04
Epoch 285/350
31/31  0s 4ms/step - loss: 7.3727e-04 - val_loss: 5.7140e-04
Epoch 286/350
31/31  0s 3ms/step - loss: 7.0063e-04 - val_loss: 5.7145e-04
Epoch 287/350
31/31  0s 4ms/step - loss: 6.8961e-04 - val_loss: 5.7049e-04
Epoch 288/350
31/31  0s 4ms/step - loss: 6.5630e-04 - val_loss: 5.7349e-04
Epoch 289/350
31/31  0s 4ms/step - loss: 6.7309e-04 - val_loss: 5.7201e-04
Epoch 290/350
31/31  0s 3ms/step - loss: 6.4513e-04 - val_loss: 5.7040e-04
Epoch 291/350
31/31  0s 3ms/step - loss: 6.8965e-04 - val_loss: 5.7257e-04
Epoch 292/350
31/31  0s 4ms/step - loss: 7.5192e-04 - val_loss: 5.7336e-04
Epoch 293/350
31/31  0s 4ms/step - loss: 6.5996e-04 - val_loss: 5.6850e-04
Epoch 294/350
31/31  0s 4ms/step - loss: 6.4314e-04 - val_loss: 5.7440e-04
Epoch 295/350
31/31  0s 4ms/step - loss: 6.5353e-04 - val_loss: 5.7100e-04
Epoch 296/350
31/31  0s 4ms/step - loss: 7.2492e-04 - val_loss: 5.7014e-04
Epoch 297/350
31/31  0s 5ms/step - loss: 7.4983e-04 - val_loss: 5.6864e-04
Epoch 298/350
31/31  0s 4ms/step - loss: 6.9108e-04 - val_loss: 5.6596e-04
Epoch 299/350
31/31  0s 4ms/step - loss: 6.5284e-04 - val_loss: 5.6752e-04
Epoch 300/350
31/31  0s 4ms/step - loss: 7.1888e-04 - val_loss: 5.7256e-04
```

```
Epoch 301/350
31/31  0s 4ms/step - loss: 7.5145e-04 - val_loss: 5.6867e-04
Epoch 302/350
31/31  0s 4ms/step - loss: 6.9928e-04 - val_loss: 5.6738e-04
Epoch 303/350
31/31  0s 4ms/step - loss: 7.2237e-04 - val_loss: 5.7326e-04
Epoch 304/350
31/31  0s 4ms/step - loss: 6.4127e-04 - val_loss: 5.7441e-04
Epoch 305/350
31/31  0s 4ms/step - loss: 6.5799e-04 - val_loss: 5.7343e-04
Epoch 306/350
31/31  0s 4ms/step - loss: 6.7468e-04 - val_loss: 5.7567e-04
Epoch 307/350
31/31  0s 4ms/step - loss: 6.7632e-04 - val_loss: 5.7237e-04
Epoch 308/350
31/31  0s 4ms/step - loss: 6.9157e-04 - val_loss: 5.7145e-04
Epoch 309/350
31/31  0s 4ms/step - loss: 6.7008e-04 - val_loss: 5.7113e-04
Epoch 310/350
31/31  0s 3ms/step - loss: 6.9951e-04 - val_loss: 5.6989e-04
Epoch 311/350
31/31  0s 4ms/step - loss: 6.4539e-04 - val_loss: 5.6870e-04
Epoch 312/350
31/31  0s 4ms/step - loss: 6.6928e-04 - val_loss: 5.7360e-04
Epoch 313/350
31/31  0s 4ms/step - loss: 6.7048e-04 - val_loss: 5.7327e-04
Epoch 314/350
31/31  0s 4ms/step - loss: 6.4485e-04 - val_loss: 5.7562e-04
Epoch 315/350
31/31  0s 4ms/step - loss: 6.9467e-04 - val_loss: 5.7438e-04
Epoch 316/350
31/31  0s 3ms/step - loss: 6.4320e-04 - val_loss: 5.7521e-04
Epoch 317/350
31/31  0s 4ms/step - loss: 6.7703e-04 - val_loss: 5.6928e-04
Epoch 318/350
31/31  0s 4ms/step - loss: 7.2475e-04 - val_loss: 5.7233e-04
Epoch 319/350
31/31  0s 4ms/step - loss: 6.8130e-04 - val_loss: 5.7276e-04
Epoch 320/350
31/31  0s 4ms/step - loss: 6.8497e-04 - val_loss: 5.6880e-04
Epoch 321/350
31/31  0s 4ms/step - loss: 6.8441e-04 - val_loss: 5.7140e-04
Epoch 322/350
31/31  0s 4ms/step - loss: 6.7678e-04 - val_loss: 5.7315e-04
Epoch 323/350
31/31  0s 4ms/step - loss: 6.4901e-04 - val_loss: 5.6909e-04
Epoch 324/350
31/31  0s 3ms/step - loss: 7.1097e-04 - val_loss: 5.7130e-04
Epoch 325/350
31/31  0s 4ms/step - loss: 7.2447e-04 - val_loss: 5.6986e-04
Epoch 326/350
31/31  0s 4ms/step - loss: 6.6603e-04 - val_loss: 5.6759e-04
Epoch 327/350
31/31  0s 4ms/step - loss: 7.2407e-04 - val_loss: 5.7266e-04
Epoch 328/350
31/31  0s 4ms/step - loss: 6.7521e-04 - val_loss: 5.6604e-04
Epoch 329/350
31/31  0s 4ms/step - loss: 6.7511e-04 - val_loss: 5.6804e-04
Epoch 330/350
31/31  0s 4ms/step - loss: 7.0720e-04 - val_loss: 5.7600e-04
```

```

Epoch 331/350
31/31  0s 4ms/step - loss: 6.2524e-04 - val_loss: 5.7016e-04
Epoch 332/350
31/31  0s 4ms/step - loss: 7.1132e-04 - val_loss: 5.7241e-04
Epoch 333/350
31/31  0s 3ms/step - loss: 6.7122e-04 - val_loss: 5.7555e-04
Epoch 334/350
31/31  0s 4ms/step - loss: 7.1109e-04 - val_loss: 5.6844e-04
Epoch 335/350
31/31  0s 4ms/step - loss: 6.3204e-04 - val_loss: 5.6854e-04
Epoch 336/350
31/31  0s 4ms/step - loss: 6.2831e-04 - val_loss: 5.7063e-04
Epoch 337/350
31/31  0s 4ms/step - loss: 6.6687e-04 - val_loss: 5.6860e-04
Epoch 338/350
31/31  0s 4ms/step - loss: 6.4531e-04 - val_loss: 5.6928e-04
Epoch 339/350
31/31  0s 4ms/step - loss: 6.7892e-04 - val_loss: 5.6799e-04
Epoch 340/350
31/31  0s 6ms/step - loss: 7.2527e-04 - val_loss: 5.6921e-04
Epoch 341/350
31/31  0s 5ms/step - loss: 6.7714e-04 - val_loss: 5.7220e-04
Epoch 342/350
31/31  0s 5ms/step - loss: 6.9821e-04 - val_loss: 5.6902e-04
Epoch 343/350
31/31  0s 4ms/step - loss: 7.7455e-04 - val_loss: 5.7205e-04
Epoch 344/350
31/31  0s 4ms/step - loss: 6.4468e-04 - val_loss: 5.7150e-04
Epoch 345/350
31/31  0s 4ms/step - loss: 6.8735e-04 - val_loss: 5.6835e-04
Epoch 346/350
31/31  0s 4ms/step - loss: 6.4893e-04 - val_loss: 5.6746e-04
Epoch 347/350
31/31  0s 4ms/step - loss: 6.7634e-04 - val_loss: 5.6932e-04
Epoch 348/350
31/31  0s 4ms/step - loss: 6.8348e-04 - val_loss: 5.6930e-04

```

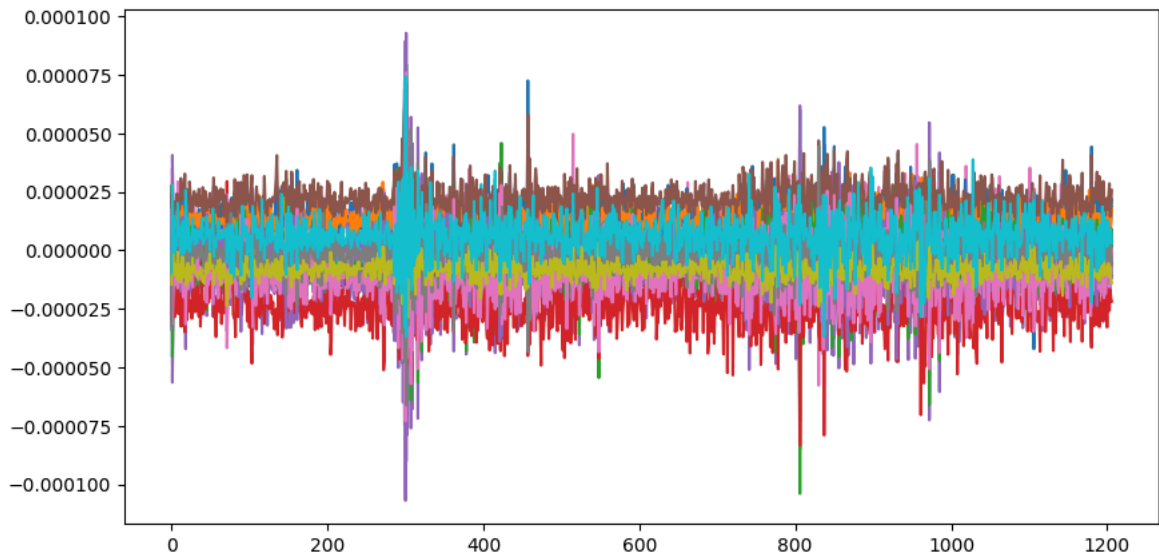
Out[8]: <keras.src.callbacks.history.History at 0x7f9295627e90>

By plotting the time series of the factors generated by the model, we can observe how the regularization term shrinks the dimensionality of the bottleneck. The model is forced to generate a number of latent factors with lower dimensionality than the data and that are relevant to explain its variation.

```

In [9]: factors = model.encoder(Y_train).numpy()
plt.figure(figsize=(10, 5))
plt.plot(factors)
plt.show()

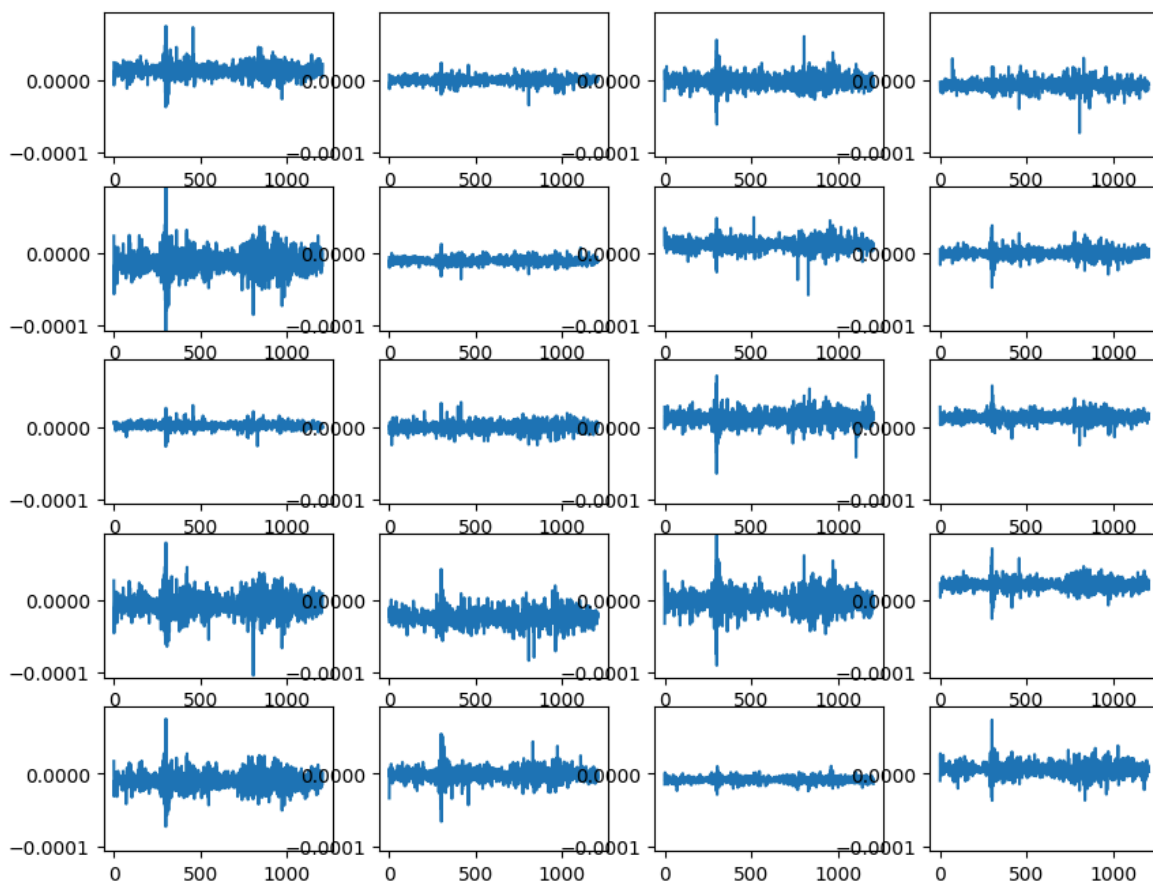
```



Below, we plot the time series of each factor. Observe how, out of 20 possible dimensions, the addition of the regularization term enables us to "drop" some dimensions. Notice the resemblance of this regularization technique with the L1-regularization of weights in MLPs. In that setup, we were interested in filtering a set of input features that are relevant for predictions. In the autoencoder setup, the activity regularization chooses a few factors that are relevant at explaining the variation in the data.

You can play with the parameter λ_1 above and observe its impact on the number of factors generated and the performance of the model. Notice also that the results depend on the random seed that is used to initialize the model's parameters while training.

```
In [10]: custom_ylim = [np.min(factors), np.max(factors)]
fig, axs = plt.subplots(5, 4, figsize=(10, 8))
for i in range(5):
    axs[i, 0].plot(factors[:, 0 + i * 4])
    axs[i, 0].set_ylim(custom_ylim)
    axs[i, 1].plot(factors[:, 1 + i * 4])
    axs[i, 1].set_ylim(custom_ylim)
    axs[i, 2].plot(factors[:, 2 + i * 4])
    axs[i, 2].set_ylim(custom_ylim)
    axs[i, 3].plot(factors[:, 3 + i * 4])
    axs[i, 3].set_ylim(custom_ylim)
```



In terms of reconstruction performance, the regularization parameter λ_1 is quite small, so the reconstruction errors from this model should be below those in Lesson 2 using an undercomplete autoencoder with three factors. This is a natural outcome because increasing the number of latent factors should increase the proportion of the variance explained by the model.

```
In [11]: Y_pred = model.predict(Y_train, verbose=0)
error = np.sum((Y_train - Y_pred) ** 2) / np.sum((Y_train - np.mean(Y_train)) ** 2)
print("Reconstruction error in training sample", error)
Y_pred = model.predict(Y_test, verbose=0)
error_oos = np.sum((Y_test - Y_pred) ** 2) / np.sum((Y_test - np.mean(Y_test)) ** 2)
print("Reconstruction error in test sample", error_oos)
```

Reconstruction error in training sample 0.9996295625952354

Reconstruction error in test sample 1.0017582451379592

3. PORTFOLIO CHOICE USING THE PREDICTIONS OF THE AUTOENCODER

Now we are going to use the trained autoencoder to choose an "optimal" portfolio of stocks. Based on the estimated exposures of each stock to the factors that determine the comovement in stock returns, we can design a diversified portfolio that maximizes the expected risk-weighted returns. Let's formally describe this.

Notice that the decoding layer of the autoencoder implies that we can model the returns of a stock i at time t as: $r_{it} = \alpha_i + \beta_{i1} f_{1t} + \beta_{i2} f_{2t} + \dots + \beta_{ik} f_{kt} + \epsilon_{it}$ where $f_{1t}, f_{2t}, \dots, f_{kt}$ are the

factors generated by the autoencoder, and $\alpha_i, \beta_{i1}, \beta_{i2}, \dots, \beta_{ik}$ are the trained parameters. ϵ_{it} are zero-mean error terms. The representation above is called a linear factor model.

If we stack the observations of returns of all the stocks $i=1, \dots, n$ at a period t in a column vector R_t , stock returns are given by: $R_t = \mathbf{a} + B F_t + \mathbf{e}_t$ where (bold faces are meant to differentiate vectors from scalars):

- $\mathbf{a}$ is an $n \times 1$ vector that includes all the trained α_i 's
- B is an $n \times k$ matrix that includes all the trained β_{ij} 's.
- F_t is a $k \times 1$ vector of realizations of the factors.
- $\mathbf{e}_t$ is an $n \times 1$ vector of idiosyncratic (stock-specific) errors ϵ_{it} 's

Assume that the variance of $\mathbf{e}_t$ is a diagonal $n \times n$ matrix Σ_e , while the variance of F_t is denoted by the $k \times k$ matrix Σ_f . These matrices capture the variation and covariation of the factors and idiosyncratic terms over time and that we can estimate, for instance, from our autoencoder model. Thus, the expectation and variance of the return vector R_t are given by $E(R_t) = \mu = \mathbf{a} + B E(F_t)$ $\text{Var}(R_t) = \Sigma = B \Sigma_f B^T + \Sigma_e$ The first component of the variance of returns is called the systematic component of the variance, which arises from the common latent factors that determine all the returns, depending on the stock-specific exposures to those factors included in B . The second component is the idiosyncratic component of the variance. This variation can be interpreted as noise, which is assumed to be stock-specific (uncorrelated across stocks).

Using the autoencoder, we can build our estimates for μ and Σ . For μ , we take the average of the reconstruction of each stock return over the training sample, $E(\hat{R}_t)$. For Σ , we first compute the variance covariance matrix of the reconstructions, $\Sigma_f = \text{Var}(\hat{R}_t)$, and then we add the idiosyncratic component from the variance-covariance matrix of the reconstruction errors, $\Sigma_e = \text{diag}(\text{Var}(\mathbf{e}_t))$. Following our assumption above, we impose a diagonal Σ_e . Although the matrix $\text{Var}(\mathbf{e}_t)$ may not be diagonal, we treat the remaining off-diagonal variation just as "noise" that is irrelevant for generalization.

```
In [12]: Y_pred = model.predict(Y_train, verbose=0)
mu = np.mean(Y_pred, axis=0)
sigma_f = np.cov(Y_pred, rowvar=False)
sigma_e = np.cov(Y_train - Y_pred, rowvar=False)
sigma_d = np.diag(np.diag(sigma_e))
sigma = sigma_f + sigma_d
```

Let's now consider building a portfolio with the modelization approach that we have derived above. A portfolio is defined as a vector $w = (w_1, \dots, w_n)^T$ where each w_i represents the proportion of the investor's wealth that we invest in an asset i . We must impose the constraint $\sum_i w_i = 1$, meaning that the investor has all of its wealth invested in the n assets. This does not rule out the possibility of short-selling, $w_i < 0$

0\$, so that the portfolio can take long positions on some assets and short positions on others.

Given a choice of portfolio w on the n assets, the return of the portfolio in period t is given by $R_{w,t} = w^T R_{t}$. Hence, the expected return and variance of this portfolio can be denoted by the following scalars: $E(R_{w,t}) = w^T \mu$ $Var(R_{w,t}) = w^T \Sigma w$

Let's define the Sharpe Ratio, SR , of a portfolio as its expected return per unit of risk, i.e.: $SR_w = \frac{E(R_{w,t})}{\sqrt{Var(R_{w,t})}} = \frac{w^T \mu}{\sqrt{w^T \Sigma w}}$

Note: Formally, to compute a Sharpe Ratio we should subtract the risk-free rate from the numerator, but we are going to abstract from it. You can read the [Investopedia entry](#) as further reference.

We are going to use the Sharpe Ratio as the performance metric that determines whether a portfolio is better than other. Thus, our objective is to find the portfolio that generates the maximum Sharpe Ratio given our estimates of expected returns and variance covariance matrix of returns.

For this, we set up an optimization problem, where the objective function is the Sharpe Ratio, given vectors of weights, expected returns, and the variance-covariance matrix. We put a minus term in front as we are going to define the optimization problem as a minimization objective.

```
In [13]: def sharpe_ratio(weights, mean, var):  
         return -np.dot(weights, mean) / np.sqrt(np.dot(np.dot(weights, var), weights
```

We then define a function where the optimization problem is performed, including the constraint that $\sum_i w_i = 1$. If we were interested in imposing a long-only portfolio (short-selling constraint) then we should change the "bound" tuple to take the value $(0,1)$, instead of $(None, None)$.

```
In [14]: import scipy.optimize as sco
```

```
In [15]: def max_sharpe(mean, var):  
         args = (mean, var)  
         num_assets = len(mean)  
         args = (mean, var)  
         constraints = {"type": "eq", "fun": lambda x: np.sum(x) - 1}  
         bound = (None, None)  
         bounds = tuple(bound for asset in range(num_assets))  
         result = sco.minimize(  
             sharpe_ratio,  
             num_assets  
             * [  
                 1.0 / num_assets,  
             ],  
             args=args,  
             method="SLSQP",  
             bounds=bounds,
```

```

        constraints=constraints,
        tol=1e-12,
    )
    return result

```

Given, the estimates of μ and Σ from the autoencoder, we get the following portfolio weights for the 20 stocks. We also verify that the constraint is roughly satisfied.

```

In [16]: opt = max_sharpe(mu, sigma)
         opt.x

```

```

Out[16]: array([ 0.13625731,  0.09874778,  0.0523024 ,  0.03819203,  0.06961968,
                 0.00350657,  0.05610333,  0.05169225, -0.00699324, -0.01166656,
                 0.07362228,  0.06228513,  0.07470726,  0.03380913,  0.04850011,
                 0.03878507,  0.02906145,  0.07333414,  0.05462749,  0.02350638])

```

```

In [17]: np.sum(opt.x)

```

```

Out[17]: 1.0

```

Actually, from portfolio choice theory, we already know the analytical expression for the portfolio with maximum Sharpe Ratio in the absence of short-selling constraints: $w^* = \frac{\Sigma^{-1}\mu}{\mathbf{1}^T \Sigma^{-1}\mu}$ where $\mathbf{1}$ is an $n \times 1$ vector of ones. See this [link](#) for a proof of this result. Below, we show that the results from the optimization problem are roughly identical (reduce the tolerance of the optimization problem to obtain closer results).

```

In [18]: # Analytic solution (only when short positions are allowed)
         weights = np.dot(np.linalg.inv(sigma), mu)
         weights = weights / np.sum(weights)
         weights

```

```

Out[18]: array([ 0.13625741,  0.09874773,  0.05230228,  0.03819205,  0.06961961,
                 0.00350667,  0.05610339,  0.05169231, -0.00699334, -0.01166649,
                 0.07362227,  0.06228522,  0.07470719,  0.03380929,  0.04850003,
                 0.03878504,  0.02906152,  0.0733341 ,  0.05462746,  0.02350625])

```

Now, let's look at the performance of the portfolio in the training sample and in the test sample. Note that the results may vary depending on the seed used while training. You can train the model several times to have a general view on the out-of-sample performance of the strategy.

```

In [19]: mu_true_test = np.mean(Y_test, axis=0)
         sigma_true_test = np.cov(Y_test, rowvar=False)
         print(
             "Sharpe ratio in training (estimated): ",
             -np.sqrt(252) * sharpe_ratio(opt.x, mu, sigma),
         )
         print(
             "Sharpe ratio in test (actual): ",
             -np.sqrt(252) * sharpe_ratio(opt.x, mu_true_test, sigma_true_test),
         )

```

```

Sharpe ratio in training (estimated):  3.3954331835003013
Sharpe ratio in test (actual):  2.07543332497095

```

Does the portfolio obtained from the autoencoder perform better than a "naive" portfolio that simply uses the mean returns and variance covariance matrix of returns over the training sample to construct the portfolio? Below, we compare the autoencoder model with that benchmark.

```
In [20]: mu_true = np.mean(Y_train, axis=0)
sigma_true = np.cov(Y_train, rowvar=False)
opt = max_sharpe(mu_true, sigma_true)
print(
    "Sharpe ratio in training: ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true, sigma_true),
)
print(
    "Sharpe ratio in test (actual): ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true_test, sigma_true_test),
)
```

```
Sharpe ratio in training:  1.9655875931884363
Sharpe ratio in test (actual):  1.6941130374559856
```

Below, we show the results from choosing the optimal portfolio using estimates from PCA, using as many principal components as those obtained from the autoencoder model. Since this method does not produce estimates of the average returns, we rely on using the principal components (which are calculated from the centered or demeaned data) to calculate the average returns over the training sample as estimates for the expected returns, μ .

Notice that the standard PCA method does not use a validation sample to avoid overfitting, which naturally leads to different results than an autoencoder approach. However, the results are independent from the choice of random seed.

```
In [21]: from sklearn.decomposition import PCA

pca = PCA(n_components=17)
pca.fit(Y_train)
pca_factors = pca.transform(Y_train)
Y_pca_pred = np.dot(pca_factors, pca.components_)
sigma_f = np.cov(Y_pca_pred, rowvar=False)
sigma_e = np.cov(Y_train - Y_pca_pred, rowvar=False)
sigma_d = np.diag(np.diag(sigma_e))
sigma_pca = sigma_f + sigma_d
```

Below we compute the Sharpe ratios of the portfolios built using PCA. The estimated Sharpe ratio from the PCA is 2.02 (in training), but in the test sample, it drops to -0.31. Notice that the optimal portfolio that arises from PCA will give greater weight to stocks with higher average returns over the training sample. Hence, the Sharpe ratio turns negative because those stocks that performed well in the training sample seem to have performed badly in the test sample.

```
In [22]: opt = max_sharpe(mu_true, sigma_pca)
print(opt.x)
print(
    "Sharpe ratio in training (estimated): ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true, sigma_pca),
)
```

```
)
print(
    "Sharpe ratio in test (actual): ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true_test, sigma_true_test),
)
```

```
[ 1.45251094  1.02761215  2.5769373  -1.79584562  0.87488948  0.49026512
  1.87165622 -0.07636212 -0.06377933 -1.14006306  0.21441525  3.11683583
  2.1610222  -2.3176693  -1.57076272 -0.84432432 -0.21190979 -3.12019528
 -0.83857693 -0.80665601]
```

```
Sharpe ratio in training (estimated): 2.016511478146714
```

```
Sharpe ratio in test (actual): 2.0072948864942446
```

5. CONCLUSION

Although the autoencoder may not generate extremely satisfactory results, its performance may improve upon other more naïve estimation procedures. The results are also largely dependent on the initialization of the autoencoder's weights. In the next module, we'll deal with ensemble methods that, at least partly, deal with this annoying variability.

Notice also that the train and test samples display very different market environments. It is quite challenging to find machine learning algorithms that adapt well to such changes in the environment.

Lastly, as you have probably grasped in the previous modules of this course, obtaining a good deep learning model for finance applications requires a great deal of tweaking and understanding of how the model is a suitable tool for the kind of data that we have at hand.

With this notebook, we finalize the descriptions of autoencoders and their application to financial time series.

In Module 7, we will study further advanced topics in Deep Learning for Finance.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Module 6: Lesson 1

Introduction to autoencoders



Outline

- ▶ Introduction to autoencoders
- ▶ Autoencoders as neural networks
- ▶ Linear autoencoders as principal component analysis (PCA)

Autoencoders

Autoencoders are a type of unsupervised ML method that is designed as a reconstruction problem.

Training autoencoders involves the minimization error from reconstructing its inputs $\mathbf{x}$ in two steps

$$\underbrace{\mathbf{x} \longrightarrow f(\mathbf{x}) \longrightarrow \mathbf{z}}_{\text{Encoding}} \underbrace{\longrightarrow g(\mathbf{z}) \longrightarrow \hat{\mathbf{x}}}_{\text{Decoding}}$$

The encoding function f transforms the input $\mathbf{x}$ into encodings $\mathbf{z}$ (latent representations of the data).

The decoding function g transforms the encodings $\mathbf{z}$ back to a reconstruction of the input $\hat{\mathbf{x}}$.

At first, the problem of reconstructing an input may be trivial. But building an autoencoder model may be useful for several tasks, such as:

- ▶ Dimensionality reduction.
- ▶ Denoising.
- ▶ Pre-processing inputs before classification (“whitening”).
- ▶ Outlier detection.

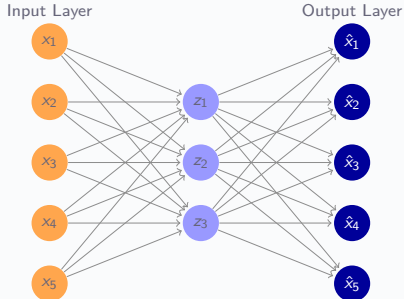
Autoencoders as neural networks

Autoencoders can be represented as a neural network with one or many fully connected hidden layers.

- ▶ The output layer is identical to the input layer, $\mathbf{x}$.
- ▶ Training the neural network implies finding an identity mapping between inputs and outputs.

The hidden activations are called encodings, latent features, or “factors,” $\mathbf{z}$.

- ▶ The encodings capture the dependencies between the different observed variables included in $\mathbf{x}$.



Autoencoders as neural networks

Suppose that we want to encode our feature vector $\mathbf{x}$, with dimension $n \times 1$, into a $k \times 1$ vector of encodings $\mathbf{z}$.

Suppose also that we have p observations of $\mathbf{x}$, say returns of n different stocks over p periods, or information of n features from p individuals.

We stack the information in an $n \times p$ matrix X .

We can represent a single-layer autoencoder as follows, where f and g are activation functions:

$$X \longrightarrow f \left(W_1 X + \mathbf{b}_1 \iota_p^T \right) \longrightarrow Z \longrightarrow g \left(W_2 Z + \mathbf{b}_2 \iota_p^T \right) \longrightarrow \hat{X}$$

where:

- ▶ Encoding weights: W_1 , $k \times n$ matrix. Decoding weights: W_2 , $n \times k$ matrix.
- ▶ Bias terms: $\mathbf{b}_1$, $k \times 1$ vector, and $\mathbf{b}_2$, $n \times 1$ vector.
- ▶ Encodings: Z , $k \times p$ matrix.
- ▶ ι_p , $p \times 1$ vector of ones.
- ▶ $\hat{X}$, $n \times p$ reconstructed input.

Autoencoders as neural networks

Given a choice of activation functions, f and g , and encoding dimension, k , we can train the autoencoder to minimize the reconstruction error computed, say, with the MSE:

$$\frac{1}{np} \|X - \hat{X}\|_F^2$$

where F denotes the Frobenius norm.

As in any other neural network, we can add regularization terms that penalize the weights and the encodings.

Depending on the number of encodings k , we can differentiate among:

- ▶ Undercomplete autoencoders ($n > k$). They gather the most salient features of the data by learning small representations (a “bottleneck”).
- ▶ Overcomplete autoencoders ($n \leq k$). They necessarily need regularization to avoid learning an identity transformation.

We can also increase the depth of the network with additional hidden layers on both sides of the encoding layer.

Linear autoencoders

Suppose you have an autoencoder model with one hidden layer, whose activation functions f and g are linear. The autoencoder can be represented as:

$$X \longrightarrow W_1 X + \mathbf{b}_1 \iota_p^T \longrightarrow Z \longrightarrow W_2 Z + \mathbf{b}_2 \iota_p^T \longrightarrow \hat{X}$$

Then, assuming an MSE loss, training this autoencoder involves the choice of weights and biases that solve:

$$\min_{W_1, W_2, b_1, b_2} \left\| X - W_2 \left(W_1 X + \mathbf{b}_1 \iota_p^T \right) - \mathbf{b}_2 \iota_p^T \right\|_F^2$$

which can be reduced to choosing W_2 to minimize $\left\| X_0 - W_2 (W_2^T W_2)^{-1} W_2^T X_0 \right\|_F^2$.

- ▶ $X_0 = X - X \iota_p / p$ is the matrix of centered observations, i.e., X minus the mean of each feature.
- ▶ W_2 is a solution to the previous problem if its columns form a basis that spans the same subspace as the principal eigenvectors of the variance-covariance matrix of X .
- ▶ Training a single-layer linear autoencoder is roughly equivalent to performing principal component analysis on X .

Principal Components Analysis (PCA)

Principal component analysis consists of a linear transformation of the data to a new coordinate system that explains its variation.

PCA finds a low-dimensional (linear) representation of a data set that contains the maximum level of variation.

For instance, the first principal component of our data X is the linear combination of the uncentered features

$$Z_1 = w_1^T X_0$$

which is the encoding with the largest variance, i.e., $w_1^T X_0 X_0^T w_1$ is maximized.

$w_1 = (w_{11}, w_{21}, \dots, w_{n1})^T$ is a normalized $n \times 1$ vector, such that $\sum_i^n w_{i1}^2 = 1$.

- ▶ The coefficients in w_1 are the *loadings* of the first principal component.
- ▶ We normalize the loadings, otherwise setting these elements to be arbitrarily large could result in an arbitrarily large variance.

PCA: Second and further principal components

The second principal component is the linear combination of X_0 with maximal variance out of all linear combinations that are uncorrelated with Z_1 .

Constraining Z_2 to be uncorrelated with Z_1 is equivalent to constraining the direction w_2 to be orthogonal (perpendicular) to the direction w_1 , $w_1^T w_2 = 0$.

- We also normalize the loadings to have unit norm $\sum_i^n w_{i2}^2 = 1$.

We can proceed in the same manner until reaching the n th principal component, such that the elements $(Z_1, \dots, Z_k)$ explain all the variance of X .

The scores and loadings from PCA are unique up to a sign flip: the variance of Z is the same as that of $-Z$.

PCA and linear autoencoders

Let Σ denote the variance-covariance matrix of X , $\Sigma = \frac{X_0^T X_0}{p-1}$. We can perform the following decomposition:

$$\Sigma = P \Lambda P^T$$

Λ is the diagonal matrix of eigenvalues, sorted from highest to lowest, and P is a matrix whose columns are orthonormal eigenvectors.

- ▶ The eigenvectors are called the *principal directions* of the data.
- ▶ PCA generates uncorrelated encodings equal to $Z_k = P_k^T X_0$, for some choice of k eigenvectors P_k .

P_k is also a solution to the following problem with orthonormality constraints, which is similar to the linear autoencoder ($I_{k \times k}$ is the $k \times k$ identity matrix):

$$\min_{W \in \mathbb{R}^{n \times k}} \|X_0 - WW^T X_0\|_F^2 \text{ s.t. } W^T W = I_{k \times k}$$

The solution P_k is not unique, $W = QP_k$ is also a solution for any orthogonal Q .

PCA and the minimum reconstruction error

Let λ_i denote the i th eigenvalue of Σ , and let w_i denote its associated eigenvector.

- ▶ The value of each λ_i corresponds to the variance of X explained in the direction of eigenvector w_i .
- ▶ As we include more principal components, λ_i 's, we explain more of the variation in X .

If $k \leq n$, the optimal reconstruction error of the single-layer linear autoencoder is bounded below by the sum of the eigenvalues beyond the k th:

$$\min_{W_1, W_2, b_1, b_2} \|X - \hat{X}\|_F^2 = \min_{W_1, W_2, b_1, b_2} \|X - W_2 (W_1 X + b_1 \ell_p^T) - b_2 \ell_p^T\|_F^2 = \sum_{i=k+1}^n \lambda_i$$

If Σ has rank k , we would have that $\lambda_{k+1} = \lambda_n = 0$ and the minimum reconstruction error would be zero.

If Σ has full rank, i.e., there are no perfect linear dependencies between any combination of features, the minimum reconstruction error would be zero only in the case where we chose $k = n$.

PCA and autoencoders

A linear autoencoder applies PCA to the input data by generating a projection of the data into a low-dimensional space. However:

- ▶ The encodings of the bottleneck are correlated and not sorted.
- ▶ The solutions to each choice of k are not nested, we need to train for each k .

When building and training an autoencoder, PCA serves as a useful benchmark.

- ▶ Autoencoders are more flexible, they may capture richer features through deep models and non-linearities.
- ▶ We can use loss functions that are not restricted to MSE.

Besides, through autoencoders, we can easily employ regularization techniques to avoid overfitting.

- ▶ PCA may detect noise as a latent encoding that is irrelevant for generalization out of sample.

Summary of Lesson 1

In Lesson 1, we have:

- ▶ Described the role and design of autoencoders.
- ▶ Represented autoencoders as neural networks.
- ▶ Described PCA and its relationship with autoencoders.

⇒ **References for this lesson:**

Chapter 7 in [Ranjan, C. *Understanding Deep Learning*](#).

[Plaut, Elad. "From Principal Subspaces to Principal Components with Linear Autoencoders." *arXiv*, 2018](#)

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement an autoencoder to compress the information contained in stock prices.

See you there!

Module 6: Lesson 2

Application of autoencoders: Extracting features from stock returns



Outline

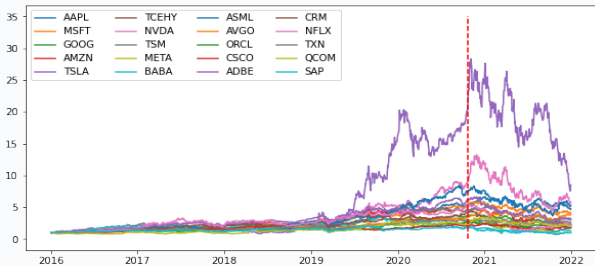
- ▶ Data
- ▶ PCA decomposition
- ▶ Building and training an autoencoder

Data

In this lesson, we illustrate the use of PCA and autoencoders for financial data.

Our data under analysis are the daily stock returns from a sample of 20 tech stocks from Jan 2017 to Dec 2022.

- The training sample covers the period up to Oct 20th, 2021 (80% of the full sample).

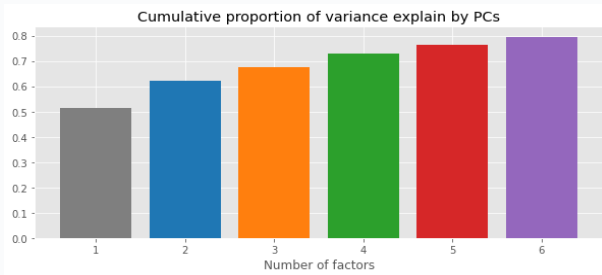


PCA decomposition

The PCA decomposition of the returns data over the training sample shows that roughly 50% of the variation arises from the first principal component.

- In other words, 50% of the variation in returns comes from a single factor that generates comovement across stocks (i.e., “the market”).

Considering 5 additional factors leads to explaining an additional 30% of all the variation in returns.



Building an autoencoder in TensorFlow

We build, via “subclassing,” a linear autoencoder, which in TensorFlow is designed as follows:

- ▶ A Dense layer (encoder) that compresses the input into a latent vector with dimension “bottleneck_dim.”
- ▶ A Dense layer (decoder) that reconstructs the original input from the latent space.

```
class autoencoder(tf.keras.Model):
    def __init__(self, output_dim, bottleneck_dim = 1):
        super(autoencoder, self).__init__()
        self.output_dim = output_dim
        self.bottleneck_dim = bottleneck_dim
        initializer = tf.keras.initializers.GlorotUniform()
        # Build the encoder layer
        self.encoder = tf.keras.Sequential([
            tf.keras.layers.Dense(bottleneck_dim,
                                   kernel_initializer=initializer,
                                   bias_initializer = tf.keras.initializers.Zeros())
        ])
        # Build the decoder layer
        self.decoder = tf.keras.Sequential([
            tf.keras.layers.Dense(output_dim,
                                   kernel_initializer=initializer,
                                   bias_initializer = tf.keras.initializers.Zeros())
        ])
    def call(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded
```

Training the autoencoder

We train the linear autoencoder with a “bottleneck” dimension equal to 3.

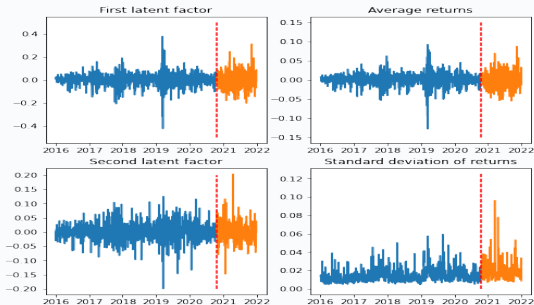
We set apart the last 20% of the training sample for validation purposes, using an early-stopping criterion.

- We use the default batch size of 32, train for 1,000 epochs, and set the patience equal to 50 epochs. We use the Adam optimizer with a learning rate of 10^{-5} .

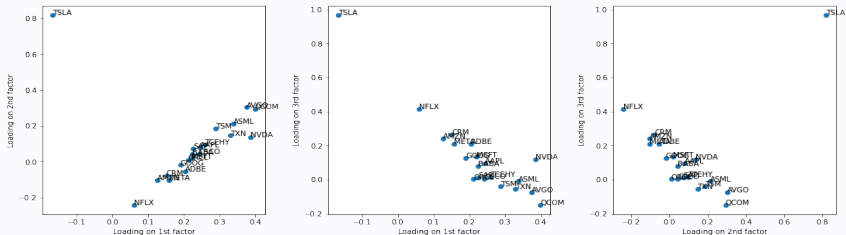
Disclaimer: The results below refer to a particular initialization of the trainable parameters and may vary if other seeds are used.

The reconstruction error in training reaches 32%, close to the lower bound set by PCA.

The first latent factor explains the average returns. The second latent factor seems to explain the dispersion of returns.



Trained loadings



A large portion of the cross-sectional dispersion in stock returns, captured by the second and third factors, is actually driven by the behavior of Tesla's stock price.

Stocks whose returns correlate more with the average return are also more exposed to the cross-sectional dispersion factor.

The third factor seems to capture a feature of stocks that are less exposed to market variations and cross-sectional dispersion.

Summary of Lesson 2

In Lesson 2, we have:

- ▶ Used PCA and autoencoders to extract the main latent factors that drive stock returns.
- ▶ Built and trained autoencoder models in TensorFlow via “subclassing.”

⇒ **References for this lesson:**

Chapter 7 in [Ranjan, C. *Understanding Deep Learning*](#).

TO DO NEXT: Please go to the Jupyter Notebook associated with Lesson 2.

In the next lesson, we will describe a variety of autoencoder architectures and several refinement tools.
See you there!

Module 6: Lesson 3

Further autoencoder topics



Outline

- ▶ Sparse autoencoders.
- ▶ Setting constraints on autoencoders.
- ▶ Autoencoders for images.

Sparse autoencoders

Sparse autoencoders represent a type of autoencoder where a penalty term is imposed on the realizations of the encodings.

This type of penalization may be of interest in overcomplete autoencoders where the number of encodings in the hidden layers is greater than the number of features.

In a sparse autoencoder, we add an L1-penalty term on the encodings Z .

If we use the MSE as the performance metric, the loss function of a sparse autoencoder can be as follows:

$$\left\| X - \hat{X} \right\|_F^2 + \lambda_1 \| \text{vec}(Z) \|_1$$

$\text{vec}(\cdot)$ is the operator that transforms a matrix into a column vector and $\| \cdot \|_1$ is the L1-norm.

In TensorFlow, we can add an activity regularizer to the layer that leads to the bottleneck:

```
activity_regularizer= tf.keras.regularizers.l1(l1weight)
```

where **l1weight** is the λ_1 hyperparameter.

Setting constraints on autoencoders

In Lesson 1, we described the architecture of an autoencoder and its association with PCA.

A single-layer linear autoencoder performs the same task as PCA, but with fewer constraints on the architecture.

Such a lack of constraints may provide too much leeway and lead to ill-posed models.

- ▶ Usual outcomes of such flexibility are the overfitting and exploding/vanishing gradient problems.
- ▶ To avoid those issues we may opt for excessively deep and complex networks, losing interpretability.

We can add the constraints from PCA to discipline the behavior of autoencoders. Namely, in PCA:

- ▶ The weight matrix is orthonormal (orthogonality + unit norm), $W^T W = I_{k \times k}$.
- ▶ The matrix of decoding weights is the transpose matrix of encoding weights, $W = W_2 = W_1^T$.
- ▶ The encodings are uncorrelated, $\text{Cov}(Z_k)$ is a diagonal matrix.

We are going to exploit TensorFlow-Keras capabilities to include constraints and loss terms in the layers of autoencoders as tools that may alleviate the under-performance of an unconstrained autoencoder.

Constraints: Unit norm weights

In PCA, the eigenvectors are constrained to have a unit norm.

The elements of the weight matrices may grow arbitrarily large with the norm of its vectors, leading to arbitrarily large variances.

- ▶ A unit-norm constraint may solve exploding-gradient problems.

TensorFlow provides a built-in method to constrain the weights to have a unit norm:

```
kernel_constraint = tf.keras.constraints.UnitNorm(axis=axis)
```

Remember that, in PCA, the encoding layer is computed as $Z_k = P_k^T X_0$, meaning that the unit norm must be imposed over the rows of the weights of the encoding layers (**axis=0**).

Thus, in the decoding layers, we must impose the unit norm over the columns in the corresponding weight matrix (**axis=1**).

- ▶ The same applies to the orthogonality constraint that we introduce next.

Constraints: Orthogonal weights

Weight orthogonality implies that weights transmit valuable information that is not shared across layers.

- An orthogonality constraint may solve vanishing-gradient problems.

Orthogonality imposes the inner products of the decoding vectors to be zero:

$$w_i^T w_j = 0, \text{ for } i, j = 1, \dots, k \text{ and } i \neq j$$

Via subclassing, we can build a custom kernel regularizer that penalizes the sum of the inner products:

kernel_regularizer=WeightsOrthogonalityRegularizer(encoding_dim, weight, axis=axis)

```
class WeightsOrthogonalityRegularizer(tf.keras.regularizers.Regularizer):
    def __init__(self, encoding_dim, weightage = 1.0, axis = 0):
        self.encoding_dim = encoding_dim
        self.weightage = weightage
        self.axis = axis
    def weights_orthogonality(self, w):
        if(self.axis==1):
            w = tf.transpose(w)
            m = tf.tensordot(tf.transpose(w), w, axes=1) - tf.math.multiply(w,tf.eye(self.encoding_dim))
            return self.weightage * tf.math.sqrt(tf.math.reduce_sum(tf.square(m)))
        def __call__(self, w):
            return self.weights_orthogonality(w)
```

Constraints: Tied weights

Another manner of constraining and regularizing the autoencoder is to tie weights across layers (we considerably reduce the number of trainable parameters).

This means setting the weight matrix in layer ℓ equal to the transpose weight matrix in layer $-\ell$ (ℓ layers before the output layer).

To implement this, we need to define a new type of layer in TensorFlow:

`decoder_l = DenseTranspose(encoder_l, activation)`
`encoder_l` refers to the layer that `decoder_l` has tied weights with.

```
class DenseTranspose(tf.keras.layers.Layer):
    def __init__(self, dense, activation=None):
        self.dense = dense
        self.activation = tf.keras.activations.get(activation)
    def build(self, batch_input_shape):
        self.biases = self.add_weight(name="bias", shape=[self.dense.input_shape[-1]], initializer="zeros")
        super().build(batch_input_shape)
    def call(self, inputs):
        z = tf.matmul(inputs, self.dense.weights[0], transpose_b=True)
        return self.activation(z + self.biases)
```

Constraints: Uncorrelated encodings

When the encodings (factors) are correlated, redundant information is transferred through the network.

We can also impose the encodings to be uncorrelated through an activity regularizer.

```
activity_regularizer = UncorrelatedFeaturesConstraint(encoding_dim, weightage)
```

```
class UncorrelatedFeaturesConstraint(tf.keras.constraints.Constraint):  
    def __init__(self, encoding_dim, weightage = 0.0):  
        self.encoding_dim = encoding_dim  
        self.weightage = weightage  
    def uncorrelated_feature(self, x):  
        covariance = tfp.stats.covariance(x)  
        m = tf.square(covariance - tf.math.multiply(covariance, tf.eye(self.encoding_dim)))  
        output = self.weightage * tf.math.sqrt(tf.math.reduce_sum(m))  
        return output  
    def __call__(self, x):  
        return self.uncorrelated_feature(x)
```

Autoencoders for images: Dense layers

Image feature extraction and recognition are the main applications of deep learning architectures. This also applies to autoencoders.

We can easily build autoencoders for images with dense layers by including a flattening layer as the input layer and a reshaping layer as the output layer.

```
encoder = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=[window_size, window_size]),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu"),
])
decoder = tf.keras.models.Sequential([
    tf.keras.layers.Dense(100, activation="relu", input_shape=[30]),
    tf.keras.layers.Dense(window_size * window_size, activation="sigmoid"),
    tf.keras.layers.Reshape([window_size, window_size])
])
autoencoder = tf.keras.models.Sequential([encoder, decoder])
adam = tf.keras.optimizers.Adam(learning_rate=1e-4)
autoencoder.compile(loss="mse", optimizer=adam)
autoencoder.fit(X_data_train, X_data_train, epochs=20, validation_data=(X_data_test, X_data_test))
```

Autoencoders for images: Convolutional layers

We can also build autoencoders with convolutional layers, which tend to reduce the number of parameters of the architecture relative to dense layers.

The layers of the decoder must be “Transpose convolutions” that perform the reverse operations of the convolutions that we studied in Modules 2 and 3.

```
conv_encoder = tf.keras.models.Sequential([
    tf.keras.layers.Reshape([window_size, window_size, 1], input_shape=[window_size, window_size]),
    tf.keras.layers.Conv2D(16, kernel_size=3, padding="SAME", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),
    tf.keras.layers.Conv2D(32, kernel_size=3, padding="SAME", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),
    tf.keras.layers.Conv2D(64, kernel_size=3, padding="SAME", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2)
])
conv_decoder = tf.keras.models.Sequential([
    tf.keras.layers.Conv2DTranspose(32, kernel_size=3, strides=2, padding="VALID", activation="relu", input_shape=[3, 3, 64]),
    tf.keras.layers.Conv2DTranspose(16, kernel_size=3, strides=2, padding="SAME", activation="relu"),
    tf.keras.layers.Conv2DTranspose(1, kernel_size=3, strides=2, padding="SAME", activation="sigmoid"),
    tf.keras.layers.Reshape([window_size, window_size])
])
conv_ae = tf.keras.models.Sequential([conv_encoder, conv_decoder])
```

Summary of Lesson 3

In Lesson 3, we have:

- ▶ Described the concept of sparse autoencoder.
- ▶ Enumerated several tools to regularize autoencoders based on PCA.
- ▶ Outlined the use of autoencoders for image data.

⇒ **References for this lesson:**

Ranjan, C. "Build the Right Autoencoder — Tune and Optimize using PCA Principles (Parts I and II)." *Medium*. July 2019.

Geron, A. "Machine Learning Notebooks: Autoencoders and GANS". *Github*. Mar 2020).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will train a sparse autoencoder and implement it for portfolio choice.

See you there!

Module 6: Lesson 4

Portfolio choice with a sparse autoencoder



Outline

- ▶ Building and training a sparse autoencoder.
- ▶ Autoencoder-based portfolio choice.

Building a sparse autoencoder

In this lesson, we build and train a sparse autoencoder to extract the latent factors that explain the variation of the 20 tech companies' stock returns that we already analyzed in Lesson 2.

We use the autoencoder model to design a portfolio that optimizes the risk-return trade-off as measured by the Sharpe Ratio.

The model is designed as an overcomplete linear autoencoder with a single hidden layer, whose dimension is equal to the number of stocks, $k = n = 20$.

The model shrinks the dimensionality of the encoding vector by including a Lasso-type regularization term in the loss function:

$$\left\| X - \hat{X} \right\|_F^2 + \lambda_1 \left\| \text{vec}(Z) \right\|_1$$

The value of the hyperparameter λ_1 determines how narrow is the bottleneck of the autoencoder.

Remember that we can implement the sparsity term in TensorFlow by specifying an activity regularizer:

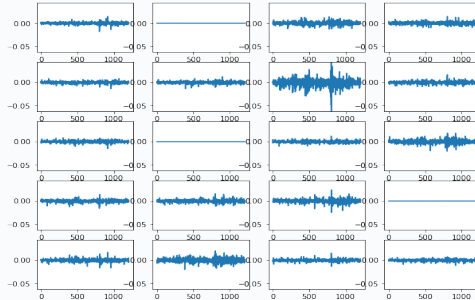
```
activity_regularizer= tf.keras.regularizers.l1(l1weight)
```

Training a sparse autoencoder

We train the autoencoder using the data from the 20 stocks, with the usual 80-20 training-test sample split.

We set aside an additional 20% of the training sample left for validation using an early stopping criterion.

- ▶ We train the model for 1,000 epochs, setting $\lambda_1 = 0.0025$.
- ▶ Training leaves 17 encodings as “relevant,” generating a small reconstruction error of roughly 5%.



Autoencoder-based portfolio choice: A model for stock returns

The trained model can be used as a basis for the design of investment strategies.

- ▶ As a byproduct, the autoencoder generates estimates for the expected returns of each stock, the exposures to systematic factors, and the amount of noise.
- ▶ We can use all this information to design diversified portfolios.

The decoding layer of the autoencoder implies that we can model the returns of stocks at time t as:

$$R_t = \hat{R}_t + \mathbf{e}_t = \mathbf{a} + B\mathbf{F}_t + \mathbf{e}_t$$

where

- ▶ $\mathbf{a}$ is an $n \times 1$ vector that includes all the trained biases in the output layer.
- ▶ B is an $n \times k$ matrix that includes all the trained weights of the output layer.
- ▶ $\mathbf{F}_t$ is a $k \times 1$ vector of realizations of the factors, which in a single-layer model are just the encodings, $\mathbf{Z}_t$.
- ▶ $\mathbf{e}_t$ is an $n \times 1$ vector of reconstruction errors, that we assume of zero-mean and uncorrelated.

Autoencoder-based portfolio choice: Risk and expected return

Using the model for stock returns from the autoencoder, we can recover estimates for the expected returns (μ) and the variance-covariance matrix of returns (Σ) from the training sample.

- ▶ $\mu = E(\hat{R}_t) = \mathbf{a} + BE(F_t)$: This is the average of the reconstructions in the training sample.
- ▶ $\Sigma = \text{Var}(R_t) = \text{Var}(\hat{R}_t) + \text{Var}(\mathbf{e}_t)$: This is the variance-covariance matrix of returns of the reconstructions, generated by the factors and the exposures, plus the diagonal (by assumption) variance-covariance matrix of reconstruction errors.

Let a portfolio be defined by a vector $w = (w_1, \dots, w_n)^T$ of weights, such that $\sum_i^n w_i = 1$.

The expected return and variance of a portfolio w can be expressed as:

$$\begin{aligned} E(R_t^w) &= w^T \mu \\ \text{Var}(R_t^w) &= w^T \Sigma w \end{aligned}$$

And the Sharpe Ratio is given by $SR^w = \frac{w^T \mu}{(w^T \Sigma w)^{1/2}}$.

Autoencoder-based portfolio choice: Performance

We devise an optimization problem, whereby we want to choose a portfolio that maximizes the return per unit of risk, as measured by the Sharpe Ratio, given the estimated μ and σ from the autoencoder.

- ▶ The solution is analytical if we do not impose short-selling restrictions.
- ▶ The optimal portfolio yields an estimated, annualized, Sharpe Ratio of 2.89 in the training sample.

We evaluate the performance of the resulting portfolio in the test sample (**Disclaimer:** These results refer to a particular initialization of the trainable parameters and may vary if other seeds are used).

- ▶ The out-of-sample Sharpe Ratio is just 0.09.
- ▶ Portfolios based on other strategies (using the sample averages and variance-covariance matrix or PCA) yield substantial negative Sharpe ratios.

The test sample features a quite different market environment than the training sample, which naturally leads to generalization issues.

However, the autoencoder seems to extract informative features that significantly outperform other, more naïve, estimation methods.

Summary of Lesson 4

In Lesson 4, we have:

- ▶ Built and trained a sparse autoencoder to extract the latent factors from a sample of stock returns.
- ▶ Used the trained model to design a portfolio that maximizes the Sharpe Ratio.

⇒ **References for this lesson:**

Conlon, T. et al. "Machine Learning and Factor-Based Portfolio Optimization." *arXiv*, 2021.

TO DO NEXT: Please go to the Jupyter Notebook associated with Lesson 4.

In the next module, we will describe transformer methods.

See you there!



- [Atom](#)
- [RSS](#)
- [Blogs](#)
- [Alpha Scientist Consulting](#)

Stock Prediction with ML: Walk-forward Modeling

Author: Chad Gray

Wed 18 July 2018

Key Takeaways:

- Traditional methods of validation and cross-validation are problematic for time series prediction problems
- The solution is to use a "walk-forward" approach which incorporates new information as it becomes available.
- This approach gives us a more realistic view of how effective our model would truly have been in the past, and helps to avoid the overfitting trap.
- It's often important to exclude data which is too far in the past using a rolling window.

Introduction

This is the fourth post in my series on transforming data into alpha. If you haven't yet see the [framework overview](#), [feature engineering guide](#), or [feature selection guide](#), please take a minute to read those first. This post will build on the framework and philosophy presented there.

This post is going to delve into the mechanics of *walk-forward modeling* which is, in my view, the most robust way to train and apply machine learning models in inherently sequential domains like finance. If you'd like to replicate and experiment with the below code, *you can download the source notebook for this post by right-clicking on the below button and choosing "save link as"*

ipynb notebook

The domain of market prediction presents several unique challenges for machine learning practitioners which do not exist in spam detection, natural language processing, image recognition, or other common areas of machine learning success, including:

- Low signal-to-noise ratio
- Non-stationarity (aka regime switching)
- Market adaptation (aka reflexivity)

The combination of these challenges make it difficult to train and tune models which *generalize* to the future, where it really matters. This, in my view, has led many to conclude that markets cannot be predicted with machine learning techniques.

This is a flawed conclusion. Instead, we should conclude that it's imperative to follow a modeling process which (1) avoids the trap of overfitting models to noise, (2) takes into account the sequential dimension of time series data, and (3) allows us to get an accurate view about how various models would have performed in the past *given the information we would have had at that time*.

"But wait!" I hear you saying, "Overfitting is not unique to market prediction!". True, but the commonly used techniques of *train-test split* and *cross-validation* each have major flaws when applied to an inherently sequential set of financial time series data.

The overriding objective of the methods described here is to **overcome the issues inherent in traditional cross validation approaches**.

Issues with typical approaches

Train-Test Split: One frequently recommended approach in finance ML tutorials is to split the data (commonly, by time) and to train on the first chunk of data and test out-of-sample on the second. This is not wrong per se, but creates two problems:

1. Only a fraction of data can be used to generate out-of-sample performance of the model. If you split 80/20, then you only get results for 2 of every 10 years of data. Yuck.
2. You're implicitly biased towards the most recent period. Since we're likely going to use the older data to train and newer data to test (for good reason...) you are implicitly searching for a model which was in favor during the most recent couple of years. Maybe last year is a good approximation for next year, but maybe it's not. Maybe 2008 would hold some important lessons for us to evaluate, but these would be lost to us.

You might consider train/test split with random (or non-random) sampling but this creates other fairly obvious issues with peeking into the future.

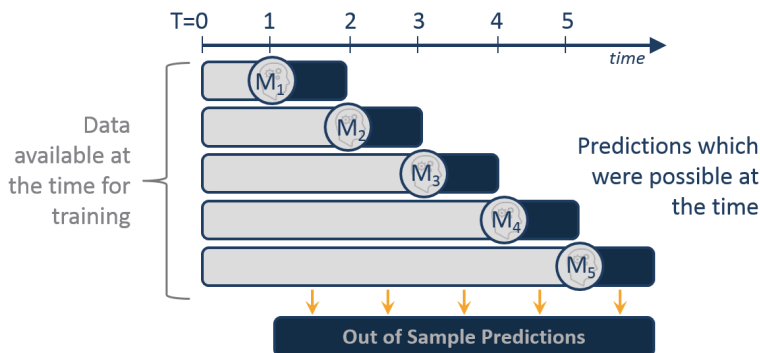
Cross Validation: In normal ML usage, cross-validation methods, such as the [K-fold cross validation](#), provide an elegant way to have our digital cake and eat it too. However, with inherently sequential datasets, it's really not kosher to train a model on 2016 and 2018 to predict 2017.

Another common method, [leave one out cross-validation](#), creates even more extreme peeking into the future. *These issues are especially intolerable if your model predicts values multiple bars in the future*, which results in your models being trained on data which significantly overlaps with your test data - a subtle but deadly form of data spillover.

The Solution: Walk-forward Train/Test

But we are not stuck with either of these problematic approaches. The better - and I think much more intuitive - approach is to simulate models in a "walk-forward" sequence, periodically re-training the model to incorporate all data available at that point in time.

The below picture is worth at least 1000 words. Say we have six years of data to train + test. At the end of the first year ($T=1$) we could have built model M_1 , which was trained on the first year's data (the grey bar). After the 5th year, we could use up to five years of data to train model M_5 .



Getting Started

As in prior posts, we'll begin by fetching some real stock price data and then generating from it a synthetic dataset which will make it easier to follow the principles of this tutorial. Please refer to the previous post [Feature Selection](#) for further explanation of the reasoning and mechanics of creating synthetic data.

```
In [1]: from IPython.core.display import HTML, Image
import sys
sys.path.append('/anaconda/')
import config

import numpy as np
import pandas as pd
pd.core.common.is_list_like = pd.api.types.is_list_like
# Needed for this issue: https://github.com/pydata/pandas-datareader/issues/534
import pandas_datareader.data as web
%matplotlib inline

def get_symbols(symbols, data_source, begin_date=None, end_date=None):
    out = pd.DataFrame()
    for symbol in symbols:
        df = web.DataReader(symbol, data_source, begin_date, end_date)[['AdjOpen', 'AdjHigh', 'AdjLow', 'AdjClose', 'AdjVolume']].reset_index()
        df.columns = ['date', 'open', 'high', 'low', 'close', 'volume'] #my convention: always lowercase
        df['symbol'] = symbol # add a new column which contains the symbol so we can keep multiple symbols in the same dataframe
        df = df.set_index(['date', 'symbol'])
        out = pd.concat([out, df], axis=0) #stacks on top of previously collected data
    return out.sort_index()

prices = get_symbols(['AAPL', 'CSCO', 'AMZN', 'YHOO', 'MSFT'], data_source='quandl', begin_date='2012-01-01', end_date=None)
# note, we're only using real price data to get an accurate date/symbol index set.
```

Similar to the [previous post](#), we will generate some random data to serve as features *and then derive* an outcome (response) variable from the features (to ensure that our made-up features will have ability to partially predict the target variable).

However, this go around we'll add two non-random patterns to the data. First, we will add some amount of "memory" (autoregression) to the white noise with a short function called `add_memory()`. This will make the data somewhat less random and more similar to actual markets - which I think we can all agree is not a purely gaussian random walk...

Second, we'll add trend to the importance of each feature to the target variable to simulate the reality that various features - and the market factors they represent - move in and out of favor over time (aka "regime shift").

- Feature f_{01} is becoming more important
- Feature f_{02} is becoming less important
- Feature f_{03} is oscillating between very important and unimportant (using a sine wave pattern)
- Feature f_{04} is not changing

```
In [2]: num_obs = prices.close.count()

def add_memory(s,n_days=50,memory_strength=0.1):
    ''' adds autoregressive behavior to series of data'''
    add_ewm = lambda x: (1-memory_strength)*x + memory_strength*x.ewm(n_days).mean()
    out = s.groupby(level='symbol').apply(add_ewm)
    return out

# generate feature data
f01 = pd.Series(np.random.randn(num_obs),index=prices.index)
f01 = add_memory(f01,10,0.1)
f02 = pd.Series(np.random.randn(num_obs),index=prices.index)
f02 = add_memory(f02,10,0.1)
f03 = pd.Series(np.random.randn(num_obs),index=prices.index)
f03 = add_memory(f03,10,0.1)
f04 = pd.Series(np.random.randn(num_obs),index=prices.index)
f04 = f04 # no memory

## now, create response variable such that it is related to features
# f01 becomes increasingly important, f02 becomes decreasingly important,
# f03 oscillates in importance, f04 is stationary, finally a noise component is added

outcome = f01 * np.linspace(0.5,1.5,num_obs) + \
           f02 * np.linspace(1.5,0.5,num_obs) + \
           f03 * pd.Series(np.sin(2*np.pi*np.linspace(0,1,num_obs)*2)+1,index=f03.index) + \
           f04 + \
           np.random.randn(num_obs) * 3
outcome.name = 'outcome'
```

So, we've now got four features which contain random and non-random elements, and we have an outcome target variable which can be partially explained by these features.

Let's run a quick linear model to verify that this worked as expected:

```
In [3]: from sklearn.linear_model import LinearRegression
model = LinearRegression()
features = pd.concat([f01,f02,f03,f04],axis=1)
features.columns = ['f01','f02','f03','f04']
model.fit(X=features,y=outcome)
print('RSQ: '+str(model.score(X=features,y=outcome)))
print('Regression Coefficients: '+str(model.coef_))

RSQ: 0.26872811401332664
Regression Coefficients: [0.97587665 1.04585218 1.01461189 0.99374324]
```

Indeed. A simple, linear model run on the entire dataset shows a remarkable (for this domain) RSQ of >0.25 and approximately equal coefficients for each of the features, suggesting that they're all contributing strongly to the model.

However, this overlooks the very important reality that our data is non-stationary and what matters is how the model will *generalize* to future, unseen data.

To illustrate, I'll re-run the same model after splitting the data into train/test sets, with the first 80% used to train and the last 20% used to test.

```
In [4]: split_point = int(0.80*len(outcome))

X_train = features.iloc[:split_point,:]
y_train = outcome.iloc[:split_point]
X_test = features.iloc[split_point:,:]
y_test = outcome.iloc[split_point:]

model = LinearRegression()
model.fit(X=X_train,y=y_train)

print('RSQ in sample: '+str(model.score(X=X_train,y=y_train)))
print('RSQ out of sample: '+str(model.score(X=X_test,y=y_test)))
print('Regression Coefficients: '+str(model.coef_))

RSQ in sample: 0.29476591841646305
RSQ out of sample: 0.13667733307962338
Regression Coefficients: [0.8361922 1.15876943 1.20372473 1.01599709]
```

Here, the results tell a different story. We see that the model's RSQ on the unseen test data degraded to only about half of the value on the training data. Even if we were still satisfied with the degraded RSQ value, we would carry concern that perhaps the next period of time the models would degrade even further.

This is where our walk-forward process comes in.

A better approach...



Here, I'll outline the basic design pattern for a robust approach to walk-forward modeling. This is simplified. In real-world usage, we'd likely add many enhancements but this will illustrate the concept.

Training models

The first step of the walk-forward approach is to train our model *using only the data available at any given point in time*. Theoretically, we have new data every bar so could retrain the model daily if we wished. However, for simplicity we'll retrain much less frequently - at the end of each calendar year.

We'll first create a list of timestamps for all end-of-year dates in our dataset. These will become our days on which to retrain models. Then, on each of the recalculation dates, we'll follow a simple procedure:

1. slice out only the data which has happened up until that point
2. train a model with that data
3. save a copy of that fitted model object in a pandas Series called `models` which is indexed by the recalculation date.

We could just as easily have used a dict or other data structure, but I find that pandas Series works well for this purpose. Pandas is not just for numbers, after all...

```
In [5]: recal_dates = features.resample('Q', level='date').mean().index.values[:-1]
        #print('recal_dates:')
        #print(recal_dates)
        #print()

        models = pd.Series(index=recal_dates)
        for date in recal_dates:
            X_train = features.xs(slice(None, date), level='date', drop_level=False)
            y_train = outcome.xs(slice(None, date), level='date', drop_level=False)
            model = LinearRegression()
            model.fit(X_train, y_train)
            models.loc[date] = model

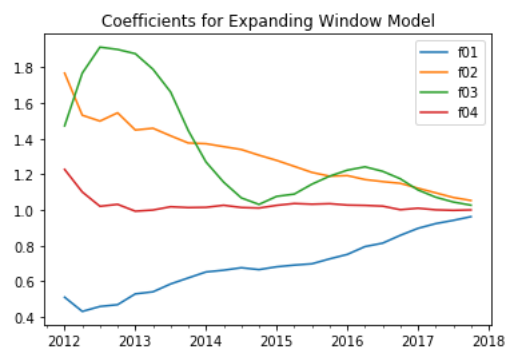
            print("Training on the first {} records, through {}"\
                  .format(len(y_train), y_train.index.get_level_values('date').max()))
            #print("Coefficients: {}".format(model.coef_))
            #print()
```

```
Training on the first 310 records, through 2012-03-30 00:00:00
Training on the first 625 records, through 2012-06-29 00:00:00
Training on the first 940 records, through 2012-09-28 00:00:00
Training on the first 1250 records, through 2012-12-31 00:00:00
Training on the first 1550 records, through 2013-03-28 00:00:00
Training on the first 1870 records, through 2013-06-28 00:00:00
Training on the first 2190 records, through 2013-09-30 00:00:00
Training on the first 2510 records, through 2013-12-31 00:00:00
Training on the first 2815 records, through 2014-03-31 00:00:00
Training on the first 3130 records, through 2014-06-30 00:00:00
Training on the first 3450 records, through 2014-09-30 00:00:00
Training on the first 3770 records, through 2014-12-31 00:00:00
Training on the first 4075 records, through 2015-03-31 00:00:00
Training on the first 4390 records, through 2015-06-30 00:00:00
Training on the first 4710 records, through 2015-09-30 00:00:00
Training on the first 5030 records, through 2015-12-31 00:00:00
Training on the first 5335 records, through 2016-03-31 00:00:00
Training on the first 5655 records, through 2016-06-30 00:00:00
Training on the first 5975 records, through 2016-09-30 00:00:00
Training on the first 6290 records, through 2016-12-30 00:00:00
Training on the first 6600 records, through 2017-03-31 00:00:00
Training on the first 6905 records, through 2017-06-30 00:00:00
Training on the first 7155 records, through 2017-09-29 00:00:00
Training on the first 7403 records, through 2017-12-29 00:00:00
```

To visualize how the model is changing over time, I'll create a simple plotting function of coefficients:

```
In [6]: def extract_coefs(models):
        coefs = pd.DataFrame()
        for i,model in enumerate(models):
            model_coefs = pd.Series(model.coef_,index=['f01','f02','f03','f04']) #extract coefficients for model
            model_coefs.name = models.index[i] # name it with the recalc date
            coefs = pd.concat([coefs,model_coefs],axis=1)
        return coefs.T
        extract_coefs(models).plot(title='Coefficients for Expanding Window Model')
```

Out[6]: <matplotlib.axes._subplots.AxesSubplot at 0xc6e68d0>



Notice a few things about how the model's coefficients change over time:

- The first feature gets increasing weight as it becomes a more important explanatory factor in the target variable,
- the second feature does just the opposite, as it should,
- the third feature oscillates before converging on a relatively lower endpoint, and
- the fourth feature is mostly unchanged

But, in a world of constantly shifting regimes, we can do better.

We know that, by the end of the dataset, f01 is actually much superior to f02 however they end at about equal weights. This is because in our final model building instance, we are using *all* of our data to train, and this entire set of data correctly shows that *on average* f02 is as useful as f01, ignoring that f02's best days are long past.

```
In [7]: recalc_dates = features.resample('Q', level='date').mean().index.values[:-1]

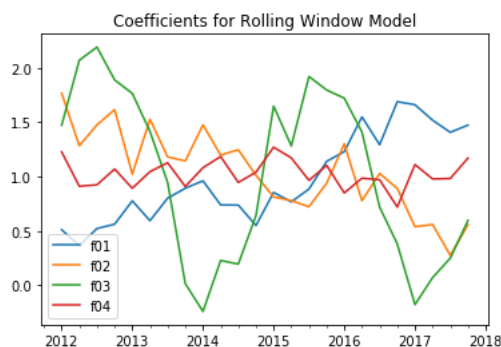
models = pd.Series(index=recalc_dates)
for date in recalc_dates:
    X_train = features.xs(slice(date-pd.Timedelta('90 days'), date), level='date', drop_level=False)
    y_train = outcome.xs(slice(date-pd.Timedelta('90 days'), date), level='date', drop_level=False)
    model = LinearRegression()
    model.fit(X_train, y_train)
    models.loc[date] = model

    print("Training on the most recent {} records".format(len(y_train)))
    #print("Coefficients: {}".format(model.coef_))

extract_coefs(models).plot(title='Coefficients for Rolling Window Model')
```

```
Training on the most recent 310 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 305 records
Training on the most recent 305 records
Training on the most recent 320 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 310 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 310 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 305 records
Training on the most recent 320 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 310 records
Training on the most recent 305 records
Training on the most recent 250 records
Training on the most recent 248 records
```

```
Out[7]: <matplotlib.axes._subplots.AxesSubplot at 0xa611a58>
```



This approach of only considering the 90 days' data prior to each model re-training has had a big impact. We now see that f_{01} has overtaken f_{02} in significance by the end of the time period, as we'd hope. The on-again, off-again feature f_{03} has risen and fallen in prominence over time, again just as we'd hope.

Using models

The second stage of the process is *using* these walk-forward models. The process is similar and equally simple.

First, we create two arrays, `begin_dates` and `end_dates`, which contain the dates on which each model is used. For instance, for the first model ($i=0$), we will apply this model to features beginning on the date we trained the model (`recalc_date`) until the day the next model is trained. The `end_dates` array therefore drops the 0th element and appends to the end a far-off date (in the year 2099).

We can equally easily use this pattern to apply the models to true out of sample features. As long as it's not later than the year 2099, we'll simply apply the latest and greatest model we have.

```
In [8]: begin_dates = models.index
end_dates = models.index[1:].append(pd.to_datetime(['2099-12-31']))

predictions = pd.Series(index=features.index)

for i,model in enumerate(models): #Loop thru each models object in collection
    X = features.xs(slice(begin_dates[i],end_dates[i]), level='date', drop_level=False)
    p = pd.Series(model.predict(X), index=X.index)
    predictions.loc[X.index] = p

predictions.shape
```

```
Out[8]: (7639,)
```

So, this looped through each of the models we had trained at various points in time and, for each, used that model to predict the period of time until the next model became available. It runs!

Now we can confirm that the rolling model is, in fact, better than the "expanding" model by making predictions with each and comparing to truth.

```
In [9]: models_expanding_window = pd.Series(index=recalc_dates)
        for date in recalc_dates:
            X_train = features.xs(slice(None,date),level='date',drop_level=False)
            y_train = outcome.xs(slice(None,date),level='date',drop_level=False)
            model = LinearRegression()
            model.fit(X_train,y_train)
            models_expanding_window.loc[date] = model

        models_rolling_window = pd.Series(index=recalc_dates)
        for date in recalc_dates:
            X_train = features.xs(slice(date-pd.Timedelta('90 days'),date),level='date',drop_level=False)
            y_train = outcome.xs(slice(date-pd.Timedelta('90 days'),date),level='date',drop_level=False)
            model = LinearRegression()
            model.fit(X_train,y_train)
            models_rolling_window.loc[date] = model

        begin_dates = models.index
        end_dates = models.index[1:].append(pd.to_datetime(['2099-12-31']))

        predictions_expanding_window = pd.Series(index=features.index)
        for i,model in enumerate(models_expanding_window): #Loop thru each models object in collection
            X = features.xs(slice(begin_dates[i],end_dates[i]),level='date',drop_level=False)
            p = pd.Series(model.predict(X),index=X.index)
            predictions_expanding_window.loc[X.index] = p

        predictions_rolling_window = pd.Series(index=features.index)
        for i,model in enumerate(models_rolling_window): #Loop thru each models object in collection
            X = features.xs(slice(begin_dates[i],end_dates[i]),level='date',drop_level=False)
            p = pd.Series(model.predict(X),index=X.index)
            predictions_rolling_window.loc[X.index] = p
```

```
In [10]: from sklearn.metrics import r2_score

        common_idx = outcome.dropna().index.intersection(predictions_expanding_window.dropna().index)
        rsq_expanding = r2_score(y_true = outcome[common_idx],y_pred=predictions_expanding_window[common_idx])
        rsq_rolling = r2_score(y_true = outcome[common_idx],y_pred=predictions_rolling_window[common_idx])

        print("Expanding Window RSQ: {}".format(round(rsq_expanding,3)))
        print("Rolling Window RSQ: {}".format(round(rsq_rolling,3)))

        Expanding Window RSQ: 0.256
        Rolling Window RSQ: 0.296
```

Great! All of that work added about 3 points to the RSQ, which is certainly worth the effort.

Avoiding complexity

The value of this walk-forward methodology is greatest when it helps you to avoid the scourge of overfitting. The linear regression model used up to this point is relatively resistant to overfit, since it has few parameters.

Let's say that, instead of a simple linear regression, we used the much more overfit-prone DecisionTree, which has tendency to "memorize the past" rather than to recognize patterns in it.

```
In [11]: from sklearn.tree import DecisionTreeRegressor

        split_point = int(0.80*len(outcome))

        X_train = features.iloc[:split_point,:]
        y_train = outcome.iloc[:split_point]
        X_test = features.iloc[split_point:,:]
        y_test = outcome.iloc[split_point:]

        model = DecisionTreeRegressor(max_depth=3)
        model.fit(X=X_train,y=y_train)

        print('RSQ in sample: '+str(round(model.score(X=X_train,y=y_train),3)))
        print('RSQ out of sample: '+str(round(model.score(X=X_test,y=y_test),3)))

        RSQ in sample: 0.179
        RSQ out of sample: 0.008
```

This leads to overfit disaster! Now, we'll do the same with our walk-forward framework:

```
In [12]: recalc_dates = features.resample('Q',level='date').mean().index.values[:-1]

models_rolling_window = pd.Series(index=recalc_dates)
for date in recalc_dates:
    X_train = features.xs(slice(date-pd.Timedelta('365 days'),date),level='date',drop_level=False)
    y_train = outcome.xs(slice(date-pd.Timedelta('365 days'),date),level='date',drop_level=False)
    model = DecisionTreeRegressor(max_depth=3)
    model.fit(X_train,y_train)
    models_rolling_window.loc[date] = model

predictions_rolling_window = pd.Series(index=features.index)
for i,model in enumerate(models_rolling_window): #Loop thru each models object in collection
    X = features.xs(slice(begin_dates[i],end_dates[i]),level='date',drop_level=False)
    p = pd.Series(model.predict(X),index=X.index)
    predictions_rolling_window.loc[X.index] = p

common_idx = y_test.dropna().index.intersection(predictions_rolling_window.dropna().index)
rsq_rolling = r2_score(y_true = y_test[common_idx],y_pred=predictions_rolling_window[common_idx])
print("RSQ out of sample (rolling): {}".format(round(rsq_rolling,3)))

RSQ out of sample (rolling): 0.108
```

Much, much better. While the RSQ result ain't great (and actually worse than the linear regression model), that's much preferred to being fooled by an extraordinarily overfitted result like above. We could have guessed that DecisionTree would be a silly model type to try in this situation but it illustrates the point.

Summary

That, in a nutshell, is the walk-forward modeling framework. The key points to keep in mind are:

- Traditional methods of validation and cross-validation are problematic for time series prediction problems
- The solution is to use a "walk-forward" approach which incorporates new information as it becomes available.
- This approach gives us a more realistic view of how effective our model would truly have been in the past, and helps to avoid the overfitting trap.
- In addition to excluding any future data from the training process, it's often important to exclude data which is too far in the past.

In the next post of this series, I will present a framework for scoring and comparing models which is more complete and useful than generic machine learning methods.

One last thing...

If you've found this post useful, please follow [@data2alpha](#) on twitter and forward to a friend or colleague who may also find this topic interesting.

Posted by Chad Gray Wed 18 July 2018 [Blog tutorial, modeling](#)

Post

Comments

ALSO ON ALPHASCIENTIST.COM

Stock Prediction with ML: Feature ...

8 years ago • 17 comments

If you haven't yet seen the first post for this series, please take a minute to ...

ML for Stock Market Prediction (part 1): ...

8 years ago • 4 comments

About this tutorial¶ This is the first in a six-part series on the mechanics of ...

Our Own Worst Enemy — The ...

8 years ago • 5 comments

Key Takeaways¶ ETF investors appear to be consistently bad at timing ...

Reddit for Fun and Profit [part 2] — ...

5 years ago • 1 comment

In the prior post Tracking Posts on WallStreetBets - Part I, we demonstrated ...

Stock Prediction with ML: Data ...

8 years ago • 10 comments

About this tutorial¶ This is the first in a six-part series on the mechanics of ...

G

Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name



2

Share

Best Newest Oldest**Robert Morris**

7 years ago edited

Really great and easy to follow tutorial on walk forward validation. You've presented a much simpler and useful method the the TimeSeriesSplit library provided in sklearn, in my opinion Thank you for sharing.

3 0 Reply

**Alexander Rios**

5 years ago

Hi Chad, would it make sense to implement a walk forward matrix to select the model?

I mean an array as described in the following link:

<https://tradingtact.com/dev...>

0 0 Reply

CP

Chris Pa

7 years ago

Hej Chad,

I really appreciate your series as I find it very useful and well-explained. I have got one question about the walk-forward modeling. For my understanding it's main purpose is to re-optimize the values after each "year" so to say.

I'm using an ANN to train my model and I thought this ANN is already doing that kind of re-optimization after each batch. So the question is: Does it make sense to use the walk-forward modeling using an artificial neural network at all?

Much appreciation from Germany,
Chris

0 0 Reply

A

Arbitrage- Technologies

8 years ago

Hi Chad,

thx for all these part on machine learning.

i am convinced that we can do much moore better than forward walk.

from my small experience in ml there is a very thin frontier between overfitting and robust leanring, which lies only in training set.

i thinks that the best path is to asses the distributions of your features, as you have been doing in other post, then fit them to the best distrib.

once you got the parameters of the distrib of each features, you just simulates a huge huge number of path and you train on that!

may be you have tried it in the past...

the overall question after that is how to build a real stock simulator engine which stick to reality...

0 0 Reply

**alphascientist** Mod

→ Arbitrage- Technologies

8 years ago

... And one common mistake I see made (not exactly related to this but sort of...) is using cross validation of randomly shuffled data on models trained for X days ahead prediction, thinking that you've estimated out of sample performance.

In other words, take 3000 days of data, each training models for a 50 day ahead horizon. Split that into 10 CV splits of 300 days each. Best case, if you keep sequential days together (days 1 to 300, 301 to 600 etc...) you are actually peeking into your held-out fold with each CV iteration. When holding out days 301 to 600 for testing, the feature values and outcome values for day 300 and day 301 are almost identical (since the target variable is 49/50ths overlapped). You're unintentionally peeking somewhat for 49 of the 300 days in each test fold.

Even worse, if randomly shuffled (fold 1 is days 1,19,22,38, ...) then there's heavy peeking for every single data point. Hopefully this makes sense w/o diagrams!

I mention this in conjunction with walk-forward because it's an example of how you can fool yourself into thinking you've built an exceptional model that backtests very well if using traditional ML techniques like K-fold CV. If you do whatever you do *inside of* a walk forward

framework, you're automatically guaranteed not to peek.

o o Reply ↗



alphascientist Mod → Arbitrage- Technologies

8 years ago

Hi JC, I think I understand what you're saying: develop huge amounts of synthetic data from the estimated distributions of each feature. I assume this would include modeling the distribution of the feature value itself as well as a distribution of the covariance of each feature with each other feature, since that's maybe the most critical parameter to get right if you want to successfully use those features in combination for prediction.

I haven't gone down this road, but know others have - and it seems to make sense if (a) you have confidence that you can get the distributions right and (b) you use algos that need that much data (e.g., complex DL networks).

Even if doing this, you would still need to walk-forward in the sense that you would want to simulate what your models would have looked like a year ago, or two years ago, or three years ago based on what you would have estimated the distributions to be *at that time* based on history to that point.

Walk forward is really just a way of forcing yourself to be honest about whether a given approach would have been successful in the past. Force yourself to retrain your models - using whatever approach - based on data which would have been available to you at a given point in time. Use predictions from those models to see whether you've found a durable approach or just a hindsight bias approach.

The flip side of this coin is that whatever model you trade live can benefit from all data which has happened in all of history, since the WF model you train today has access to all data which happened prior to today. If you choose to do monte carlo, synthetic data (as you suggest), or plain old actual history, you still make use of all data up to this point.

- Chad

o o Reply ↗



顏聰文

8 years ago

Hi Chad, have you tried using other Neural Network models, such as RNN, LSTM, or A3C? Is the performance better than a linear regression model? Thanks.

o o Reply ↗



alphascientist Mod → 顏聰文

8 years ago

I've dabbled (fully connected networks, LSTMs) and have generally found those models to be less robust out of sample than those limited by linearity constraints. That's not to say it can't be successfully done - and I haven't given up on trying.

I do think the right approach is to build a very simple model as baseline, then benchmark more exotic approaches against that. It's very good to use a framework like I present here (true walk forward, custom evaluation metrics) to get a more accurate idea of how complexity helps/harms performance

1 o Reply ↗

D

David Toohey

8 years ago

Great series - just a few simple edits are required: all of the hyperlinks to previous articles in the series link to this current article instead.

o o Reply ↗

C

Chad Gray → David Toohey

8 years ago

Hi David, thanks for pointing this out. I'm mobile right now but will try to fix next time I'm able to re-publish.

o o Reply ↗



alphascientist Mod

8 years ago

Thanks Dustin - glad you're enjoying them!

o o Reply ↗



Dustin Espersen

8 years ago

Fantastic series Chad! I'm really enjoying reading through these!

o o Reply ↗

[Subscribe](#)

[Privacy](#)

[Do Not Sell My Data](#)

Recent Posts

- [Reddit for Fun and Profit \[part 2\]](#)
- [Reddit for Fun and Profit \[part 1\]](#)
- [COVID By The Numbers](#)
- [Contribute to The Alpha Scientist blog!](#)
- [Stock Prediction with ML: Ensemble Modeling](#)

Tags

[data](#), [web scraping](#), [COVID](#), [administrative](#), [tutorial](#), [modeling](#), [keras](#), [ETFs](#), [market anomaly](#), [feature engineering](#), [pandas](#), [seasonality](#), [SPY](#)

Receive email notifications for future posts

Email Address

First Name

Subscribe

Social

- [RSS](#)
- [Atom](#)
- [Linkedin](#)
- [Twitter](#)

Latest Tweets

[Tweets by data2alpha](#)

- Status updating...

Follow @data2alpha

Blogroll

- [Quantocracy](#)
- [Quant-at-Risk](#)
- [Robot Wealth](#)
- [QuantStart](#)
- [Simon Ouellette](#)
- [Marcos Lopez de Prado](#)
- [Financial Hacker](#)
- [Knoema](#)
- [AlternativeData.org](#)
- [Eagle Alpha](#)
- [Neudata](#)
- [Cloudquant](#)
- [Quiver Quant](#)
- [Real Vision](#)
- [Cryptoquant](#)
- [Glassnode](#)
- [Diffbot](#)
- [TheGraph](#)
- [Mnemonic](#)
- [Yewno](#)

- [Yipit](#)